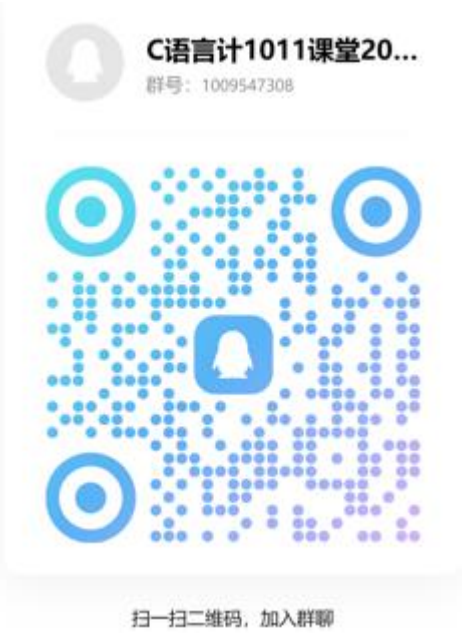
C语言与程序设计

The C Programming Language

第1章 概述

华中科技大学计算机学院

毛伏兵



第1章 概论

- 本章重点

1.2 学习C语言的第一个例子
1.4 计算机系统及内存编址
1.5 数和字符的编码表示
1.6 算法及其表示

1.1 程序设计语言与程序设计

- 了解什么是程序设计语言，了解什么是程序，了解什么是程序设计，就是了解本门课程的基本研究对象。这些对于学习语言和程序设计来讲是非常必要的。
- 计算机语言包括机器语言、汇编语言和高级语言这三类语言。如果不涉及汇编语言，程序设计语言往往就是指高级语言。

1.2 例1--输出:Hello,world!

```
/* printf “Hello,world !” in screen */
#include <stdio.h>
int main( )
{
    printf ( "hello, world!\n " ) ;
    return 0;
}
```

/* printf “Hello,world !” in screen */
Comment
注释
#include <stdio.h>
Include information about standard library
文件包含
int main( )
Main function
主函数
{
Body
函数体
printf ("hello, world!\n ") ;
}
Library function
库函数
Arguments
函数参数

进一步： 输入名字，计算机问候自己.

名字---字符串，

- 如何输入字符串?
- 如何保存?
- 如何输出保存的信息?

```c
#include <stdio.h>
int main(void)
{
    char name[20];

    printf("Input your name please!\n");
    scanf("%s", name );       //  vs(Visual Studio)   scanf_s
    printf("Hello %s!\n", name);
    return 0;
}
```

如何输入成绩?

```
#include <stdio.h>
int main(void)
{
    char name[20];
    int  score;

    printf("Input your name and score, please!\n");
    scanf("%s", name );       //  fgets(name,20,stdin); 最多读19个字符
    scanf("%d", &score);    //  decimal     string
    printf("Hello %s!, your score is %d\n", name, score );
    return 0;
}
```

【思考】scanf的%s格式和fgets使用中的区别

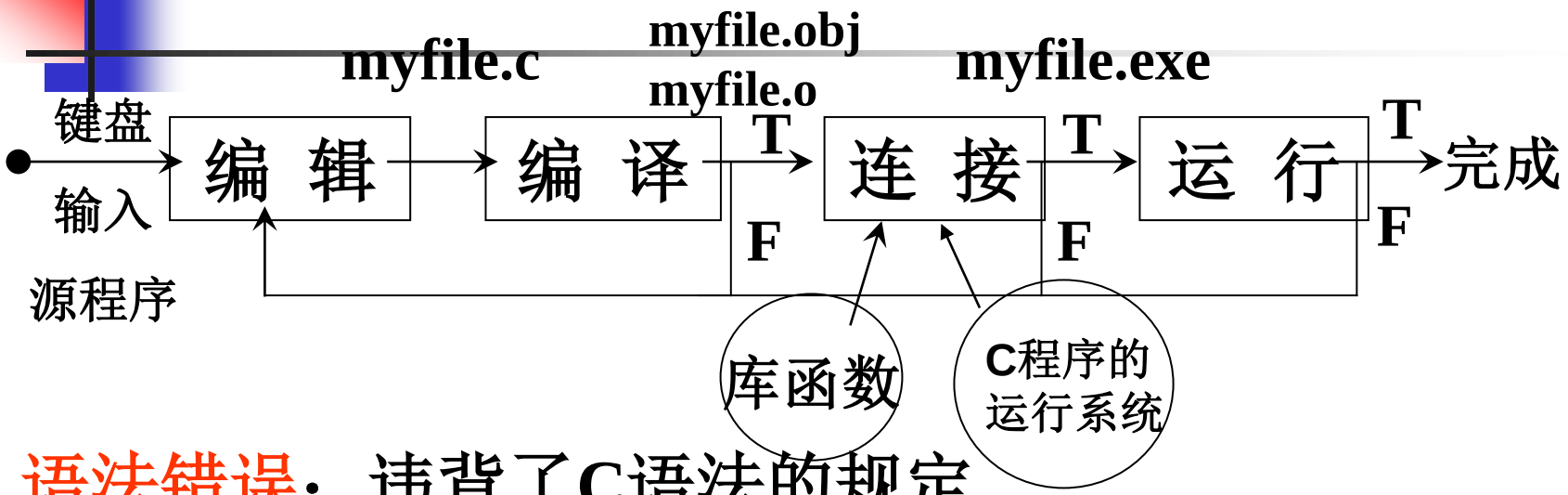
【练习】不同类型数据的输入输出 %c %d %f %s

如何定义函数？让计算机问候某人.

```
#include <stdio.h>
void show(char str[ ]);          // 函数说明

int main(void)
{    char name[20];
     printf("Input your name please!\n");
     scanf("%s", name );
     show(name);                 // 函数调用
     return 0;
}
void show ( char str[ ] )        // 函数定义
{
   printf("Hello %s!\n", str);
}
```

Code::Blocks

运行C程序的步骤



语法错误：违背了C语法的规定

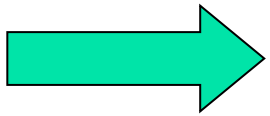
实际错误可能出现在编译程序指定错误位置前面很远处；一个实际错误有时会使编译程序产生许多出错信息行 。

应当注意系统给出的警告信息（WARNING）。警告常预示着隐藏较深的实际错误，必须认真弄清原因

逻辑错误：程序无违背语法规则，但程序执行结果与原意不符，这是因为程序设计人员通知给系统的指令与原意不相同，即出现了逻辑上的混乱

作业

- 1、安装VC或C:B编译器或Dev C++，熟悉运行C程序的步骤。



1.3 C语言的产生、发展与语言特征

1.3.1 C语言的产生与发展

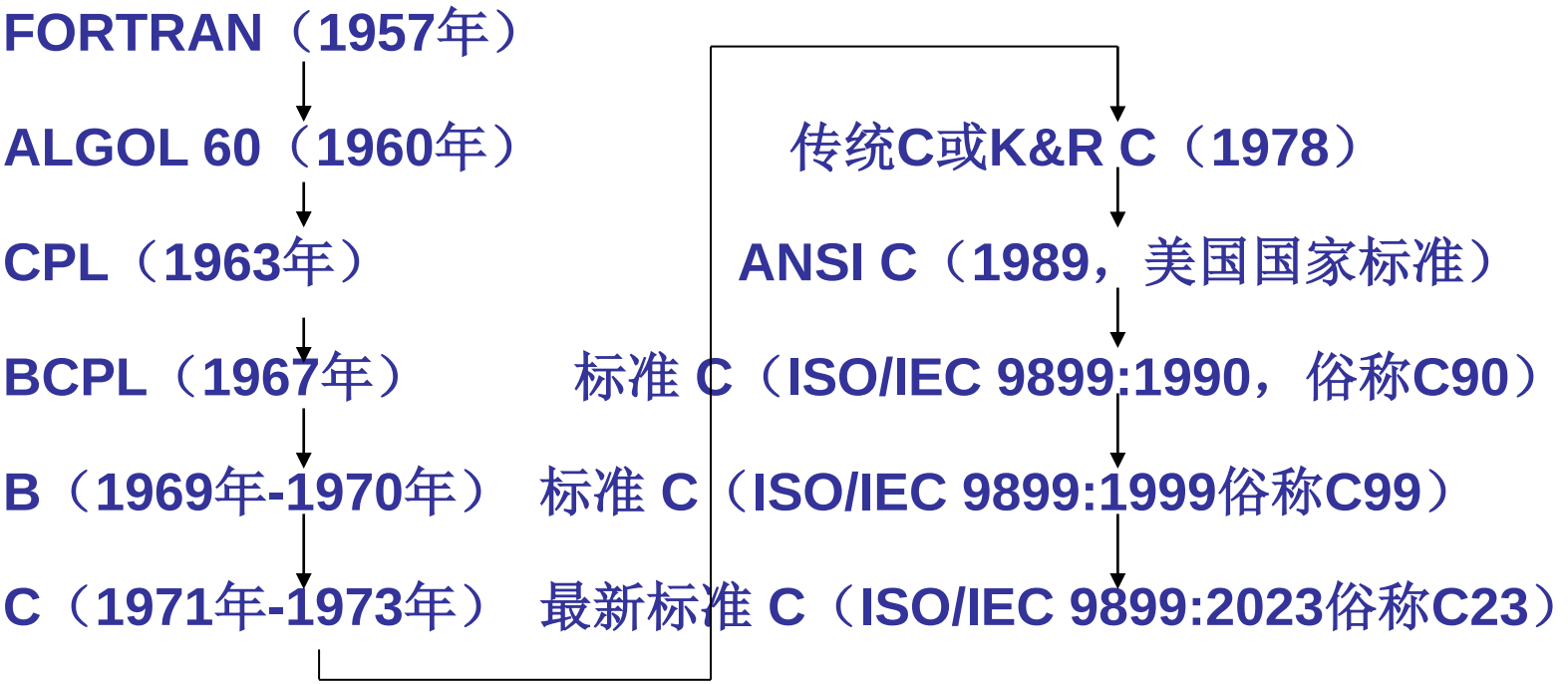


图1.1 C语言的继承、产生与发展历程

1.3.2 C语言的标准化

- 以1978年K&R C为代表的C语言被称为传统的C语言
- 1989年底公布美国第一个C语言的国家标准ANSI 89，简称C89
- 1990年，国际标准化组织ISO将其接受为C语言的国际标准，称为ISO/IEC 9899-1990。它是C语言的第一个国际标准，也称为标准C，简称C90。
- ISO/IEC在1995年公布了一个新的C语言标准草案，称为C95，供讨论和征求意见。
- 接着ISO/IEC在1998年又公布新标准的草案WG14/N843和WG14/N897，进一步就C语言标准的完善征求意见。
- ISO/IEC于1999年12月公布了C语言国际标准ISO/IEC 9899:1999 (E)，C99。它是C语言国际标准9899的第二版。
- ISO/IEC于2011年12月公布了C语言国际标准ISO/IEC 9899:2011，C11。它是C语言国际标准9899的第三版。

1.3.3 C语言的特征

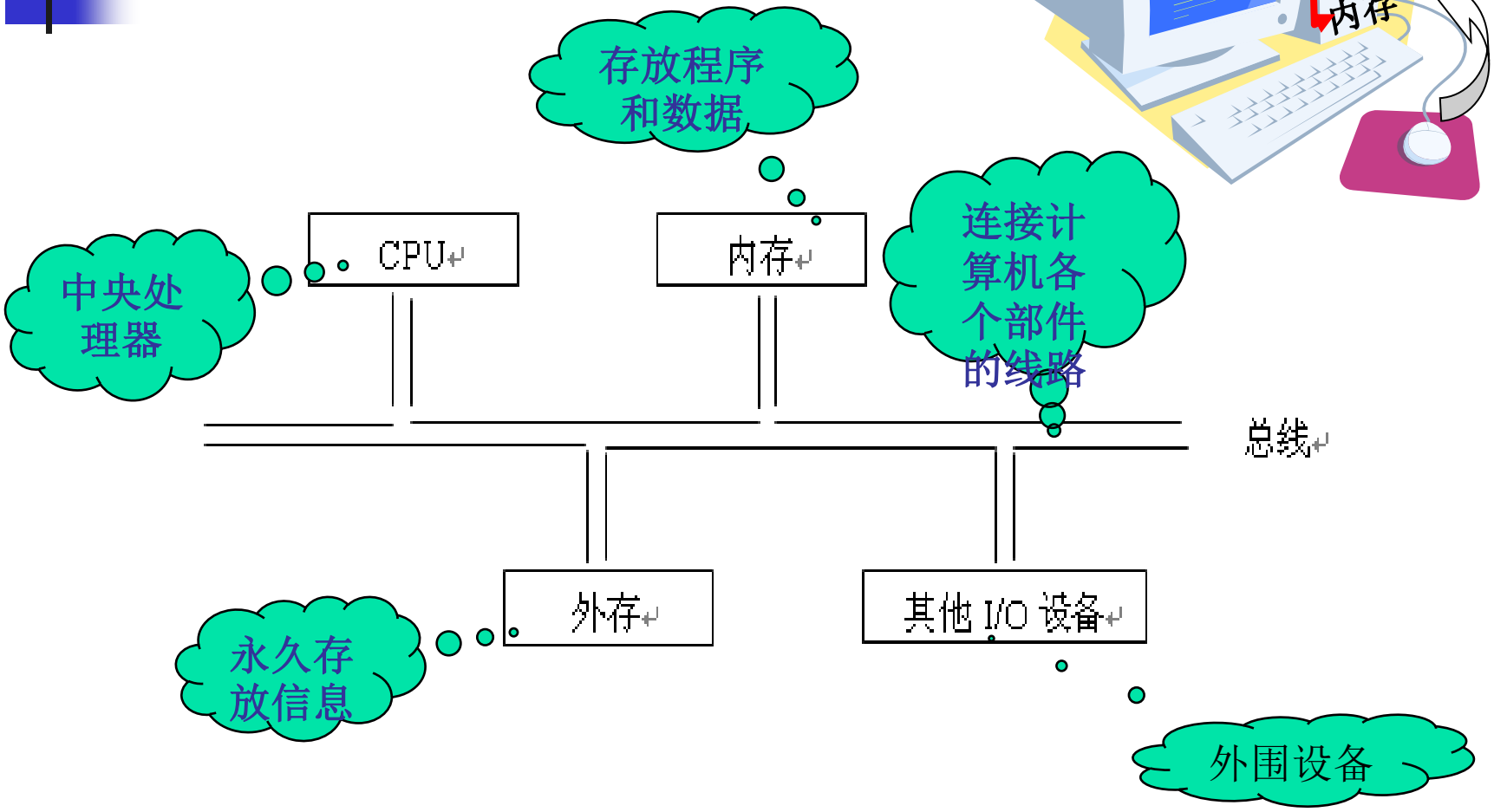
1) 语言简洁紧凑
2) 目标代码质量高
3) 语言表达能力强
4) 流程控制结构化
5) 弱类型
6) “中级语言”特性
7) 书写自由、使用灵活
8) 可移植性好

1.4 计算机系统及内存编址

- 一台计算机硬件系统由CPU、内存、外存、其他I/O设备和总线组成。
- CPU 又称为中央处理器 。 CPU又是由运算器、控制器、指令计数器、内部寄存器、标志寄存器等部件组成。
- 内存又称为内存储器，用以存放程序和数据。
- 总线是用以连接计算机各个部件的线路，它完成各个部件之间的信息传送。根据传送信号种类的不同，总线又分成地址总线、数据总线和控制总线。

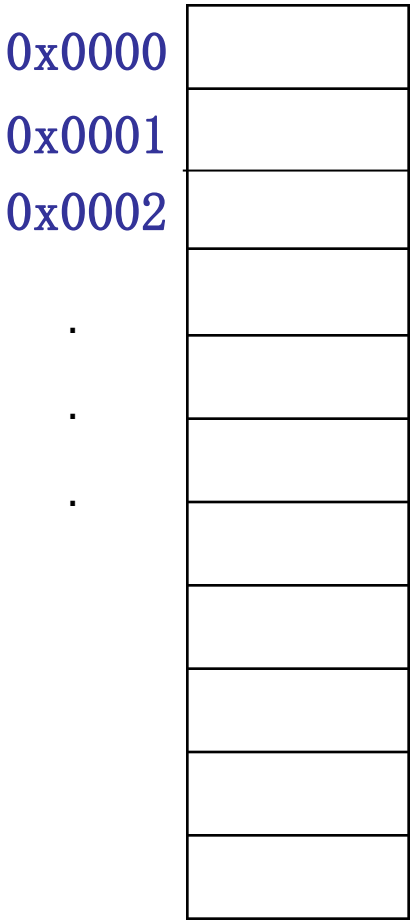
计算机硬件系统

执行结果
硬盘
CPU
内存
程序和数据


存放程序和数据
中央处理器
CPU
内存
连接计算机各个部件的线路
总线
永久存放信息
外存
其他 I/O 设备
外围设备

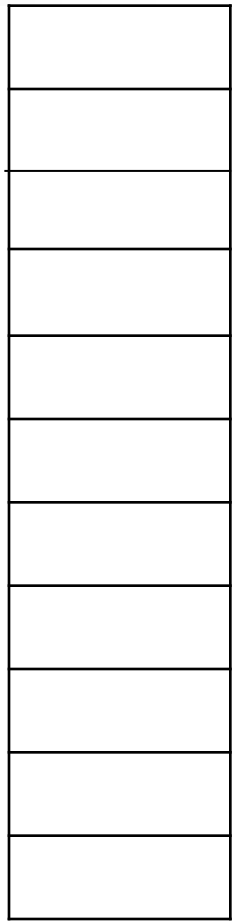
内存地址

- 内存以字节为单位线性连续编址。即按照0x0000，0x0001，0x0002，…的方式；从低地址端开始向高地址端为每一个内存字节进行顺序连续编号。



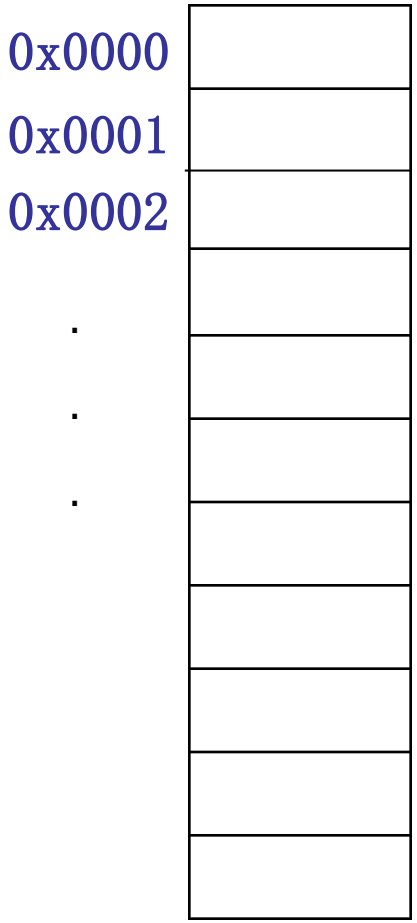
b, B, KB, MB, GB, TB

- 一个位（bit）有多大？
  - 只能是“0”或者“1”，这叫二进制
  - 1b
- 一个字节（Byte）有多大？
  - 1B=8b
  - 保存一个字符（英文字母、数字、符号）
    - ASCII（美国标准信息交换码）编码
  - 两个字节保存一个汉字
    - GB编码



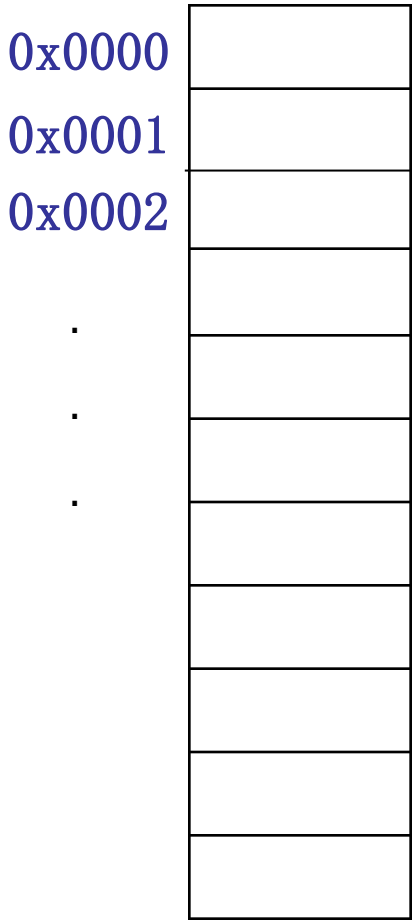
b, B, KB, MB, GB, TB

- 1B=8b
- 1KB（Kilobyte）=1024B
- 1MB（Megabyte）=1024KB
- 1GB（Gigabyte） =1024MB
- 1TB（Terabyte） =1024GB



字长

- CPU数据总线的宽度（bit数）称为计算机的机器字长。对８位ＣＰＵ，机器字长为１字节；对１６位ＣＰＵ，机器字长为２字节；对３２位ＣＰＵ，机器字长为４字节。



1.5 数和字符的编码表示（自学为主）

计算机采用二进制表示法

- 计算机为什么用二进制呢？
- 为什么不用我们日常熟悉的十进制呢？
  - 二进制在电器元件中容易实现
  - 计算机进行二进制运算比进行十进制运算要简单得多

但是C中没有二进制数（不能直接输入输出二进制数），只支持十进制（%d）、八进制（%o）和十六进制（%x）数的表示。

- **进位制数的运算**
- **进位制数之间的转换？**

1.5.1 进位计数制

二进制数：只有0、1， 逢2进位1

八进制数：0~7， 逢8进位1

在C语言中，通过加前导零的方式来表示一个数是八进制数。如：0136表示的是八进制数$(136)_8$。

十六进制数：0~9 a~f(或A~F)，逢16进位1

在C语言中，通过加前导0x或前导0X的方式来表示一个数是十六进制数。如：0x2d5b表示的是十六进制数$(2d5b)_{16}$。

1.5.2 进位制数之间的转换

1、二、八、十六进制转换为十进制

用按权展开的方式，以二进制为例：

$$\underset{\uparrow}{1}\ \underset{\uparrow}{1}\ \underset{\uparrow}{0}\ \underset{\uparrow}{1}\ \underset{\uparrow}{1}\ .\ \underset{\uparrow}{1}\ \underset{\uparrow}{0}\ \underset{\uparrow}{1}\ _{(2)}$$

$$2^4\ 2^3\ 2^2\ 2^1\ 2^0\ \ 2^{-1}\ 2^{-2}\ 2^{-3}$$ 权值

$$= 1\times 2^4 + 1\times 2^3 + 0\times 2^2 + 1\times 2^1 + 1\times 2^0 +$$

$$1\times 2^{-1} + 0\times 2^{-2} + 1\times 2^{-3}$$

$$= 27.625$$

2. 十进制整数转换为二、八、十六进制整数

用除2（8、16）取余法

例1.6 用除2取余法求十进制整数189的二进制表示。

解

```
2 | 1 8 9
  2 | 9 4      ……余 1（最低位）
    2 | 4 7    ……余 0
      2 | 2 3  ……余 1
        2 | 1 1……余 1
          2 | 5……余 1
            2 | 2……余 1
              2 | 1……余 0
                0……余 1
```

∴ $(189)_{10}=(10111101)_2$

3、二、八、十六进制整数的相互转换

二 —> 八进制数： 3位分组法

二 —> 十六进制 ： 4位分组法

$(10111101)_2$: $(275)_8$

$(10111101)_2$: $(bd)_{16}$

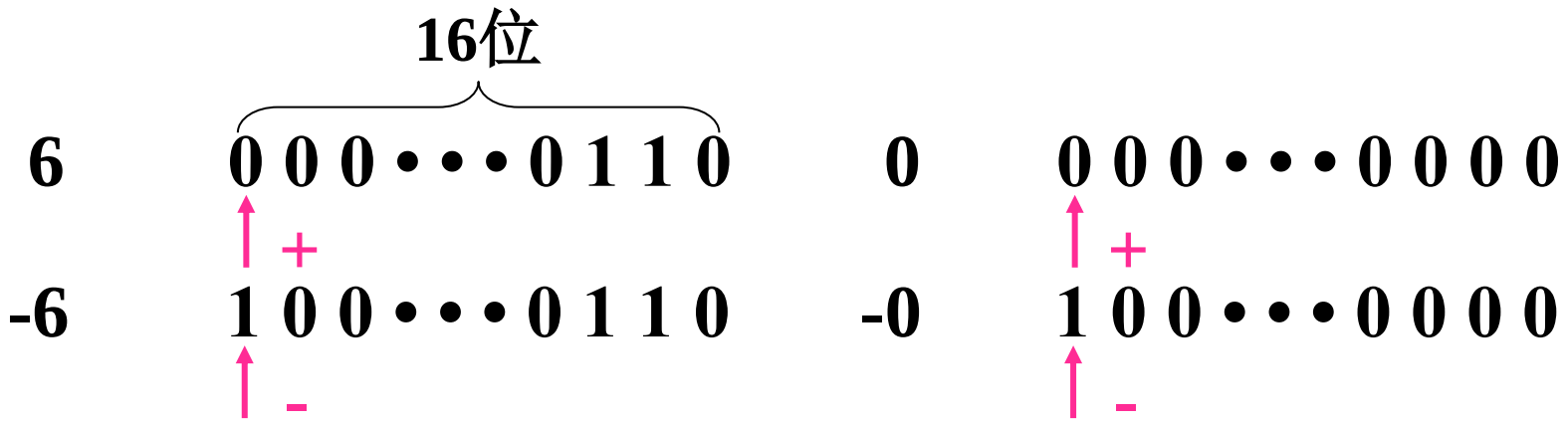
1.5.3 数的机器码表示

1. 机器数与真值

- 机器数：在计算机中，将一个数的最高位定义为符号位，其余各位为数值位。并且规定符号位之值为0表示正，符号位之值为1表示负。用这种方法表示的数称为机器数。
- 真值：机器数的数值称为该机器数的真值。也就是正、负号后跟二进制数的绝对值就构成真值。

2. 原码

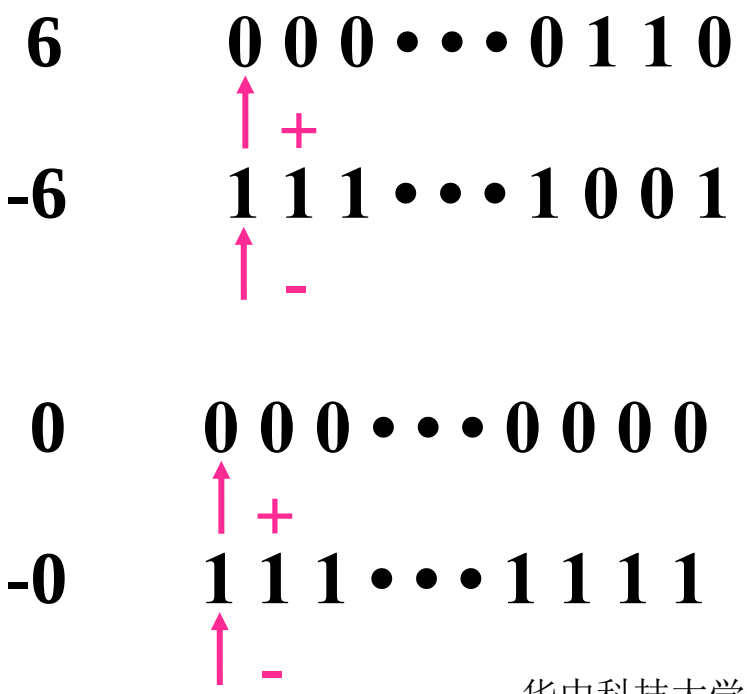
最高位为符号位，其余各位为该数的绝对值。



0的原码表示不惟一

3. 反码

- 正数：同原码
- 负数：最高位为1，其余各位对原码取反



4. 补码

正数：同原码

负数：反码 + 1

6	0 0 0 • • • 0 1 1 0	0	0 0 0 • • • 0 0 0 0
-6	1 1 1 • • • 1 0 0 1	-0	1 1 1 • • • 1 1 1 1
	+1		+1
	1 1 1 • • • 1 0 1 0		0 0 0 • • • 0 0 0 0

计算机是以补码形式存放数据的

（钟表原理）

5. 码的内存存储形式

- 码是指计算机内存中存储的数据。根据存储数据的性质，码可以分为数值码、字符码和图象码。
- 数值码用于表示数的大小，常见的有整数和浮点数。

  整数: 补码形式存放

  浮点数:见第2章
- 字符码用于表示英文字母、标点符号、控制字符以及汉字和其它文字的编码。
- 图象码是用于表示图象中象素的颜色或灰度。虽然它们都是以二进制码的表现形式存储在内存之中，但物理意义却各不相同。
- C语言的基本处理对象是字符和数字。在16位计算机中，整型数占2个字节，在32位计算机中，整型数占4个字节。

1.5.4 字符的编码表示

- 1. 字符的编码的概念
- 计算机在屏幕上能够输出英文、汉字，乃至各种各样的民族文字，其基础在于对字符进行编码以及构造相应的点阵字模。
- 用户的字符输入通过编码在内存中以二进制数码存储，机器内部对字符的处理实际上是对字符编码的处理。
- 输出的时候则是通过字符的编码去索引对应的点阵字模到内存中，然后进一步通过内存中的点阵字模去控制显象管，在屏幕的指定位置将字符显示出来。

2．ASCII码

- 西文字符的编码国际上采用的是**ASCII码**（**American Standard Code For Information Interchange**）,是美国国家信息交换标准字符码的英文缩写。
- 完整的**ASCII码**字符集请参阅**附录1**。
- **ASCII码**采用单字节编码，并且规定字节的最高位留做校验位，只有低**7**位参与编码。

3．汉字编码

- 根据标准规定和计算机的输入、存储和显示过程，汉字编码有许多方式。常用的有国标码、区位码、拼音码、五笔字型码和汉字的机内码。
- 拼音码和五笔字型码用于汉字的输入。
- 国标码：1981年颁布的《通用汉字字符集(基本集)及其交换码标准》GB2312–80。
- 采用两字节编码方案。

1.6 算法及其表示

- **1.6.1 算法的定义**

  算法是指为解决某个问题所采取的方法和步骤，并且要具备下列性质。

(1) 有零个或多个执行算法所需要的数据作为输入。

(2) 有一个或多个经算法处理的数据作为输出。这种数据也称为算法的执行结果。

(3) 应只包含有限个步骤，且执行算法所需的时间也是有限时间。算法的这种性质称为算法的有限性或有穷性。

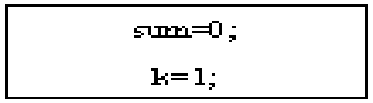
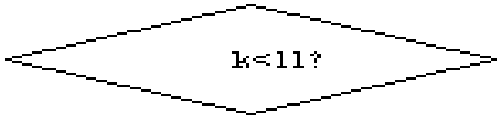
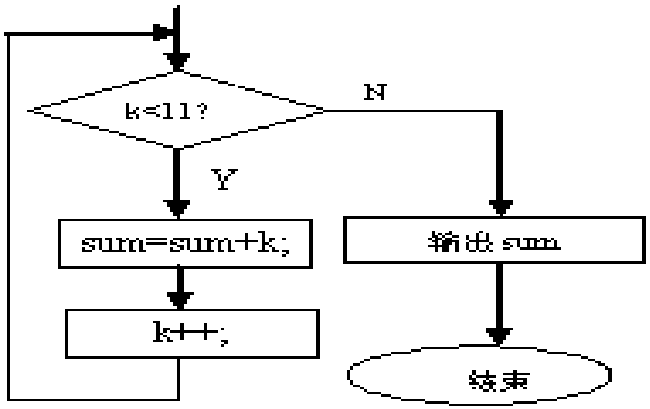
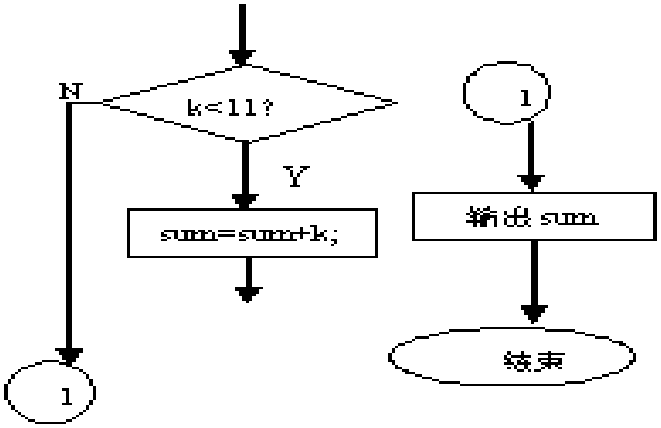
(4) 算法每一步所规定的动作应该是唯一确定的，没有二义的。算法的这种性质称为算法的确定性。

1.6.2 算法的表示

- 选用一种合适的工具或方法来表达算法设计者的算法思想称为算法的表示或算法的描述。
- 算法的表示方式多种多样，如自然语言方式，流程图方式、表格方式、伪码方式,等等。
- 自然语言方式显然非常容易理解，但是在将算法转换为具体程序方面不够直观，它最适合表达算法的设计思想。
- 流程图方式则非常具体，易于将它描述的算法编写成具体的程序，但是在程序规模变大的情况下，用流程图来完整描述算法不仅多余，而且不太可能。因此，流程图方式尤其适合初学者。本书重点介绍流程图方法。

1. 流程图的图符

- 流程图又称为框图。它是用它的图符来表示算法的每一步骤以及这些步骤之间的相互联系。学习用流程图来表达自己的算法思想首先要从认识组成流程图的图符的功能入手。流程图图符及其功能由表1.4 描述。

		的结束	
	处理框	表示按顺序执行操作步骤	
	判断框	表示判断选择。根据框中列出的二值判断条件满足与否，从两种备选步骤中选择其一执行	
	流程线	连接两个相邻的步骤。表示执行顺序从从头尾向箭头。	Y：Yes，N：No 
	连接点	表示一条流程线北断开后的两个端点。连接点必须相同的形式成对出现。	

2. 用流程图描述算法

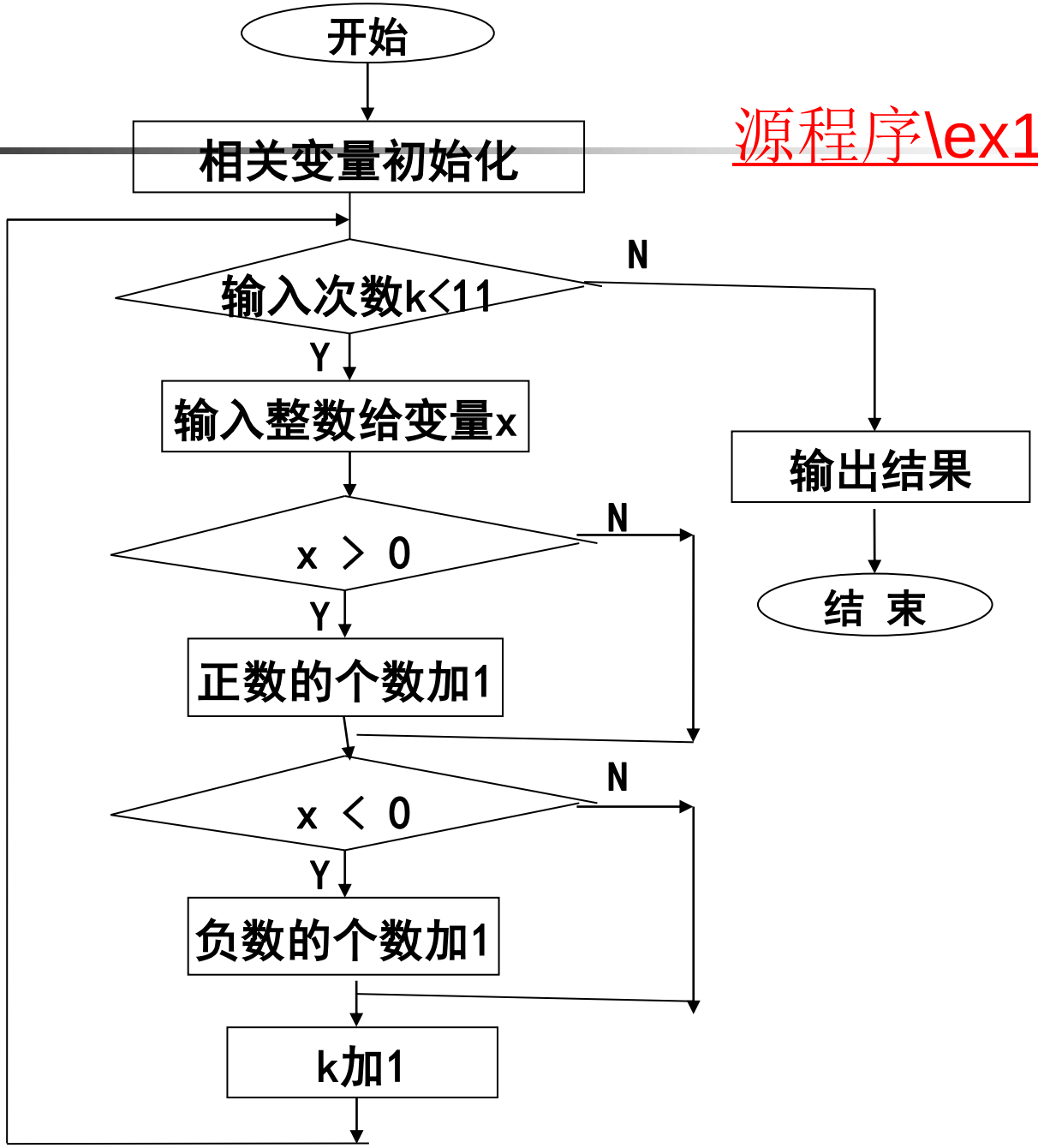
- 问题：输入10个整数，统计正数和负数的个数。
- 算法设计

  变量x：存储输入的整数，

  k：表示输入的次数，

  positive：表示正数的个数，

  negtive：表示负数的个数。


开始
相关变量初始化
输入次数k<11
N
Y
输入整数给变量x
输出结果
x > 0
N
Y
结 束
正数的个数加1
x < 0
N
Y
负数的个数加1
k加1

源程序\ex1_15.c

1.7 学习C语言与程序设计的方法

作为初学者，如何学会、掌握、以及熟练使用C语言？如何用C语言去解决实际工作中的应用问题呢？这主要涉及到四个方面的学习和训练。

一、学习并理解C语言的语法和语义

二、学习并掌握一些基本数据结构和常用算法的设计

三、学习并熟悉C语言的集成开发环境

四、熟悉C语言相关的库函数

1）学习并理解C语言的语法和语义

- 语义指的是构成C语言的各种符号的含义。
- 由于C程序是由C语言的符号所组成，因此只有准确理解各种符号的语义，才有可能合适的使用各种符号来表达自己的编程思想。
- 另一方面，C的语法规定了C语言符号集中各种符号的结合方式，规定了各种表达式、语句、函数、乃至程序结构等的构造规则。
- 只有理解和掌握C语言的语法，才有可能正确的用各种符号去构造程序的各种成分，并进而构成满足算法要求、能解决实际应用问题的程序。

2) 学习并掌握一些基本数据结构和常用算法的设计

- C语言对机器硬件有很强的操纵能力，在大多数情况下可以代替汇编语言。因此，要学好用好C，需要对字符、整型变量、数组元素在内存中的存储形式，对原码、补码，对字节中的位（bit）的操作做到熟练掌握。
- 深刻理解各类指针及所指对象的声明、使用以及物理含义。对结构，结构数组、链表都要熟练掌握。
- 由于文本是一个基本研究对象，对文本操作的各种算法，如：统计字符个数、单词个数、行数，字符串的各种操作所涉及的各种算法，以及教材例子中给出的各种算法（如排序、查找等）都应该熟练掌握。这样才能奠定程序设计的扎实基础。

3) 熟悉集成开发环境

- 几乎所有的程序设计语言都有自己的集成开发环境。集成开发环境将源程序的创建、编辑，以及其后的编译、链接，乃至程序的调试与跟踪全部都集成于一体。为编程者提供了方便的程序开发环境。因此，编程者要能够进行程序的设计和开发，调试与跟踪，就必须学会使用相应的集成开发环境。编程者只有学会使用集成开发环境提供的这些功能，才能进行程序的设计与开发。

4) 熟悉库函数

- 在C语言中，编译系统提供了一些标准的库函数。如输入/输出的标准库函数、字符串处理的标准库函数、数学计算的标准库函数等。编程者只要知道这些标准库函数的功能、调用方式、返回何种结果，并且能够在自己的程序中调用这些标准库函数就行了。完全没有必要自己去编写这些函数。

本章小结

- 本章首先通过介绍学习C语言程序设计的第一个例子，说明了如何创建并运行第一个C程序同时，对第一个C程序进行了解释分析。
- 从C语言的产生与发展，C语言的标准化，以及C语言的特征三个方面对C语言进行了概述。
- 简单介绍了计算机硬件系统的概念。
- 从进位计数制、数的机器码表示，以及字符的编码表示三个方面介绍了数和字符的机器码表示。
- 尤其是ASCII码，要熟练掌握。
- 在算法方面，介绍了算法的定义、算法的表示、算法的程序实现。
- 最后，从学习并理解C语言的语法和语义、学习并掌握一些基本数据结构和常用算法的设计、熟悉集成开发环境，以及熟悉库函数四个方面介绍了C语言的学习方法。

C语言与程序设计

The C Programming Language

第2章 基本数据类型与表达式

华中科技大学计算机学院

毛伏兵

第2章 基本词法语法规则与程序元素

本章重点

2.3 标识符、关键字及分隔符

2.4 基本数据类型

2.5 常量与变量

2.6 运算符和表达式

2.7 位运算符和位表达式

2.8 类型转换

2.9 枚举类型

(2.1 、2.2 ， 2.10了解)

2.1 字符及词法元素

2.1.1 字符集

C源程序由字符序列构成，其字符集为：

- 英文字母：a~z 和A~Z
- 数字字符：0~9
- 特殊字符：！ “ # % & ‘ ( ) * + ， - . /：
  ； < > = ? [ ] \ ^ _ { } | ~
- 空白字符：空格、换行符、水平制表符（HT）、
  垂直制表符（VT）、换页符（FF）

2.1.2 词法元素

词法元素称为记号(token)，记号是程序中具有语义的最基本组成单元。记号共分5类：标识符、关键字、常量、运算符和标点符号。编译器从左至右收集字符，总是尽量建立最长的记号，即使结果并不构成有效的C语言程序。相邻记号可以用空白符或注释语句分开。

词法分析(记号)举例

- 例2.1 sum=x+y

  分解成sum、=、x、+和y 共5个记号。

- 例2.2 int a，b=10;

  分解成int、a、，、b、=、10和； 共7个记号

- 例2.3 x+++++y

  分解成x、++、++、+、y 共5个记号

2.2 语法规则

如何描述计算机语言的语法规则？

2.2.1 BNF （Backus-Naur Form）范式

BNF:巴科斯范式是由 **John Backus** 和 **Peter Naur** 首先引入的用来描述计算机语言语法的符号集。

现在，几乎每一位新编程语言书籍的作者都使用巴科斯范式来定义编程语言的语法规则。

FORTRAN之父--John Backus (1924.12.3 - 2007.3.17)

早年在HillSchool学习的时候因为讨厌学习，成绩一踏糊涂而不得不在暑假补课。

1943年到维吉尼亚大学学习化学，随后参军、照顾头部受伤的伤员、在医学学校学习治疗，可是最后又都放弃了。

战后Backus进入纽约哥伦比亚大学学习数学，于1949年毕业。之后，成为了一名IBM的程序员。

在IBM，Backus的才华得到了施展，发明了人类历史上第一个高级语言——FORTRAN。接着，又提出了规范描述编程语言语法的Backus-Naur Form(BNF)。

这位当年的“差生” 于1977年获“图灵奖”。

获奖原因：可实用高级编程系统设计，特别是其在FORTRAN语言方面的设计，以及其在编程语言规约的形式化描述方面的，深奥的，杰出影响力的和持续的贡献。

2005年图灵奖获得者 Peter Naur

生于1928年10月25日于丹麦。

1949年从Copenhagen Unviersity(哥本哈根大学）天文学的硕士学位

1957年获天文学的博士学位。

1959年，Naur加盟丹麦第一个计算机公司。并且领导了Algol 60语言的定义。

1969年，Naur成为哥本哈根大学的教授，直到1998年退休。

2005年A.M.图灵奖的获得者。

获奖原因：他设计的Algol 60语言，由于其定义的清晰性，成为了许多现代程序设计语言的原型。在语法描述中广泛使用的BNF范式，其中的“N”便是来自Peter Naur的名字。

BNF范式的符号

- 尖括号(< >)内包含的为必选项。
- 竖线(|)表示在其左右两边任选一项，相当于“OR”的意思。
- ::= 是“被定义为”的意思。

BNF范式示例--标识符的BNF范式

- <标识符>::= <字母>|<标识符><字母数字>
  <字母数字> ::= <字母>|<数字>
- <字母> ::= _|<大写字母>|<小写字母>
- <小写字母> ::= a|b|c|d|…|z
  <大写字母> ::= A|B|C|D|…|Z
- <数字>::= 0|1|2| 3 | 4 | 5 | 6 | 7 | 8 | 9

2.2.2 EBNF

EBNF是BNF的一种扩充。EBNF中符号有：

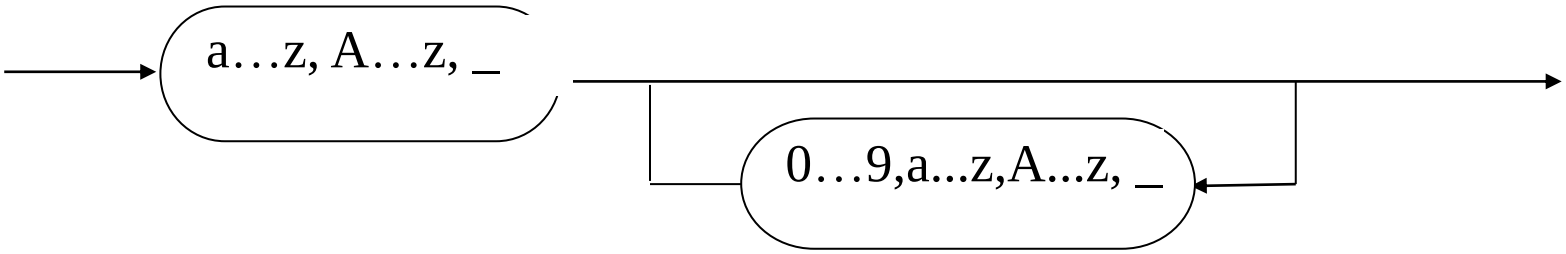
- { }：括起来的部分重复0次或多次。
- [ ]：括起来的部分出现0次或1次。
- ( )：表示结成一组。

标识符的EBNF范式：

- <标识符> ::= <字母>{<字母数字> }

2.2.3 语法图

- 语法图是另一种表示语法的常见方式。
- 标识符的语法图



2.3 标识符、关键字及分隔符

2.3.1 标识符

- 标识符是用来标识用户定义的常量、变量、数据类型和函数等名字的符号。其命名规则:
- 以一个字母(a~z, A~Z)或下划线(_)开头，后跟字母、下划线或数字(0~9)

例： K，_id , month, time1 √

20_sum , not#me ， void ×

注意

- 大小写字母表示不同意义。
- 不能使用类似 **int** 和 **void** 这样的C关键字为自己的对象命名，也要避免使用C程序库中函数和常量的名称，例如 **scanf** 。
- 良好的编程风格是选择有助于记忆且有一定含义的标识符，这样可增强程序的可读性和程序的文档性。

2.3.2 关键字

- 是被系统赋予特定含义并有专门用途的标识符，不能作为普通标识符，但可以作为宏名。

  int 、void、return、while、if

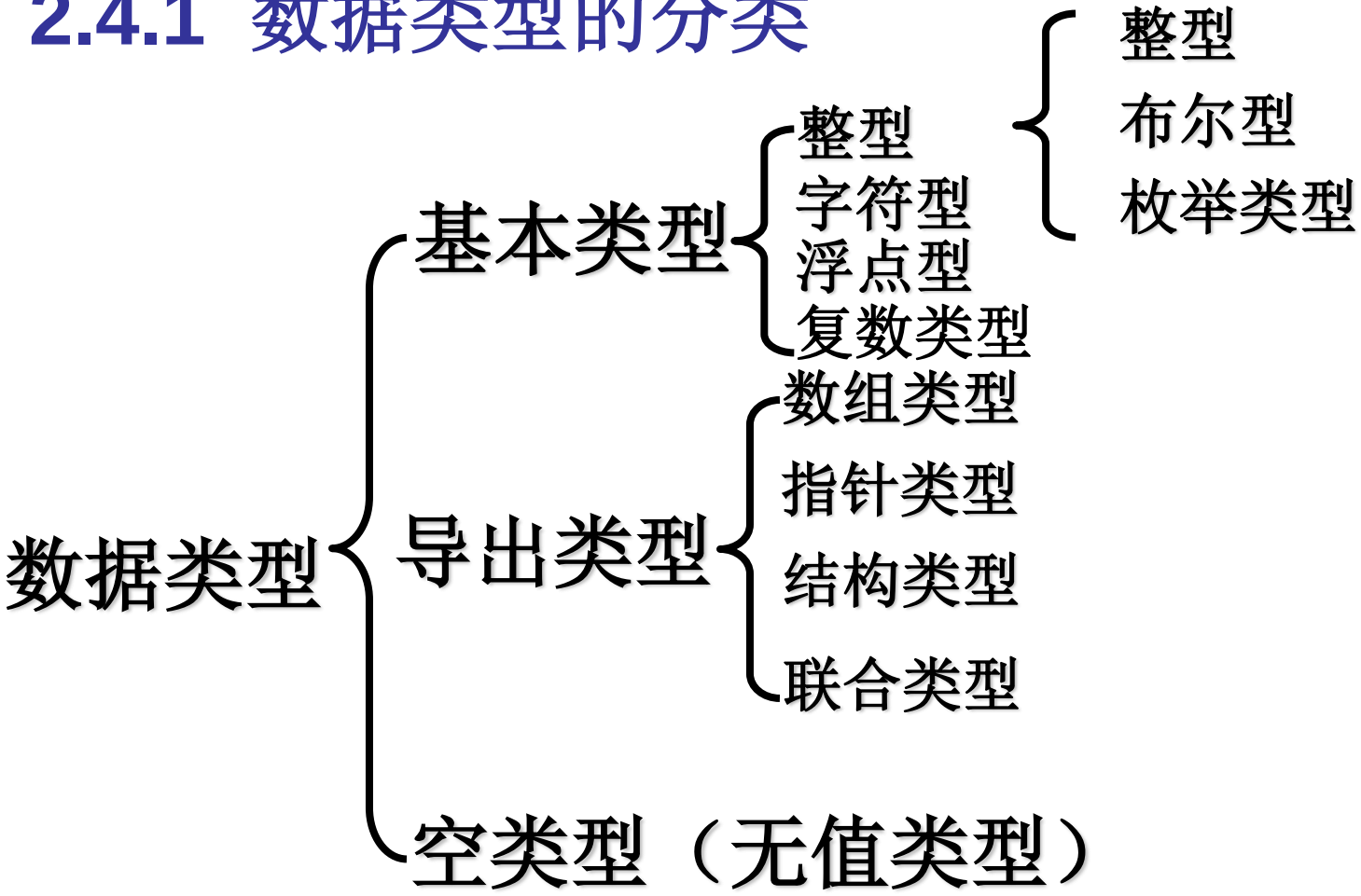
2.3.3 分隔符

- 分隔符统称为空白字符(包括空格符、制表符、换行符、换页符及注释符)，在语法上仅起分隔单词的作用。
- 当程序中两个相邻的单词之间如果不用分隔符就不能区分开时则必须加分隔符（通常用空格符)。
- 例如，int x,y；不能写成 intx,y；

  能写成 int x , y ；

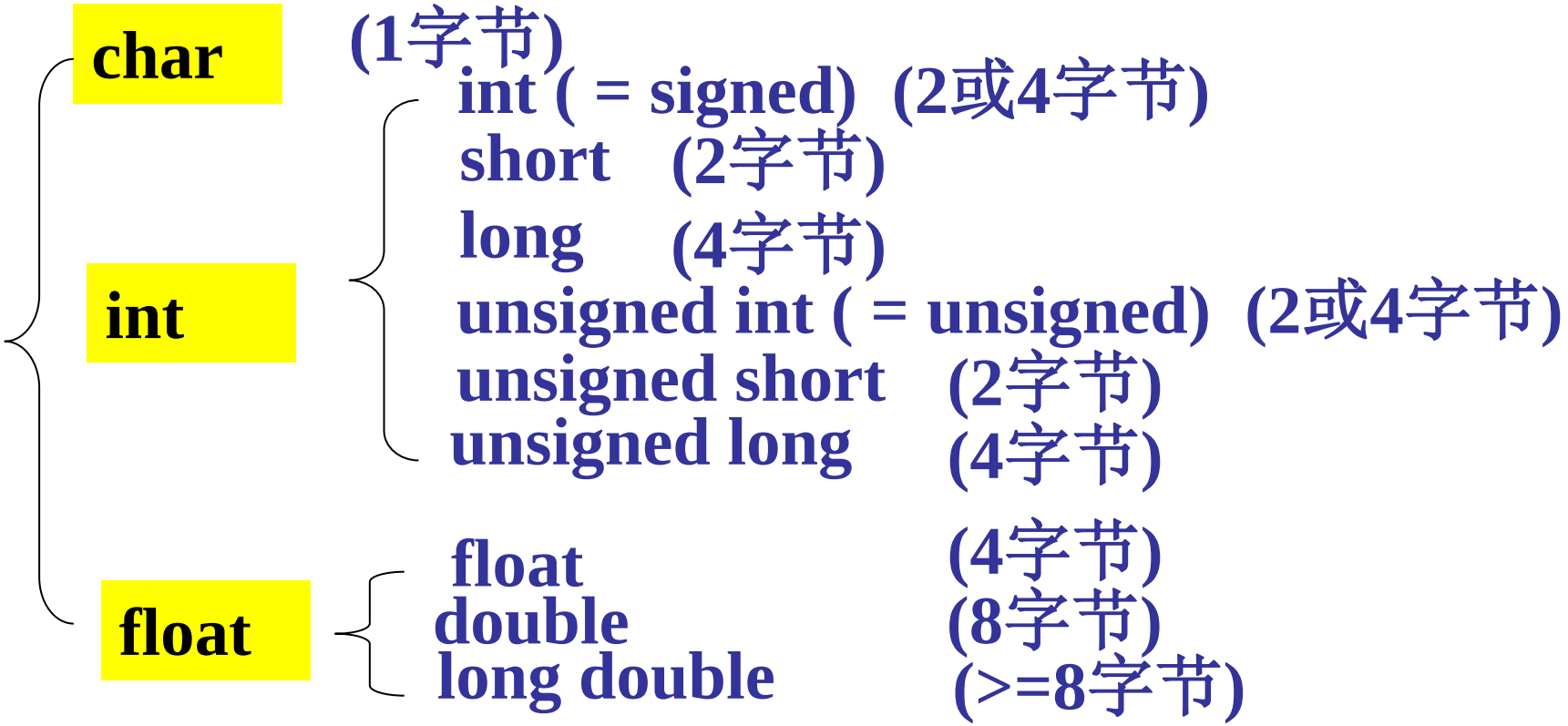
2.4 基本数据类型

2.4.1 数据类型的分类



2.4.2 基本类型的名字

- 本小节介绍字符型、整型、浮点型 (P_{24}, P_{25})



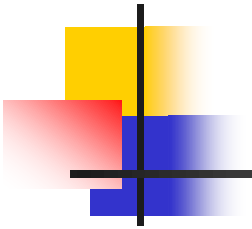
2.4.3 字符类型char

- char的存储长度是一字节 。
- 多数系统中char与signed char同(-128~127).
- 字符数据以ASCII码存储在内存中 。
- 在不要求大整数的情况下，可用字符型代替整型 。

2.4.4 整型类型

- int型值存储在一个机器字中．
- 假设字长为2B，int取值范围为-32768~32767,
  unsigned取值范围为0 ~ 65535
- 下面的代码是否正确

```
#define BIG 30000
int main(void)
{  short x,y,z;
   x=y=BIG;
   z=x+y;          /*  短整数溢出  */
    ……
}
```

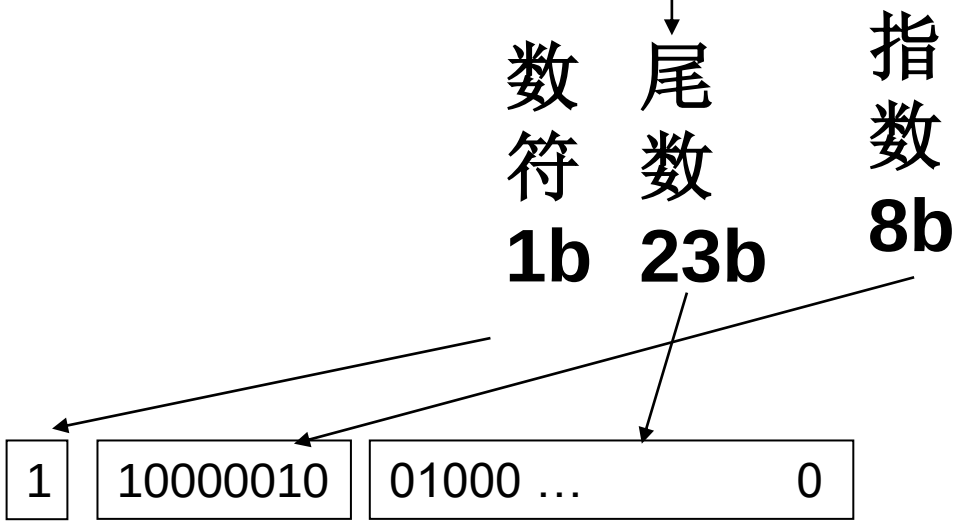


- 程序员必须时刻保证整数表达式的值在合理范围内。
- 引入short和long的目的是为了提供各种满足实际要求的不同长度的整数。int通常反映特定机器的自然大小，short一般为2B，long一般为4B。因此，当关心存储时，用short；当需要较大的整数值时，用long。

2.4.5 浮点类型

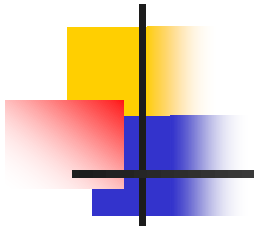
- 一个浮点数N可表示为：

$$V = (-1)^S \times M \times 2^E$$



-10.0=-1.010*2^3=

（书P27）



- 浮点数的表示可能只是近似的。其值与表示法之间的差称为“可表示误差”。
- 计算也可能造成可表示误差。不能使用= =和！=运算符比较浮点数据。
- 可以用两个数值之差同一个预定的小正数epsilon比较的方法解决这个问题。

2.5 常量与变量

2.5.1 文字常量

1. 整型常量

有三种表示方法（通过前缀字符区分）：

- 十进制：无前缀
- 八进制：前缀为0
- 十六进制：前缀为0x或0X时。

例如，31可写成037，也可写成0x1f或0X1F

整型常量可以带有后缀，用以指定其类型：

- 字母u或U表示unsigned
- 字母l或L表示long
- 字母ul或UL表示unsigned long
- 字母ll或LL表示long long (C99)
- 字母ull或ULL表示unsigned long long (C99)
- 无后缀时，一般表示int
- 当常量值超出指定类型的范围时，其实际类型取决于数值大小、前缀等，确定类型的规则很复杂，在标准化前的C语言、C89和C99中各不相同

2. 浮点型常量

有两种表示方式：

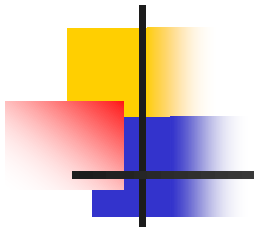
（1）带小数点的十进制数形式（可以小数点开头，也可以小数点结尾）

如23.7，14.，.126

（2）指数形式（科学计数法）

将指数部分跟在尾数部分后面。尾数部分的书写规则与第一种相同，但可以没有小数点，指数部分e(E)±n，代表10±n。

如 45e-3 = 45×10^{-3}， .15e5 = 0.15×10^{5}。



可以使用后缀来指定其类型

- 无后缀：double，
- 后缀f或F：float，
- 后缀l或L：long double。

3. 字符常量

(1) 用单引号包含的一个字符是字符常量

(2) 只能包含一个字符

‘a’，’A’，‘1’

‘abc’ 、“a”

转义序列

- 以\开头的特殊字符称为转义序列,有两种形式：
- 一种是“字符转义序列”，即反斜线后面跟一个图形符号，用于表示字符集中的非图形符号和一些特殊的图形字符。

\n　换行　　\t　水平制表符

\\　反斜杠　　\‘　单引号

\“　双引号　　\0　空字符

\?　问号

\源程序\ex2_4.c

数字转义序列

- 转义序列的另一种是“数字转义序列”，即
  \ooo（1~3个八进制数字）
  \xhh （ 1~2个十六进制数字）
- 例如，

'A'、'\101' 和'\x41'　　字符A;

'\t'、'\11'、'\011'、'\x9'和'\x09'　水平制表符

4. 字符串常量

- 写成用一对双引号括住0至多个字符的形式。

"string\n"　/* 包含7个字符的字符串 */

""　/* 包含0个字符的空字符串*/

- 字符串中的单引号可以用图形符号表示，但双引号和反斜线必须用转义序列表示。例如：

"3'40\""　/* 表示5个字符的字符串：3'40"　*/

“c:\tc”　/* 表示4个字符的字符串 */

“c:\\tc”　/* 表示5个字符的字符串　*/

如何将一个较长的字符串写成多行？

有两种方法：

（1）行连接：在前一行的末尾输入续行符（\） 再换行。

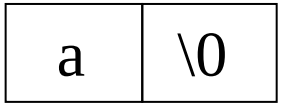
```
"Hello,\
  how are you“   /* 换行后应紧靠行首 */
```

（2）字符串连接：将字符串分段，分段后的每个字符串用双引号括起来。

```
"Hello, "
"how are you"    /* 换行后不必紧靠行首*/
```

‘a’与“a”有何区别？

- ‘a’：字符常量，占1 B内存空间
  “a”：字符串常量，占2B内存空间



- 存储时，系统自动在后面补上\0（空字符，ASC11值为0，作为字符串结束标志）
- *字符串的存储长度比字符串的实际长度大1*

求字符串的长度

```
#include <stdio.h>
int main( )
{
   char s[20]=“world ”;    /* 用于保存串 */
   int i;          /* 计数器变量，用于保存串的长度 */
   i=0;            /* 计数器变量清零 */
   while(s[i] !='\0')
         ++i;
  printf（“%d”，i）；
  return 0；
}
```

2.5.2 符号常量

用一个标识符表示一个常量.

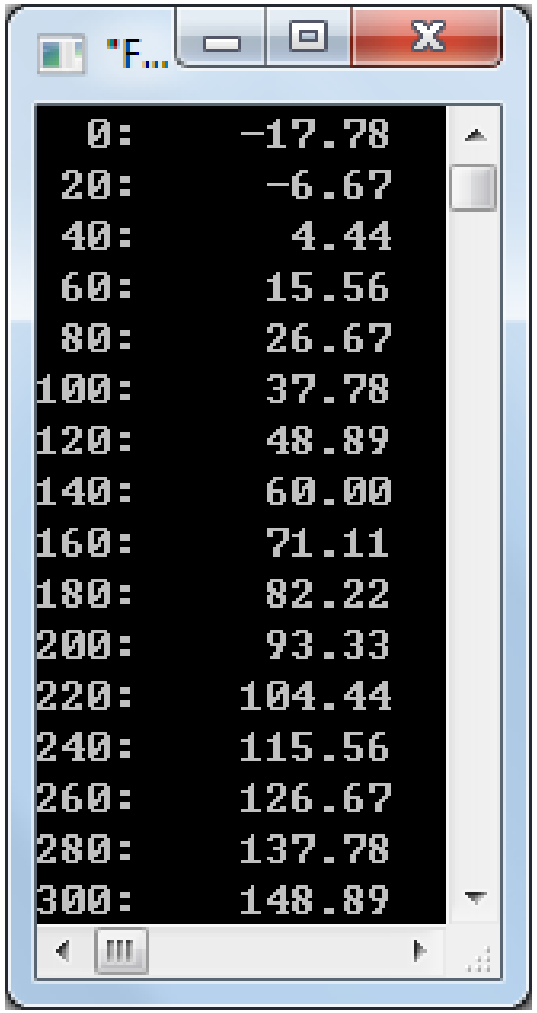
C语言中有三种定义符号常量的方法：

(1) 用#define指令（常用）

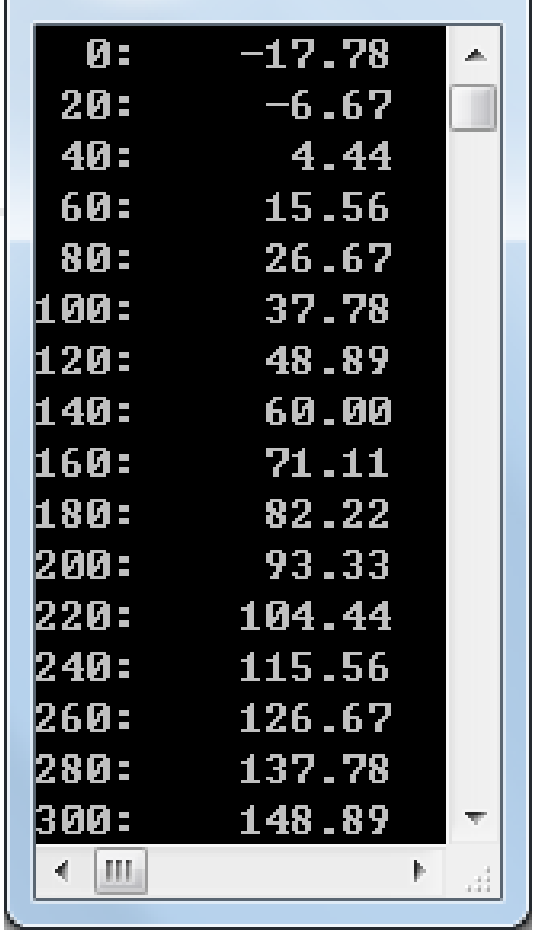
(2) 用const声明语句 （只读变量、常变量 ）

(3) 用枚举类型（在2.9节介绍）

例2.6 打印华氏和摄氏温度对照表，温度转换公式为：°C=(5/9)(°F-32)

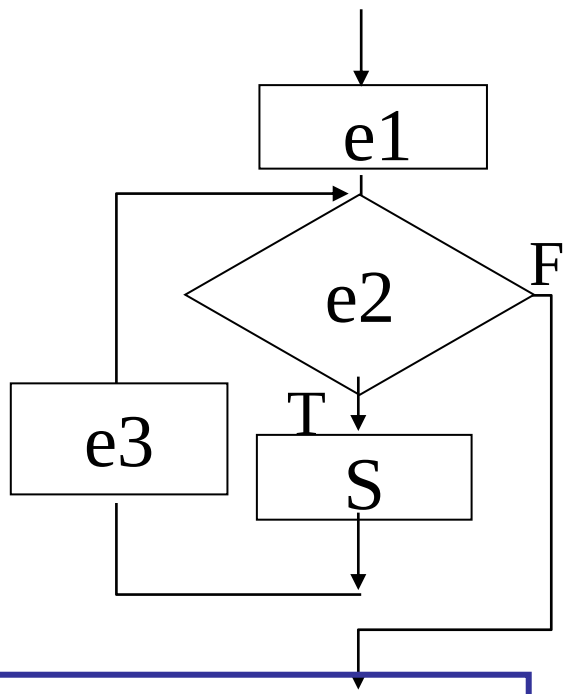


例2.6 打印华氏和摄氏温度对照表，温度转换公式为：°C=(5/9)(°F-32)




```
#include <stdio.h>
/* print Fahrenheit-Celsius table */
int main(void)
{
    int fahr;

    fahr=0;
    while（fahr <= 300） {
        printf(“%3d： %10.2f\n”, fahr, (5.0/9)*(fahr - 32) );
        fahr = fahr + 20;
    }
    return 0;
}
```

```
#include <stdio.h>
/* print Fahrenheit-Celsius table */
#define LOWER  0      /* 表的下限 */
#define UPPER  300    /* 表的上限 */
#define STEP   20     /* 步长     */
int main(void)
{
    int fahr;

    for(fahr = LOWER; fahr <= UPPER; fahr = fahr + STEP)
        printf("%3d %10.2f\n", fahr, (5.0/9)*(fahr - 32) );
    return 0;
}
```




```
fahr=LOWER;
while（fahr <= UPPER） {
      printf( "%3d %10.2f\n", fahr, (5.0/9)*(fahr - 32) );
      fahr = fahr +STEP;
}
```

1. 用#define定义符号常量

#define是一种编译预处理指令,格式为:

#define 标识符 常量

符号常量(一般用大写,以区分变量)

2. 用const定义符号常量

const是关键字，称为类型限定符。格式为：

const 类型名 标识符=常量；

例如：

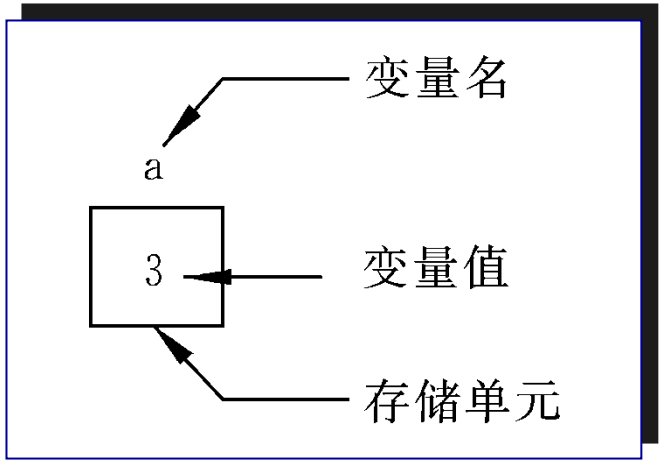
```
const double PI=3.14159;
const int DOWN=0x5000; /* 下光标键的扫描码 */
const int YES=1, NO=0;
```

用const和#define定义的符号常量的区别？

- const声明的标识符是一个只读变量，编译时系统会根据定义的类型为该标识符分配存储单元，并把对应的常量值放入其中，该值不能再被更改，此后，程序中每次出现该标识符都是对所代表存储单元的访问。
- #define定义的标识符没有对应的存储单元，只是在编译之前由预处理程序进行简单的文本替换。

2.5.3 变量定义

- 变量代表内存中具有特定属性的一个存储单元，它用来存放数据，这就是变量的值，在程序运行期间，这些值是可以改变的。



- 要求对所有用到的变量作定义，也就是“先定义，后使用”。

  类型名 变量表；

  int total，average；

- 变量在声明时可以同时赋一个初值（称为变量的显示初始化），每个变量必须分别显示初始化。

  int count=0，sum=0；

  char alert='\a'， c ；

  int count=0,sum=0；（不能 **int count=sum=0;**）

2.6 运算符和表达式

- 运算符是运算的符号表示，执行对运算对象（称为操作数）的各种操作。
- 单个的操作数（包括常量、变量和函数调用）是表达式，由运算符和操作数组成的有意义的计算式子更是表达式，如：
- **sqrt(b*b-4*a*c)**
- **x=x*PI/180**
- **fabs(an)>=EPS**

2.6.1 C运算符简介

学习每一种运算符都应掌握以下四点：

（1）运算符的运算功能

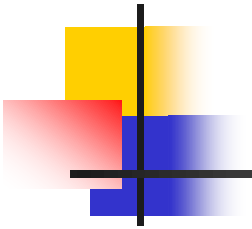
（2）操作数的个数和类型要求

单目（或一元）运算符

双目（或二元）运算符

三目（或三元）运算符

- 运算符都对操作数的类型有规定，比如%的操作数不能为浮点型。



（3）运算符的优先级和结合性

（4）运算所得结果的类型

- 运算的结果是一个具有确定类型的值，这个类型称为表达式值的类型。尤其当两个不同类型的操作数进行运算时，会引起数据类型的转换，特别要注意结果值的类型。

2.6.2 运算符的优先级和结合性

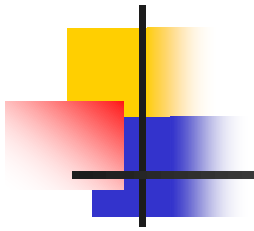
- 当表达式中包括多个运算符时，C语言会先按优先级规则解释表达式的意义。如：

  1+2*3　等价于　1+（2*3）

- 当一个操作数两侧的运算符优先级别相同时，则按“结合性”规则。

  1+2-3　等价于（1+2）-3

  ---从左至右的结合性（左结合）



-a++　等价于-（a++）

----从右至左的结合性（右结合）

- **所有运算符的优先级和结合性规则见书中P41**

45个 所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	（） [ ] -> .	圆括号 下标 间接引用结构体成员	左结合
2	! ~ ++、-- +、- （数据类型） &、* sizeof	逻辑非 按位取反 自增、自减 取正、取负 强制类型转换 取地址、间接引用 数据长度	右结合
3	*、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	==、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10	|	按位或	左结合
11	&&	逻辑与	左结合
12	||	逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、|=	赋值	右结合
15	,	逗号	左结合

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	（） [ ] -> .	圆括号 下标 间接引用结构体成员	左结合
单目运算符	! ~ ++、--	逻辑非 按位取反 自增、自减	
2	+、-	取正、取负	右结合
	（数据类型） &、* sizeof	强制类型转换 取地址、间接引用 数据长度	
3	*、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	==、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10	|	按位或	左结合
11	&&	逻辑与	左结合
12	||	逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、|=	赋值	右结合
15	,	逗号	左结合

2.6.3 算术运算

- *运算符:*

	双目	单目		
+	加法	正值	3+6, +3	
–	减法	负值	6– 4, –5	
*	乘法		3*8	
/	除法		8 / 5	
%	求余		7 % 4	值为3

【例2. 7】求出所有的水仙花数

- “水仙花数”是一个三位数，其各位数字立方和等于该数本身。例如，153是一个“水仙花数”，因为153=1^3+5^3+3^3。
- 找出“水仙花数”的关键是怎样从一个三位数中分离出百位数、十位数和个位数.

```
i=x/100;        /* 分解百位数 */
j=(x-i*100)/10; /* 分解十位数 */
k=x%10;         /* 分解个位数 */
```

```
#include<stdio.h>
int main(void)
{  int x,i,j,k;
   for(x=100;x<=999;x++)   {  /* 枚举每一个3位数 */
       i=x/100;             /* 分解百位数 */
       j=(x-i*100)/10;   /* 分解十位数 */
       k=x%10;           /* 分解个位数 */
       if(x==i*i*i+j*j*j+k*k*k) /* 是水仙花数 */
              printf("%5d",x);
   }
   return 0;
}
```

```
#include<stdio.h>
int main(void)
{  int x;
    for(x=100;x<=999;x++)    {
          if( isNarcissus（x）） /*x是水仙花数*/
               printf("%5d",x);
    }
    return 0;
}
```

如何定义函数**isNarcissus**，判断一个数**x**是否水仙花数.是，返回**1**，不是返回**0**。

```
int  isNarcissus（int  m）
{
   int i,j,k;
   i=m/100;              /* 分解百位数 */
   j=(m-i*100)/10;   /* 分解十位数 */
   k=m%10;              /* 分解个位数 */
  if(m==i*i*i+j*j*j+k*k*k) /* 是水仙花数 */
       return 1;
  else  return  0；
}
```

注:

两个整型数据相除 （结果为整, 舍去小数部分）

–5/ 3 ⇒ – 1　　1/ 2 ⇒ 0　　1. / 2 ⇒ 0. 5

使用时千万注意 int / int 出现数据丢失。

%操作数必需为整数

2.6.4 关系运算

- 有6个关系运算符：
- > 大于
- >= 大于等于
- < 小于
- <= 小于等于
- == 等于
- != 不等于

常见的C语言编程错误

- 将运算符= = 写成运算符=（赋值）。可能会因为运行时的逻辑错误而导致不正确的结果。例如，

```
if (grade = = 'A')  printf("Very Good! ");
 /*当成绩的等级为A等时，输出Very Good!*/
```

而：

```
if (grade ='A')  printf("Very Good!");
  /*不论成绩的等级是几等，总输出Very Good!*/
```

关系表达式的类型和值

- 关系表达式的类型： int
- 关系表达式的值：
- 关系成立，值为1（代表“真”）
- 关系不成立， 0（代表“假”）

注意

- 数学上判断x是否在区间[a,b]中时，习惯写成:

  a<x<b

- C中“a<x<b”的含义与数学中的含义不同，应写成：

  a<x && x<b

2.6.5 逻辑运算

- 有3个逻辑运算符
- ! 逻辑非
- && 逻辑与
- || 逻辑或

逻辑表达式：用逻辑运算符将关系表达式或逻辑量连接起来的式子

逻辑表达式的类型和值

- 逻辑运算的操作数可以是0和任何非0的数值，
- 系统最终以0判断属于“假”（以0代表“假”），
  以非0判断属于“真”（以1代表“真”）。
- 逻辑表达式的值: 1 (“真”)
  0 (“假”)

⑩ 逻辑表达式的类型： int

& 和 || 的真值表

e1	e2	&&	||
0	0	0	0
0	非0	0	1
非0	0	0	1
非0	非0	1	1

! 的真值表

e	!
0	1
非0	0

熟练掌握C语言的关系运算符和逻辑运算符后，可以巧妙地表示一个复杂的条件。

- （1） 整数a是偶数

  !(a%2) 或 a%2= =0

  （2）字符c的值是英文字母。

- c>='a' && c<='z' || c>='A' && c<='Z'

  （3）某一年year是闰年。如果某一年的年份能被4整除但不能被100整除，那么这一年就是闰年，此外，能被400整除的年份也是闰年。

- !(year%4) && year%100 || !(year%400)

```
/* 判断闰年,year 是闰年，返回1；否则，返回0 */
int isleap（int  year）
{
   if（!(year%4) && year%100 || !(year%400)）
      return 1;
   else
      return 0;
}
```

main函数怎么调用？

【例2.10】编程判断闰年

- 输入年份year，如果year是闰年，输出“yes”，否则，输出“no”。

```
#include<stdio.h>
int main(void)
{  int year;
   scanf("%d",&year);
   if (  isleap(year) )
        printf("%d is a leap year\n",year);
   else
        printf(%d is not a leap year\n",year);
   return 0;
 }
```

函数调用

注意

- 编译程序在处理含有&& 、|| 表达式时，
- 往往采用优化算法(提高速度)。
- **e1&&e2** 一旦发现**e1=0**，不再计算 **e2**
- **e1 || e2** 一旦发现**e1=1**，不再计算 **e2**
- 例如，
- **x>=0.0 && sqrt(x)<=7.7**
- **/* 如果x值为负，不求x的平方根 */**

2.6.6 自增和自减运算

++: 自增, 使内存中存储的变量值加1

-- : 自减, 使内存中存储的变量值减1

- ++和--的奇特之处: 前缀式和后缀式都行

++x

x++ 相当于 x=x+1;

--x

x-- 相当于 x=x-1;

2.6.6 自增和自减运算

- 运算符多、操作灵活是C语言的一大特色。
- 在诸多运算符中，最难理解、最容易出错的是自增、自减运算符。
- ++、-- 运用得非常广泛，且具有一定的使用技巧和难度。
- 合理使用++、--，可以节省代码，提高效率；否则，容易造成错误。

注意前缀与后缀的区别

```
x = 10 ;
y = ++x ;      /* y=11, x=11  */
```

---- 先执行对操作数的加运算，
再使用该操作数的值

```
y = x++        /*  y=10, x=11  */
```

---- 先使用该操作数的值，
再对它作加运算，

【例2.12】计算1+2+3+…+n，n从键盘输入

```
#include<stdio.h>
int main(void)
{
        int i,sum,n;
        scanf("%d",&n);
        i=1;  sum=0;
        while(i<=n) {
            sum=sum+i;
            i++;
        }
        printf("sum=%d\n",sum);
        return 0;
}
```

sum+=i++;

sum=sum+i++;

序列点

- 后缀++(或--)计算延迟的终止点称为序列点。在序列点之前，用原值，序列点之后，该操作数是更改后的新值。下列条件出现序列点：
- (1) &&、||、?: 或，运算符，即这些运算符的第一个操作数之后
- (2) 完整表达式结束时，即表达式语句、return语句中的表达式、if、switch或循环语句中的条件表达式（包括for语句中的每个表达式）之后

【例2.13】后缀式++(或--)表达式举例

int a=1,b=0;

（1） b++ + b++

表达式值为0，b为2

（2） a--&&a

表达式值为0，a为0

（3）b++ ? b : -b

表达式值为-1，b为1

序列点

- 后缀++(或--)计算延迟的终止点称为序列点。
  - 在序列点之前，用原值，序列点之后，该操作数是更改后的新值。
- 下列条件出现序列点：

(1) &&、||、?:或，运算符

在这些运算符的第一个操作数之后。

如：a--&&a 、 b++?b:-b

(2) 完整表达式结束时，即表达式语句、return语句中的表达式、if、switch或循环语句中的条件表达式（包括for语句中的每个表达式）之后。

如：b++ + b++

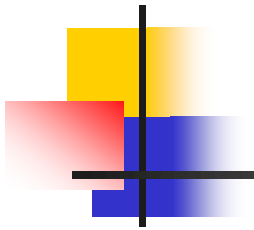
几点注意:

- 1. 只能用于变量（即左值表达式）.
  如 5+ +，(a+b)- - 均不合法.

2. 结合性为从右至左.

  如 –i++ 相当于 –(i++)

  若 i=3， 则该表达式结果为 –3， i为4

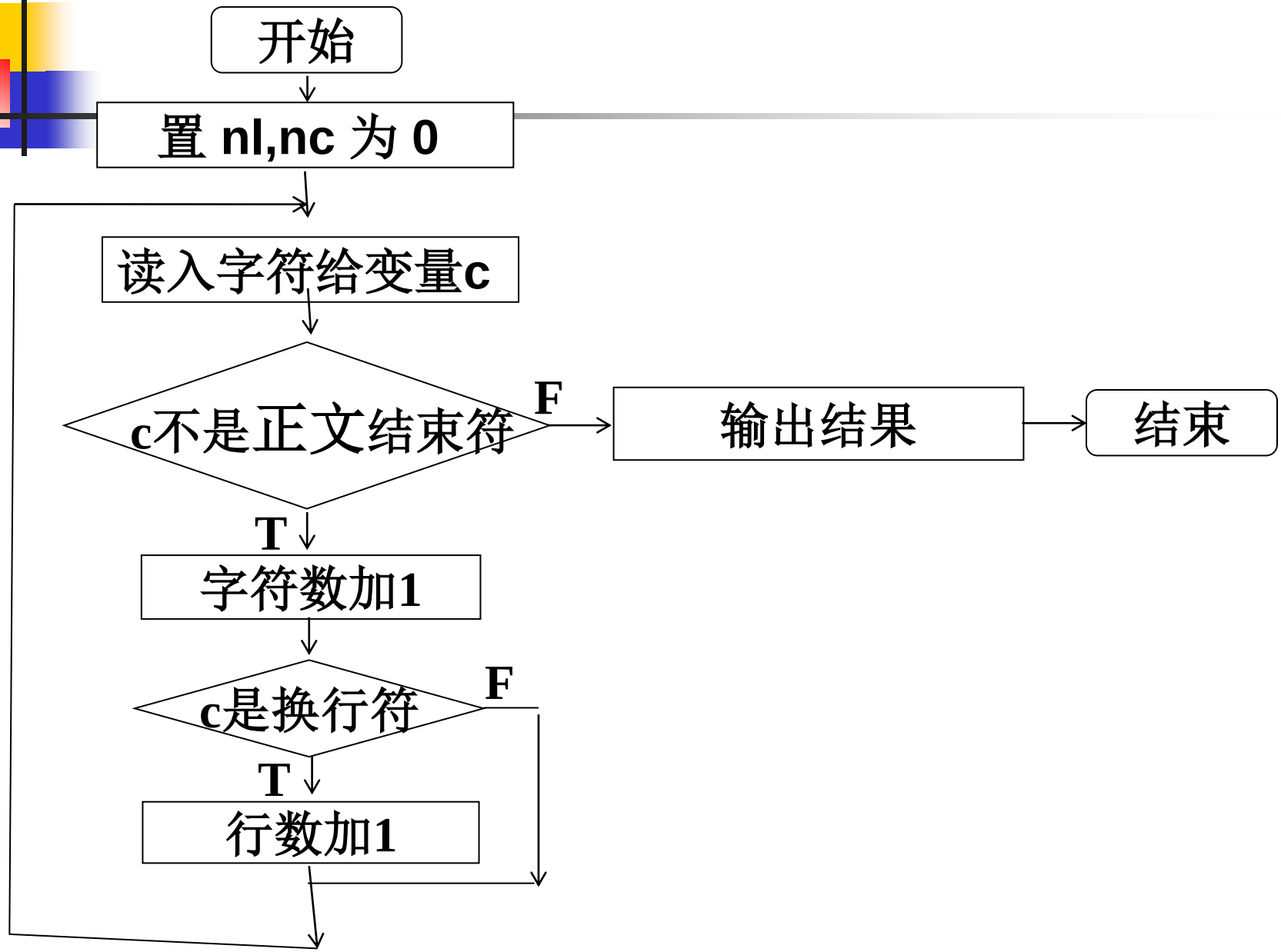


- 要慎重使用自增、自减运算符
- 在一个表达式中不要多处出现变量的自增、自减等运算。
- 尽量不要在函数参数中用自增减表达式

【例2.11】 统计输入正文的字符数和行数

- 正文是一行行字符组成的字符序列，每一行是以换行符为结束标志的一串字符.

Upload file↙
Special pages↙
Page information↙
Ctrl+z


开始
置 nl,nc 为 0
读入字符给变量c
c不是正文结束符
F
输出结果
结束
T
字符数加1
c是换行符
F
T
行数加1

2.6.7 赋值运算

1. 简单的赋值运算

赋值运算符：=

赋值表达式一般形式为：<变量>=<表达式>

右侧的“表达式”的值赋给左侧的变量

左值（lvalue）：赋值运算符左侧的标识符

- **变量可以作为左值**
- **而表达式就不能作为左值(如a+b)**
- **常量也不能作为左值**

赋值表达式的值和类型

- 与左操作数的值和类型相同 。

  int x;

  x=5.6；// 等价于 x=（int）5.6；

- 强制类型转换运算符

  （类型名）操作数

2. 复合的赋值运算

- 在“＝”号之前加一个双目运算符：

  +=， -=，*=，/=，%=

如

i+=2　　等价于　i=i+2

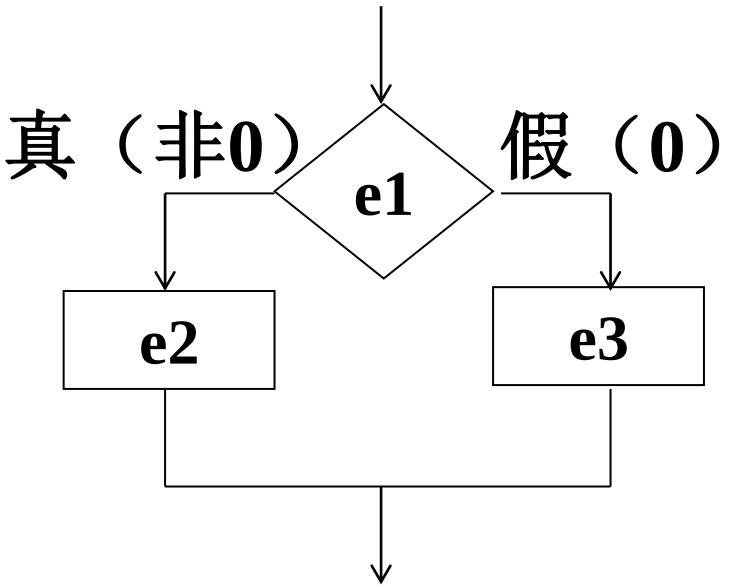
y/=x+10　　等价于　y=y / (x+10)

x*=k=m+5　　等价于　x=x * (k=m+5)

s[i++]+=1　和s[i++] = s[i++] +1不等价

2.6.8 条件运算

- e1 ? e2 : e3




```
if(e1)
    值=e2;
else
    值=e3;
```

统计正数、负数、零的个数

```
k=1; positive=negative=zero=0;
while(k<11) {
  scanf("%d",&x);
    x>0? positive++：(x<0? negative++：zero++);
  k++;
}
```

按照结合性，括号可省略

2.6.9 逗号运算

- e1，e2
- 计算规则:
- 先计算e1，再计算e2，逗号表达式的值和类型与e2的值和类型相同

举例

- （1）x=(i=4，i%3)
- 表达式的值为：1， x为1

- （2）x=i=4，i%3
- 表达式的值为：1， x为4

扩展形式

逗号表达式的一般形式可以扩展为

e1，e2，e3，……，en

它的值为表达式 n 的值。

注意

并不是任何地方出现的逗号都是作为逗号运算符。例如函数参数也是用逗号来间隔的。

printf(“%d,%d”, 2, 3); /* 输出

printf(“%d,%d **输出：3, 3**

***/**

“2，3”并不是一个逗号表达式，它是printf函数的2个参数

“（2，3）”是一个逗号表达式，它的值等于3。

反转字符串

```
i=0;      // 首字符位置
j=strlen(s)-1; // 最后一个字符位置 */
while(i<j) {
    mid=s[i];
    s[i]=s[j];
    s[j]=mid;   /*交换s[i]、s[_
    i++;
    j--;     /* i、j向中间靠拢 */
}
```

```
for(i=0,j=strlen(s)-1;i<j;i++,j--)
    mid=s[i],s[i]=s[j],s[j]=mid;
```

【例2.14】输入一串数字字符，将其转换为一个十进制整数赋值给x（模拟scanf("%d",&x）的功能）。

- 当用scanf("%d",&x）输入一个整数时，实际上输入的是组成该整数的各位数字的一个字符串，scanf函数读到的每个数字都是该字符的字符码。为了将字符串转换为整数，必须先将每个数字字符转换成数字对应的一位整数，然后按相应的数位拼成一个十进制整数，最后赋给变量x。

```c
int getint(void)
{
    char c;    /* 用于存放当前输入的字符 */
    int x;     /* 用于存放转换的整数 */
    for(x=0,c=getchar();c>='0'&&c<='9';c=getchar())  /* 遇非数字符结束
        x=10*x+c-'0';     /* c-'0'; 将数字字符转换成对应的一位整数 */
    return x;
}

int main(void)
{
    int score;
    score=getint();
    printf("%d\n",score);
    return 0;
}
```

2.7 位运算符和位表达式

6个位运算符：

～ (求反)

& (按位与)

| (按位或)

^ (按位异或，或按位加)

<< (左移)

>> (右移)

2.7.1 按位求反（～）

- 对操作数的每个二进制位取相反值
- 例如：
- **short a=5; unsigned short b=5;**
- **a和b的二进制为：00000000 00000101**
- **~a的二进制为： 11111111 11111010**
- **~a的值为short型 –6。**
- **~b的值为unsigned short型 65530。**

2.7.2 按位与(&)、或(|)、加(^)运算

表达式	二进制表示	十六进制值	说明
x	01101000 11010001	0x68d1	待处理数据
mask	11111111 00000000	0xff00	逻辑尺
x & mask	01101000 00000000	0x6800	将x的低字节置为0，保留高字节
x | mask	11111111 11010001	0xffd1	将x的低字节保留，高字节置为1
x ^ mask	10010111 11010001	0x97d1	将x的低字节保留，高字节翻转

2.7.3 左移(<<)和右移(>>)

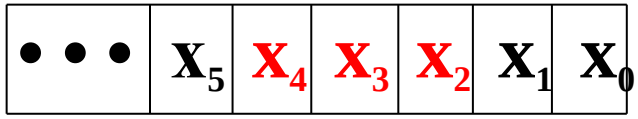
- e<<n　将e的值向左移n位，低n位填入0。
  左移1位相当于该数值乘以2
- e>>n　将e的值向右移n位，而高n位可能填入？。
  --e是无符号类型，填入0；
  --e是有符号类型，一些机器填入0（即“逻辑移位”），
  而另一些机器则填入符号位（即“算术移位”）。
- 在使用右移运算符时应经常用无符号类型。
- 右移一位相当于该数值除以2

【例2.15】写一个表达式

将整数x的第m位起的右n位取出来，作为另一个数

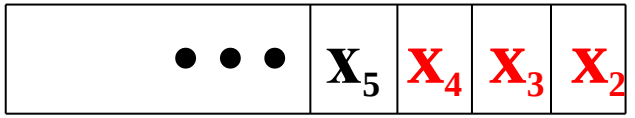
注：一个整数的各个二进制位从右至左依次编号为第0位、第1位、第2位、……

如：m=4　n=3



① 将要取出的那n位移到最右端

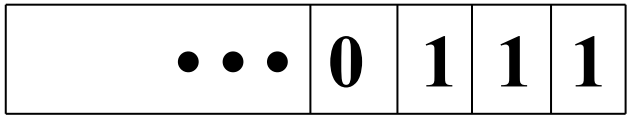
x >> m-n+1



② 设计一个逻辑尺：低n位全为1，其余位全为0

~(~0<<n)

(1<<n)-1



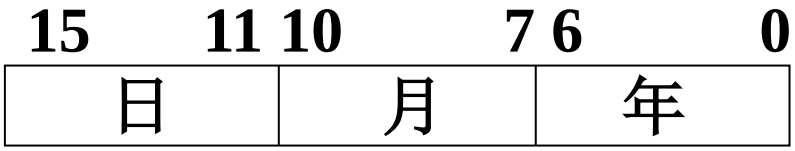
③ 将上面二者进行&的运算

x>>(m-n+1) & ~(~0<<n)

2.7.4 位运算符应用举例

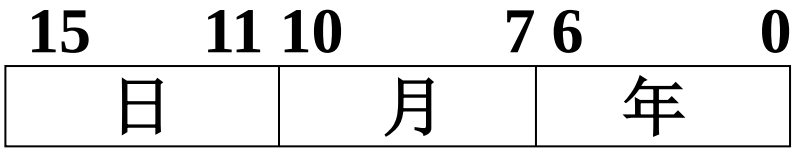
【例2.16】 压缩和解压示例

可以把表示21世纪日期的日、月和年3个整数压缩成一个16位的整数(short)。写一个表达式，实现压缩存储日、月和年。



day<<11 | month<<7 | year

- 分析：
- 可以把表示21世纪日期的日、月和年3个整数压缩成1个16位的整数，因为日有31个值，月有12个值，年有100个值，所以可以在一个整数中用5b表示日，用4b表示月，用7b表示年.



day<<11 | month<<7 | year

编写一个压缩存储日、月和年的函数。

函数名：**pack**

输入参数： **day-日**

month-月

year-年

返回值： 压缩后的整数

```
short pack(short day, short month, short year)
{
   return(day<<11 | month<<7 | year) ;
}
```

压缩存储日、月和年

```
short pack(short day, short month, short year); //函数声明语句
int main()
{ short day,month,year,dat
   printf("Input year,month d
   scanf("%hd%hd%hd", &year,&month,&day);
   date=pack(day,month,year);      //函数调用
   printf("%hx\n",date);
   return 0;
}
short pack(short day, short month, short year) //函数定义
{    return(day<<11 | month<<7 | year) ;
}
```

输入/出 short 数据 用 %hd
输入/出 unsigned short 用 %hu

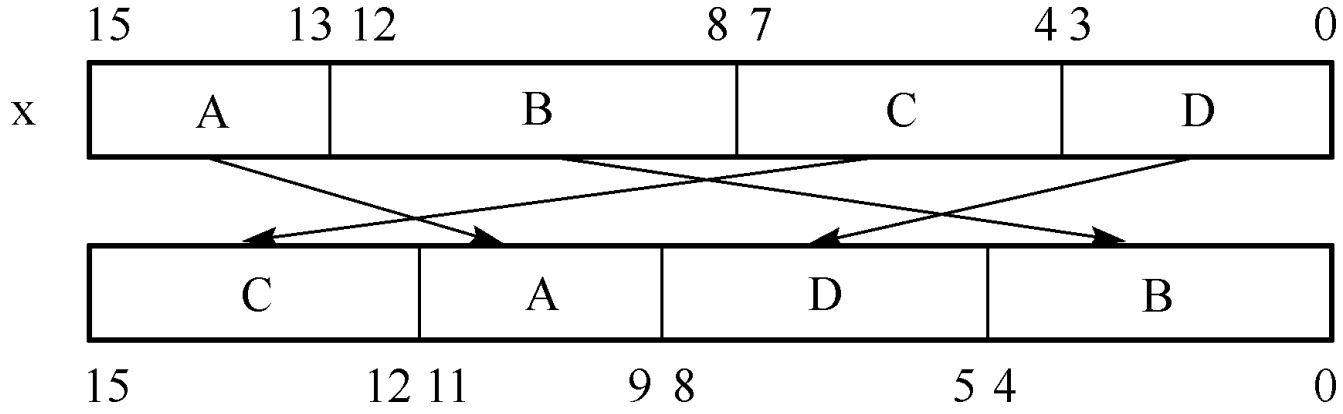
练习：编写解压函数

```
void  unpack(short x)
{
   将x解压为 day，month，year
    printf ：  day，month，year
 }
```

【例2.17】简单加密示例

将一个短整型数x分成4个长度不等的部分：

A（3b）、B（5b）、C（4b）和D（4b），然后将它们按照CADB的顺序重新拼凑在一起，实现对其加密的功能。写一个表达式，要求其值为x的密文。



【例2.18】 计算ASCII码字符的奇偶校验位

二进制数据在传送中会发生误码（1变成0或0变成1），这就有如何发现及纠正误码的问题。解决方法是在原始数据（数码位）基础上增加几位校验（冗余）位。

奇偶校验：分奇校验和偶校验。

奇校验：整个码字中奇数个“1”

偶校验：整个码字中偶数个“1”

data=‘A’

	1	0	0	0	0	0	1

校验位 parity

校验位在每个字符的最高位

```
int main()
{ unsigned char data,backup,t;
  int parity=0;    /* 奇偶校验位,0-偶校验，1-奇校验 */
  data=getchar();  /* 被校验的数据 */
  backup=data;
  while (data)   {
        t=data&1;     /* 取该数的最低位 */
        parity^=t;     /* 进行异或操作 */
        data>>=1;     /* 为取下一位做准备 */
  }
  data=backup|(parity<<7);    /* 生产校验码 */
  printf("The data is %#x\n",backup);
  printf("Parity-Check Code is %#x\n",data); /* 输出校验码 */
  return 0;
}
```

2.8 类型转换

- C语言允许双精度、单精度、整型及字符数据之间混合运算

10 + '1' + 6.5

但有一个规则: 先转换成同一类型,再计算。

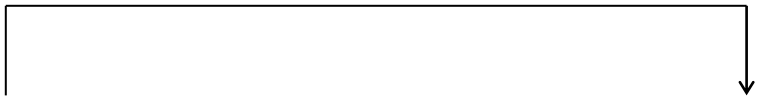
2.8.1 整数提升

- 任何表达式中的char、unsigned char、short和unsigned short都要先转换成int或unsigned，如果原始类型的所有值可以用int表示，则转换成int，否则转换成unsigned，把这称为“整数提升”。
- short a;

  sizeof(a)=?　　sizeof(-a)=?

  2　　4

2.8.2 一般算术转换



char / short → int → unsigned → long → unsigned long → float → double → long double

2.8.3 赋值转换

- 右操作数的值被转换为左操作数的类型
- 例如：

```
short s=5；  double d=2.9；long l=0x11118001；
```

则

```
s = d；     /*把d转换为short，再赋给s。值为2*/
d = s；     /*把s转换为double，再赋给d。值为5.0*/
s = l；     /* s值为 ? */
printf("%hx %hd %hu\n",s,s,s);
        // 输出   8001   -32767    32769
```

2.8.4 强制类型转换

- （类型名）操作数
- 例如，

```
(double) i  /* 将i转换成double，i的类型保持不变*/
(long)('a'-32)   /*  'a' 被自动转换成int，
                     相减的结果被强制转换为long*/
(float)x+y       /*等价于( (float)x)+y*/
(double)x=10  /*错误*/
```

2.9 枚举类型

- 使代码更具可读性，理解清晰，易于维护。
- 它是由用户定义的若干枚举常量的集合
- 枚举常量用标识符命名,缺省状态下，其值是所列举元素的序号，序号从0开始

```
enum color { WHITE,YELLOW,RED,BLUE};
enum week { SUN, MON, TUE, WED, THU, FRI, SAT };
```

9.1 枚举类型的定义

```
enum week { SUN, MON, TUE, WED, THU, FRI, SAT };
```

关键字　　枚举名　　枚举常量

在未指定值的缺省情况下，第一个枚举常量的值为0，以后的值依次递增1 .

可以指定一个或多个枚举常量的值，未指定值的枚举常量的值比前面的值大1 .

```
enum sizes { SMALL, MEDIUM=10, BIG , TOO_BIG=20 } ;
```

enum 后面也可以不出现枚举名 .

```
enum { WIN, LOSE, TIE, ERROR};
```

2.9.2 用枚举类型定义符号常量

```
#define   WIN        0
#define    LOSE       1
#define    TIE        2
#define    ERROR    -1
```

- 可用下面的枚举类型定义来代替。

  enum { WIN, LOSE, TIE, ERROR=-1};

2.9.3. 枚举变量的声明

(1) 在定义枚举类型的同时说明枚举变量

```
enum color { RED,GREEN,BLUE} c1,c2;
```

(2) 利用枚举名来说明枚举变量

```
enum color { RED,GREEN,BLUE} c1;
enum color c2;
```

或者

```
enum color { RED,GREEN,BLUE};
enum color c1,c2;
```

- 一个枚举变量的值是int型整数，但值域仅限于列举出来的范围。枚举变量值的输入和输出都只能是整数。
- **c1=BLUE;**
- **/* *等价于 c1=2; 使用有意义的标识符有助于读者理解程序* */**
- **printf(“%d”,c1); /* *输出2，而不是BLUE* */**
- **scanf(“%d”,c2); /* *输入0，不能输入RED* */**
- 下面的语句是错误的：
- **c1=3; /* *变量c1的值域为：0、1和2* */**
- **printf(“%s”, GREEN);**
- **/* *输出错误的结果，而不是GREEN* */**
- **if(c1= =RED) printf(“Red”);**

【例2.22】用枚举变量day控制循环，输出存储在数组weekName中的英文星期名。

- 一个枚举变量的值是一个int整数，它可以出现在整数允许出现的任何地方。如果要输出与枚举值相对应的枚举常量标识符或代表其含义的完整字符串，可以定义一个字符串数组，以枚举值作为下标。
- 源程序\ex2_22.c

```
/*简要描述：用枚举变量控制循环，输出存储在数组weekName中的英文星期名。*/
#include<stdio.h>
enum week { SUN, MON, TUE, WED, THU, FRI, SAT } ;
int main(void)
{
    enum week day;
    char *weekName[ ]={ "Sunday",  "Monday", "Tuesday",
          "Wednesday", "Thursday", "Friday",   "Saturday"
     };
    for(day=SUN;day<=SAT;day++)
          printf("\n%3d\t%s",day,weekName[day]);
    return 0;
}
```

2.10 新增数据类型 (自学）

- C99标准增加了支持64位的整数类型long long，还增加了布尔类型和复数类型。CodeBlocks编译器能够较好地支持C99标准，本节中的例子程序均由CodeBlocks运行通过。

2.10.1 long long类型

- long long类型的长度至少应为64位。和其他整型类型一样，有带符号和无符号两种。signed long long的数值范围为-2exp(63)～ 2exp(63)-1，unsigned long long的数值范围为0～ 2exp(64) -1。long long类型的常量用后缀“LL”或“ll”表示。
- 在所有整型类型中，类型long long的值域最宽，因此，C99标准的自动转换所遵循的转换方向为：

  …→unsigned long→long long→unsigned long long→float→…

【例2.23】输出斐波那契（FIBONACCI）数列的前60个数。

- FIBONACCI数列是由1和1 开始，之后的数是它前面两数的和，即 1，1，2，3，5，8，……
- 分析：由于FIBONACCI数列从大约第45个数起就超过了long类型的范围，为避免溢出，应将结果变量说明为long long类型，从输出结果可以看到，它的长度为8字节。
- 源程序\ex2_23.c

2.10.2 布尔类型

- C语言中可以用任何整数类型表示布尔值，0表示假，所有非0表示真。布尔表达式为假时值为0，为真时值为1。例如，i=(a<b)，在a小于b时对整型变量i赋值1；在a不小于b时对变量i赋值0。
- C99标准引入了真正的布尔类型_Bool，_Bool类型长度为1，只能保存数值0和1（分别表示“false”与“true”）。虽然也可以使用其他整数类型表示布尔值，但如果C语言实现支持C99标准，那么使用_Bool类型更清晰。
- 另外，C99标准为了让C和C++兼容，增加了一个头文件stdbool.h。里面定义宏名bool为_Bool的同义词，定义false与true分别为0和1。因此，只要在源文件中包含stdbool.h这个头文件，就可以在C语言里像C++那样使用bool定义布尔类型变量，通过true和false对布尔变量进行赋值。
- 将任意非零值赋值给_Bool类型，都会先转换为1，表示真。将零值赋值给_Bool类型，结果为0，表示假。

【例2.24】捕鱼和分鱼问题

- A、B、C、D、E五人在某天夜里合伙捕鱼，到第二天凌晨时都疲惫不堪，于是各自找地方睡觉。A第一个醒来，他将鱼分为五份，把多余的一条扔掉，拿走自己的一份。B第二个醒来，也将鱼分为五份，把多余的一条扔掉，拿走自己的一份。C、D、E依次醒来，也按同样的方法拿鱼。问他们合伙捕了多少条鱼?

- 分析：总共将所有的鱼进行了5次平均分配，每次分配的策略是相同的，即扔掉一条后剩下的分为五份，然后拿走自己的一份，余下其他的四份。
- 假定鱼的总数为X，则X可以按照题目的要求进行五次分配：X-1后可被5整除，余下的鱼为4*(X-1)/5。若X满足上述要求，则X就是题目的解。
- 源程序\ex2_24.c

2.10.3 复数类型

- C99标准新增了2个复数类型关键字：_Complex 和_Imaginary，其中，_Imaginary表示只有一个实数组成的纯虚数类型（即bi）。由于复数的实部和虚部均为实数，在说明复数类型时，需用浮点类型名进一步指定对应实数类型，因此复数或纯虚数各有以下3种类型：
- float _Complex、double _Complex、long double _Complex
- float _Imaginary、double _Imaginary和long double _Imaginary

- 复数类型_Complex表示为对应实数类型的二元数组，第1个元素表示复数的实数部分，第2个元素表示复数的虚数部分。而复数类型_Imaginary是实数部分总为0的纯虚数类型。
- C99标准还引入了头文件complex.h，其中定义complex宏为关键字_Complex的同义词，定义imaginary宏为关键字_Imaginary的同义词，定义_Complex_I(或I)为复数类型的虚数常量表达式，取值为虚数单位i，或。因此，复数型常量用包含了虚数常量 _Complex_I(或I)的浮点型常量表达式来表示，如1.0+2.0*I，1.0+2.0*I，4.5*I等。
- 复数的运算必须要有相应的库函数支持。在C99标准中，复数的库函数很多，如三角函数、双曲函数、指数函数和其他函数等，这些函数的原型在头文件complex.h中说明。

复数类型转换规则

（1）一个复数类型转换成另一个复数类型时，实数和虚数浮点数成员按照浮点类型转换规则分别转换。

（2）从Complex类型转换为_Imaginary类型时，放弃实数部分；从_Imaginary类型转换为_Complex类型时，将实数部分设置为0。

（3）将一个实数（整数或浮点数）转换成复数类型时，该实数转换成复数值的实数部分，而虚数部分为0。

（4）将一个复数类型转换成实数（整数或浮点数）时，虚数部分放弃。

【例2.25】求两个复数的乘积。

- 源程序\ex2_25.c
- 程序中定义了3个双精度的复数a、b和c，第一条printf函数调用语句输出复数a所需的内存大小，它为double的2倍。接着，给复数变量a和b分别赋一个复数常量，复数a和b的乘积赋值给复数变量c。函数creal和cimag分别为取得复数的实部和虚部。

本章小结

- 记号是程序中具有语义的最基本组成单元，共分5类：标识符、关键字、常量、运算符和标点符号。标识符是程序员用于对变量和函数等命名的符号。关键字有明确的含义，不能被用作普通标识符。常量包括字符常量、字符串常量，以及各种类型的整型常量、浮点型常量，必须熟练掌握各种类型常量的表示方法。C语言提供的基本数据类型有：字符型、整型、短整型、长整型，以及单精度、双精度和高精度浮点型。在C99标准中增加了布尔类型、长长整型、复数类型等。枚举类型是用标识符命名的整型常量的集合，其中的标识符实际上是自动设置值的符号常量。

- C语言提供了很多运算符，除通常的算术、关系、逻辑和赋值运算符外，还有一些C语言特有的运算符：自增、自减、条件、逗号、复合赋值、sizeof和位运算符。自增自减运算符有前缀式和后缀式，但效果可能不同，需要特别注意。位运算符使程序员可以访问字节或字中的实际二进制位，要重点掌握位运算符的应用。每个表达式都有一个值和类型，运算符的优先级和结合性规则决定了表达式求值的顺序，在表达式的计算过程中，会涉及类型之间的转换问题，有自动类型转换和强制类型转换两种规则。

C语言与程序设计

The C Programming Language

第3章　基本的标准输入与输出

华中科技大学计算机学院

毛伏兵

第3章 基本的标准输入与输出

主要内容

- C语言提供了一些标准的输入/出函数——系统函数

  putchar、puts、printf

  getchar、gets、scanf

注意： 用这些函数时,须用预编译指令:

#include <stdio.h>

3.1 字符输出与输入

3.1.1 单字符输出/入

- **putchar**：字符输出函数
- **getchar**：字符输入函数

putchar

函数原型： int putchar(int);

功能： 向标准输出设备(显示器)输出一个字符，函数正确执行时返回该字符码，否则返回EOF。

调用方式： putchar (c) ;

char 或 int

举例： 欲输出字符A：

```
putchar ('A') ;
putchar (65) ;
putchar ('\x41') ;
```

getchar

函数原型：int getchar(void) ;

功能：从标准输入设备(键盘)输入一个字符，并将该字符作为函数的值。

调用方式: getchar( );

举例：

```
char c;
c=getchar( );
putchar(c);
```

嵌套调用

```
putchar(c=getchar( ));
```

或

```
putchar(getchar( ));
```

3.2 字符串输出/入

- **puts** ：字符串输出函数
- **gets**： 字符串输入函数

puts

函数原型：int puts（const char *s）；

功能: 将s指向的字符串输出到标准输出设备，并换行。

正确执行：返回一个非负整数值

出错：返回EOF

调用方式： puts(s);

char [ ] 或 char *

```
char  name[]="Rita";
puts（name）；
```

gets

函数原型：char *gets(char *s);

功能: 读取一行字符存放到s指定的内存缓冲区

正确执行：返回该内存缓冲区的首地址

出错：返回空指针NULL

调用方式： gets(s);

char [ ] 或 char *

```
char  name[15];
gets(name);
puts（name）；
```

3.3 格式化输出/入

- **printf**：*格式化输出函数*
- **scanf**：*格式化输入函数*

printf

是C语言中使用得最多的一种输出函数，它可一次按格式输出多个不同类型的数据。

例：

```
printf ( “This is the first program.\n” );
x=10;  y=10.512;
printf ( “x=%d, y=%.1f ” , x, y);
```

```
This is the first program.
x=10,  y=10.5
```

printf 的调用方式

printf（“•••”，参数1，参数2，•••）；

格式控制串

1. 普通字符：原样输出
2. 格式说明：由%开始，转换字符结尾，

   如：%f，%d 等

格式说明的基本组成

% [特征符] [域宽] [长度修饰符] 转换字符

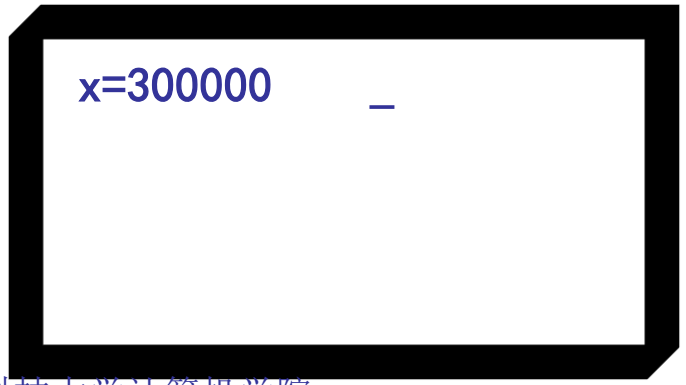
表3.2：前5行

输出数据类型和格式
表3.1

指出输出参数的类型
表3.2:h l L

设置显示的最小宽度及小数位数
*表3.2: m . n *

```
long x=300000;
printf("x=%-10ld ", x);
```



◆ 长度修饰符

h：加在 d、o、x、u 之前，表示输出 short

l：加在 d、o、x、u 之前，表示输出 long

L：加在 f、e、g 之前，表示输出 long double

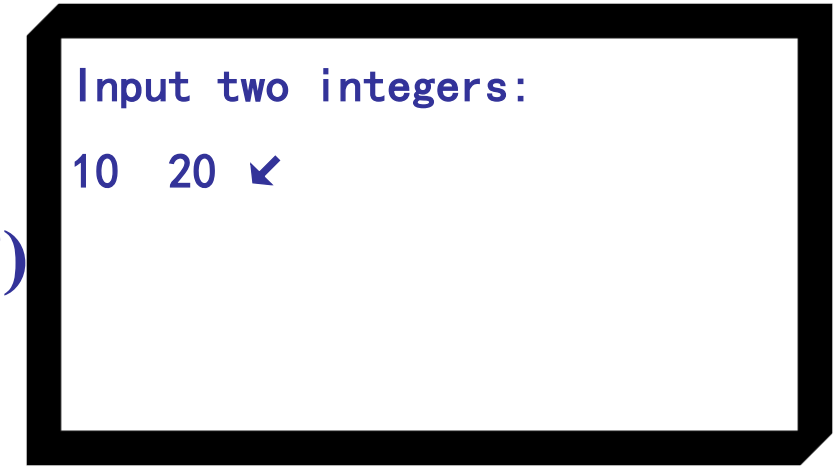
```
short a;
long b;
long double y;
printf("a=%hd,b=%ld,y=%Lf\n",a,b,y);
```

scanf

在标准输入设备上按指定格式输入各种类型的数据到内存中。

例：

```
int x, y;

printf ("Input two integers:\n");

scanf("%d %d", &x, &y);
```



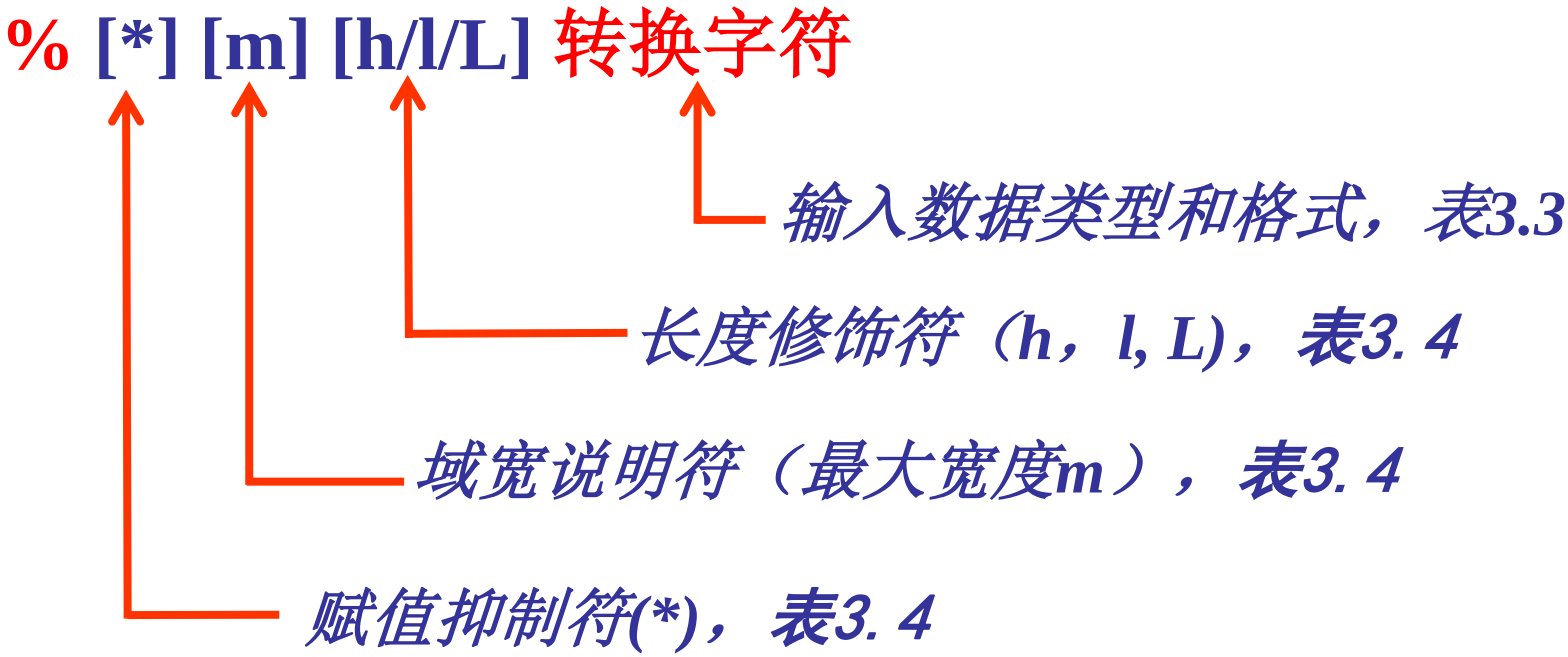
scanf 的调用方式

scanf (“ • • • ” ， 地址参数1，地址参数2，• • •) ;

格式控制串

1. 格式说明
2. 空白字符：空格，\n , \t
3. 非空白字符：最好不要用

格式说明的基本组成


% [*] [m] [h/l/L] 转换字符
输入数据类型和格式，表3.3
长度修饰符（h，l, L)，表3. 4
域宽说明符（最大宽度m），表3. 4
赋值抑制符(*)，表3. 4

◆ 长度修饰符

```
long a;
double x;
scanf( “ %ld  %lf ”, &a , &x) ;
```

转换说明与输入参数类型不匹配时可能导致的后果：

读入的数据值不正确或程序非正常终止；

输入数据的形式（即分隔数据项的方法）

自学

1. **用隐含的分隔符**，如空格、回车键

   （只分隔 整数、浮点数、字符串，字符不需分隔）

2. 根据转换字符的含义从输入流中取得数据

3. 根据转换项中指定的**域宽**分隔出数据项

4. 使用显示的分隔符

实例\源程序\实例.c

```
#include<stdio.h>
void main()
{
  int a,b;
  char op;
  printf(“Input two integers : \n");
  scanf("%d%d", &a, &b);
  printf("Input an operator:\n");
  scanf("%c", &op);
  printf("%d%c%d\n ", a, op, b);
}
```

```
Input two integers :
2   3↙
Input an operator:
2
3
```

用隐含的分隔符分隔整数、浮点数、字符串

字符不需分隔，根据转换字符的含义自动分隔

```
#include<stdio.h>
void main()
{
  int a,b;
  char op;
  printf("Input two integers :\n ");
  scanf("%d%d", &a, &b);
  printf("Input an operator:\n");
  scanf("%*c%c", &op);
  printf("%d%c%d\n ", a, op, b);
}
```

```
Input two integers :
2   3↙
Input an operator:
+↙
2 + 3
```

*：表示跳过相应的数据，不赋值

```
#include<stdio.h>
void main()
{
  int a,b;
  char op;
  printf("Input two integers :\n ");
  scanf("%d%d", &a, &b);
  printf("Input an operator:\n");
  scanf("%1s", &op);
  printf("%d %c %d \n ", a, op, b);
}
```

```
Input two integers :
2   3↙
Input an operator:
+↙
2 + 3
```

根据指定的域宽分隔出数据项

```
#include<stdio.h>
void main()
{
  int a,b;
  char op;
  printf("Input two integers : ");
  scanf("%d ,%d", &a, &b);
  printf("Input an operator:");
  scanf("%1s", &op);
  printf("%d %c %d \n ", a, op, b);
}
```

程序执行时，如果输入：

Input two integers :

2 , 3↙

Input an operator:

+

2 +3

使用显示的分隔符，即用户定义的分隔符（要有提示）

如按点分十进制输入IP地址

C语言与程序设计

The C Programming Language

第4章　流程控制

华中科技大学计算机学院

毛伏兵

主要内容

if语句
switch语句
while语句
for语句
do-while语句
goto语句和标号语句
break语句
continue语句
常用算法：枚举、筛法、递推

4.1 C语句分类

- C语句
  - 说明语句
    - 变量说明、函数说明、
    - 常量定义、类型定义、
    - ……
  - 可执行语句
    - 表达式语句、复合语句、
    - 选择语句（if 和 switch）、
    - 循环语句（while, do-while和for）、
    - 转移语句（break, continue, goto和return）、
    - 标号语句

4.2 表达式语句

表达式;

例：x = y + 1　　x = y + 1;

printf(”hello”)　　printf(”hello”);

;　　/* 空语句 */

4.3 复合语句

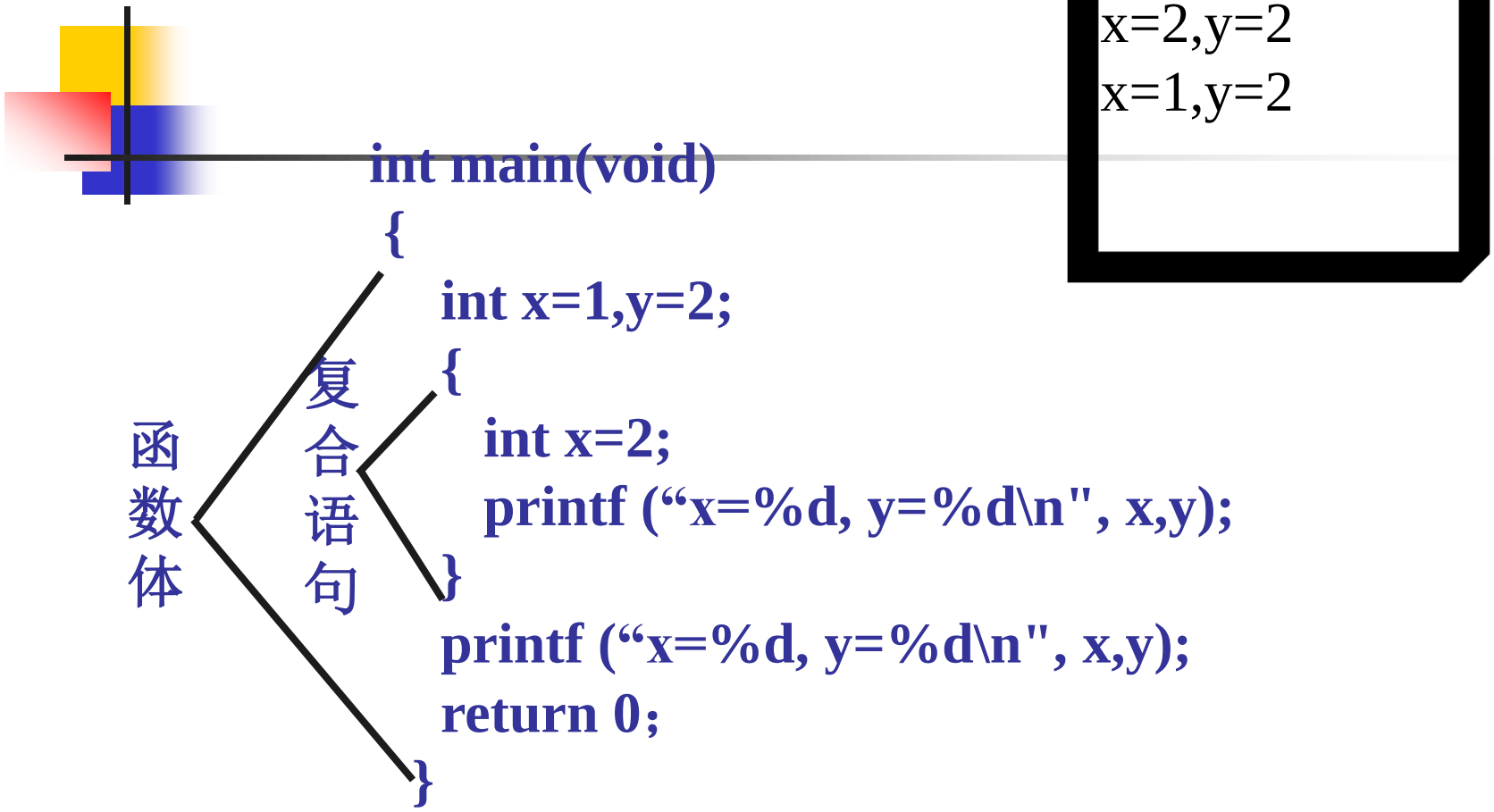
{

语句部分 说明部分

}

复合语句又称块。函数体是一个块。

注意变量的作用域。

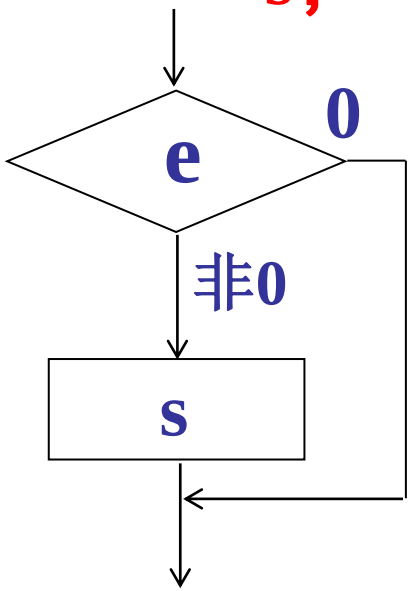

x=2,y=2
x=1,y=2
int main(void)
{
int x=1,y=2;
{
int x=2;
printf (“x=%d, y=%d\n", x,y);
}
printf (“x=%d, y=%d\n", x,y);
return 0;
}
函数体
复合语句

4.4 if语句

1. if 语句的一般形式

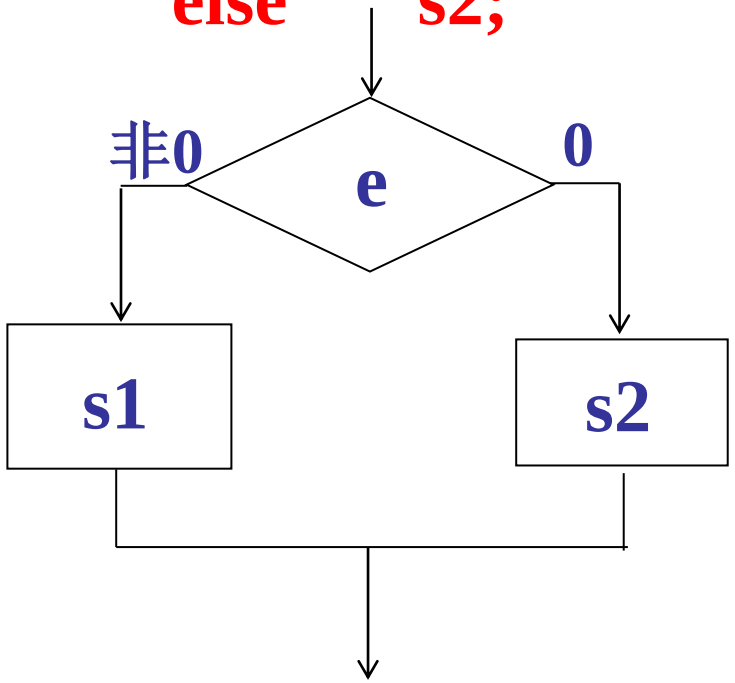
形式1：（单分支）

```
if （e）
   s;
```



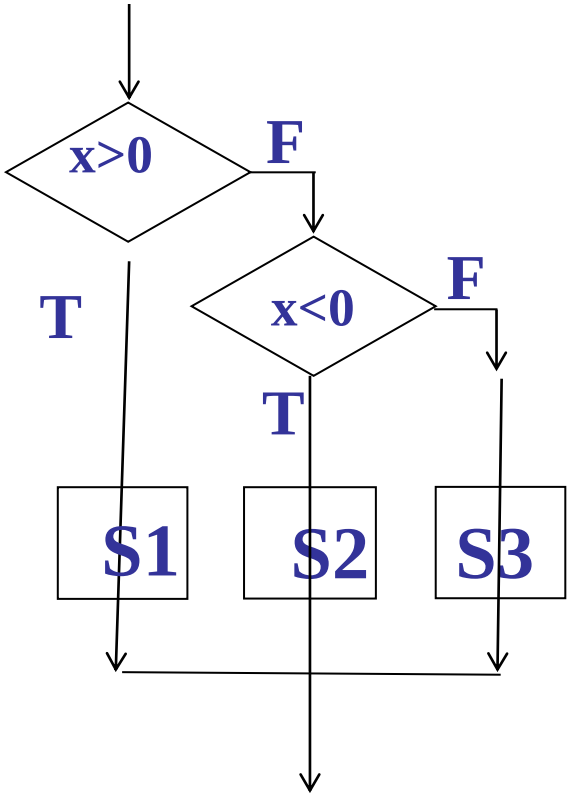
形式2：（双分支）

```
if （e） s1;
else     s2;
```



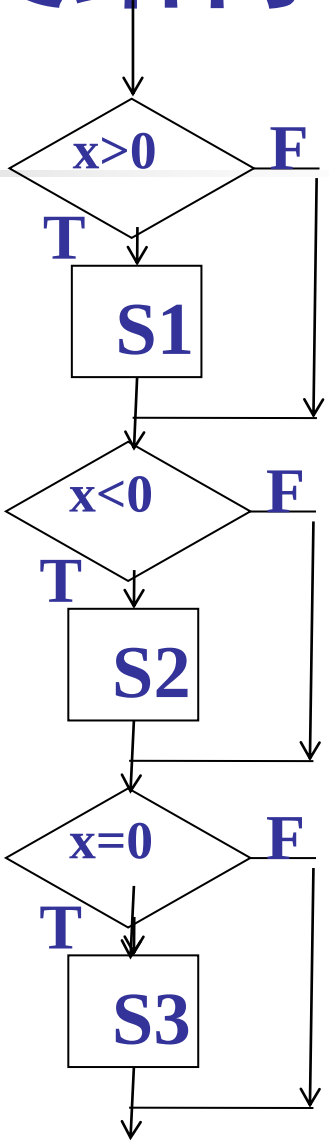
如何用if语句实现多分支结构?

```
//统计正数、负数、零的个数
positive=negative=zero=0;
for(k=1;k<11;k++) {
    scanf("%d",&x);
    if(x>0) positive++;
     else if(x<0) negative++;
      else  zero++;
}
```

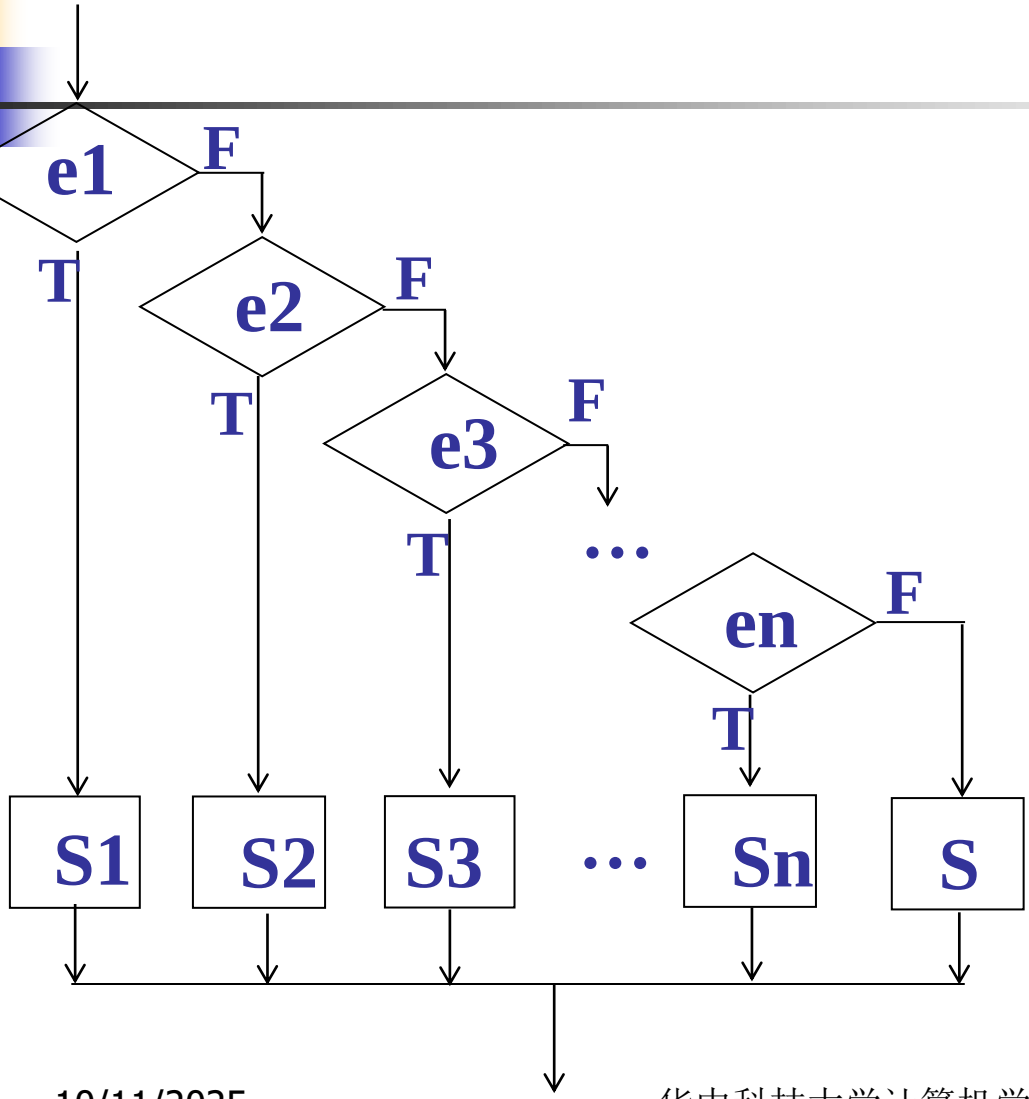


如何用if语句实现多分支结构?

```
//统计正数、负数、零的个数
positive=negative=zero=0;
for(k=1;k<11;k++) {
    scanf("%d",&x);
    if(x>0) positive++;
     if(x<0) negative++;
     if(x==0)  zero++;
}
```



形式3： if-else-if结构（多分支）




```
if(e1)
    S1
else if (e2)
    S2
else if (e3)
    S3
  …
else  if(en)
    Sn
else  S
```

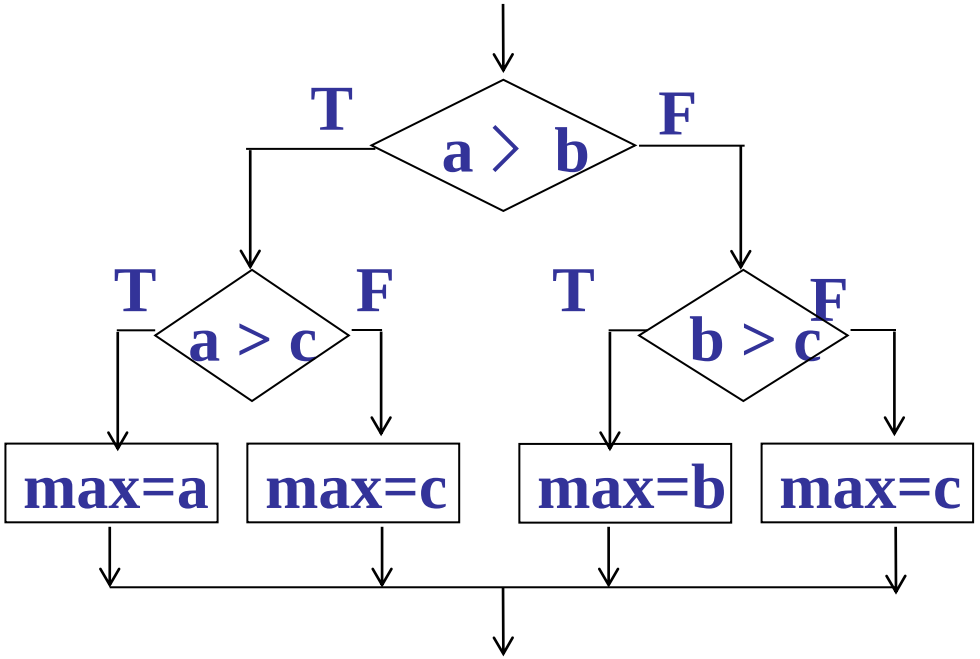
2. 嵌套的if语句

1）一般形式

```
if (e)
    if 语句
else
    if 语句
```

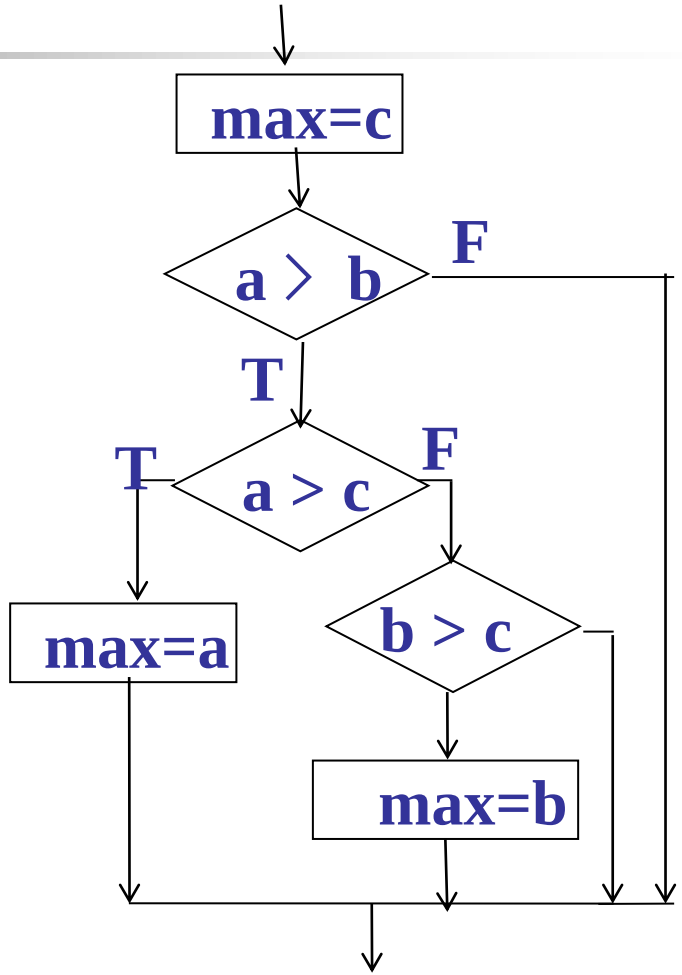
用嵌套的if语句求a，b，c三个数中最大值。

```
if ( a > b )
   if ( a > c ) max = a;
   else  max = c;
else
   if ( b > c ) max = b;
   else max = c;
```



下面代码能实现求a，b，c中最大值吗？

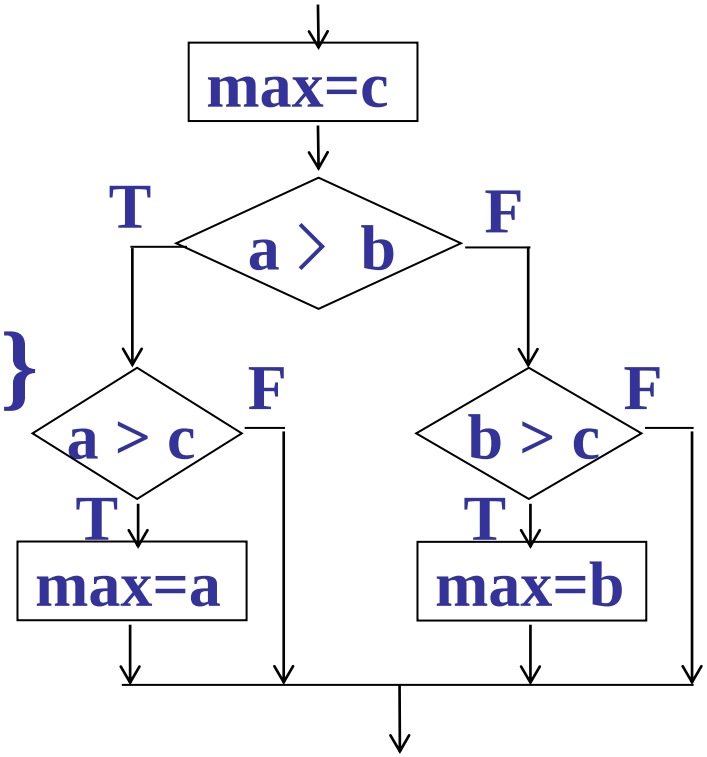
```
max = c;
if ( a > b )
   if ( a > c ) max = a;
else
   if ( b > c ) max = b;
```



2）嵌套if 语句中else的配对规则

else与其同一程序块中最靠近的还未配对的if配对。

```
max = c;
if ( a > b )
    { if ( a > c ) max = a; }
else
    if ( b > c ) max = b;
```



举例：输入百分制成绩，输出英文等级。

分数范围	等级英文名
90≤x≤100	excellent（优）
80≤x＜90	good（良）
60≤x＜80	middle（中）
x＜60	bad（差）

方案一：用多路选择结构 if-else-if

```
#include<stdio.h>
int main (void)
{
    float x;
     printf(”input the score : ”);
     scanf(”%f ”, &x);          // 输入double：  %lf
     if ( x > 100 || x < 0)
          printf(”input error!\n”);
     else {
       if (x>=90) printf(” excellent! \n”);
       else if (x>=80) printf(” good! \n”);
       else  if (x>=60) printf(” middle! \n”);
       else printf(” bad \n”);
     }
     return 0;
}
```

可无

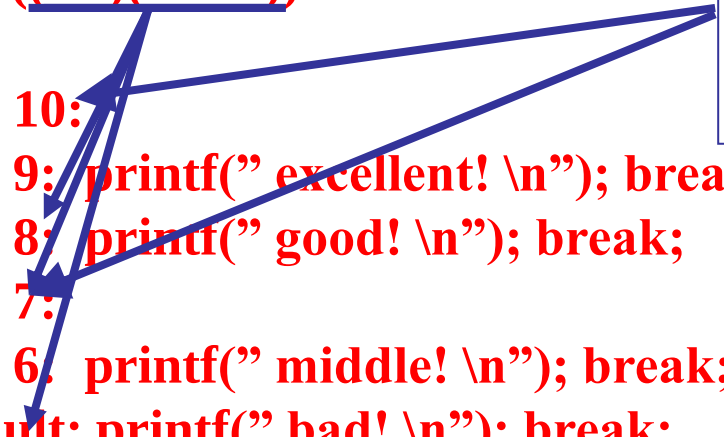
方案二：用多条 if 语句

```
#include<stdio.h>
int main (void)
{
    float x;
    printf(”input the score : ”);
    scanf(”%f ”, &x);
    if ( x > 100 || x < 0)
        printf(”input error!\n”);
    else {
        if (x>=90)  printf(” excellent! \n”);
        if (x<90&&x>=80)  printf(” good! \n”);
        if (x<80&&x>=60)  printf(” middle! \n”);
        if(x<60)  printf(” bad \n”);
    }
    return 0;
}
```

必有

方案三：用*switch*语句

```
#include<stdio.h>
int main (void)
{
    float x;
    printf(”input the score x( 0 <= x <= 100):\n”);
    scanf(”%f ”, &x);
    if ( x > 100 || x < 0)    printf(”input error!\n”);
    else
       switch ((int)(x / 10))
       {
          case 10:
          case 9:  printf(” excellent! \n”); break;
          case 8:  printf(” good! \n”); break;
          case 7:
          case 6:  printf(” middle! \n”); break;
          default: printf(” bad! \n”); break;
       }
    return 0;
}
```



空的case语句，用于几种情况合并执行一组语句。

注意：当多个case执行相同的语句时，这些case可以写成一行。

case 7: case 6: printf(" middle! \n"); break;

case 7， case 6: printf(" middle! \n"); break; ✗

case 7， 6: printf(" middle! \n"); break; ✗

4.5 switch 语句——开关语句

一般形式:

```
switch (e) {
   case 常量1: 语句序列1
               break;
   case 常量2: 语句序列2
               break;
          …
   case 常量n: 语句序列n
               break;
   default     : 语句序列
               break;
}
```

功能:

计算 e,

如e =常量i , 则执行其后的语句,直到**break**语句止

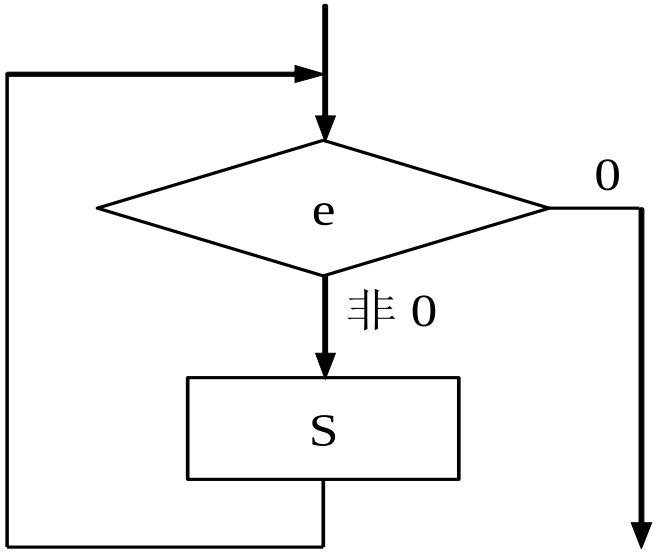
✸ 若与所有常量值不相等,则从**default**后的语句开始执行

下面语句执行时的输出？

```
i = 1;
switch ( i ){
   case 0： printf("%d\t", i);
   case 1： printf("%d\t", i++);
   case 2： printf("%d\t", i++);
   case 3： printf("%d", i++);
   default： printf("\n");
}
printf("%d\n", i);
```

4.6 while语句

while (e)
S



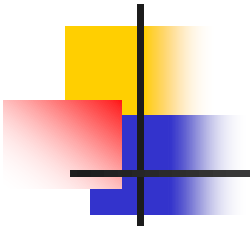
例：将输入正文每一行的前导空格删除后，将后面部分进行输出

分析：用有限状态机（FSM--Finite State Machine）方法。

状态机的4个要素：

一句话描述：当系统处于某状态时，如果发生了什么事件，就执行某动作，然后系统变成新状态

```
while ( 事件 )
  switch（当前状态）{
    case 状态i:
          if (某条件/事件)
               执行动作，迁移至状态j
          break;
     case 状态j:
          .......
          .....
}
```



使用状态机解决问题，主要有两个步骤：

（1）确定系统总共有几个状态

（2）确定状态之间的迁移过程

状态迁移图：是一种描述系统的状态、以及相互转化关系的图形方式

```
while ((ch = getchar()) != EOF)
{
    switch(当前状态) {
      case 前导空格:
            if (ch不为空白符)
                输出ch;
                状态迁移至"拷贝态"
            break;
      case 拷贝 :
            输出ch;
            if(ch为换行符)
                状态迁移至"前导空格态"
            break;
      }
}
```

```c
enum { HeadSpace,COPY };
int main()
{
  int ch, state;        /* state存当前状态 */

  state=HeadSpace;      //  初始前导空格状态
  printf("Input text,   end of Ctrl+z\n") ;
  while ((ch = getchar()) != EOF)
  {
    switch(state) {
        case HeadSpace:
            if (ch!=' '&& ch!='\t')     /* 不为空格时*/
              { putchar(ch); state=COPY; }
             break;
        case COPY :
             putchar(ch);
             if (ch=='\n')  state=HeadSpace;
             break;
    }
  }
  return 0;
}
```

例4.11　输入一个C程序(一段正文)，按原来格式复制输出，复制过程中删去所有的注释（标准C的注释）。

为了删去C程序中所有的注释，关键在于如何区分注释部分和需要复制的部分。为此，可将复制过程划分为4种状态：

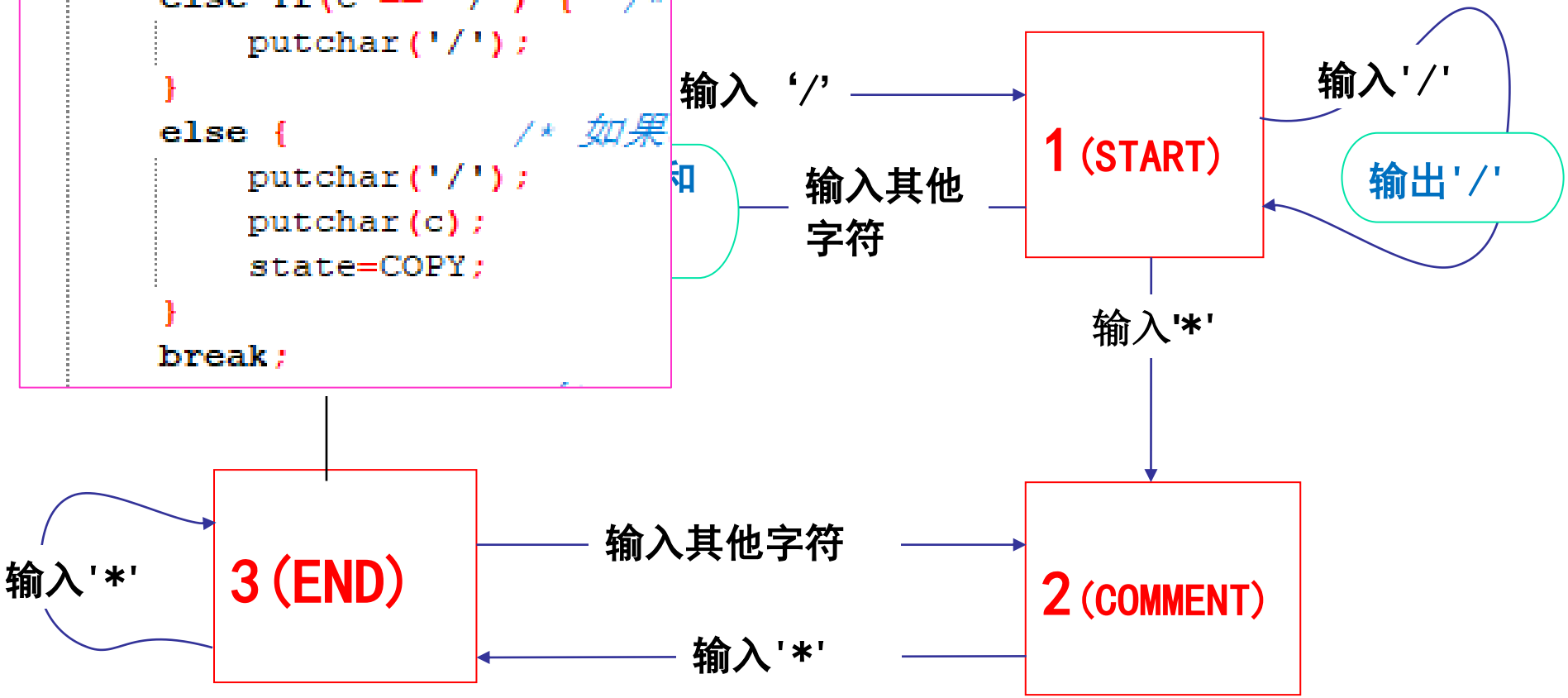
- 0--复制状态(COPY)
- 1--开始注释状态 (START)
- 2--注释状态 (COMMENT)
- 3--结束注释状态 (END);

初始状态为COPY。

状态迁移图

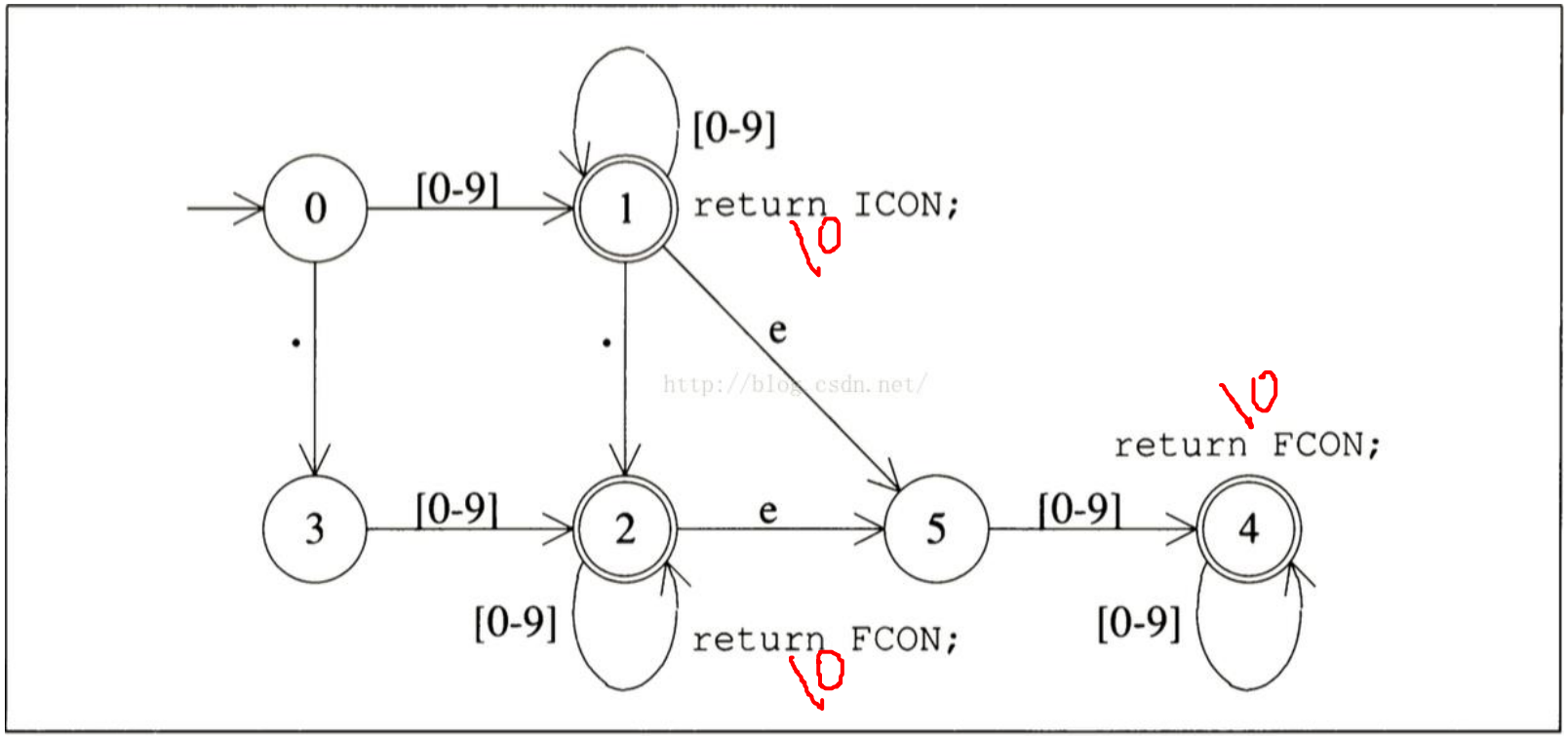
ex4-11

```
case START:        /* 在START状
    if (c == '*')    /* 如果
        state = COMMENT;
    else if(c == '/') {   /*
        putchar('/');
    }
    else {              /* 如果
        putchar('/');
        putchar(c);
        state=COPY;
    }
    break;
```



自主练习

3、整形/浮点型数值识别器: 判断给定的字符串是否是十进制整数和实数（含科学记数法）表示的数值。

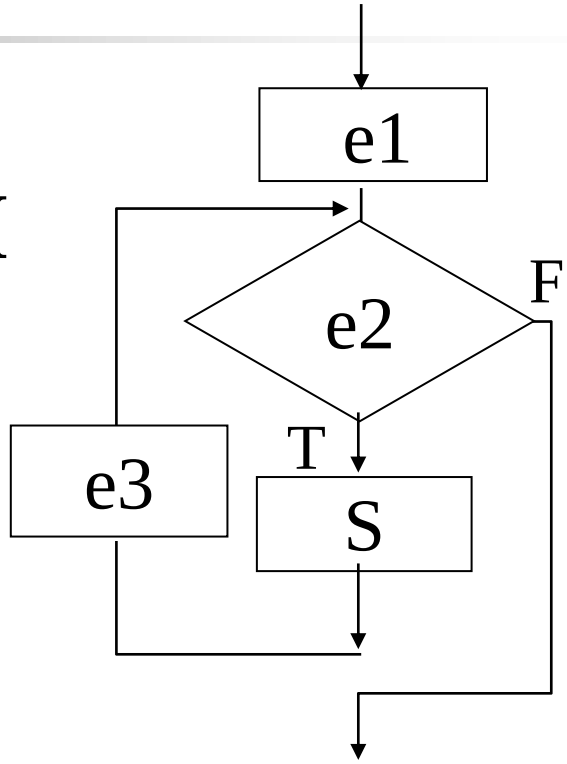


4.7 for 语句

for(e1; e2; e3) ↔

S

```
e1;
while(e2) {
   S
   e3;
}
```



for语句的几种特例

- e2为空

  for(e1; ;e3)　S ⟷ for(e1; 1 ; e3)　s

- 只有e2

  for(; e2 ;)　S ⟷ while (e2)　s

- e1,e2,e3全为空

  for(; ;)　S ⟷ while(1)　s

举例：输入一批正整数，以0为结束。输出其中最大的一个值。

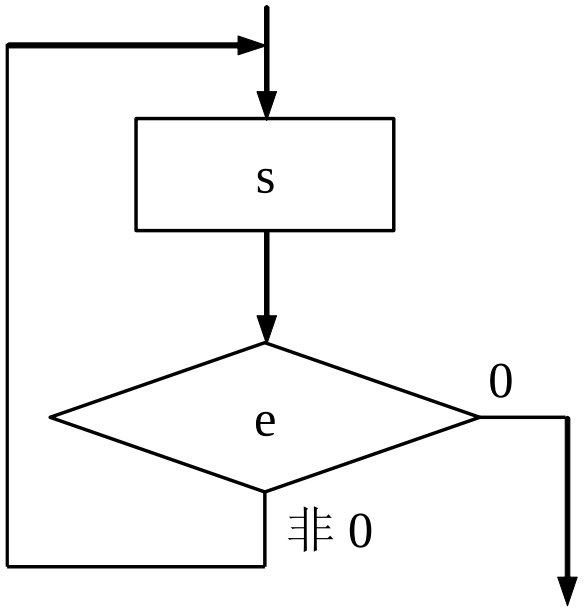
```
#include<stdio.h>
void main(void)
{
   int x, max;
   printf("input a group integer end of 0：\n");
   scanf("%d", &x);
   max = x;
   for( ; x ; ) {    /*   x 等价于 x!=0  */
      scanf("%d", &x);
      if (max < x) max = x;
   }
   printf("max=%d\n", max);
}
```

4.8 do-while语句

```
do
   s
while (e);
```

等价于

```
   s
while (e)
   s
```



举例：把输入的正整数按反方向输出。例如，输入的数是12345，要求输出结果是54321。

分析：输入：从高位到低位。

输出：从低位到高位

关键：如何获取一个整数（x）的个位数字。

算法：(1) x%10

(2) x/=10，不停循环语句直至x=0

```
/*把输入的正整数按反方向输出*/
#include<stdio.h>
int main(void)
{
    int x, digit;
    printf("input an integer x>0:\n");
    scanf("%d", &x);
    do {
        digit = x % 10;     /* 求个位数字 */
        printf("%d", digit);
        x /= 10;                /* 十位数字变个位数字 */
    }while (x);
    return 0;
}
```

```
/*把输入的整数按反方向输出*/
#include<stdio.h>
int main(void)
{
    int x, digit;
    printf("input an integer:\n");
    scanf("%d", &x);
    while (x) {
        digit = x % 10;     /* 求个位数字 */
        printf("%d", digit);
        x /= 10;                /* 十位数字变个位数字 */
    }
    return 0;
}
```

```
int main(void)
{   char ch;
lop: do{ /* 构造提示菜单，供用户选择 */
        printf("\t 1: 输入成绩\n");
        printf("\t 2: 找最高分  \n");
        printf("\t 3: 统计各分数段人数\n");
        scanf("%c%*c",&ch);  // ch=getchar(); getchar();
     }while(ch < '1' || ch > '3');
     switch(ch){    /* 根据用户输入的选择转移 */
        case '1': inputScore(int x[ ],int n); break;
        case '2': maxScore(int x[ ],int n);  break;
        case '3': statistics(int x[ ],int n); break;
     }
     printf("coutinue ? yes or no?\n");
     ch = getchar(); getchar(); /* 是否继续处理 */
     if(ch == 'y' || ch =='Y')
        goto lop;
     return 0;
}
```

4.9 goto语句和标号语句

goto 标号;

标号：语句

功能: 无条件转向标号处。

用途：（1）与if语句一起构成循环语句

```
loop: if(e){
          语句序列
          goto loop;
      }
```

（2） 从循环体中跳转到循环体外

```
        for(e1; e2; e3) {
            ……
            for(e1; e2; e3){
                ……
                if(e) goto stop;
            }
            ……
        }
stop:   ……
```

为了使程序的结构化功能强，应尽量少用goto语句。

4.10 break、contiune语句

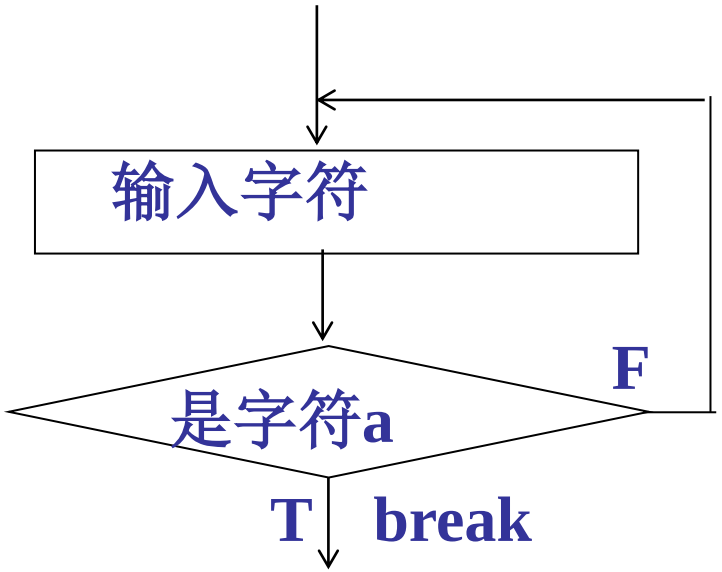
一、break语句

break;

用途：

- 在switch语句中，用于终止case子句的执行
- 用在循环体内部，强迫立即终止循环，跳过循环条件检测

```
for(; ;) {
    ch = getchar ( ) ;
    if (ch = = 'a')    break ;
}
```



1、

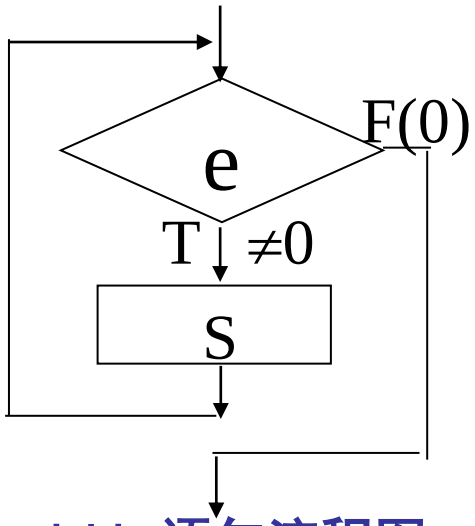
```
while ((ch = getchar ( ))!='a') ;
```

2、

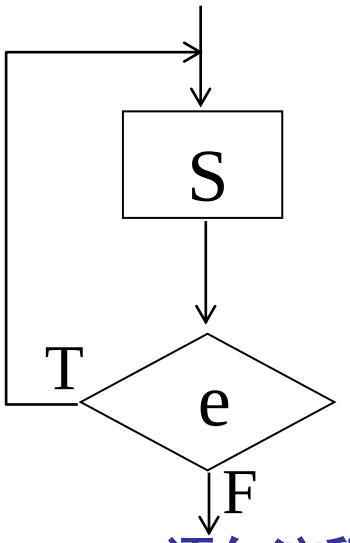
```
do
   ch = getchar ( ) ;
while (ch ! = 'a') ;
```

3、

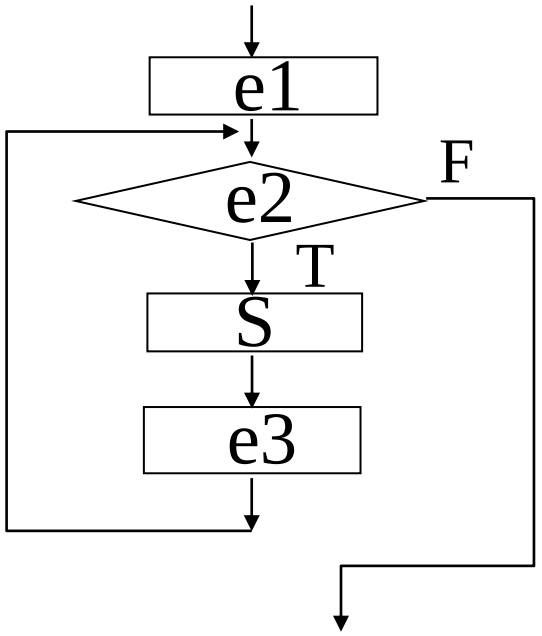
```
for(ch = getchar ( ) ; ch ! = 'a'; ch = getchar ( ) ) ;
```



while语句流程图



do-while语句流程图



for语句流程图

例　判断m是否素数.

凡不能被自身及1以外的整数整除的自然数

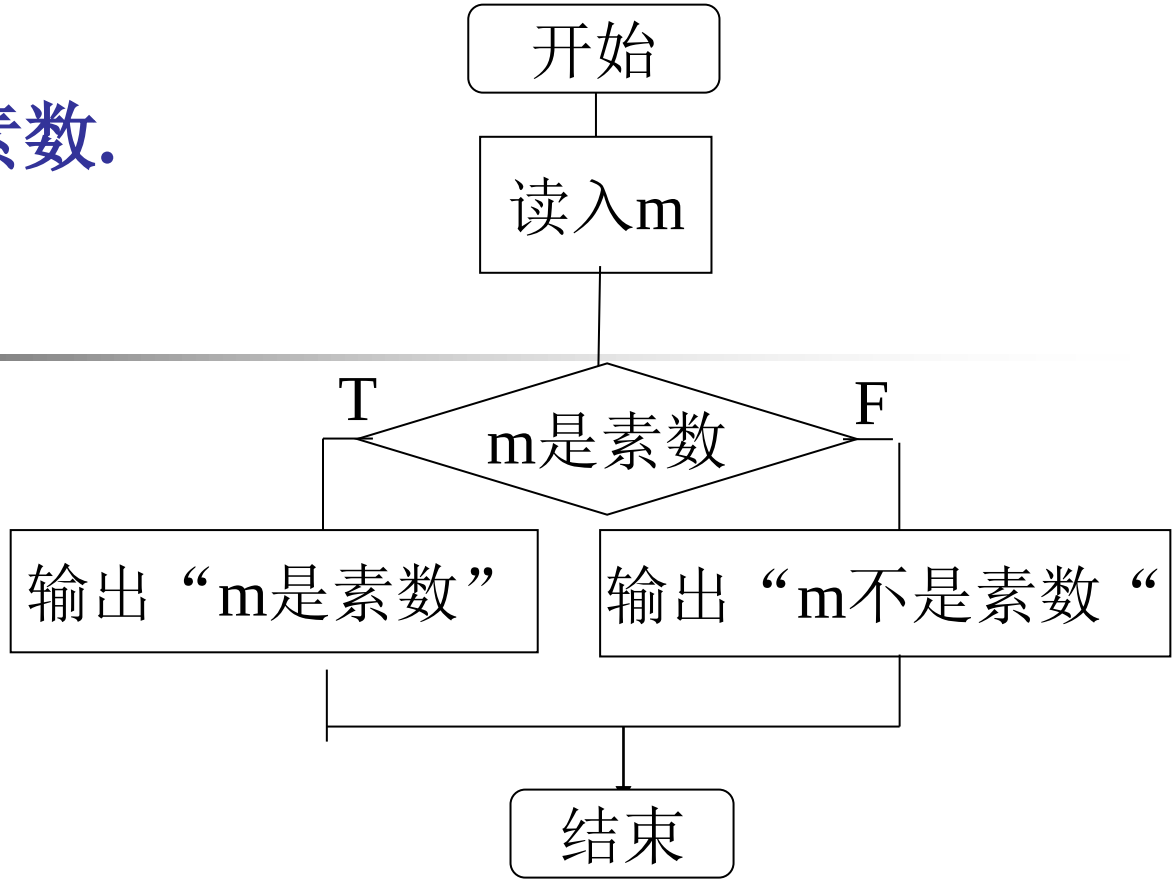
判一个数m是否为素数的方法:

当用 2，3, … ，(m-1) 的整数去除它时均不能除尽，则为素数。

2，3, … ，(m/2)

2，3, … ，$\sqrt{m}$

例　判断m是否素数.

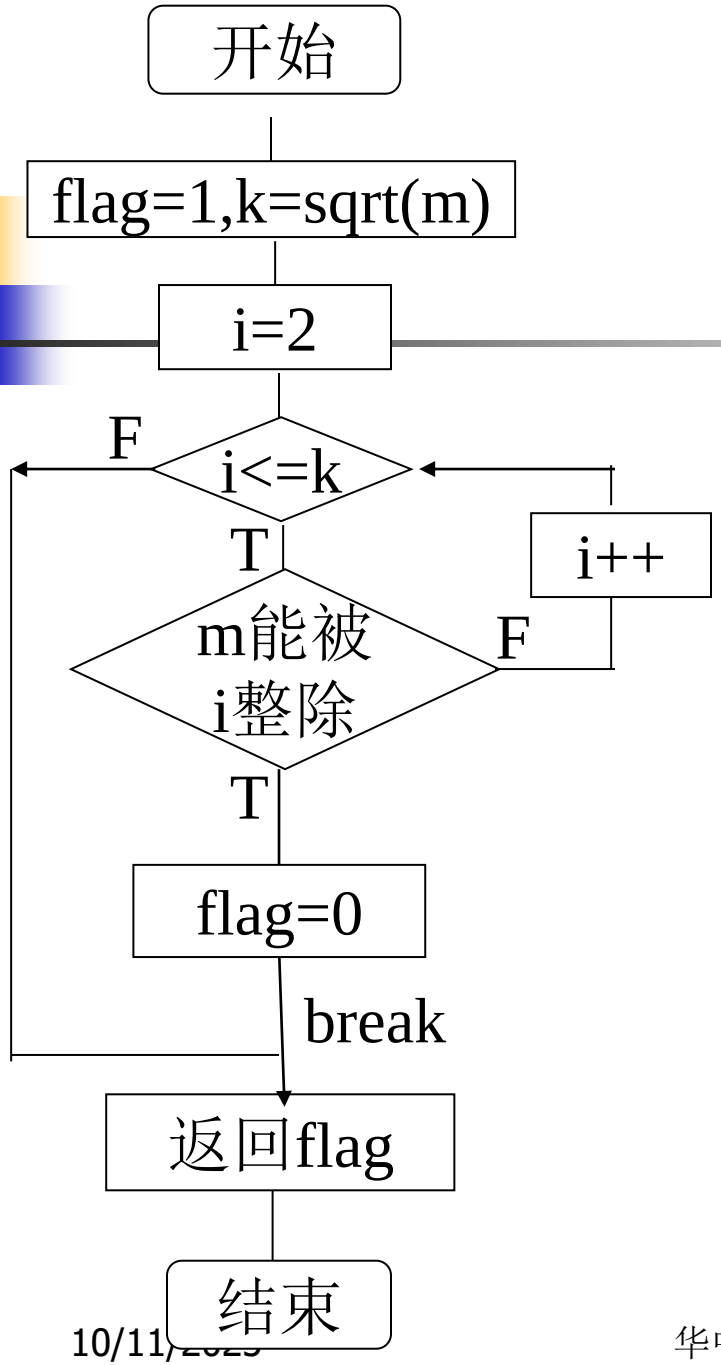


将判断一个整数是否素数定义成函数，

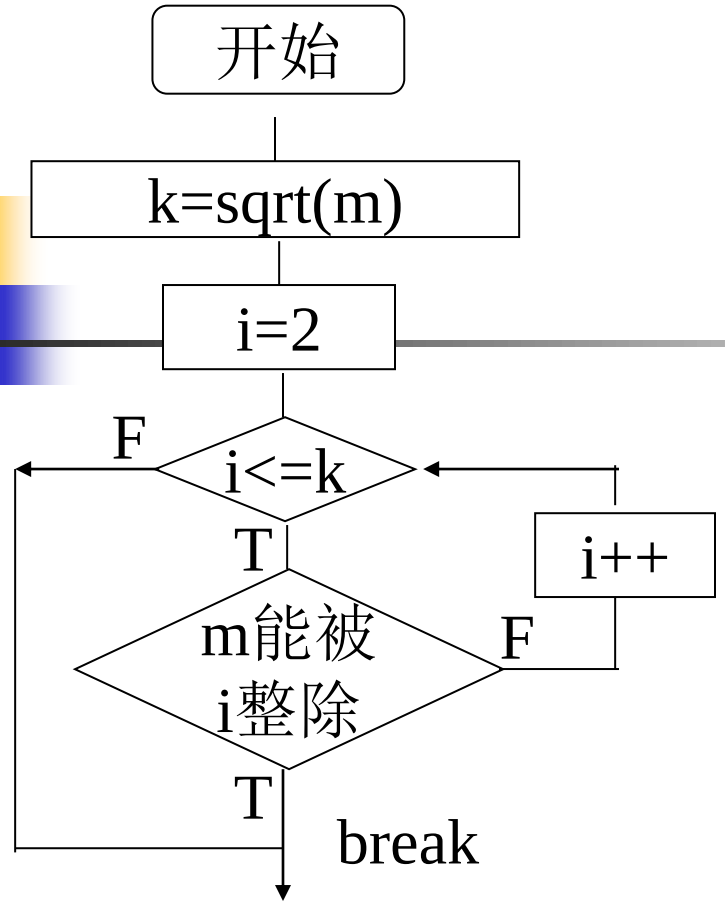
是素数函数返回1，否则，返回0。

```
int  isprime(int m);
```

```
#include<stdio.h>
 int  isprime(int m)；/*函数原型 */
int main(void)
{ int m;
  do{
    printf(“input m:”);
    scanf(“%d”,&m);
   } while(m<2);  // 输入一个大于等于2的整数
   if(isprime(m))        /*函数调用 */
     printf(“%d is a prime\n”,m);
  else
     printf(“%d isn’t a prime\n”,m);
   return 0;
}
```




```
#include<math.h>
int  isprime(int m)
{
  int flag,k,i;
  flag=1;   // 初始是素数
  k=sqrt(m);
  for(i=2;i<=k;i++)
    if (!(m% i)) {
      flag=0;
      break;
    }
   return flag;
}
```




```
#include<math.h>
int  isprime(int m)
{
  int k,i;
  k=sqrt(m);
 for(i=2;i<=k;i++)
    if (!(m% i)) {
            break;
     }
    if(i>k) return 1;
    else  return 0;
}
```

```
#include<math.h>
int  isprime(int m)
{
  int k,i;
  k=sqrt(m);
  for(i=2;i<=k;i++)
    if (!(m% i)) {
          break;
    }
   if(i>k) return 1;
   else  return 0;
}
```

```
for (i=2;i<=k&&m%i;i++)
    ;
```

例4.22：输出2~100的所有素数，每行输出10个数 .

每输出10个数, 换一次行。

```
#include<stdio.h>
#include  N  10
int  isprime(int m);      /* 函数原型  */
int main( )
{
   int n,k=1;
   printf(“%5d”,2);
   for (n=3; n<100; n+=2  ) {
      if( isprime(n) )      /* 函数调用  */
      {  printf(“%5d”,n);
         if(!(++k/N))    putchar('\n');
      }
   }
   return 0;
}
```

二、continue语句

continue;

用在循环体内，跳过其后代码，继续下一次循环。

```
/* 输入10个数，求其中正数的个数及平均值 */
for (sum=num=i=0; i<10; i++) {
    scanf（“%d”， &x);
    if (x<=0)  continue;
    num++;
    sum+=x;
}
if(num)
    printf(“numbers:%d, average:%f ”,num,1.0*sum/num);
```

```
if (x>0) {
    num++;
    sum+=x;
 }
```

4.11 return语句

return语句可以是如下两种形式之一：

（1）return ；　　/* void函数 */

导致从当前函数立即退出，返回到调用函数处

void函数也可以不包含return语句。如果没有return语句，当执行到函数结束的右花括号时，控制返回到调用处，把这种情况称为离开结束。

（2）return e； /* 非void函数 */

使流程返回到调用函数处，并送表达式的值到调用处。

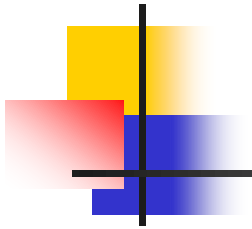
表达式值的类型应该与函数定义的返回值类型一致，如果不相同，就把表达式值的类型自动转换为函数的类型 。

4. 12　嵌套循环程序设计

嵌套循环指循环体是一个循环语句，或循环体包含循环语句。

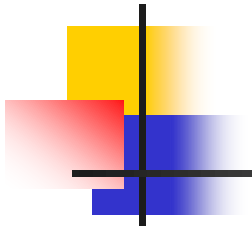
嵌套循环又称为多重循环，其中二重循环应用最为普遍。

```
for(……) {
  ……
 循环语句
  ……
}
```



```
for (i=1;i<5;i++) {
   for(j=1;j<=i;j++)
      putchar(‘*’);
   putchar(‘\n’);
}
```

```
*
**
***
****
```



```
for (i=1; i<5; i++)  {
   for( i=1; i<=5; i++ )
      putchar('*');
   putchar('\n');
}
```

```
*****
```

```
for (i=1; i<8; i++)  {
   for( i=1; i<=5; i++ )
      putchar('*');
   putchar('\n');
}
```

```
*****
*****
*****
*****
    .
    .
    .
```

◆ 内外循环变量不应同名，

否则将导致死循环或结果出错，

```
for (i=1; i<5; i++)
   for( j=1; j<=i; j++ )
      putchar('*');
   putchar('\n');
```

```
**********
```

◆ 不要忘记内、外循环体中必须的{ }，

否则将导致死循环或结果出错，

[例4.26] 编制程序：

求 $s=1^1+2^2+3^3+\cdots+n^n$ ，n由终端输入。

分析：

（1） 输入n，求和变量s置初值0;

（2） for(i=1;i<=n;i++) {

term= i^i ;

s+=term;

}

（3）输出s

[例4.26] 编制程序：

求 $s=1^1+2^2+3^3+\cdots+n^n$ ，n由终端输入。

分析：

（1） 输入n，求和变量s置初值0;

（2） for(i=1;i<=n;i++) {

```
for(term=1, j=1; j<=i; j++)  term*= i;
s+=term;
}
```

（3）输出s

```
#include<stdio.h>
/* compute  s= 1^1 + 2^2 + … + n^n  */
int main( )
{  int    i, j, n;
   long  s, term;
   printf ( “Input n:” );
   scanf ( “%d” , &n);
   s=0;
   for ( i=1; i<=n; i++) {
     for(term=1, j=1; j<=i; j++)    /*  term=i^i  */
          term*=i;
     s+=term;
    }
    printf( “s=%ld\n” , s);
}
```

◆ 注意给与循环有关的变量赋初值的位置。只需赋一次初值的操作应放在最外层循环开始执行之前，给内循环的有关变量赋初值应放在外循环体内、内循环开始执行之前

```
#include<stdio.h>
/* compute  s= 1^1 + 2^2 + … + n^n  */
int main( )
{  int    i, j, n;
   long  s, term;
   printf ( “Input n:” );
   scanf ( “%d” , &n);
   s=0;    term=1;
   for ( i=1; i<=n; i++) {
     for(j=1; j<=i; j++)
          term*=i;
     s+=term;
    }
    printf( “s=%ld\n” , s);
}
```

◆ 会怎样？

// term=$1^1 * 2^2 * (i-1)^{i-1} * i^i$

```
#include<stdio.h>
/* compute  s= 1^1 + 2^2 + … + n^n  */
long  power(int x,int n);
int main( )
{  int    i,  n;
   long  s;
   printf ( “Input n:” );
   scanf ( “%d” , &n);
   s=0;
   for ( i=1; i<=n; i++) {
     s+=power(i,i) ;
    }
    printf( “s=%ld\n” , s);
}
```

```
//计算 x^n  (n>0)
long  power(int x,int n)
{
    int i;
    long p=1;
    for(i=1;i<=n;i++)
       p*=x;
    return p
}
```

例4.26：输入一个字母，在屏幕正中输出由这个字母决定其高度的字符"金字塔"。例如输入小写字母d，则输出下列左边图形，如果输入大写字母D，则输出右边图形。

```
    a                       A
   a b a                   A B A
  a b c b a               A B C B A
 a b c d c b a           A B C D C B A
```

（1）定义变量top保存塔顶字母，变量c保存输入的字母，根据c的值设置top

（2）for(c1= top ; c1<=c; c1++) {

输出左侧空格（关键是空格数）

输出从top到c1的每个字符（递增）

输出从(c1-1)到top的每个字符（递减）

输出换行符

}

```
#define AXIS  40
#include<ctype.h>
int main（）
{  char c,c1,c2,top;
int i;
c=getchar();
top=isupper(c)?'A':(islower(c)?‘a’:'\0');
if(top)  {   /*  如 top 非零，输出图形  */
  for(c1= top ;  c1<=c;  c1++) {
      for(j=1;j<=AXIS-2*(c1-top);j++)    /*  输出左侧空格 */
            putchar(' ');
      for(c2=top;c2<=c1;c2++)  /*  输出左半段 */
            printf("%2c",c2);
      for(c2=c1-1;c2>=top;c2--)  /*  输出右半段 */
            printf("%2c",c2);
      printf("\n");              /*  换行 */
    }
}
reurn 0;
}
```

```
        a
      a b a
    a b c b a
  a b c d c b a
```

枚举（穷举）法

穷举算法是程序设计中使用得最为普遍、必须熟练掌握和正确运用的一种算法。它利用计算机运算速度快、精确度高的特点，对问题的所有可能情况，一个不漏地进行检查，从中找出符合要求的答案。

用穷举算法解决问题，通常可从两方面进行分析：

一、问题所涉及的情况：问题所涉及的情况有哪些，情况的种数可不可以确定。把它描述出来。

二、答案需要满足的条件：分析出来的这些情况，需要满足什么条件，才成为问题的答案。把这些条件描述出来。

[例4.27] 下列乘法算式中，每个汉字代表1个0～9的数字，不同的汉字代表不同的数字。试编程确定使得整个算式成立的数字组合，如有多种情况，请给出所有可能的答案。

赛软件 * 比赛 = 软件比拼

赛软件 * 比赛 = 软件比拼

a b c d a b c d e

分析：这个算式中有5个不同的汉字，分别用变量a、b、c、d、e来代表，此算式可表示为：

(a*100+b*10+c) * (d*10+a)

== b*1000+c*100+d*10+e。

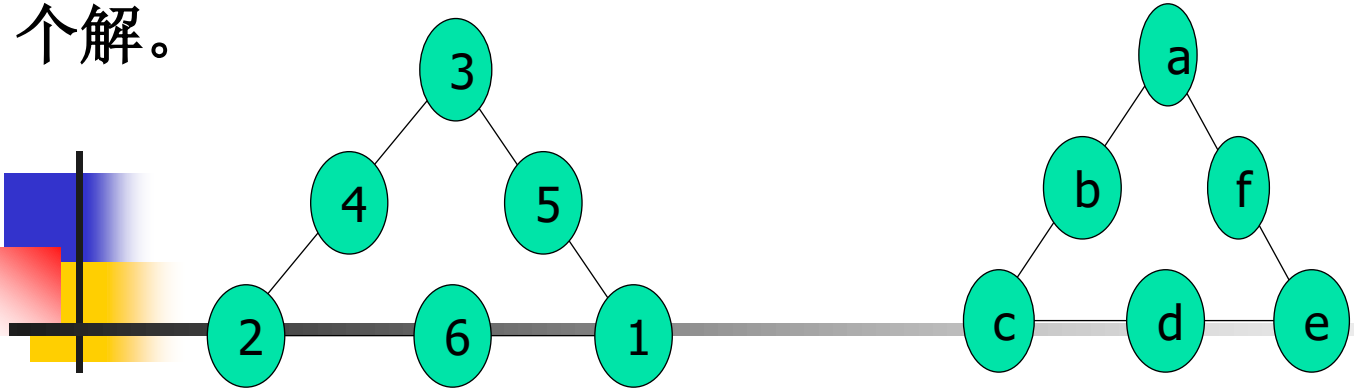
a: 0~9 b: 0~9 c: 0~9 d: 0~9 e: 0~9

且取值不得重复，共有10*9*8*7*6=30240。可采用枚举算法，依次测试每一种取值情况，如果满足上式，则得到一个解，直到测试完毕，得到所有解。

```
for(a=0;a<=9;a++){  /* 枚举 a，循环10次  */
  for(b=0;b<=9;b++){  /*枚举 b,循环9次*/
    if(b和a相等)  continue;
    for (c=0; c<10; c++) { /*枚举 c,循环8*/
      if(c和a相等或者c和b相等)  continue;
       for (d=0; d<10; d++) {/*枚举 d,循环7*/
        if(d和a~c中之一相等)  continue;
          for (e=0;e<10; e++) {/*枚举e,循环6
          if(e和a~d中之一相等)  continue;
          if (算式成立)  输出算式
          }
        }
      }
    }
}
```

枚举法练习

1、将**1~6**这六个整数排成如图所示的三角形，使得三角形三条边上的数字之和相等，求满足条件的的全部解。如图就是一个解。



穷举 a、b、c、d、e，算出 f=21-(a+b+c+d+e)

2、输入正整数x（2<=x<=79），输出所有形如 abcde/fghij=x 的表达式，其中 a～j由不同的**0~9**九个数组成的。例如，x=32时，输出为：

75168/02349=32　（75168/2349=32 ）

a/b=x　　穷举 b=1234~98765，　算出 a=b*x

if(a<=98765 且 a和b没有相同的数字）

输出 算式

筛选法例子

【例4.28】 找出10000以内的所有素数，建立素数表。

```
int a[100001];    // 以空间换时间
```

标记x是否素数？　x作为下标值，

a[x]:1是素数

:0不是素数

筛法找素数

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

(1) 2是素数，将（2*2 ～ N）之间所有2的倍数都划掉

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

(2) 3是素数，将（3*3 ～ N）之间所有3的倍数都划掉

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

(3) 4不是素数（不在筛子里），不处理

(4) 5是素数，将（5*5 ～ N）之间所有5的倍数都划掉

....

直至 N/2 或 $\sqrt{N}$

```
for (i=2; i*i<=N; i++)
  if( a[i] ) {
    for(j=i*i;j<=N;j+=i)
       a[i]=0;
  }
```

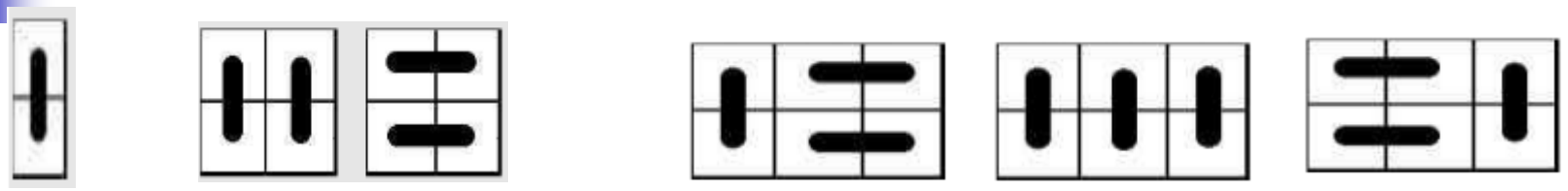
【例4. 28】 用筛法构造2~10000之间的素数表，并输出该素数中的素数。

```
#define N 10000
int main()
{
   short i, j,a[N+1]={0，0};

   for (i=2; i<=N; i++)  /* 数组初始化，先假定2~N都是素数 */
      a[i] = 1;     /* 数组下标值是被测数，用1标记素数 */
   for (i=2; i*i<=N; i++)  /* 从2开始，“筛掉”每个素数的倍数 */
      if (a[i]) {    /* 如果a[i]值为1，则i为素数 */
         for (j=i*i; j<=N; j+=i)  /* （ 可以是 2*i），从i倍的i开始“筛”,因之前的倍数已被“筛掉” */
            a[j] = 0;   /* 元素值为0表示该元素下标值不是素数 */
      }
   for (i=2,j=0; i<=N; i++)  /* 输出2~N间的素数 */
      if (a[i]) {  /* “筛”过之后，值为1的元素对应下标为素数 */
         printf("%9d", i);
         if (++j == 8) {  /* 为了输出整齐，每行输出8个素数 */
            j = 0;
            printf("\n");
         }
      }

   return 0;
}
```

递推

递推算法是一种简单的算法，不断通过已知某个条件或结果，逐步推出下一个结果或条件，直至得到问题的解。

1、用1*2的骨牌去铺满一个2*n的长方形方格，共有几种铺法. 输入n值，输出铺法种数.



n=1
1种

n=2
2种

n=3
2+1=3种

思路：设2*n的长方形方格共有f（n）种铺法，显然
f(1)=1, f(2)=2, f(3)=3,
对于f（n），考察最后一列：如果最后一列是竖着放的，那么方案数为f(n-1);如果最后一列是横着放的，那么必是两个横着放，此时方案数为f(n-2). 因此，f(n)=f(n-1)+f(n-2), n>2, 即为Fibonacci数。

- **例4.29** 盒子里有n(n<10000)个球，A、B两人轮流从盒中取球，游戏规则为：

  （1）每次从盒中取出的球的数目必须是1，3，7或者8个；

  （2）A先取球，B接着取球，双方交替；

  （3）取走盒中最后一个球的一方为输方。

  假设取球过程中A和B都不存在失误，即不会将赢的局面变为输。

- 编程实现，输入小球的数目n，按规则取球，如果A为赢方则输出1，A为输方则输出0。

- 只有1个球--- A输
  递推出 2,4,8,9个球----A赢
  （1+1=2,1+3=4,1+7=8,1+8=9)
- 只有3个球--- A输
  递推出 4,6,10,11个球----A赢
  （3+1=4,3+3=6,3+7=10,3+8=11)
- 只有5个球--- A输
  递推出 6,8,12,13个球----A赢
  ...

在递推中需要记录得出的结论？

定义数组tab[i]：下标i代表球数，元素值代表输赢。

即tab[i]表示球数为i个时，谁赢，1--A赢；0-A输

(1) 输入小球的数目n
（2）初始化数组tab，设置为0（A输）
（3）for (i=1; i<=n; i++) {//从只有1个球开始递推
    if (i个球时，A输)
        则 i+1，i+3，i+7，i+8个球，A赢
        推出n个球A赢，退出整个递推过程，输出结果
    }
（4）输出tab[n]

将每次能取出的球数保存在一个数组中。
int ins[]={1，3，7，8};

```
for (j=0; j<4; j++){
    num=i+ins[j];
    tab[num] = 1;
}
```

数组tab不能越界！！！

C语言与程序设计

The C Programming Language

第5章 函数与程序结构

华中科技大学计算机学院

毛伏兵

第5章 函数与程序结构

- 结构化编程和C程序的一般结构
- 函数的机制，包括函数定义、函数声明、函数调用、变量的存储类型等。
- 递归

5.1 C程序的一般结构

5.1.1 结构化程序设计

2条编程标准：

(1) 程序中的控制流尽可能简单。

(2) 把一个问题逐步细化分解为若干子问题，用函数实现子问题。

5.1.2 蒙特卡罗模拟：猜数游戏

- 计算机随机产生一个数（1~1000）。
- 游戏者猜直到正确为止，计算出猜数的次数。
- 发出提示信息“Too high”或“Too low”。

5.1.2 蒙特卡罗模拟：猜数游戏

- **模拟算法**：编程实现现实世界中的随机事件，例如，抛硬币、掷骰子和玩牌等。
- **蒙特卡罗模拟**：使用随机数来模拟。
- **随机数**：具有不确定性和偶然性特点，应用领域：

  软件测试--测试数据，加密系统--密钥，网络--验证码
- **随机数发生器**：生成随机数的函数

```
int  rand（void）；   // 在 stdlib.h中
```

- **伪随机数**：依靠计算机内部算法产生的“随机”数

主程序结构

```
do {
        计算机产生一个1到1000的随机数;
       游戏者猜数，直至猜对;
        printf("Play again? (Y/N) ");
        scanf("%1s", &cmd);
} while (cmd == 'y' || cmd == 'Y');
```

int GetNum（void);

void GuessNum(int x);

主程序结构

```
do {
        magic = GetNum( );   /* 产生随机数*/
        GuessNum(magic);     /* 猜数   */
        printf("Play again? (Y/N) ");
        scanf("%1s", &cmd);
} while (cmd == 'y' || cmd == 'Y');
```

rand是接口stdlib.h中的一个函数，
它返回一个非负并且不大于常量RAND_MAX的随机整数，
RAND_MAX（32767）的值取决于系统。

int GetNum(void) ;

```
/*****************************************************************
   函数名称：GetNum
   函数功能：产生一个1到MAX_NUMBER之间的随机数，供游戏者猜测。
   函数参数：无
   函数返回值：返回产生的随机数
*****************************************************************/
int GetNum(void)                        /* 注意：后面无分号  */
{
    int  x;
    printf("A magic number between 1 and %d has been chosen.\n",
            MAX_NUMBER);
    x=rand();                           /* 调用标准库函数rand产生一个随机数 */
    x= x % MAX_NUMBER + 1;              /* 将这个随机数限制在1～MAX_NUMBER之间 */
    return(x);
}
```

函数 void GuessNum(int x)

```
/*游戏者猜数，直至猜对*/
for(;;) {
   输入猜测的数给变量guess
    if（猜对了） 结束函数
    else  if（小了） 输出太小的提示
    else   输出太大的提示
}
```

源程序\ex5_1_12.c

main函数

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX_NUMBER 1000
int GetNum (void);          /*  函数原型   */
void GuessNum(int) ;        /*  void GuessNum(int x) ;函数原型   */
int main(void)
{
    char command;
    int magic;
   do
   {
       magic = GetNum( );     /*调用GetNum产生随机数*/
       GuessNum(magic);     /* 调用GuessNum猜数   */
       printf("Play again? (Y/N) ");
       scanf("%1s", &command);     /*   询问是否继续   */
    } while (command == 'y' || command == 'Y');
    return 0;
}
```

行吗？

伪随

```
int rand( )
{
    seed=（A*seed+B）% M;
    return seed;
}
```

- 伪随机数是指用数学递
- 应用最广的递推公式：

$a_0 = seed$

$a_n =（A*a_{n-1}+B）\% M，n>=1$

其中 A,B,M是产生器设定的常数,用户不能更改。

seed：种子参数，该发生器从称为种子（一个无符号整型数）的初始值开始用确定的算法产生随机数。

- 产生器给定了初始值
- seed应该在哪儿定义并初始化？
- 允许用户设置初始值（修改）
- 怎么修改？

初始化随机

```
void srand(unsigned int x )
{
   seed=x;
}
```

- 初始化随机数发生器：改变种子seed值的函数

  void srand（unsigned ）;

- 用什么值做种子？

  用系统时间（由time函数得到）作为种子。

  函数time返回自1970年1月1日以来经历的秒数。

  srand（time（NULL））；

  该初始化语句放在程序什么位置？

- 函数srand和rand的原型在stdlib.h中，

  函数time的原型在time.h中

源程序\ex5_1.c

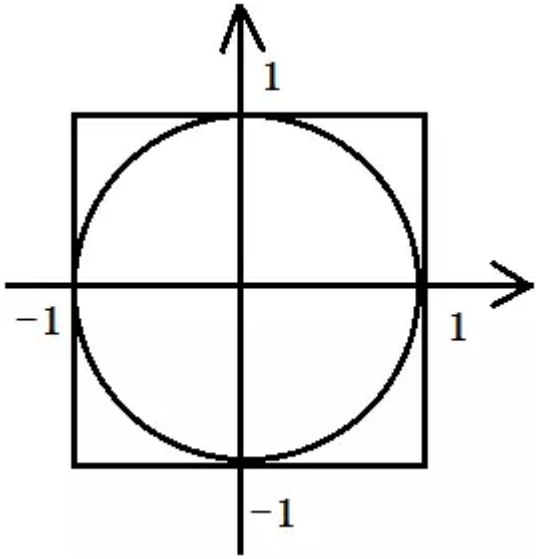
```
unsigned int seed; // 外部变量
```

```
int rand( )
{
    seed=（A*seed+B）% M;
    return seed;
}
```

```
void srand(int x )
{
    seed=x;
 }
```

练习----蒙特卡罗法求圆周率近似值

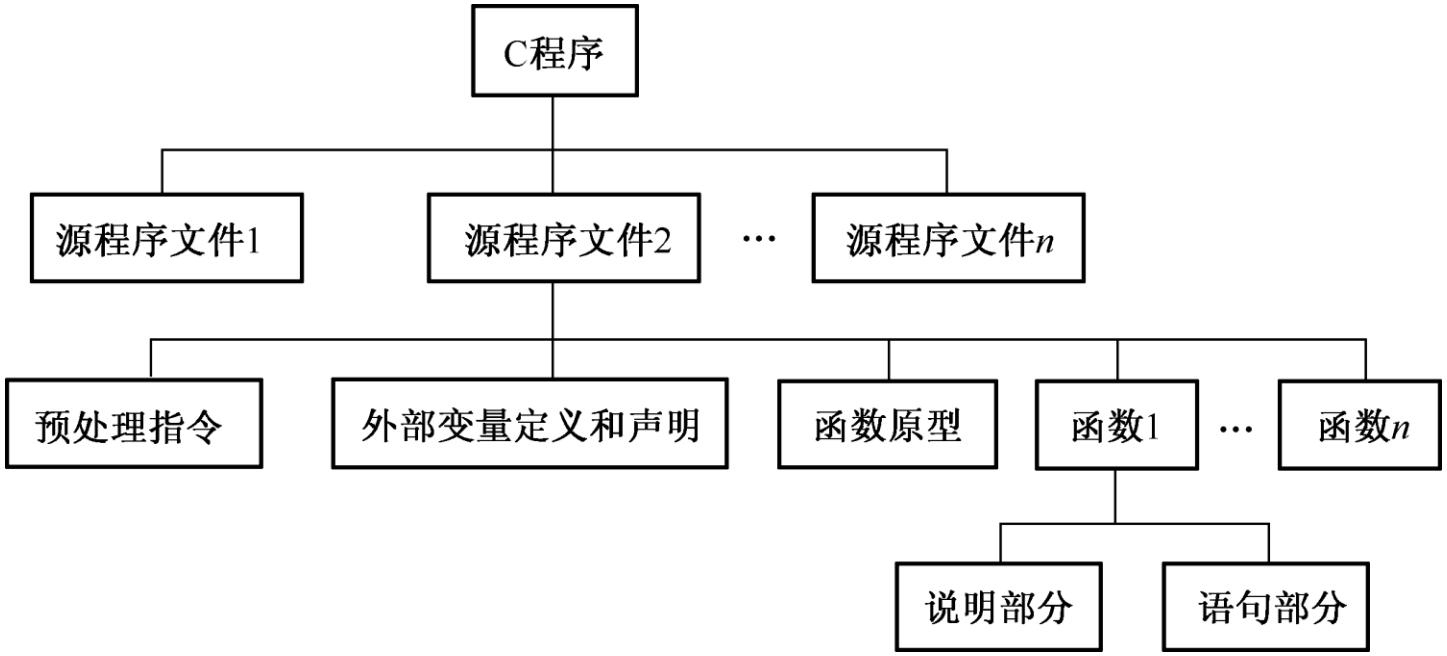
- 有一个如图所示正方形及其内切圆，往正方形内扔飞镖n次，当n足够大，比值“落在圆内的次数/落在正方形内的次数n”将无限接近“圆的面积/正方形的面积”，即π/4。n越大，π的近似结果越精确。



结构化程序设计的益处

- 使程序编制方便，易于管理、修改和调试。
- 增强了程序的可读性、可维护性和可扩充性，方便于多人分工合作完成程序的编制。
- 函数可以公用，避免在程序中使用重复的代码。
- 提高软件的可重用性，软件的可重用性是转向面向对象程序设计的重要因素。

5.1.3 C程序的结构


C程序
源程序文件1
源程序文件2
…
源程序文件n
预处理指令
外部变量定义和声明
函数原型
函数1
…
函数n
说明部分
语句部分

5.2 函数的定义与函数的声明

- 程序中若要使用自定义函数实现所需的功能，需要做三件事：

  ① 按语法规则编写完成指定任务的函数，即定义函数；

  ② 有些情况下在调用函数之前要进行函数声明；

  ③ 在需要使用函数时调用函数

5.2.1 函数的定义

函数定义的一般形式为：

类型名 函数名（参数列表）

{

声明部分

语句部分

}

形式参数（简称形参）
无参数：有void或无

无返回值：类型为void

5.2.2 函数的返回值

- return语句可以是如下两种形式之一：

（1）return ；　　/* void函数 */

（2）return 表达式 ；　/* 非void函数 */

- void函数也可以不包含return语句。如果没有return语句，当执行到函数结束的右花括号时，控制返回到调用处，把这种情况称为离开结束。
- 表达式值的类型应该与函数定义的返回值类型一致，如果不相同，就把表达式值的类型自动转换为函数的类型 。

函数返回的值，程序可以使用它，也可以不使用它

```
while(…) {
  getchar();          /*  返回值不被使用 */
  c=getchar();        /*  返回值被使用   */
  …
}
```

5.2.3 函数的声明

- 函数定义出现在函数调用后
- 被调用函数在其它文件中定义

---- 必须在函数调用之前给出函数原型

类型名 函数名（参数类型表）；

int max(int x ， int y) ;

或

int max(int ， int) ;

无参数：必须写void

良好的编程风格

- 在函数调用之前必须给出它的函数定义、函数原型或两者都给出。
- 引入标准头文件的主要原因是它含有函数原型。

5.3 函数调用与参数传递

5.3.1 函数调用

1. 函数调用的形式

函数名（实参列表）

实参是一个表达式
无参函数：为空

函数调用的形式

(1) 作为表达式语句出现。

```
GuessNum( );
putchar(c);
```

(2）作为表达式中的一个操作数出现。例如：

```
c=getchar()
magic = GetNum( );
```

(3）作为函数调用的一个实参出现，即嵌套调用。

```
printf("%10.0f",sqrt(x));
while(putchar (getchar())!='#' );
```

2. 函数调用的执行过程

- 为了说明函数的调用过程，请看下面程序。

【例5.3】编写程序实现如下功能：分列整齐地显示整数1到10的2至5次幂。输出结果如下所示：

Int	Square	Cube	Quartic	Quintic
1	1	1	1	1
2	4	8	16	32
3	9	27	81	243
4	16	64	256	1024
5	25	125	625	3125
6	36	216	1296	7776
7	49	343	2401	16807
8	64	512	4096	32768
9	81	729	6561	59049
10	100	1000	10000	100000

- 源程序\ex5_3.c

3．实参的求值顺序

- 由具体实现确定，有的从左至右计算，有的按从右至左计算。

a=1;

power(a,a++)

----从左至右：power(1,1)

----从右至左：power(2,1) (多数)

为了保证程序清晰、可移植，应避免使用会引起副作用的实参表达式 .

5.3.2 参数的值传递

- 参数的传递方式是“值传递”，实参的值单向传递给相应的形参。如果实参、形参都是x，被调用函数不能改变实参x的值。

【例5.4】改写例5.3来说明值传递概念。

```
#include<stdio.h>
double  power(int,int);  /*  函数原型 */
int main(void)
{
        int  x, n;
            /* 分列整齐地显示整数1到10的2至5次幂 */
        for(x=1;x<=10;x++)
        {
                    for(n=2;n<=5;n++)
                                printf("%10.0f", power(x,n));
                    printf("\n");
        }
        return 0;
}

double  power(int x, int n)  // 计算x的n次方

{
            double  p;
            for (p=1;n>0;n--)
                  p*=x;
            return  p;
}
```

传地址（引用）调用

- 传地址（引用）调用是将变量的地址传递给函数，函数既可以使用，也可以改变实参变量的值。标准库函数scanf就是一个引用调用的例子，通过地址实参返回一个或多个数据。程序员也可以定义这种带有地址形参的函数，这部分内容将在第9章中介绍。
- **int x;　scanf("%d",&x);**

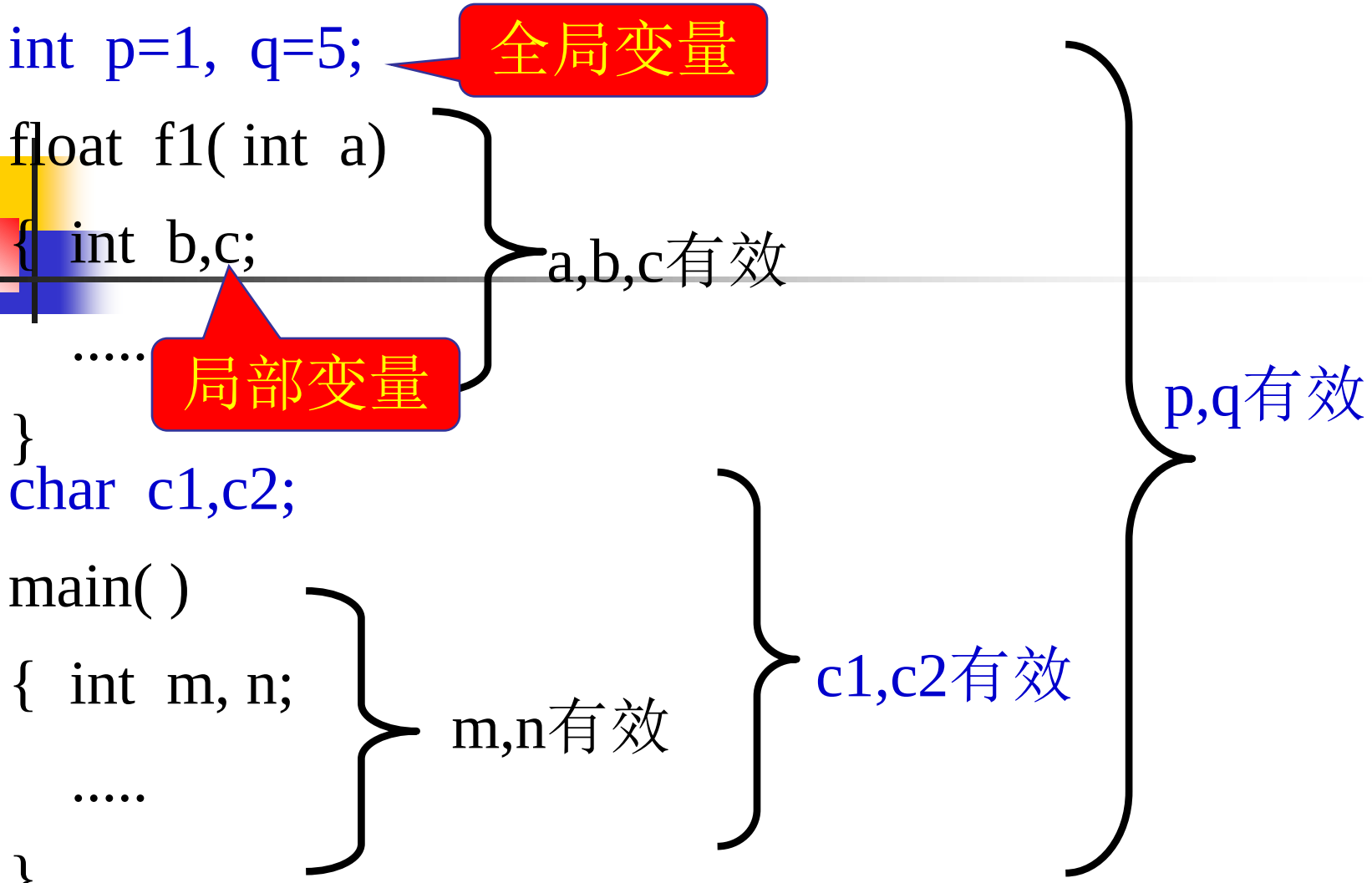
5.4 作用域与可见性

- 作用域是指标识符（变量或函数）的有效范围，也就是指程序正文中可以使用该标识符的那部分程序段。
- 可见性：块多重嵌套时，同名外层变量不可见
- 按照作用域，变量分：

  局部变量和全局变量

5.4.1 局部变量和全局变量

- **局部变量**：在函数内部定义的变量，作用域是定义该变量的程序块，程序块是带有说明的复合语句（包括函数体）。不同函数可同名，同一函数内不同程序块可同名。**形式参数是局部变量。**
- **外部变量**：在函数外部定义的变量，其缺省作用域从其定义处开始一直到其所在文件的末尾，由程序中的部分或所有函数共享。


int p=1, q=5;
全局变量
float f1(int a)
{ int b,c;
.....
局部变量
}
a,b,c有效
char c1,c2;
main( )
{ int m, n;
.....
}
m,n有效
c1,c2有效
p,q有效

extern声明

```
srand(int x)
{
   seed=x;
}
int seed=C;
int rand( )
{
   seed =（A*seed+B）% M ;
   return seed;
}
```

引用出错：无作用

外部变量的定义性声明语句从定义点开始有效直至文件尾

extern声明

```
srand(int x)
{
   extern int seed;
   seed=x;
}
int seed=C;
int rand( )
{
    seed =（A*seed+B）% M ;
    return seed;
}
```

外部变量的引用性声明语句

外部变量的定义性声明语句
从定义点开始有效直至文件尾

定义性声明和引用性声明的区别

- 外部变量的定义性声明---- 分配存储单元
  - 位于所有的函数之外
  - 定义一次
  - 可初始化
- 外部变量的引用性声明----通报变量的类型
  - 位于在函数内或函数外
  - 出现多次
  - 不能初始化

外部变量 PK 形式参数

- 函数之间可以通过外部变量进行通信。
- 一般地，函数间通过形式参数进行通信更好。这样有助于提高函数的通用性，降低副作用。

```
int main(void)
{  int m;
   m = rand( )%10+1;
   guess(m);
   guess(12);
}
void guess(int x)//猜x
{  ......  }
```

```
int magic;
int main(void)
{ int m;
   m = rand( )%10+1;
   magic=m;
   guess();
 } guess();
}
void guess( ) //猜magic
{ ...... }
```

```
int  min;
int maxmin(int x,int,y);
int  maxmin (int x, int  y)
{   int  z;
    min=(x<y)?x : y;
    z=(x>y)? x : y ;
    return  z;
}

int main (void)
{    int   a=4,b=1,max;
     max=maxmin (a , b) ;
     printf("The max is %d\nThe min is %d",max,min);
    return 0;
}
```

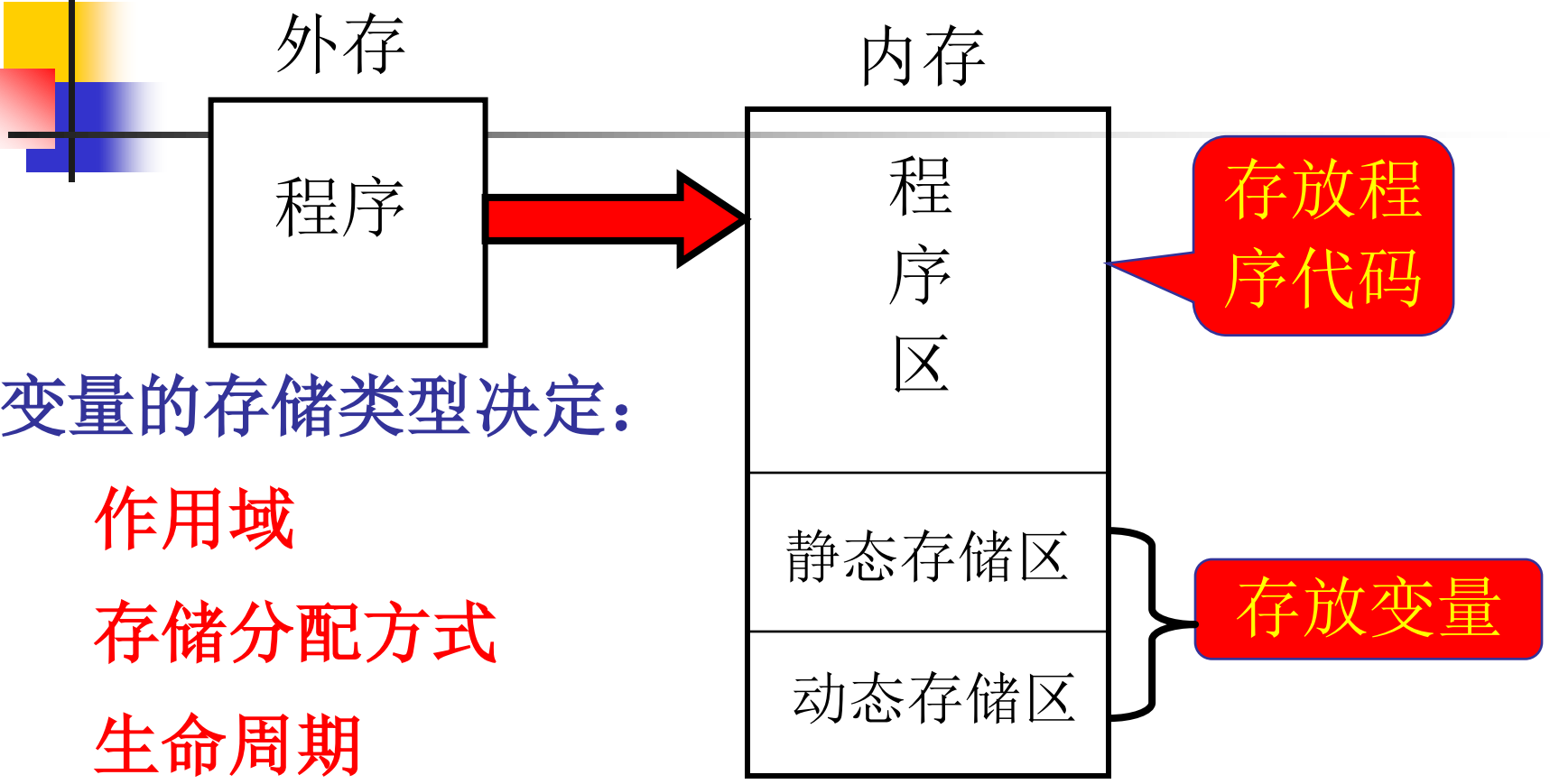
外部变量

函数值为4

The max is 4

The min is 1

5.5 变量的存储类别



变量的存储类型决定：

作用域

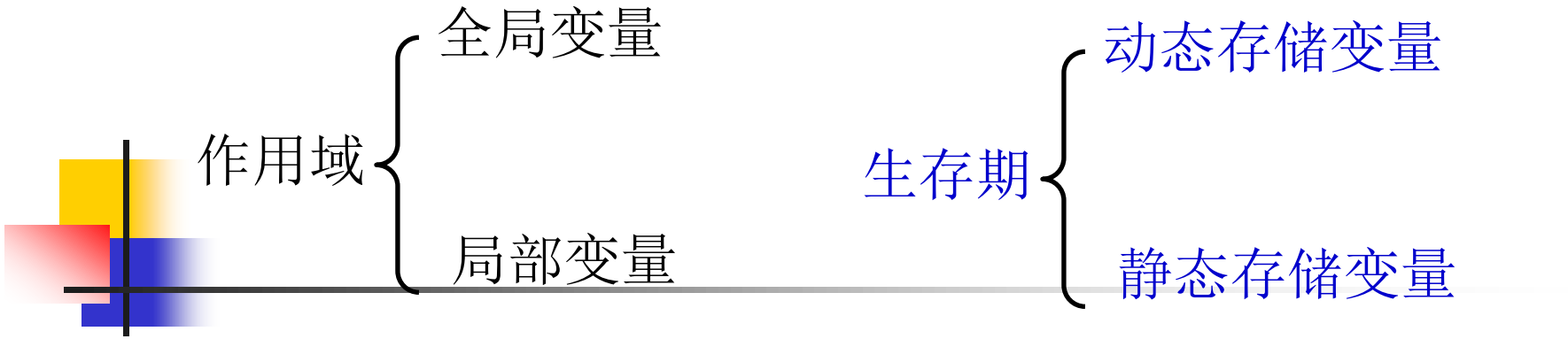
存储分配方式

生命周期

初始化方式

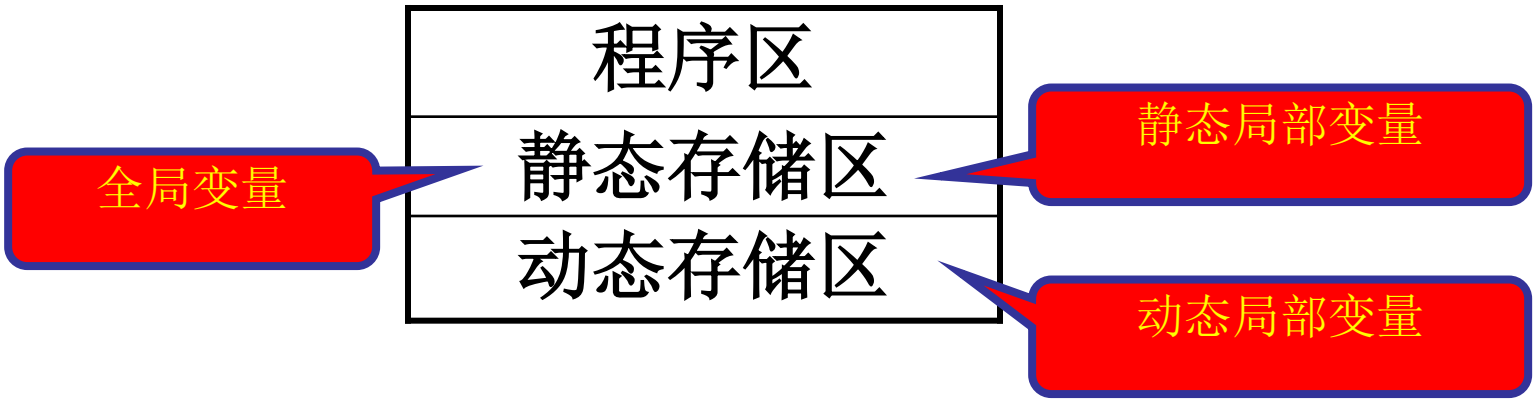
需要区分变量的存储类型

关键字有：auto、extern、static和register



静态存储：在文件运行期间有固定的存储空间，直到文件运行结束。

动态存储：在程序运行期间根据需要分配存储空间，函数结束后立即释放空间。若一个函数在程序中被调用两次，则每次分配的单元有可能不同。



5.5.1 存储类型auto

- 局部变量的缺省存储类型是auto，称自动变量
- auto int a; /*等价于int a; 也等价于auto a; */
- 作用域：局部于定义它的块，从块内定义之后直到该块结束有效。
- 存储分配方式：动态分配方式,即在程序运行过程中分配和回收存储单元。
- 生命周期：短暂的，只存在于该块的执行期间。
- 初始化方式：定义时没有显示初始化，其初值是不确定的；有显示初始化，则每次进入块时都要执行一次赋初值操作。

5.5.2 存储类型extern

- 外部变量的存储类型是extern，但在定义时不使用关键字extern。
- 外部变量的作用域:从定义之后直到该源文件结束的所有函数，通过用extern做声明，外部变量的作用域可以扩大到整个程序的所有文件。
- 存储分配方式:静态分配方式,即程序运行之前，系统就为外部变量在静态区分配存储单元，整个程序运行结束后所占用的存储单元才被收回。
- 生命周期：永久的，存在于整个程序的执行期间。
- 初始化方式：定义时没有显示初始化，初值0；有显示初始化，只执行一次赋初值操作。

定义点前或其它文件引用：需用extern作引用声明

```
srand(int x)
{
   extern int seed;
   seed=x;
}
int seed=C;
int rand( )
{
    seed =（A*seed+B）% M ;
    return seed;
}
```

引用在前，定义在后

文件file1.c中的内容为：

```
int   a;
int main(void)
{  extern  int  fun (int);
   int  b=3, c, d, m;
   ...
   c=a*b;
   d= fun(m);
   ...
 }
```

外部变量定义

外部变量说明

引用本文件外定义的全局变量

文件file2.c中的内容为：

```
extern  int   a;
int fun (int n )
{  int  i, y=1;
   ...
   y*=a;
   return   y;
}
```

5.5.3 存储类型static

关键字static有两个重要而截然不同的用法：

(1) 用于定义局部变量，称为静态局部变量。

(2) 用于定义外部变量，称为静态外部变量。

存储分配方式:静态分配方式

生命周期: 永久的，

缺省初值: 0,只执行一次

静态局部变量和自动变量有根本性的区别。

静态外部变量和外部变量区别:作用域不同。

```
srand(int x)
{
  seed=x;   // seed在此无作用
}

int rand( )
{
    int seed=D;  // 自动变量，存于栈，每次调用初始化为D
    seed =（A*seed+B）% M ;// 不能产生随机系列
    return seed;
}
```

```
srand(int x)
{
  seed=x;   // seed在此无作用，用户无法改变种子参数
}

int rand( )
{
    static int seed=D;  // 静态局部变量，存于静态区
                        // 初始化只执行1次
    seed =（A*seed+B）% M ; // 可以产生随机系列
    return seed;
}
```

```
int seed=D;  // 外部变量，存于静态区， 初始化只执行1次
srand(int x)
{
  seed=x;  // seed在此有作用，用户可改变种子参数
}

int rand( )
{
    seed =（A*seed+B）% M ; // 可以产生随机系列
    return seed;
}
```

1. 静态局部变量

- 作用域：自动变量一样

【例5.7】 编程计算1!+2!+3!+4!+…n!

- 将求阶乘定义成函数，要求使用static使计算量最小。

```
    scanf("%d",&n);
    for(i=1;i<=n;i++)
          sum+=fac(i);
    printf("1!+2!+..+%d!=%ld\n",n,sum);
    return 0;
}
/*****************************************************************
函数名称：fac
函数功能：利用静态局部变量的特性求一个整数的阶乘。
函数参数：形参n是int型
函数返回值：返回n的阶乘，类型long
*****************************************************************/
long  fac(int  n)
{
    static  long f=1;       /* 静态局部变量 */
    f *=n;
    return f;
}
```

2. 静态外部变量

```
static int seed=D;
srand(int x)
{
    seed=x;
}
int rand( )
{
   seed =（A*seed+B）% M ;
   return seed;
}
```

静态外部变量
只能作用于定义它的文件，
其它文件中的函数不能使用

2. 静态外部变量

- 静态外部变量和外部变量的区别是作用域的限制。
- 静态外部变量只能作用于定义它的文件，其它文件中的函数不能使用，
- 外部变量用**extern**声明后可以作用于其它文件。
- 使用静态外部变量的好处在于：当多人分别编写一个程序的不同文件时，可以按照需要命名变量而不必考虑是否会与其它文件中的变量同名，保证文件的独立性。
- 和局部变量能够屏蔽同名的外部变量一样，一个文件中的静态外部变量能够屏蔽其他文件中同名的外部变量。在静态外部变量所在的文件中，同名的外部变量不可见。

5.5.4 存储类型register

- 用来定义局部变量, register建议编译器把该变量存储在计算机的高速硬件寄存器中，除此之外，其余特性和自动变量完全相同。
- 使用register的目的是为了提高程序的执行速度。程序中最频繁访问的变量，可声明为register。

```
register int  i;                /* 等价于register  i; */
for (i=0;i<=N;i++)  {       …}
```

- 不可多，必要时使用。
- 无地址，不能使用&运算。

C语言与程序设计

The C Programming Language

第12章 递 归

华中科技大学计算机学院

毛伏兵

本章讲授内容

递归（recursion）是一项非常重要的编程技巧，可以使程序变得简洁和清晰，是许多复杂算法的基础。本章介绍

- 递归、递归函数的概念；
- 递归的执行过程；
- 典型问题的递归函数设计；
- 分治法与快速排序；（了解）
- 回溯法；（不要求）
- 递归在动态规划等算法中的应用。（不要求）

```
#include<stdio.h>
// 输出 n ~ 1
void  prn_int(int n)
{   if (n>0)  {
        printf ("%d ",n);
        prn_int(n-1);
    }
 }
int main(void)
{  prn_int(5);
   return 0;
 }
```

递归函数：在定义中含有递归调用

递归调用：调用自己

12.1 递归概述

- 递归是一种函数在其定义中直接或间接调用自己的编程技巧。递归策略只需少量代码就可描述出解题过程所需要的多次重复计算，十分简单且易于理解。
- **递归调用**：函数直接调用自己或通过另一函数间接调用自己的函数调用方式

  f() { … f(); … }
- **递归函数**：在函数定义中含有递归调用的函数

【例12. 1】 用递归法计算阶乘n！

- 阶乘的计算是一个典型的递归问题。n！定义为：

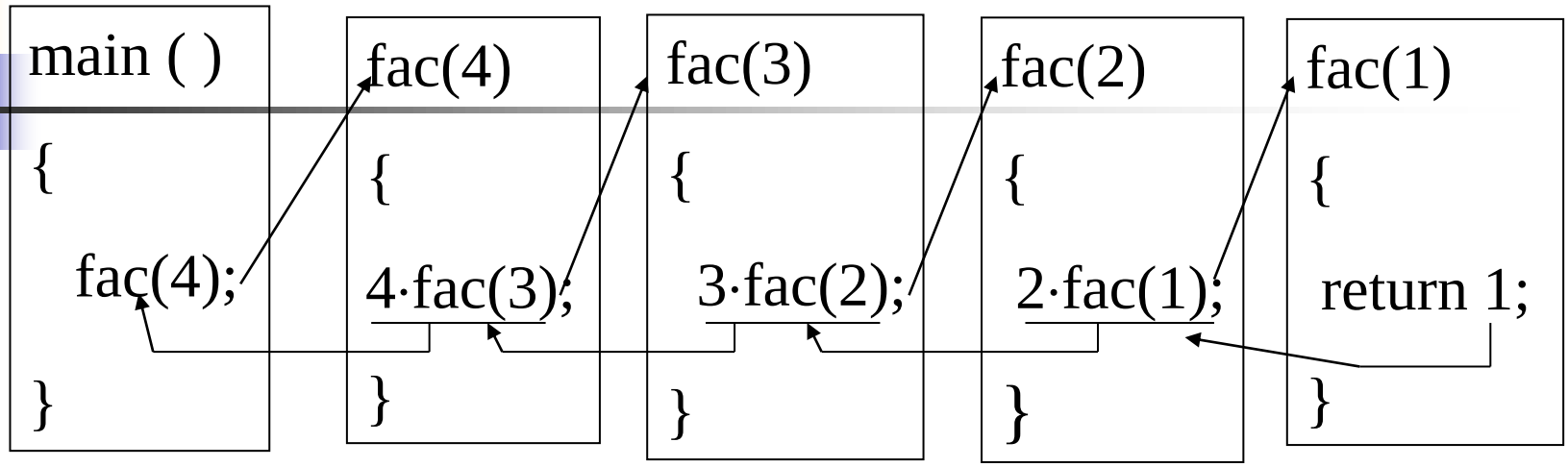
$$n!=\begin{cases}1 & n=0,1\\ n\times(n-1)! & n>1\end{cases}$$

- 这是递归定义式，对于特定的k，k!只与k和(k-1)！有关，上式的第一式是递归结束条件，对于任意给定的n！，将最终求解到1！或0！。

n！的递归实现

```
/*
函数功能：递归法求一个整数的阶乘。
函数参数：参数n为int,表示要求阶乘的数。
函数返回值：n的阶乘值，类型为long。        */
long factorial(int n)
{
 if (n==0||n==1)          /*  递归结束条件 */
        return 1;
 else
        return (n*factorial(n-1));  /* 递归调用 */
}
```

4！的递归执行过程



- 把求解问题转化为规模较小的子问题，通过多次递归一直到可以得出结果的最小解，然后通过最小解逐层向上返回调用，最终得到整个问题的解。
- 将递归概括为一句话：“能进则进，不进则退”
- 简化算法（易于理解)、但不节省存储空间，运行效率也不高(运行时开销大，效率低）

n！ 的迭代实现

```
/* 迭代法计算n！ */
long factorial_iteration( int n )
{
    int result = 1;
    while( n>1 ) {
        result *=n;
        n--;
    }
    return result;
}
```

对比

- **两种方式的比较**

两种实现方式都非常简单易懂，在实际项目中，为了效率，应该优先选择迭代。

- **什么情况下使用递归**

用递归能容易编写和维护代码，且运行效率并不至关重要

递归算法的特点

- 递归算法的运行效率较低，耗费的计算时间较长，占用的存储空间也较多。
- 递归算法结构紧凑、清晰、可读性强、代码简洁。
- 大多数的简单递归函数都能改写为等价的迭代形式。什么情况下使用递归呢？如果用递归能容易编写和维护代码，且运行效率并不至关重要，那么就使用递归。例如，像二叉树这样的数据结构，由于其本身固有的递归特性，特别适合于用递归处理；像回溯法等算法，一般也用递归来实现。

递归的要素

（1）每次调用在规模上有所缩小。

（2）递归结束条件

当子问题的规模足够小时，必须能够直接求出该规模问题的解。

例 编写一个递归函数计算Fibonacci数列的第n项。

```
/*  Recursive fibonacci : Compute the n item  */
long fibonacci (long n )
{
  if ( n = = 1 || n = = 2)  return 1;
  else
    return  fibonacci(n-1)+fibonacci(n-2);
 }
```

这种递归方式有损于运行效率——如何改进？

12.2 递归函数设计

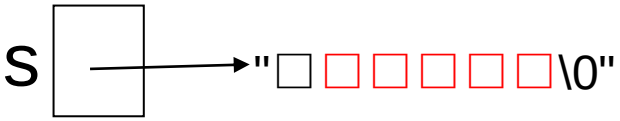
- 递归是一种强大的解决问题的技术，其基本思想是将复杂问题逐步转化为稍微简单一点的类似问题，最终将问题转化为可以直接得到结果的最简单问题。在较高级的程序中，递归是一个很重要的概念。

12.2.1 字符串的递归处理

- 字符串是以空字符（'\0'）结尾的字符序列。因此，可以把字符串看成“一个字符后面再跟一个字符串”，或者仅有一个空字符组成的空串。这个字符串的定义说明字符串是一种递归的数据结构，可以用递归的方法对一些基本的处理字符串函数进行编写。

【例1】 用递归实现标准库函数strlen(s)

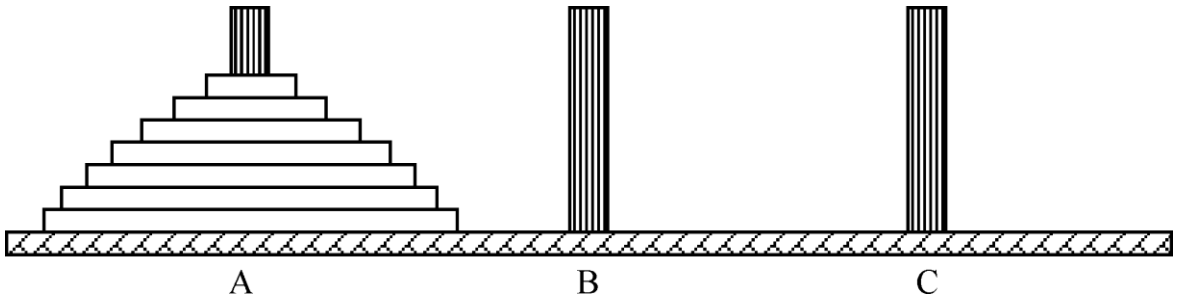
- 字符串看成“一个字符后面再跟一个字符串”




```
int strlen(char s[ ])
{
    if(s[0]=='\0')
        return 0;
    else
        return(1+strlen(s+1));
}
```

12.2.2 汉诺塔问题

- 问题：木桩A上有64个盘子，盘子大小不等，大的在下，小的在上。把木桩A上的64个盘子都移到木桩C上，条件是一次只允许移动一个盘子，且不允许大盘放在小盘的上面，在移动过程中可以借助木桩B。



【例12.3】 设计一个求解汉诺塔问题的算法。

- 这是一个典型的用递归方法求解的问题。要移动n个盘子，可先考虑如何移动n-1个盘子。分解为以下3个步骤：

  （1）把A上的n-1个盘子借助C移到B。

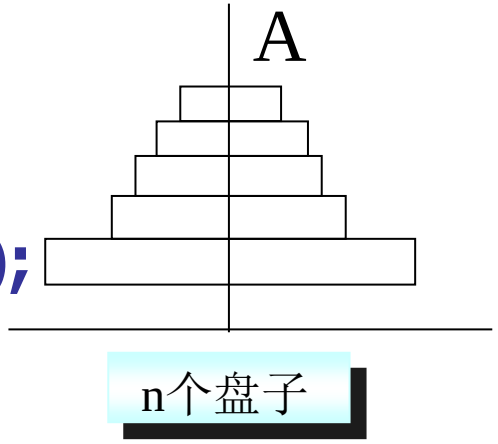
  （2）把A上剩下的盘子（即最大的那个）移到C。

  （3）把B上的n-1个盘子借助A移到C。

- 其中，第（1）步和第（3）步又可用同样的3步继续分解，依次分解下去，盘子数目n每次减少1，直至n为1结束。这显然是一个递归过程，递归结束条件是n为1。

函数move(n, a, b, c)

```
/* 将n个盘从a借助b，移至c */
void move(int n,int a,int b,int c )
 {
   if (n= =1) printf(" %c– –>%c\n ", a, c);
   else {
       move (n–1, a,c, b);
        printf(" %c– –>%c\n ", a, c);
        move (n–1, b, a, c);
    }
 }
```



移动次数：1个盘子：移动1次

2个盘子：移动2*1+1=3次

3个盘子：移动2*3+1=7次

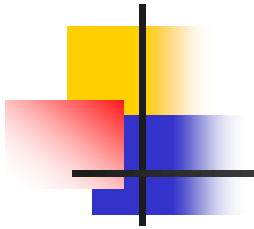
n个盘子：移动（2^n-1）次

64个盘子：移动（2^{64}-1）次

12.3 分治法与快速排序

- **分治法**：将一个大问题划分成若干互相独立的子问题，这些子问题与原问题相同但规模更小。
- **递归求解子问题**：分治与递归像一对孪生兄弟，经常同时应用在算法设计之中。

quick排序算法



- quick排序法是C.A.R.Hoare于1962年发明的。
- 基于分治策略

quick排序算法

（1）分解：将数组分为两部分

给定数组a[left]~a[right]，从中选择一个元素（称为分区元素），并把其余元素划分为两个子集合：

a[left]～a[split-1]、a[split]、a[split+1]~a[right]

左边部分的所有元素都比右边部分的元素小。

（2）递归求解

对两个子集合递归应用同一过程，当某个子集合中的元素个数小于2时，递归结束。

函数QuickSort

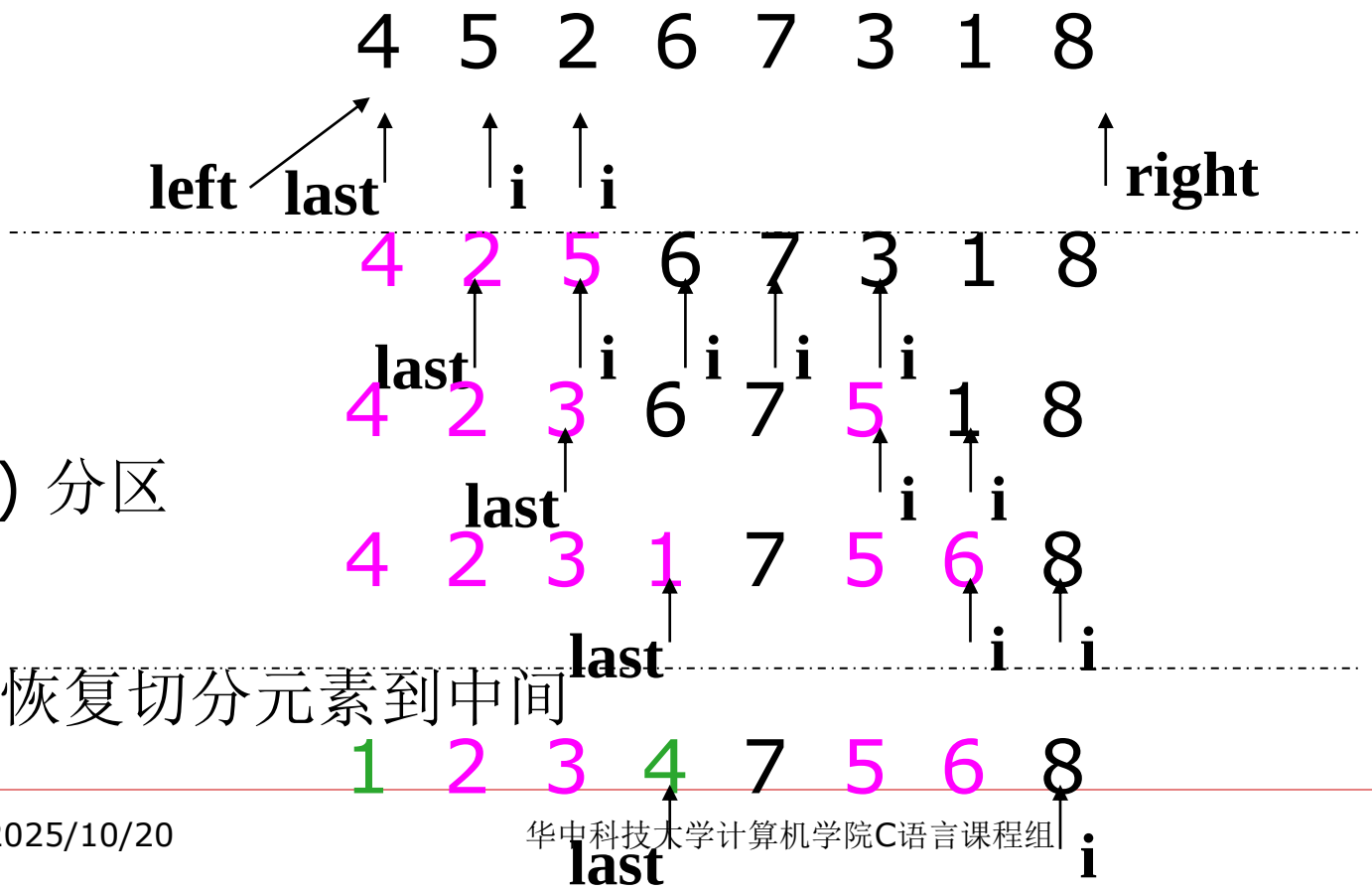
```
/* quick排序法 */
void QuickSort(int a[ ],int left,int right)
{
   int split;     /* 分区位置 */
   if(left<right) /* 待排序数组的元素个数至少为2 */
   {
        split=partition(a,left,right); /* 将数组元素分成两部分*/
        QuickSort(a,left,split-1);     /* 对左边部分递归排序 */
        QuickSort(a,split+1,right);    /* 对右边部分递归排序 */
   }
}
```

分区

- 选择中间元素作为切分元素

v[]={ 6 5 2 4 7 3 1 8}

(1) 移切分元素到最左边位置



(2) 分区

(3) 恢复切分元素到中间

```
/* 将数组a中的元素a[left ]至a[right]分成左右两部分，
   返回切分点的下标。  */
int partition_v1(int a[ ],int left,int right )
{  int i, last;     /*last为左边部分最后一个元素的位置*/
   int  split=(left+right)/2;  /*选择中间元素作为切分元素 */
   swap(a, left, split);  /*  移切分元素到最左边位置  */
   last = left;      /* 初始化last */
   for (i = left + 1; i <= right; i++)   /* 分区：从左至右扫描  */
       if (a[i] < a[left])   /* 小的数移到左边*/
           swap(a, ++last, i);
    swap(a, left, last);   /* 将切分元素移到两部分之间*/
    return last;
}
```

```
/* swap:  交换 v[i] and v[j] */
void swap(int v[], int i, int j)
{
    int temp;
     temp = v[i];
    v[i] = v[j];
    v[j] = temp;
  }
```

- stdlib.h 中有 函数 qsort 能对任意类型的对象排序。

分区

- 选择中间元素作为切分元素

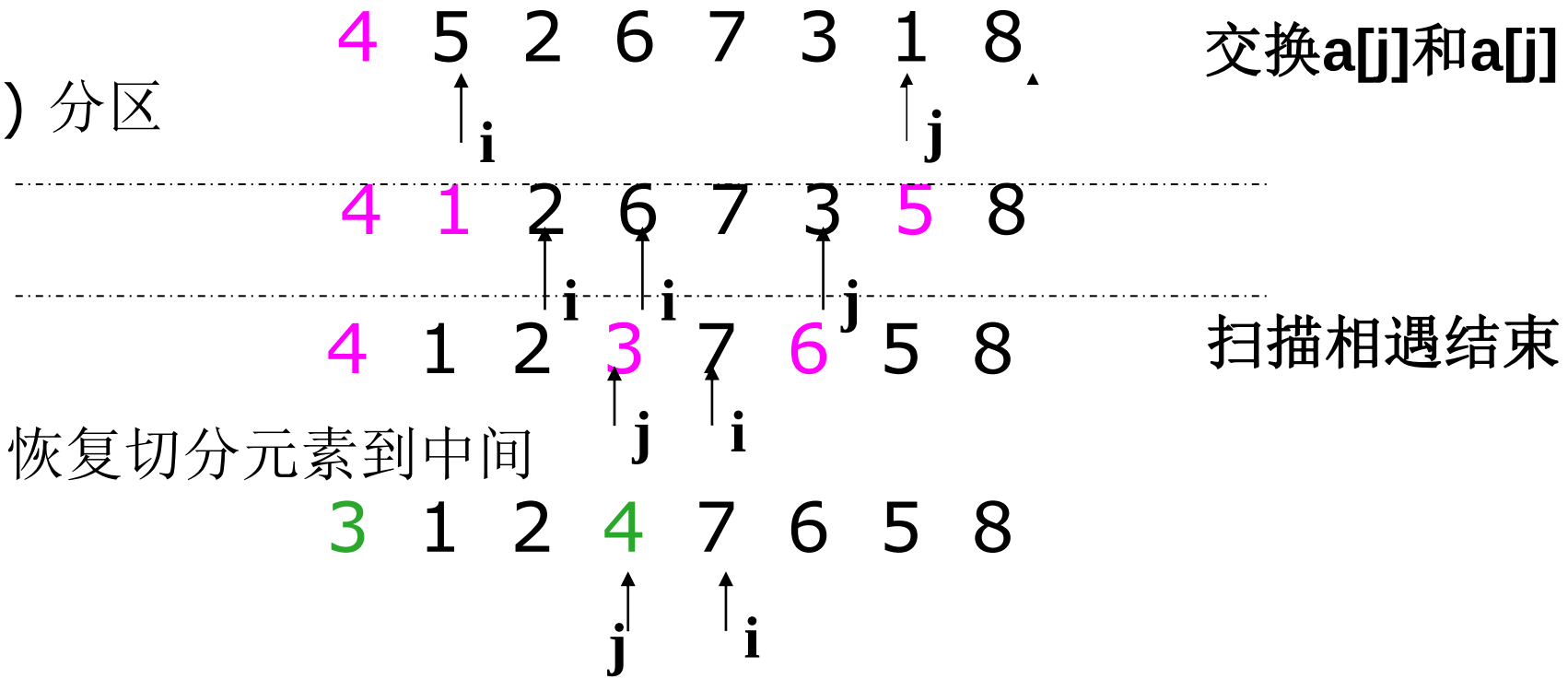
v[]={ 6 5 2 4 7 3 1 8}

i 从左至右扫描，找到比切分元素大的数停止

j从右至左扫描，找到比切分元素小的数停止

(1) 移切分元素到最左边位置

(2) 分区



(3) 恢复切分元素到中间

函数partition

```
int partition(int a[ ],int left,int right )
{
   int  i=left,j=right+1;
   int  split=(left+right)/2;  /*  选择中间元素作为切分元素  */
   swap(a,left,split);   /*  将切分元素移到数组的开头  */
   for ( ; ; )
   {
        while(a[++i]<=a[left] && i <= right);   /*从左至右扫描 */
        while(a[--j]> a[left]);          /*  从右至左扫描 */
        if(i>=j) break;     /*  扫描相遇（或交叉）结束循环  */
                swap(a,i,j);  /*  交换左右两边的元素  */
   } /*  j 是切分元素的位置  */
   swap(a,left,j);    /*  将切分元素重新移到中间  */
   return j;     /*  返回切分元素的下标  */
}
```

切分元素的选择

- 选择切分元素有很多种策略，最简单的方法是选用数组的第一个元素，该法对随机排列的数组很好，如果数据基本有序，则执行效率很差。上述程序中的方法可极大提高对有序或基本有序数组排序的效率。更加完善的策略是选择中间值，或至少是介于最大值和最小值之间的数值。
- 编写测试主函数用于输出排序结果，具体代码如下。
- \源程序\ex12_5.c

C语言与程序设计

The C Programming Language

第6章　编译预处理

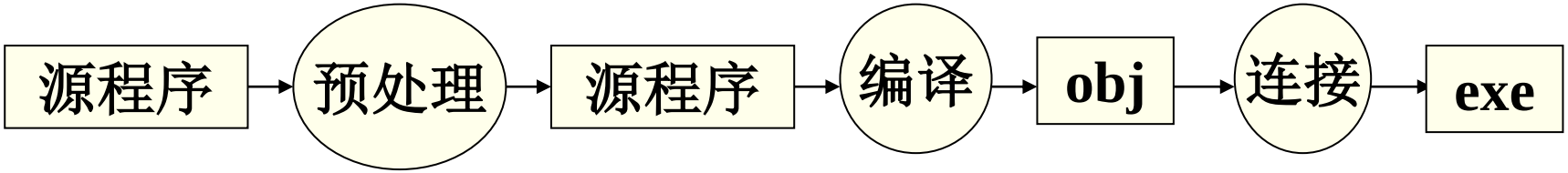
华中科技大学计算机学院

毛伏兵

本章重点

- 宏定义#define
  - 无参宏定义
  - 带参宏定义 √

编译预处理



编译预处理：对源程序进行编译之前所作的工作，它由预处理程序负责完成。编译时，系统将自动引用预处理程序对源程序中的预处理指令进行处理。

预处理指令：以“#”号开始的指令。

6.1 文件包含#include

用指定文件的内容取代该预处理指令行，有2种一般形式：

（1） #include <文件名>

在指定的标准目录下寻找被包含文件

（2） #include "文件名"

首先在用户当前目录中寻找被包含文件，若找不到，再在指定的标准目录下寻找

6.2 宏定义#define

用一个标识符来表示一个字符串.

#define 标识符 字符串

宏名：被定义的标识符。

宏代换（宏展开）：在编译预处理时，用字符串去取代宏名.

预处理前

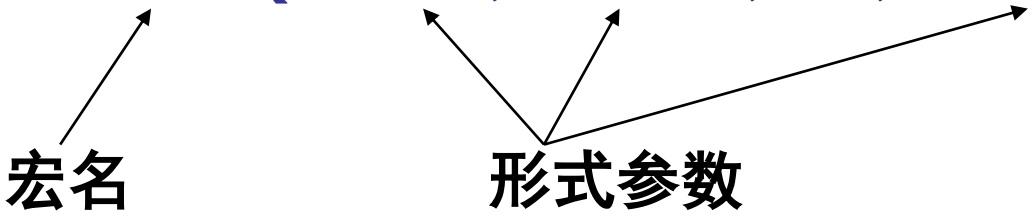
```
#define  M  (y*y+3*y )
void main(void)
{  int  s,y;
   printf("Input a number: ");
   scanf("%d",&y);
   s=3*M+4*M+y*M;
   printf("s=%d\n",s);
}
```

预处理后

```
void main(void)
{  int  s,y;
   printf("Input a number: ");
   scanf("%d",&y);
   s=3* (y*y+3*y ) +4* (y*y+3*y )
        +y* (y*y+3*y );
   printf("s=%d\n",s);
}
```

6.3 带参数的宏定义

#define 标识符(标识符，标识符，…，标识符) 字符串



宏调用：给出实参

宏展开：（1）用字符串替换宏，

（2）用实参去替换形参

例　定义计算x^2的宏

```
#define  SQ(x)  ((x)*(x))
```

宏调用：SQ(a+1)

宏展开：((a+1) * (a+1))

宏调用：SQ(SQ(a))

宏展开：((((a)*(a))) * (((a)*(a))))

为什么要这么多的括号？

考虑：#define SQ(x) x*x

宏调用：SQ(a+b)

宏展开：a+b*a+b /* 与(a+b)*(a+b)不同 */

再考虑：#define SQ(x) (x)*(x)

宏调用：27/SQ(3)

宏展开：27/(3)*(3) /* 值27, 与 $27/3^2$ 不同 */

定义带参数的宏时，为了保证计算次序的正确性，表达式中的每个参数用括号括起来，整个表达式也用括号括起来。

注意：宏名和左括号之间不能有空格

#define SQ（x）((x)*(x))

被认为是无参宏定义 。

宏调用：SQ(3)

宏展开：(x) ((x)*(x)) (3) /*显然错误的*/

带参的宏虽被认为不安全，但还是很有价值

```
#define  SQ(x)  ((x)*(x))
```

宏调用：SQ(++a)

宏展开：((++a)*(++a))　　/*a加2次

如是函数调用，将不会有问题 */

- 宏节省了函数调用的开销，程序运行速度更快，形式参数不分配内存单元，不必作类型说明。但是，宏展开后使源程序增长。
- 宏比较适合于经常使用的简短表达式，以及小的可重复的代码段；当任务比较复杂，需要多行代码才能实现时，或者要求程序越小越好时，就应该使用函数。

6.4 取消宏定义#undef

终止宏名的作用域，形式为：

#undef 标识符

何时使用#undef指令?

- 防止宏名的冲突

```
#include "everything.h"
#undef SIZE   /*everything.h中定义了SIZE，就取消它；
                否则该指令不起作用*/
#define SIZE 100
```

- 保证调用的是一个实际函数而不是宏

```
#undef getchar
int getchar(void) {…}
```

6.5 条件编译

- 预处理程序提供了条件编译指令，用于在预处理中进行条件控制，根据所求条件的值有选择地包含不同的程序部分，因而产生不同的目标代码。 这对于程序的移植和调试是很有用的。对源程序的各部分有选择地进行编译称为条件编译。
- 条件编译指令三种形式，控制流与if-else语句类似。如表6.1所示 .

例 利用R计算圆或正方形的面积

预处理前

```
#define R
void main(void)
{
    float c, s;
    printf ("input a number: ");
    scanf("%f", &c);
    #ifdef R
    s=3.14159*c*c;
    printf("%f\n", r);
    #else
    s=c*c;
    printf("%f\n", s);
    #endif
}
```

预处理后

```
void main(void)
{
    float c, s;
    printf (“input a number: ”);
    scanf(“%f”, &c);
    s=3.14159*c*c;
    printf(“%f\n”, s);
}
```

生成的目标程序较短

例 利用R计算圆或正方形的面积

预处理前

```
void main(void)
{
    float c, s;
    printf ("input a number: ");
    scanf("%f ", &c);
    #ifdef R
    s=3.14159*c*c;
    printf("%f\n", s);
    #else
    s=c*c;
    printf("%f\n", s);
    #endif
}
```

预处理后

```
void main(void)
{
    float c, s;
    printf (“input a number: ”);
    scanf(“%f”, &c);
     s=c*c;
    printf(“%f\n”, s);
}
```

生成的目标程序较短

6.4.2 defined运算符

- defined是预处理运算符，其形式为：

  defined（标识符）或defined 标识符

  它用来判断标识符是否被#define定义了，如被定义，值为1，否则为0

- #ifdef R

  同

  #if defined(R)

6.4.3 条件编译的应用

【例6.5】采用条件编译，避免多次包含同一个头文件。

- 为了避免一个头文件被多次包含，可以在头文件的最前面两行和最后一行加上预编译指令，让头文件在被多个源文件引用时不会多次编译。

```
#ifndef _NAME_H
#define _NAME_H        /* 定义头文件的标识符 */
……                     /* 头文件的内容 */
#endif
```

其中，NAME是头文件的名字。比如头文件为myFile.h，则其标识符可为_MYFILE_H。

- 在创建一个头文件时，用#define指令为它定义一个唯一的标识符。通过#ifndef指令检查这个标识符是否已被定义，如果已被定义，则说明该头文件已经被包含了，就不要再次包含该头文件，#ifndef就帮助编译器跳过直到#endif的所有文本；反之，则定义这个标识符，以避免以后再次包含该头文件。

调试程序时临时忽略一些代码

- 可以把代码放在注释中。然而，如果代码中也含有注释，就会导致语法错误。
- 使用条件编译能解决这个问题

```
#if 0
  不编译的代码
#endif
```

- 要让编译器编译这段代码，把原来的0改为1就可以了。

调试程序时跟踪程序的执行

- 为调试而增加的语句放在条件编译指令之间，在调试时编译这些语句。如：

```
#ifdef  DEBUG
    printf("Variable x=%d\n",x);
#endif
```

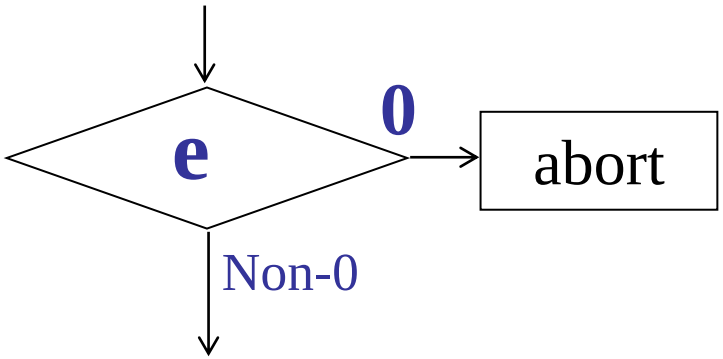
- 在调试程序时，在前面加#define DEBUG，就编译printf语句，输出供判断参考的x值。完成调试后，从程序中去掉#define指令，编译就忽略为调试而插入的printf语句，相当于它被“自动”删除了。

6.5 assert宏(C中的断言)

在头文件assert.h中，用来测试表达式的值是否符合要求，其形式如下：

assert(e)

如果e值非0，程序继续执行下一个语句。如果e值0，就输出错误信息，并通过调用实用库中的函数abort终止程序的执行。



断言与防御性编程

防御性编程使用户在运行程序（发布版本里）时，当出现意外情况时程序仍能继续工作。

```
long  Fact(int  n)
{
    int  i;
    long result = 1;
    assert(n >= 0);    /* 使用断言检查入口参数的合法性 */
    for (i=2; i<=n; i++)
        result *= i;
    return result;
}
```

如果n<0，会输出包含行号和文件名的错误信息并中断执行：

Assertion failed:n>=0, file test.c,line 32

把断言看作一种简单的制造栅栏的方法，这种栅栏能使错误在穿过自己时暴露。

C语言与程序设计

The C Programming Language

第7章 数组

华中科技大学计算机学院

毛伏兵

【例】使用一维数组计算学生的平均成绩

- **学生人数？**

  设定一个上限N

- **N个学生的成绩如何存放？**

  数据结构：数组

- **算法步骤：**

（1）输入学生人数n；

（2）输入n个学生成绩存放于数组score；

（3）成绩的累加求和存放于sum；

（4）计算平均成绩average；

（5）输出 average；

程序

```
#include <stdio.h>
#define SIZE  30     /* 学生人数上限值 */
 int main(void)
{    int score[SIZE],i,sum,n;
      double average;
      printf(“输入学生人数(<=%d):\n”, SIZE);
       scanf(“%d”,&n);   /* 输入人数存于n */
       printf(“输入%d个学生的成绩:\n”,n);
      for(i=0;i<n;i++)      /* 输入成绩存于数组score中 */
           scanf(“%d”,&score[i]);
      for(sum=0,i=0;i<n;i++)      /* 累加求和 */
           sum+=score[i];
      average=1.0*sum/n;                  /* 计算平均成绩 */
      printf("average=%f\n",average);
      return 0;
}
```

第7章 数组

用来描述一群有联系的同类型数据集合

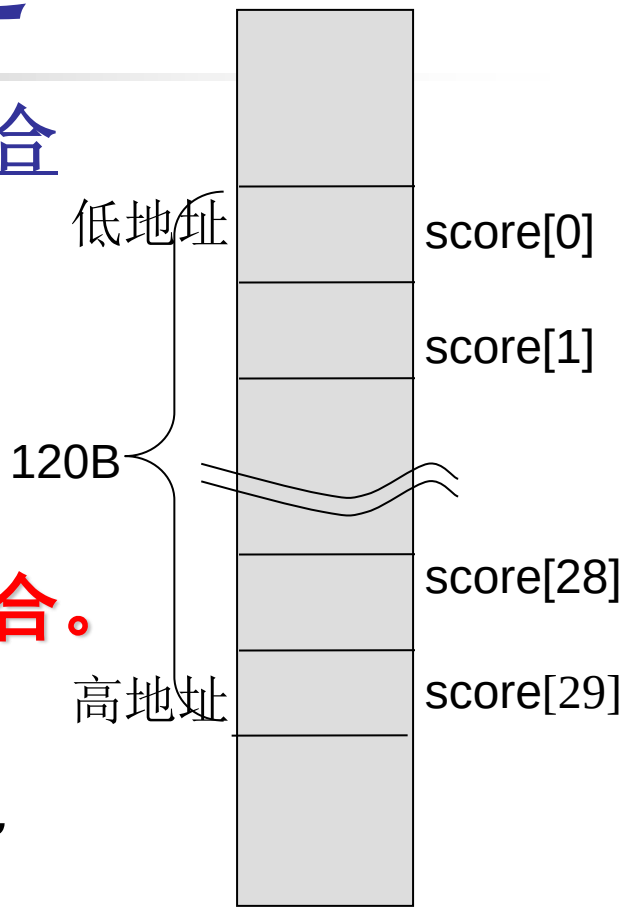
- 一维数组的声明、初始化和使用
- 数组作为函数参数的使用
- 字符串数组
- 多维数组

7.1 数组概述

用来描述一群有联系的同类型数据集合

```
#define SIZE 30
int  score[SIZE];
```

score是含有30个元素的int型数组



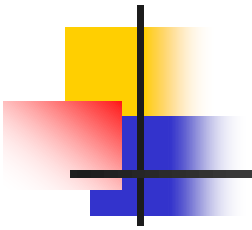
数组：是固定数量的同类型元素的集合。

- 数组元素的下标从0开始

  数组score的30个元素是：score[0] ~ score[29]

- 各个元素在内存中连续存放

  数组score所占内：sizeof（int）* SIZE



7.2 一维数组

- 只有一个下标
- 可用于表示一个线性的数据队列

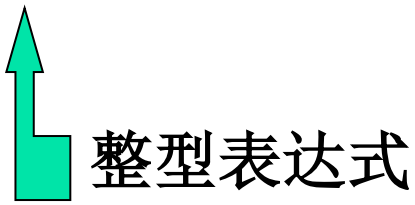
7.2.1 一维数组的声明

存储类型 类型说明符 数组名[常量]={初值表};

- **蓝色部分可选**
- 不能是 register
- int score[SIZE];
- static int y[10];
- extern double s[2];

7.2.2 一维数组元素的使用

数组名[下标表达式]



整型表达式

- 整型常量
- 整型变量
- 含有运算符的整型表达式
- 值为整型的函数调用

a[2]、a[i]、a[i+j]、a[max(a,b)]

代表了一个元素，等价一个同类型的变量。

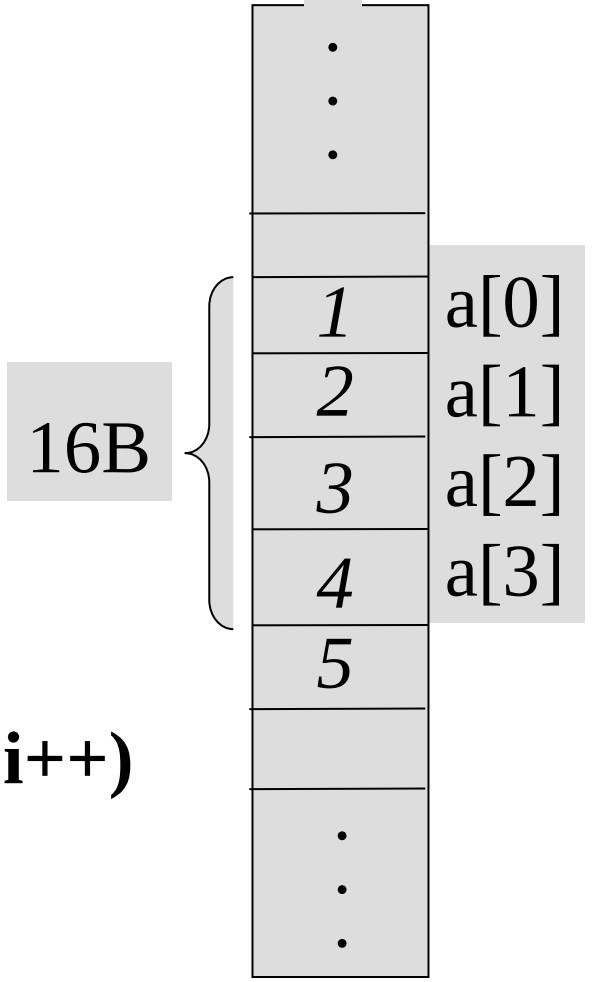
下标值不要超过数组的范围

```
int  a[4],i;
for(i=0;i<=4;i++)
    a[i]=i+1;
```

a[4]=5;

↑

下标越界



破坏数组以外的其它变量的值，可能造成严重后果

```
int  a[4],i;
for(i=0;i<4;i++)
    a[i]=i+1;
```

7.2.3 一维数组的初始化

存储类型 类型 数组名[常量表达式]={初值};

- 对全部元素赋初值（初值的个数与数组长度相同）

```
int   x[5]={1,2,3,4,5};
<=> int  x[ ]={1,2,3,4,5};  /*数组长度由初值个数确定*/
```

- 对部分元素赋初值(初值的个数小于数组长度)

```
int   z[6]={1,2,3,4};   /* 前4个下标变量赋值 */
  z[0]:1     z[1]:2    z[2]:3     z[3]:4
```

【例】使用一维数组计算学生的平均成绩

- 如何将计算n个学生的成绩定义成函数 ？

  函数名：CalAverage

  形式参数：

  若干成绩：数组 score

  人数： n

  返回值：平均成绩

- 函数原型

  double CalAverage(int score[ ] , int n);

- 算法步骤：

（1）成绩的累加求和存放于sum；

（2）计算并返回平均成绩average；

程序

```
double CalAverage(int x[ ] , int n)
{
      int sum,i;
      for(sum=0,i=0;i<n;i++)      /* 累加求和 */
          sum+=x[i];
     return (1.0*sum/n);       /* 计算并返回平均值 *
}
```

程序

```
#include <stdio.h>
#define SIZE  30     /* 学生
double CalAverage(int x[ ] , int n);
 int main(void)
{      int score[SIZE],i,sum,n;
       double average;
       printf(“输入学生人数(<=%d):\n”, SIZE);
        scanf(“%d”,&n);   /* 输入人数存于n */
        printf(“输入%d个学生的成绩:\n”,n);
       for(i=0;i<n;i++)      /* 输入成绩存于数组score中 */
            scanf(“%d”,&score[i]);
       printf(“average=%f\n”, CalAverage(score,n));
       return 0;
 }
```

double CalAverage(int [ ] , int);

&score[0]

8.2.5 一维数组作为函数参数

- 子函数
  - 给出形参数组的声明和形参数组元素的个数（char数组除外）
  - *形参是不指定大小的数组*

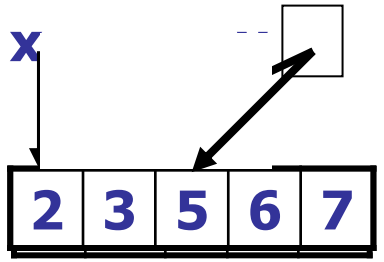
    double CalAverage (int score[ ], int n);
- 调用函数
  - 实参的值是**数组名**或数组元素的地址

```
int x[5]={ 3,2,4,1,5};
double ave;
ave=CalAverage(x,5);
ave=CalAverage(&x[1],3);
```

数组的首地址 ***x ⟺ &x[0]***

一维数组作为函数参数的例子

```
#include <stdio.h>
void fun(int y[ ], int n);
void main(void)
{  int k,x[5]={1,2,3,4,5};
   fun(x,5);
   for(i=0;i<=4;i++)  printf( “%d” ,x[i]);
   fun(&x[2],3);  printf( “\n” );
   for(i=0;i<=4;i++)  printf( “%d” ,x[i]);
}
void fun(int y[ ], int n)
{  int i;
   for(i=0; i<n; i++)     y[i]++;
 }
```



C的参数传递：传值和传址

用数组名作参数时，则为地址传送，即实参数组的首地址传递给形参数组首地址。所以，实参、形参数组共享相同的内存单元。相当于是值的双向传递

例 对n个整数采用冒泡法对其排序

排序问题——计算机处理数据的一个重要问题

- *将冒泡排序算法定义成函数*
  - *函数名：BubbleSort*
  - *形式参数：*

    带排序的若干整数：数组 a

    整数的个数： n
  - *返回值：无*
  - *函数原型*

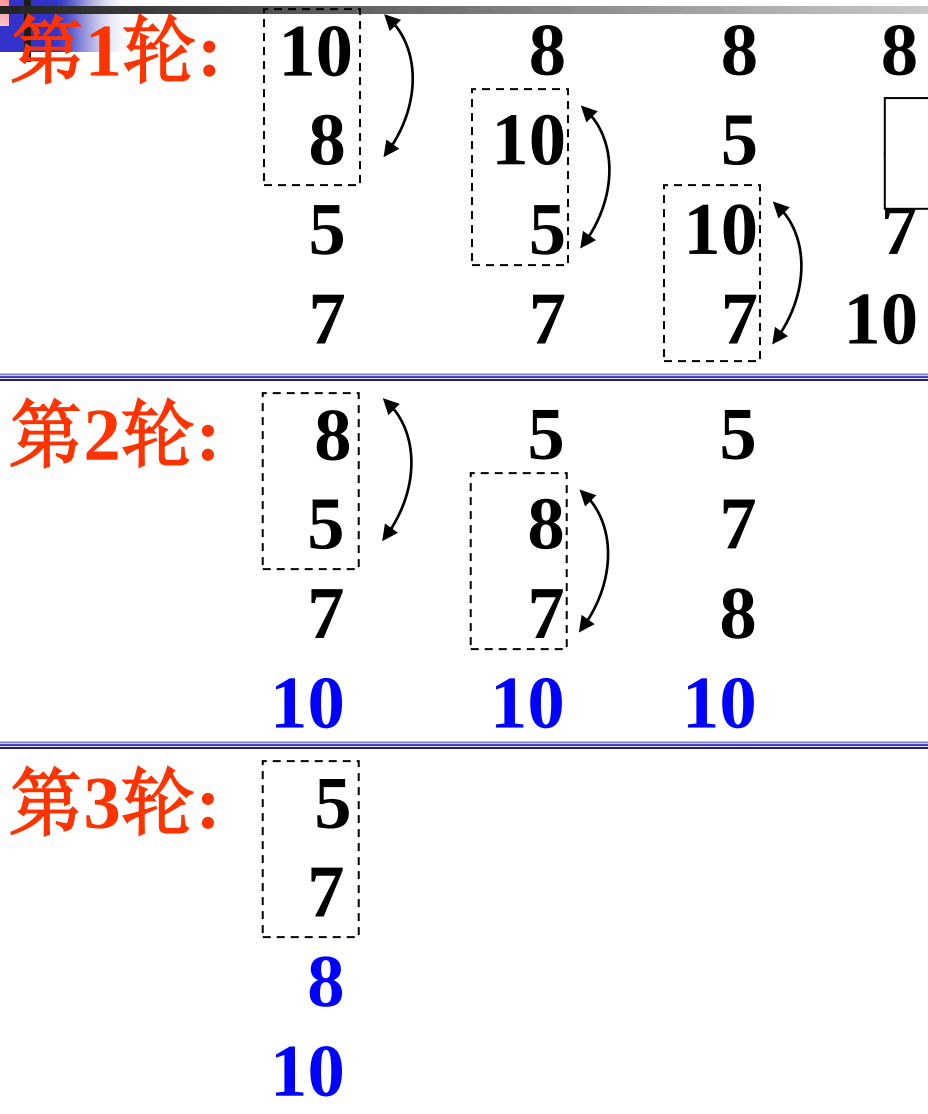
    void BubbleSort(int a[ ] , int n);
  - *算法?*

冒泡(bubble)排序法

基本思路：设有n个数，需将它们从小到大顺序排列。则：

- 1. 对n个元素，依次比较相邻的两个数，小的调到前头，比较(n-1)次后，最大的一个数“沉底”。
- 2. 对剩下的n –1个元素，比较(n-2)次后，得到次大的数。
- 3. 对上述过程重复，直至剩下一个元素。

例：int a[4] = {10　8　5　7}



n个数进行 ？轮比较

```
for(i=1;i< n;i++){
   相邻元素的一一比较, 交换
}
```

第i轮中要进行 ？次两两比较

```
for(i=1;i< n;i++) {
    for(j=0;j<n-i;j++)
      if(a[j]>a[j+1])
        a[j]与a[j+1]交换
}
```

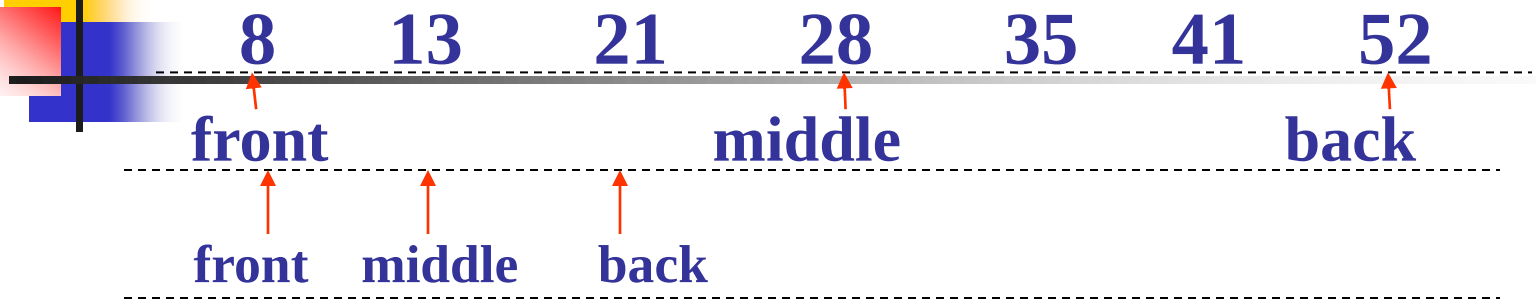
比较1次　交换0次

```
#include<stdio.h>
#define N 10
void BubbleSort ( int a[ ],int n)
{
      int i, j, t;
      for (i=1; i<n; i++)   /* 共进行n-1轮"冒泡" */
        for (j=0; j<n-i ; j++)  /* 对两两相邻的元素进行比较 */
          if (a[j]>a[j+1] ) {  t=a[j];  a[j]=a[j+1];  a[j+1]=t; }
}
int main ( )
{  int x[N], i;
   printf (" please input %d numbers: \n ", N);
   for (i=0; i<N; i++)    scanf(" %d ", &x[i]);
   BubbleSort (x , N ) ;
   printf (" the sorted numbers: \n ");
   for (i=0; i<N; i++)   printf("%d  ", x[i]);
   return 0;
}
```

7.6.2 基于分治策略的二分查找

- 二分查找（也称折半查找）算法的思路
  - 将已排好序的n个元素(存放于数组a)分成两半，取a[n/2]与x比较
    - 如果x=a[n/2]，则找到x，算法结束
    - 如果x<a[n/2]，则在数组a的前半部分继续查找x
    - 如果x>a[n/2]，则在数组a的后半部分继续查找x
- 返回值
  - 如果找到x，返回x在数组a中的位置（下标）
  - 如果没有找到，返回-1

在 n=7个数中 找 x=14



1）令 **front** *=0，back=n-1*

2）middle = (front + back) / 2

3）将x与a[middle]比较

- *x<a[middle],* x在a[0]与a[middle-1]范围内
  令back=middle-1
- *x>a[middle],* x在a[middle+1]与a[back]范围内
  令front=middle+1
- *x==a[middle],* x在数组a中的下标为middle

转2，直至front>back

二分查找函数

```
/* find x in a[0]~ a[n-1] ( sort ascending )
   return the subscript , or –1 if no match      */
int BinarySearch(int a[ ], int x, int n)
{  int  front=0,  back=n-1,  middle;
   while(front<=back) {
      middle=(front+back)/2;  /* 计算中间单元的下标 */
      if(x<a[middle])   back=middle-1;   /* 查找单元变成原来的前半部*/
      else if(x>a[middle])  front=middle+1;    /*在后半部查找*/
      else  return (middle);      /* 找到，返回下标 */
   }
   return -1;          /* 没有找到，返回-1 */
}
```

二分查找函数

```
#include<stdio.h>
int main(void)
{
    int x[ ]={1,3,5,7,9,11,13,15,17,19},  index;
    index=BinarySearch(x,11,10);
    if(index!=-1)
        printf("find %d!\n",x[index]);
    else
        printf("not find!\n");
    return 0;
}
```

运行结果是：
find 11!

7.3 字符数组

- 类型为char的一维数组
- 存放字符数据
- 字符串－－－用字符数组存放
- 在末尾加一个空字符 ‘\0’ 来构造字符串
- 字符串常量“……” 隐含有空字符 ‘\0’
- %s 也会自动加 ‘\0’
- 字符串的长度 = 字符串的存储长度−1

7.3.1 字符数组的声明和使用

```
char s[81];
```

s是字符型数组，可以放入81个字符

可以表示最多由80个字符组成的字符串

字符数组的最小长度＝该字符串的长度+1

例　编程产生26个大写英文字母的字符串和26个小写英文字母的字符串

```
#include <stdio.h>
void main(void)
{
    char Capital[27], Lowercase[27];
    int i ;
    for(i=0; i<26; i++) {
        Capital[i]='A'+i;
        Lowercase[i]='a'+i;
    }
    Capital[26]= Lowercase[26] ='\0'; /*在末尾加　'\0'　来构造字符串
     */
    puts(Capital);
    puts(Lowercase);
}
```

7.3.2 字符数组的初始化

- 通过初始化列表(直接给出字符串中的各字符)

  char s1[8]={'W','u','h','a','n','\0'};

  - '\0'必须在初始化列表中显示给出
  - 当数组长度未给出并且无'\0'时 ？

- 指定一个字符串常量 （常用）

  char s2[28]="Computer Science";

  - 末尾将自动加上一个'\0'

  当数组长度未给出时?

  char s3[]="Computer";

  - 字符数组的长度等于字符串的存储长度

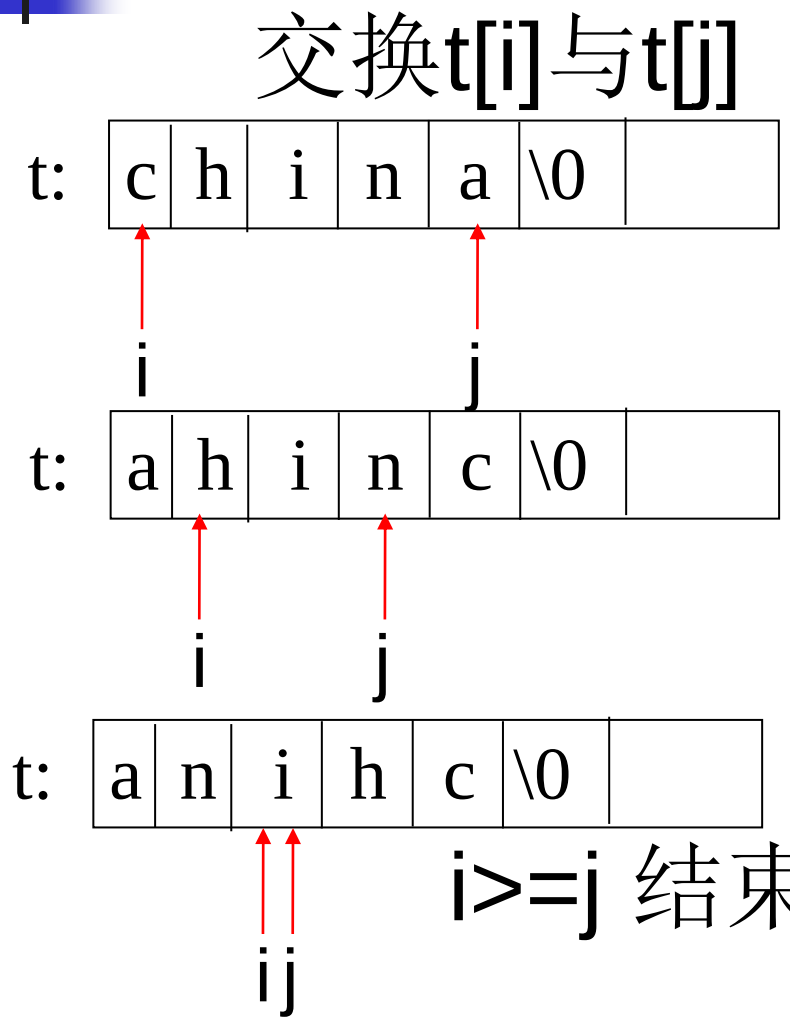
7.4 字符串处理函数

- 串操作函数（<string.h>）
  - 求字符串长度(strlen),(不计‘\0’)
  - 字符串的拷贝(strcpy)
  - 字符串的比较(strcmp)
  - 字符串的连接(strcat)
  - 求字符串的子串(strstr)
  - 删除字符串首尾空白字符
  - 从字符串中删除所有与给定字符相同的字符
  - 将字符串反转等函数(strrev)

例　求字符串长度的函数

```
/* gets length of s */
int mystrlen(char s[ ])
{     int j=0;
      while(s[j]!= ‘\0’ )       j++;
      return j;
}
int main(void)
{ char str[100];
   gets(str);   /*  scanf("%s",str);   */
   printf("length of the string is %d\n",mystrlen(str));
   return 0;
}
```

将字符串反转的函数




```
void mystrrev(char s[])
{
int  i, j;   /*前、后指示器*/
char c;
for(i=0,j=strlen(s)-1;i<j;i++,j--)
{      c=s[i];
       s[i]=s[j];
       s[j]=c;
  }
}
```

字符串拷贝的函数

```
/* copies string s to t */
void mystrcpy(char t[ ], char s[ ])
{
      int j=0;
      while( (t[j]=s[j]) !='\0' )   /* 可为 !=0 ， 或去掉 !='\0' */
          j++;
 }
int main(void)
{     char str1[30],  str2[]= "there is a boat on the lake.";
      mystrcpy(str1,str2);
      puts(str1);
      return 0;
}
```

运行结果：

there is a boat on the lake.

字符串拷贝的函数

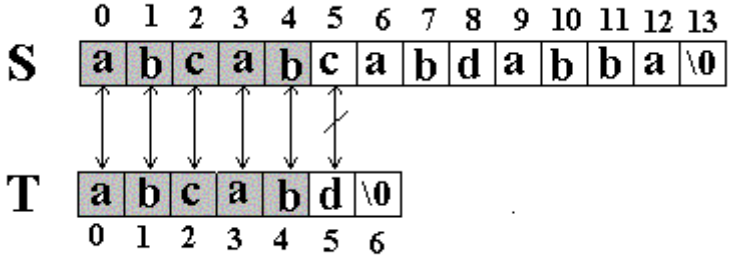
```
/* copies string s to t */
void mystrcpy(char t[ ], char s[ ])
{
    int j=0;
    while( (s[j] !='\0' )
        t[j]=s[j++];
}
int main(void)
{   char str1[30]="there",  str2[]= "is";
    mystrcpy(str1,str2);
    puts(str1);
    return 0;
}
```

对吗？

两个字符串比较函数

比较规则

从两个字符串的第一个字符起开始，按照字符ASCII码值的大小进行比较。




```
/* compares t to s
   return a value: =0  if  s=t
                   >0 if s>t
                   <0 if s<t   */
int mystrcmp(char s[],char t[])
{
  int j=0;
  while(s[j]==t[j] && s[j]!= '\0' )
       j++;
  return s[j]-t[j];
}
```

例　验证密码

```
int main(void)
{
      char pw[]="1234",s[20];
      int  count=3;
      do {
          printf("Input  password\n"）；
           scanf(“%s”,s);
           count--;
       } while(mystrcmp(pw,s)&&count));
        if(mystrcmp(pw,s)
             return 1;
         ....    // 进入系统
        return 0;
}
```

字符串的连接函数

```
/* appends s to t  */
char * mystrcat(char t[],char s[])
{    int j=0,k=0;
     while(t[j++]!='\0')  ;
     j--;
     while((t[j++]=s[k++]))  ;
     return t;
}
void main(void)
{    char s1[80]="I like ", s2[]="the C programming. “ ;
     mystrcat(s1,s2);
     printf("%s\n",s1);
}
```

运行结果：

I like the C programming.

```
/* appends s to t  */
char * mystrcat(char t[],char s[])
{    int j=0,k=0;
     while(t[j++]!='\0') ;
     /*j--;*/                怎样?
     while((t[j++]=s[k++]))  ;
     return t;
}
void main(void)
{    char s1[80]="I like ", s2[]="the C programming. “ ;
     mystrcat(s1,s2);
     printf("%s\n",s1);
}
```

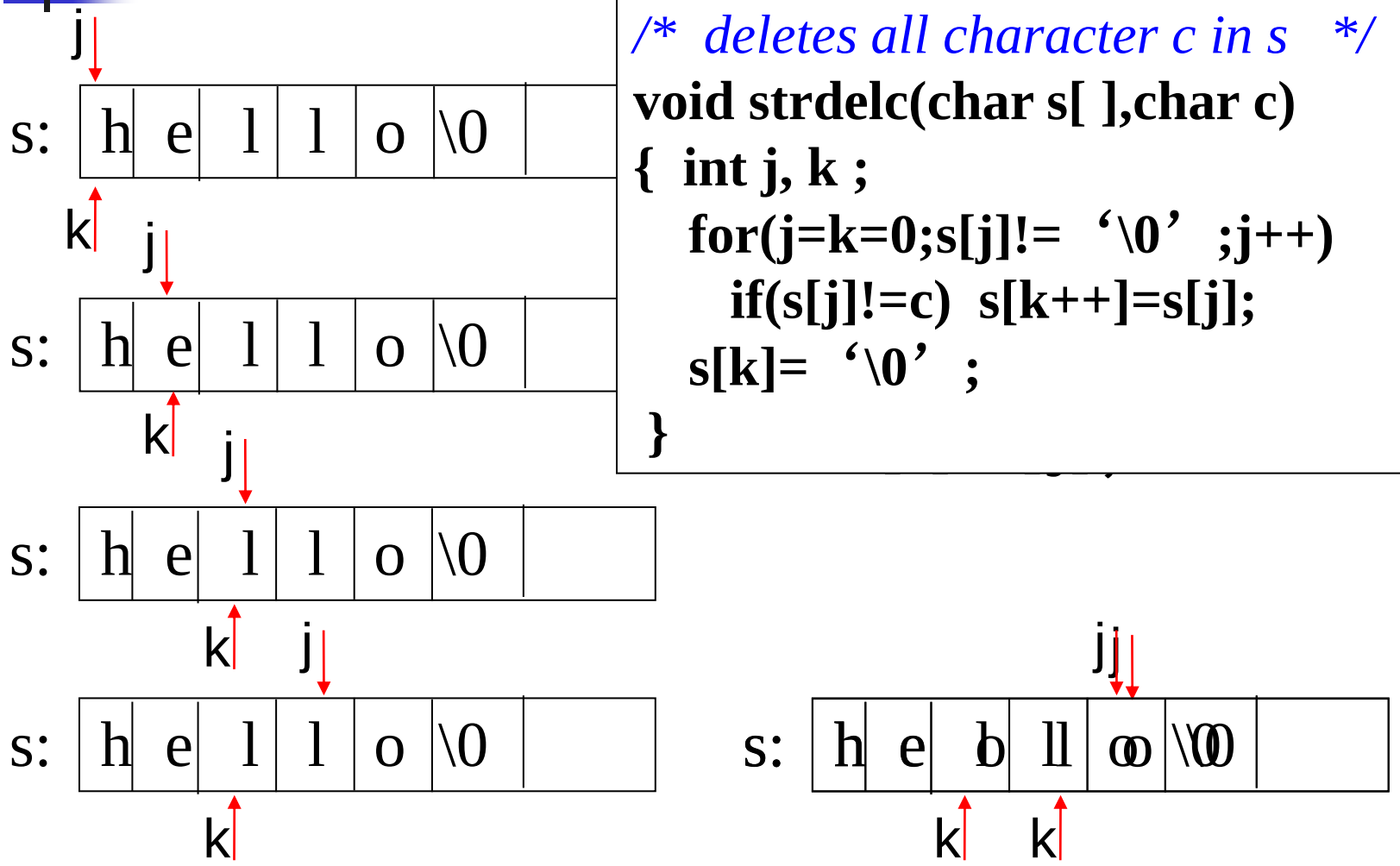
运行结果：

I like

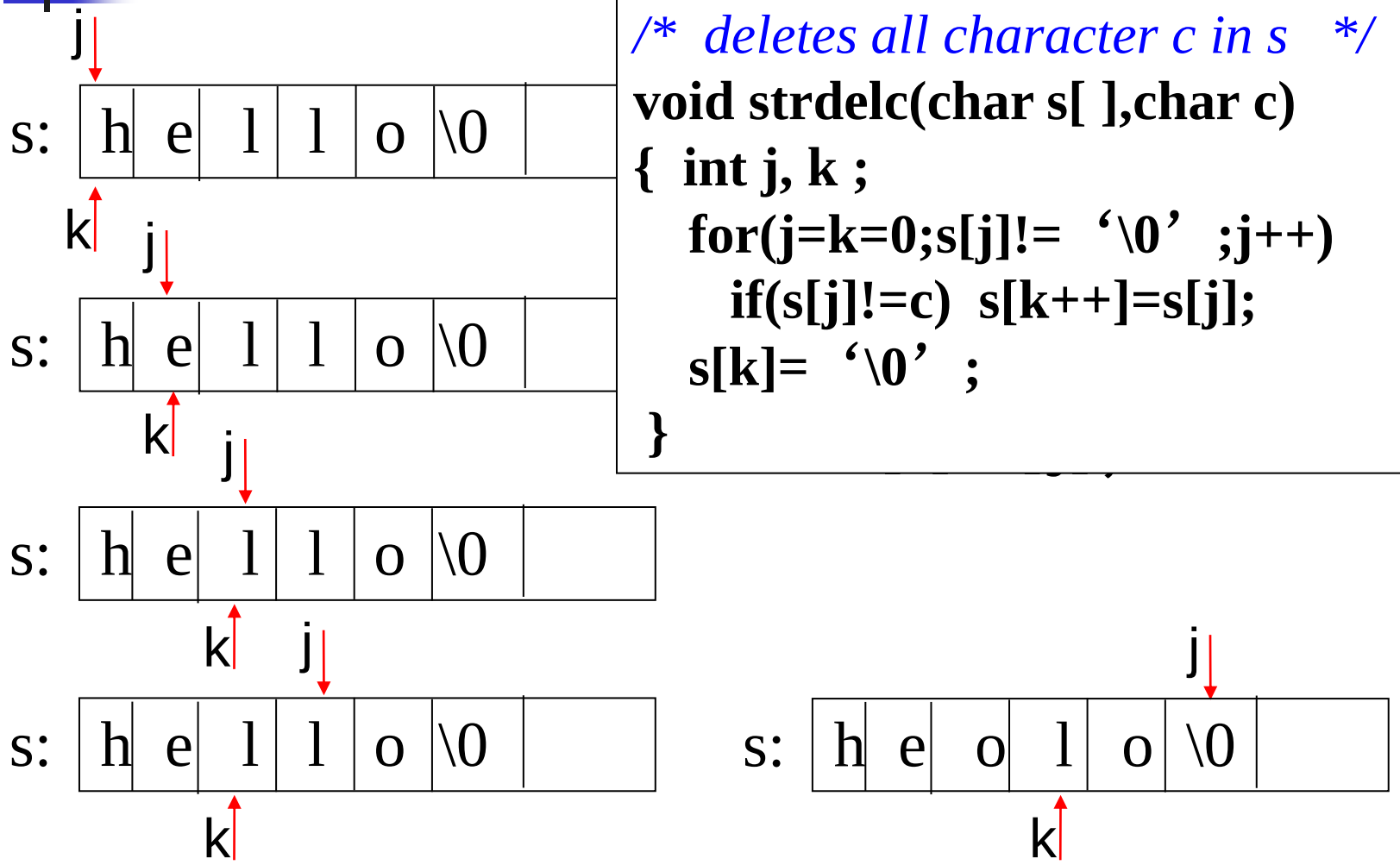
```
/* appends s to t  */
char * mystrcat(char t[],char s[])
{     int j=0,k=0;
      while(t[j++]!='\0')  ;
      j--;
      while((t[j++]=s[k++]))   ;
      return t;
}
void main(void)
{     char s1[ ]="I like ", s2[]="the C programming. “ ;
      mystrcat(s1,s2);
      printf("%s\n",s1);
}
```

缺省长度？

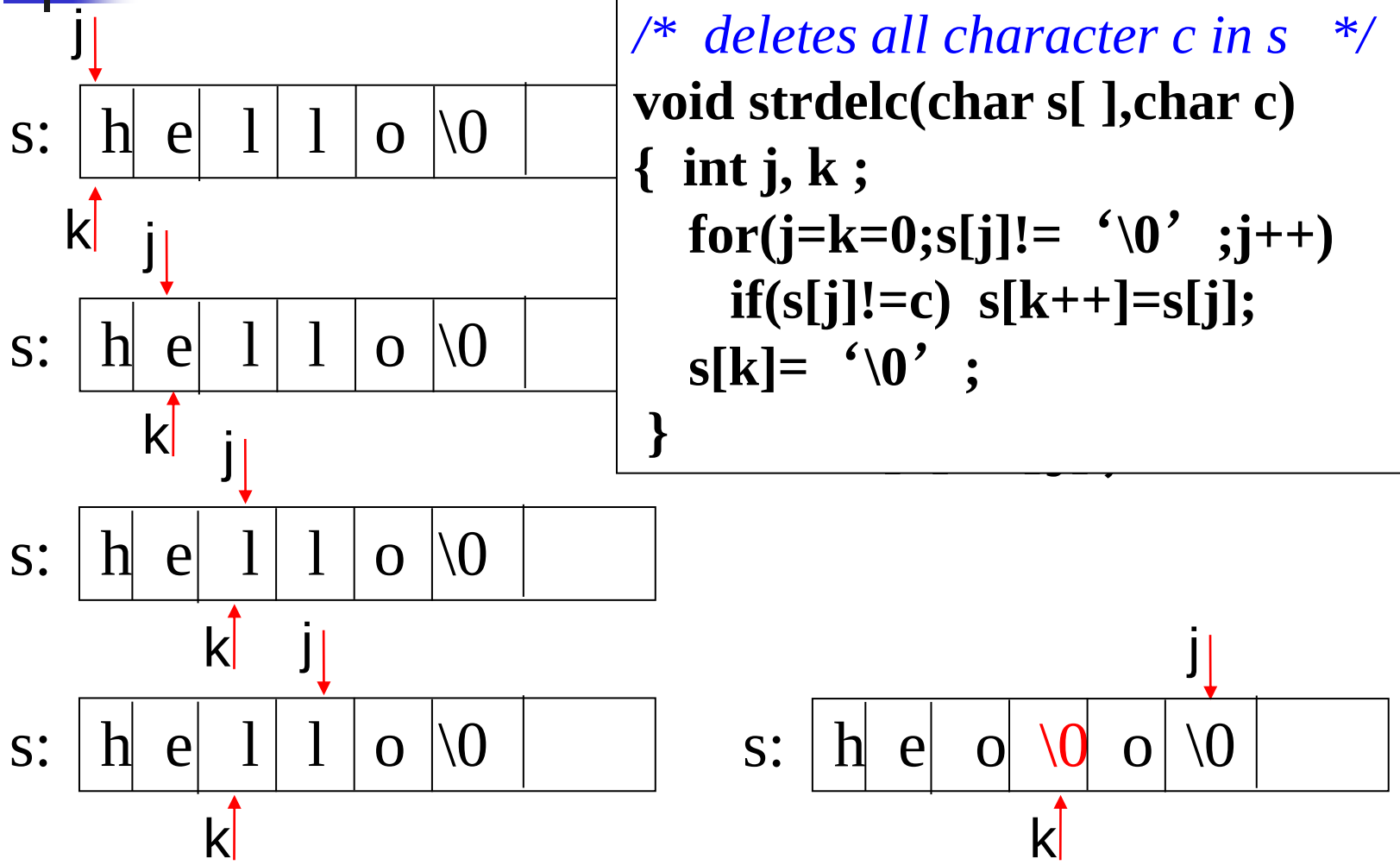
从串s中删除所有与给定字符c相同的字符




```
/*  deletes all character c in s   */
void strdelc(char s[ ],char c)
{  int j, k ;
   for(j=k=0;s[j]!= '\0' ;j++)
     if(s[j]!=c)  s[k++]=s[j];
   s[k]= '\0' ;
 }
```




```
/*  deletes all character c in s   */
void strdelc(char s[ ],char c)
{  int j, k ;
   for(j=k=0;s[j]!= '\0' ;j++)
     if(s[j]!=c)  s[k++]=s[j];
   s[k]= '\0' ;
 }
```




```
/*  deletes all character c in s   */
void strdelc(char s[ ],char c)
{  int j, k ;
   for(j=k=0;s[j]!= '\0' ;j++)
     if(s[j]!=c)  s[k++]=s[j];
   s[k]= '\0' ;
 }
```

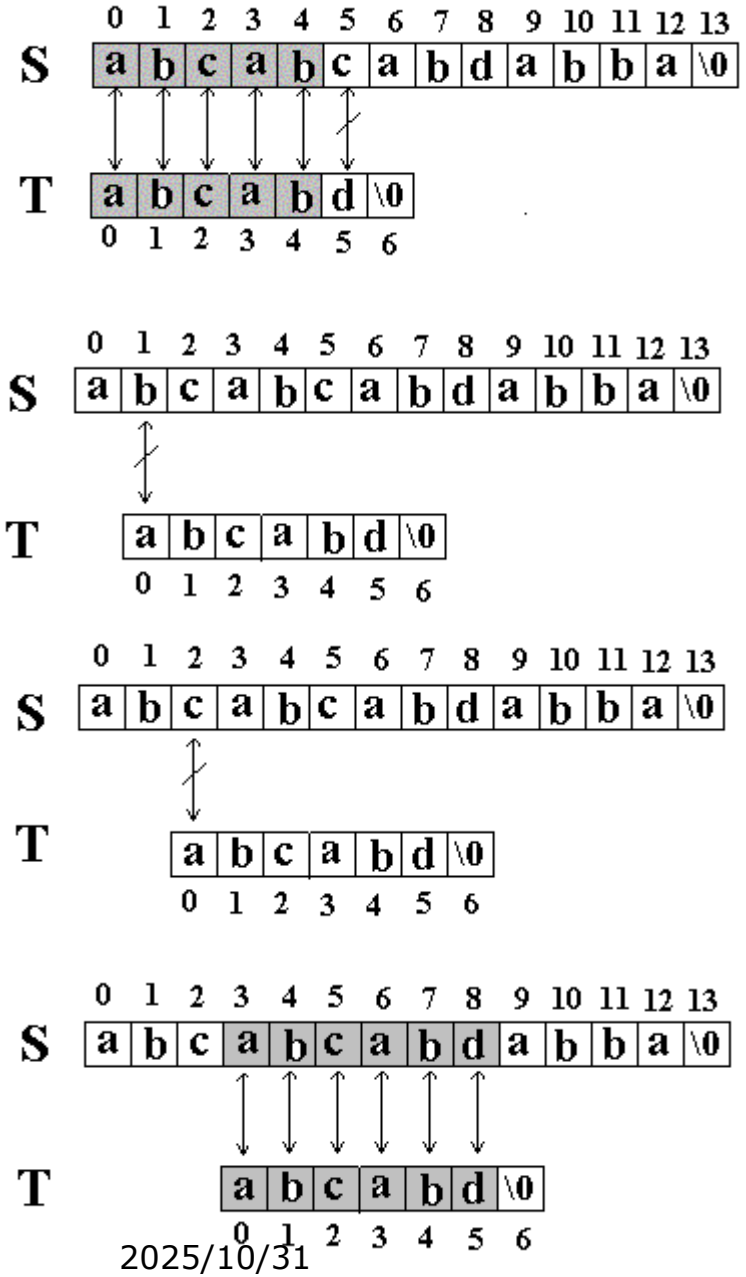
字符串匹配

字符串匹配——文本处理的一个重要领域

指从文本中找出给定字符串(称为模式)的一个或所有出现的位置----**求字符串s(文本)子串t(模式)**

- ***mystrstr函数功能： 如果字符串t是字符串s的子串，则返回t在s中第一次出现时的位置（即下标）。否则，返回-1。***

BF(Brute Force) Algorithm
暴力求解法、穷举法




```
for (i = 0; s[i] != '\0'; i++) {
  // 开始第 i 轮匹配
  for (j=i, k=0; t[k]!='\0' ; j++, k++)
    if(s[j]!=t[k])
      break;    /*结束本轮匹配*/
  // 判断本轮匹配是否成功
  if (t[k] = = '\0')
    return i;  /*匹配成功  返回下标*/
}
return -1;  // 不存在子串，返回 -1
```

KMP(Knuth-Morris-Pratt)

```
/*strstr:  return index of t in s, -1 if none */
int mystrstr(char s[ ], char t[ ])
{
    int i, j, k;
    for (i = 0; s[i] != '\0'; i++) {
        for (j=i, k=0; t[k]!='\0' ; j++, k++)
            if(s[j]!=t[k])
                break;     /*结束本轮匹配*/
        if (t[k] = = '\0')
            return i;     /*匹配成功   返回下标*/
    }
    return -1;
}
```

```
for (j=i, k=0; t[k]!='\0' && s[j]= =t[k]; j++, k++)
        ;
```

```
int main(void)
{
   char s1[ ]="I like the C programming.", s2[]="ram";
   int i;
   i = mystrstr(s1,s2);
   if ( i>=0)
      printf("the sub-string's position is %d\n",i);
   else
       printf("\"%s\" does not occur in \"%s\"\n", s2,s1);
   return 0;
}
```

7.4.2 数字串与数之间转换的函数

- 串与数值之间的转换函数（<stdlib.h>）
  - 十进制串转换为整数(atoi)
  - 十进制串转换为长整数(atol)
  - 十进制串转换为浮点型数(atof)
  - 整数转换为十进制串(itoa) -- 讲
  - 十六进制串转换为整数(htoi)（无）-- 讲

十进制串转换为整数(atoi)

将一个十进制数字串转换成为对应的整数的函数

- **atoi函数功能**
  - *将s字符数组中存放的一个十进制数字串转换成为对应的整数，并返回该整数*
- **算法：**
  - *ASCII码字符s[j]转换为对应数字*
    - *s[j]-'0'*
  - *本位乘以10加下一位的算法*
    - *54321 =（（（（5）*10+4）*10+3）*10+2*10）+1*

atoi函数

```
#define BASE 10
/* converts a string s to an int  */
int atoi(char s[])
{
    int j=0,num=0;
    for(;s[j]!='\0';j++)
        num=num*BASE +s[j]-'0';
    return num;
}
```

将一个整数转换成为十进制数字串的函数

函数名：myitoa

形式参数：

待转换的整数：n

转换之后的数字串怎么返回？函数不能返回（return）数组类型。

转换结果串：字符数组s

返回值：无

- 函数原型

  void myitoa(int n,char s[]);

- 算法：

  分解出n的每一位数字存于数组s中。

```
#define BASE 10
/* converts a integer n to a string s  */
void myitoa(int n,char s[])
{    int sign,j=0;
     if((sign=n)<0)  n=-n;
     while(n>0){
        s[j++]=n%BASE+'0';
        n/=BASE;
      }
     if(sign<0)
        s[j++]='-';
     s[j]='\0';
     strrev(s);
}
```

7.5 二维数组

- 有两个下标
- 可用于表示数据阵列(二维数据表格）

	语文	数学	英语	物理
01	85	91	80	78
02	82	95	88	92
…	…	…	…	…

如何描述上面的**课程成绩表**中的成绩数据？

用二维数组

```
int score[30][4];   /* score是30行4列的int型数组 */
```

二维数组还可以描述数学中的矩阵或行列式

7.5.1 二维数组的声明与使用

- **说明**

  存储类 类型名 数组名[常量表达式][常量表达式]

- **使用**

  数组名[行下标][列下标]

例：设全班同学修了高等数学、普通物理、c语言和英语4门课程并取得了成绩，要求计算每个同学的平均成绩，并输出成绩表。

序号	数学	物理	c	英语	平均成绩
01	91	78	91	80	78
02	95	92	95	88	92
	…	…	…	…	…

用二维数组表示上表。

```
int  s[N][M+1]；  /*  N-人数，M-课程数*/
```

```
#include<stdio.h>
#define N  10      /*  人数  */
#define M  4       /*  课程数  */
double  CalAverage（int [ ], int ）;     /*  该函数定义见7.1.3  */
int main(void)
{  int x[N][M+1];
   int i,j;
   for(i=0;i<N;i++){
        for(j=0;j<M;j++)
                scanf("%d",&x[i][j]);          /*输入成绩*/
        x[i][M]=CalAverage（x[i], M）;     /*计算学生的平均成绩*/
   }
   printf("\n");
   for(i=0;i<N;i++){ /*输出成绩表*/
        for(j=0;j<=M;j++)
                printf("%d\t",x[i][j]);
        printf("\n");
   }
   return 0;
}
// 扩展程序功能：加一行，储存每门课的平均成绩及总平均成绩
```

x[i]<=>&x[i][0]

7.5.2 二维数组的存储结构

二维数组的存放方式为：

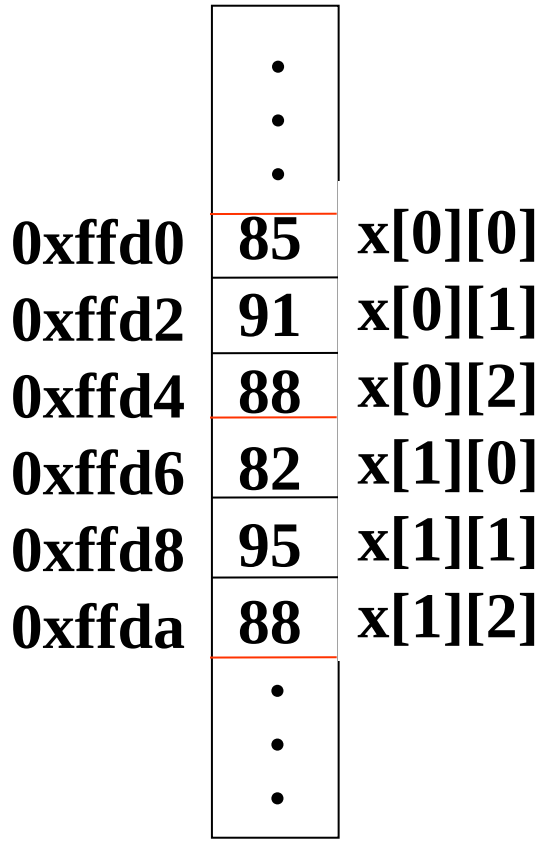
按行存放

```
short x[2][3];
```

二维数组x的逻辑结构

x[0][0] =85	x[0][1] =91	x[0][2] =88
x[1][0] =82	x[1][1] =95	x[1][2] =88

二维数组x的物理存储结构



7.5.3 二维数组的初始化

- **按照物理存储结构赋初值** （当一维数组处理）

  int a[2][2]={85,91,82,95};

  第1维大小的说明有时可以省略

  int b[ ][3]={1,3,5,6,7};

- **按照逻辑结构**（按行）**赋初值**

  可读性好

  int x[2][3]={{ 85,91,0}，{82,95，0}};

  int b[ ][3]={ {1,3}, {5,6,7} };

7.5.4 二维字符数组

使用二维字符数组可以产生字符串数组

```
char text[25][80];
```

数组text可以存放25个字符串，每个字符串的最大长度为79.

- 初始化
  - 与其它二维数组类似

```
char s[2][4]={ 'a','b','c','\0','d','e','f','\0'};
char s[2][4]={{'a','b','c','\0'},{'d','e','f','\0'}};
```

  - 用字符串对二维数组进行初始化（常用）

```
char devices[3][12]={"hard disk","CRT","keyboard"};
```

<=> `char devices[ ][12]={"hard disk","CRT","keyboard"};`

二维字符数组的使用

char devices[ ][12]={"hard disk", "CRT", "keyboard"};

h	a	r	d		d	i	s	k	\0		
C	R	T	\0								
k	e	y	b	o	a	r	d	\0			

- 可以引用单个字符元素

  devices[2][3]　　值为　　'b'

- 可以引用字符串

  devices[i]表示devices数组中第i行字符串的首地址

  printf("%s", devices[1]);　　/* 输出　CRT　*/

字符串数组的输入输出操作

```
#include <stdio.h>
int main(void)
{
    int i;
    char devices[ ][12]={"hard disk", "CRT", "keyboard"};
    devices[0][0]= 'H';    /* "hard disk"变为"Hard disk"*/
    devices[2][0]= 'K';   /* "keyboard "变为"Keyboard "*/
    for(i=0;i<3;i++)
        printf("%s\n", &devices[i][0]); /*  同devices[i] */
    scanf("%s",devices[1]);   /*  同 &devices[i][0] */
    for(i=0;i<3;i++)
        printf("%s\n", devices[i]);
    return 0;
}
```

7.7 三维数组

三维数组的用途

三维数组可以描述空间中的点集

n维数组来描述n维线性空间中的n维向量

```
int a[ ][3][3]={ { {1,2,0},{3,4,0},{5,0,0} },
                 { {0,6,7},{8,0,0},{9,0,0} }
               };
```

第1页

0	6	7
8	0	0
9	0	0

第0页

1	2	0
3	4	0
5	0	0

C语言与程序设计

The C Programming Language

第8章 指针

毛伏兵

华中科技大学计算机学院

8.1 指针的概念与指针的使用

8.1.1 指针的概念

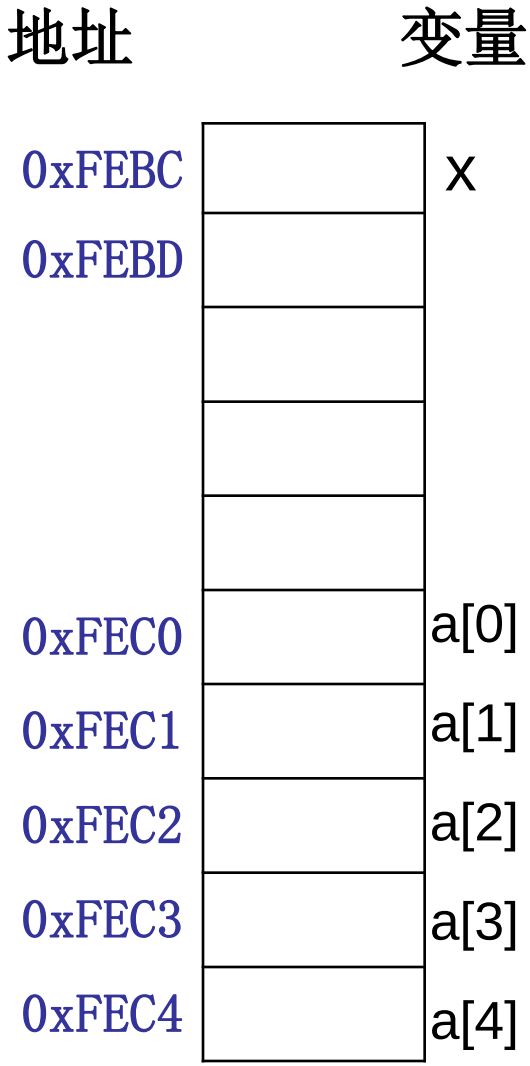
- 变量占有一定数目(根据类型)的连续存储单元;

```
short x, a[5];
```

- 变量的连续存储单元首地址称为变量的地址。

```
&x,    a <= => &a[0]
```

- 思考: 用什么类型的变量来保存地址数据?



8.1.2 指针变量的声明

数据类型 *标识符;

↓

所指(即所指向)变量的数据类型

```
short x=1, y=2, a[10],*p;
p=&x;
p=a;
p=a[0];  // X
```

思考: 为什么使用指针?

思考: 为什么使用指针?

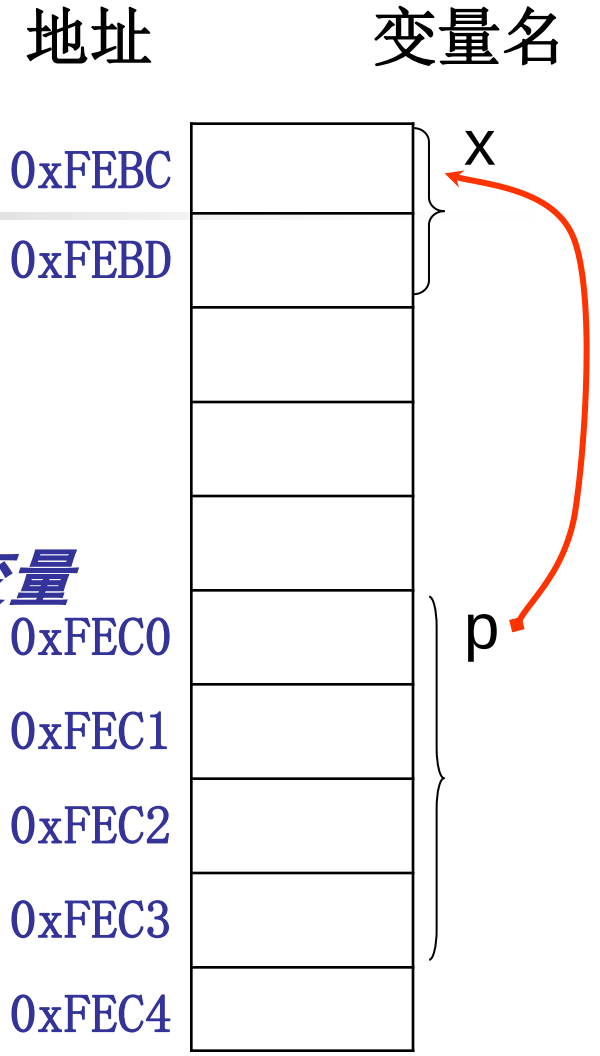
- **直接：** *通过变量名存取（或访问）变量*

  x=0x1234 ； printf(“%x”, x);

- **间接：** *通过变量的地址（即指针）存取变量*

  p=&x ；

  *printf(“%x”, *p);*

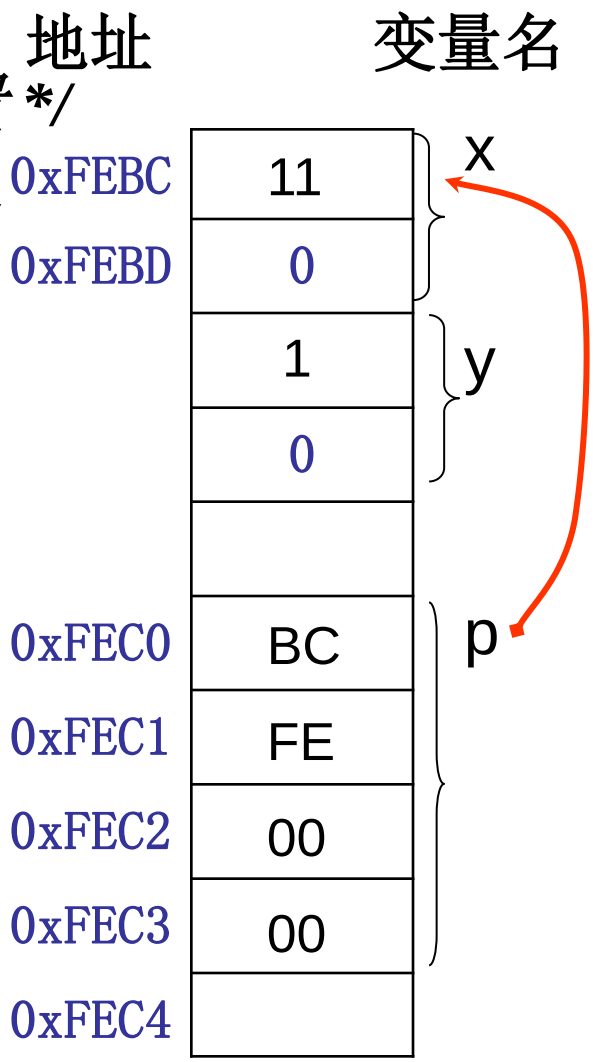


先访问 p，得到x的地址，
再通过该地址找到它所指向的单元中的值。

指出下面程序段的输出结果

```
short x=1, y=2, *p; /* p是short型指针变量 */
p=&x; /*将x的地址给p，即指针p指向x */
y=*p;     /*  *p<==>x */
*p += 10;     /*  x+=10 */
printf("x=%hd,y=%hd",x,y) ;
```

输出？ x=11,y=1

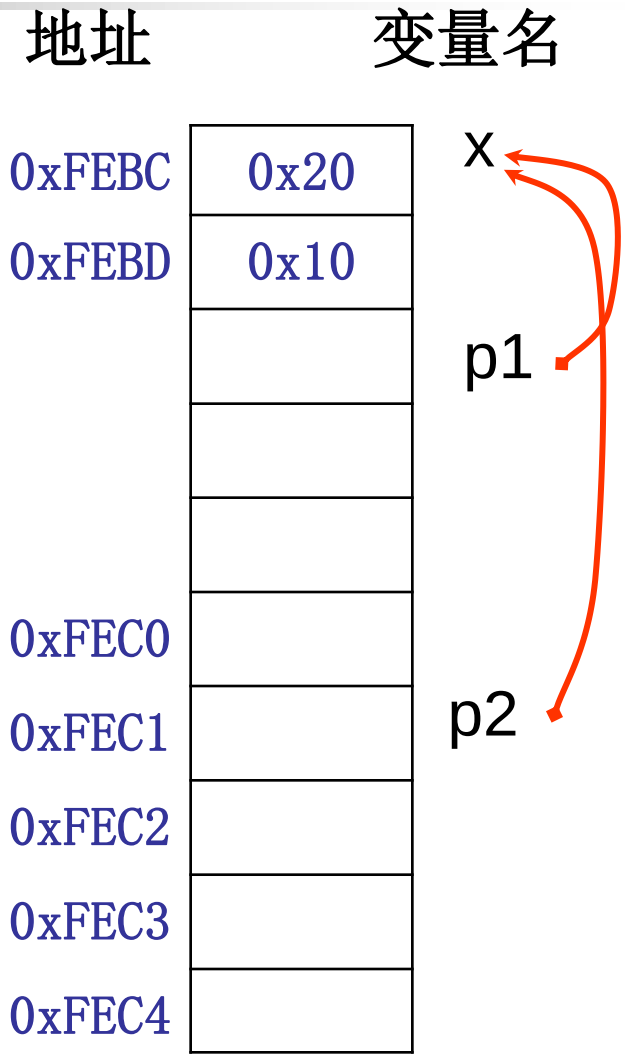


思考: 为什么指针有类型?

```
short x=0x1020, *p1=&x;
char *p2=&x;
int a,b;
a=*p1;
b=*p2;
printf("%d,%d",a,b) ;
b=*(p2+1);
printf("%d",b) ;
```

4128,32

16



8.1.3 指针的使用

1）建立指针变量与被指变量间的指向关系。

取地址运算符 &

2）通过指针变量来间接访问和操作指针所指的变量。

间访运算符 *

1、取地址运算符-单目&

&操作数

*返回其后操作数的内存地址，*操作数必须是一个左值表达式。例如

```
int b;    char s[20];
```

以下表达式哪些正确？ 指出正确表达式的类型。

&b 、 &s[2]、 &s

*int * , char *,* ×

如果右操作数的类型是T，

则表达式&操作数的类型是 T *

2、指针的赋值

```
int x,*p;
float a[5],*pf;
p=&x;      /* 取整型变量x的地址赋给整型指针变量p,
              指针p指向变量x      */
pf=a;      /* 等价于pf=&a[0];
              指针pf指向a数组的第一个元素a[0]。
              数组名 a 的数据类型是float *    */
```

悬挂指针

指出下面程序段的错误:

```
int x,*p; /* 说明p是一个整型指针变量，其值不确定. */
x=25;
*p=x;      /* 使用悬挂指针，错！  */
scanf("%d",p);   /* 使用悬挂指针，错！  */
```

- 指针的声明只是创建了指针变量，获得了指针变量的存储，但并没有给出指针变量指向那个具体的变量，此时指针的值是不确定的随机值，指针处于“无所指”的状态。称为悬挂指针（野指针）。
- 不能使用悬挂指针

3、间访运算符---单目*

操作数　/ 返回 操作数所指地址处的变量值。*/

设有声明：

char ch=’a’,*pc=&ch;

*pc=’b’;

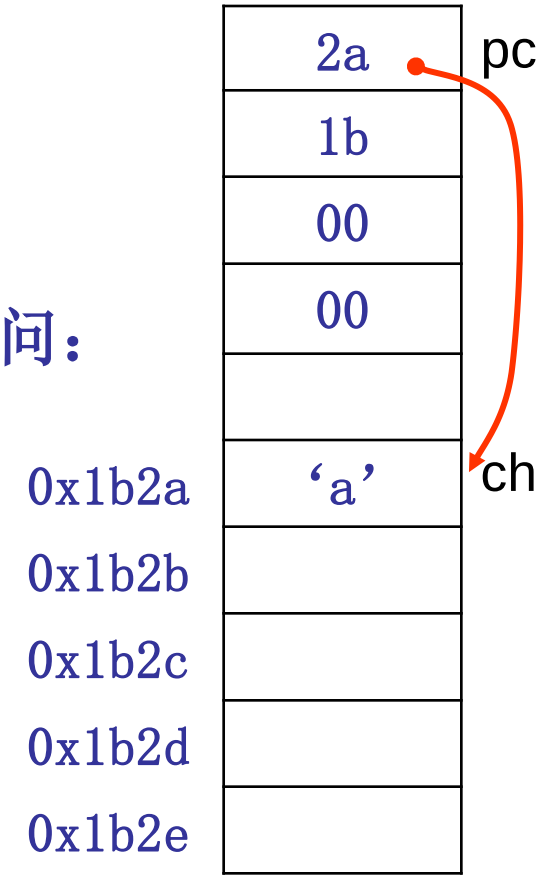
并且假设字符变量ch的地址是0x1b2a，试问：

ch，pc，以及*pc的值是什么？

ch：　‘b’　　（直接访问）

pc：　0x1b2a

*pc：　‘b’　　（间接访问）



声明指针的目的是希望通过指针实现对内存中变量的快速访问。

8.2 指针运算

指针允许进行算术运算、赋值运算和关系运算。通过指针运算可以快速定位到指定的内存单元，提高程序的执行效率。

8.2.1 指针的算术运算

(+, −, ++, −−, +=, −=)

1、指针的移动

指针的移动指指针在原有位置的基础上，通过加一个整数实现指针的前移（地址增大的方向）或者通过减一个整数实现指针的后移。

p ± k

指针　　整型

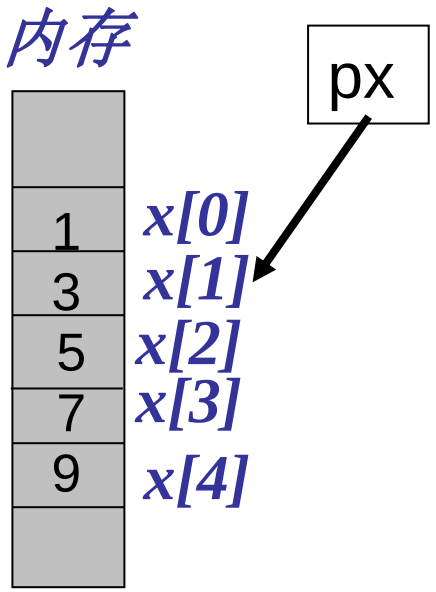
p 前移/后移 k个元素

已知声明如下，指出下面表达式的值。

(各题的表达式相互无关)

int x[5]={1，3，5，7，9}，*px=&x[1];

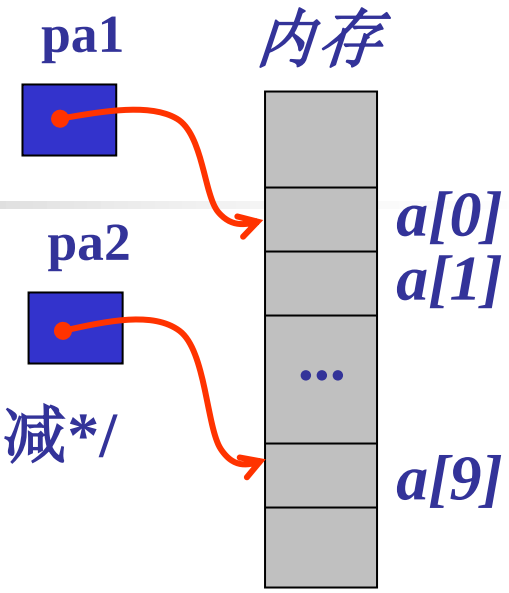
（1）	++*px	4
（2）	*++px	5
（3）	*(--px)	1
（4）	*(px--)	3
(5)	*px++	3
(6)	px+=2，*px	7



所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	( ) [ ] ->> .	圆括号 下标 间接引用结构体成员	左结合
单目运算符	! ~ ++、--	逻辑非 按位取反 自增、自减	
2	+、-	取正、取负	右结合
	(数据类型) &、* sizeof	强制类型转换 取地址、间接引用 数据长度	
3	*、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	==、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10	|	按位或	左结合
11	&&	逻辑与	左结合
12	||	逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、|=	赋值	右结合
15	,	逗号	左结合

2、两个指针相减

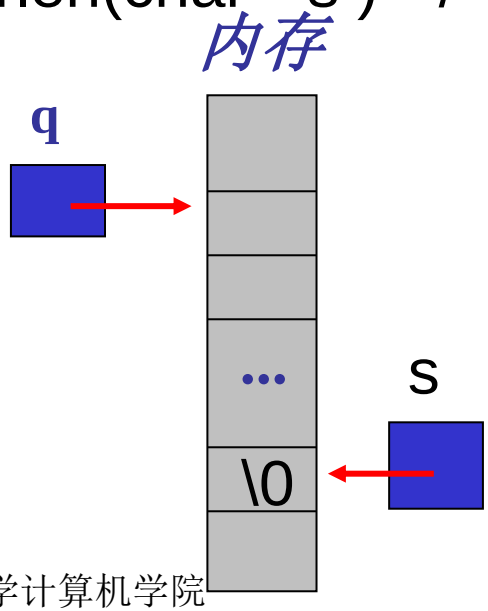


两个指针可以相减

pa2 – pa1 /*等于所指元素的下标相减*/

指向同一数组元素的指针

```
int  strlen(char  s[ ])  /*  等价于 int  strlen(char  *s )  */
{   char  *q=s;
   while(*s)  s++;
   return (s-q);
}
```



(1) 同类型的指针可以直接赋值。如：

```
int a[3]={1,2,3},x,*p,*q;
p=a;   q=&x;
```

(2) 不同类型的指针必须使用类型强制转换再赋值。

```
long x;
char *p;
p = (char *)&x;
```

9.2.3 指针的关系运算

<，<=，>，>=，==，!=

只限于同类型指针，

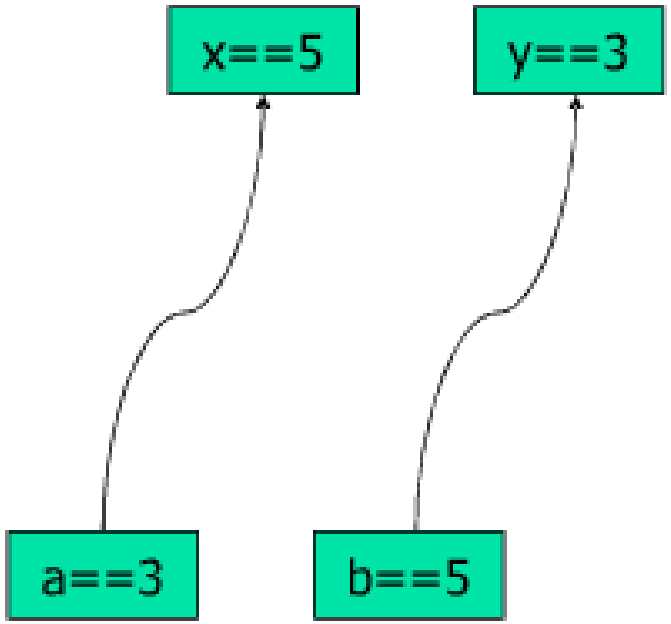
不同类型指针之间的关系运算被视为非法操作。

8.3 指针作为函数的

8.3.1 形参指针对实参变量的

例 形参的修改无法影响实参变量的

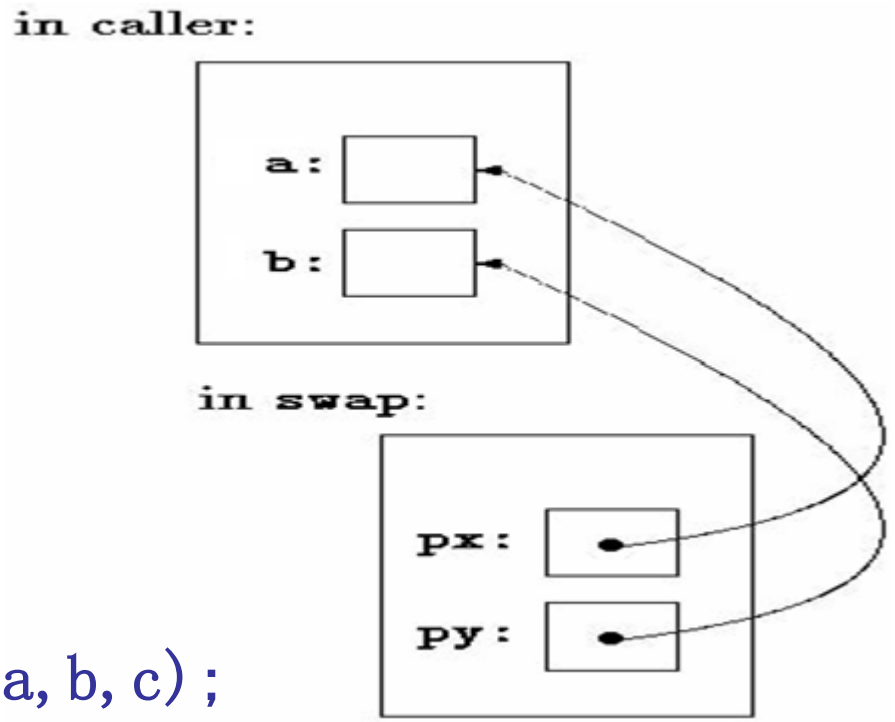
```
#include <stdio.h>
void swap(int x,int y)
{ int t;
  t=x;x=y;y=t;
}
```

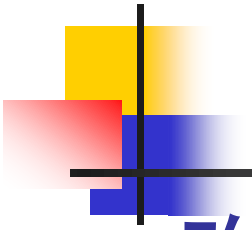



```
int main(void)
{ int a=3,b=5;
  swap(a,b);
  printf("a=%d,b=%d\n",a,b);
  return 0;
}
```

例　以指针作为函数的参数实现变量值的交换。

```
#include <stdio.h>
void swap(int *px,int *py)
{  int t;
   t=*px;*px=*py;*py=t;
}
void main(void)
{ int a=10,b=20,c=30;
  swap(&a,&b);
  swap(&b,&c);
  printf("a=%d,b=%d,c=%d\n",a,b,c);
}
```





- 改变主调函数中变量的值
- 使函数送回多个值

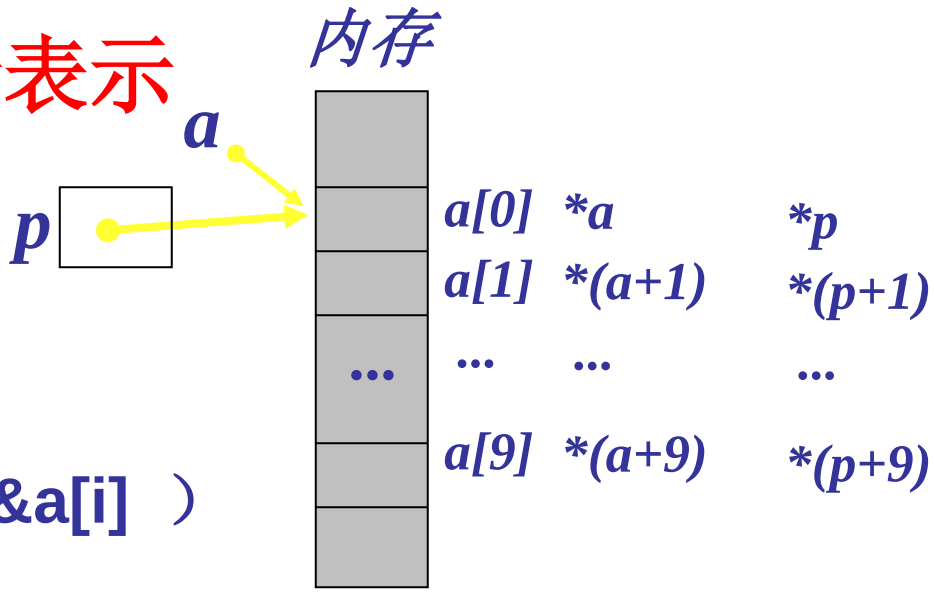
```
// implicit returned values:
void sum(int x,int y,int *result)
{
    *result=x+y;
}

// the caller
int s;
sum(3,4,&s);
```

8.4 数组的指针表示

数组元素既可以用下标表示，也可以用指针表示。

8.4.1 一维数组的指针表示

int a[10], *p=a;



数组元素的表示

- 下标法　a[i]　（地址为 &a[i] ）
- 指针法
  - 数组名　*(a+i)　（地址为 a+i ）
  - 指针变量　*(p+i)　（地址为 p+i ）

已知：　#define N

　　int a[N],*p, i;

　　p=a;

- *输入数组的全部元素*

(1)　for(i=0;i< N;i++)

　　scanf(“%d”, _____) ;

(2)　for(;p<a+ N;p++)

　　scanf(“%d”, ____p____) ;

&a[i]
a+i
p+i
p++
a++ ×

已知： int a[10],*p, i;

p=a;

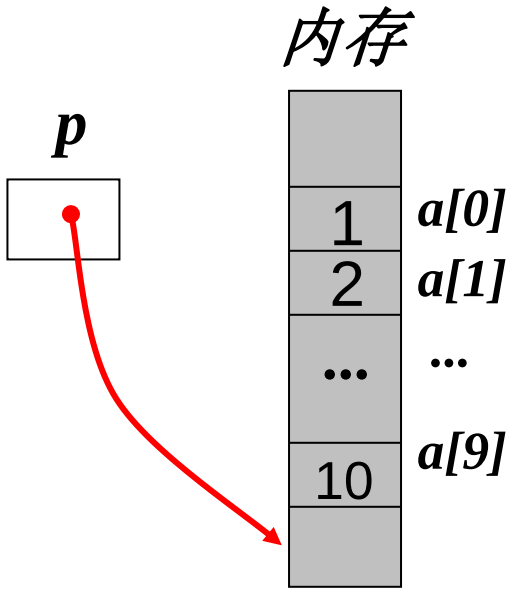
- *输出数组的全部元素*

```
for(i=0;i<10;i++)
  printf("%d", _____ ) ;

for( ; p<a+10;p++)
  printf("%d", ____*p____ ) ;
```

a[i]
*(a+i)
*(p+i)
*p++

输出数组a的全部元素（ 正确否？）

```
int a[10],*p , i;

for( p=a,i=1; i<=10; i++){
  *p=i;
   p++;
}

while(p<a+10)

   printf("%d", *p++ ) ;
```



8.4.2 一维数组参数的指针表示

```
void sort ( int a[ ], int n)
{
  ......
}
```

int *a

形参
- 不指定长度的数组
- 指针

实参
- 数组名
- 指向数组元素的指针变量

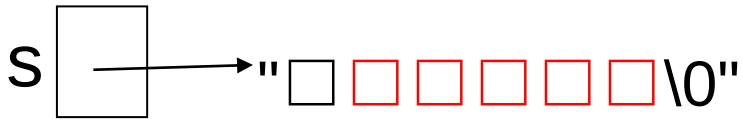
```
#include<stdio.h>
#define N 10
void BubbleSort ( int a[ ],int n)  // 形参为不指定长度的数组
{
      int i, j, t;
      for (i=1; i<n; i++)   /* 共进行n-1轮"冒泡" */
        for (j=0; j<n-i ; j++)  /* 对两两相邻的元素进行比较 */
          if (a[j]>a[j+1] ) {  t=a[j];  a[j]=a[j+1];  a[j+1]=t; }
}
int main ( )
{  int x[N], i;
   printf (" please input %d numbers: \n ", N);
   for (i=0; i<N; i++)    scanf(" %d ", &x[i]);
   BubbleSort (x , N ) ;    // 实参为数组名
   printf (" the sorted numbers: \n ");
   for (i=0; i<N; i++)   printf("%d  ", x[i]);
   return 0;
}
```

```
#include<stdio.h>
#define N 10
void BubbleSort ( int *a,int n)  // 形参为指针
{
      int i, j, t;
      for (i=1; i<n; i++)   /* 共进行n-1轮"冒泡" */
        for (j=0; j<n-i ; j++)  /* 对两两相邻的元素进行比较 */
          if (a[j]>a[j+1] ) {  t=a[j];  a[j]=a[j+1];  a[j+1]=t; }
}
int main ( )
{  int x[N], i;
   printf (" please input %d numbers: \n ", N);
   for (i=0; i<N; i++)    scanf(" %d ", &x[i]);
   BubbleSort (x , N ) ;    // 实参为数组名
   printf (" the sorted numbers: \n ");
   for (i=0; i<N; i++)   printf("%d  ", x[i]);
   return 0;
}
```

```
#include<stdio.h>
#define N 10
void BubbleSort ( int *a,int n)  // 形参为指针
{
      int i, j, t;
      for (i=1; i<n; i++)   /* 共进行n-1轮"冒泡" */
        for (j=0; j<n-i ; j++)  /* 对两两相邻的元素进行比较 */
          if (*(a+j)>*(a+j+1) ) {
             t=*(a+j);  *(a+j)=*(a+j+1);  *(a+j+1)=t;
          }
  }
int main ( )
{  int x[N], i,*p=a;
   printf (" please input %d numbers: \n ", N);
   for (i=0; i<N; i++)    scanf(" %d ", &x[i]);
   BubbleSort (p , N ) ;    // 实参为指针变量
   ..... ;
}
```

库函数strlen(s)递归实现

- 字符串看成“一个字符后面再跟一个字符串”

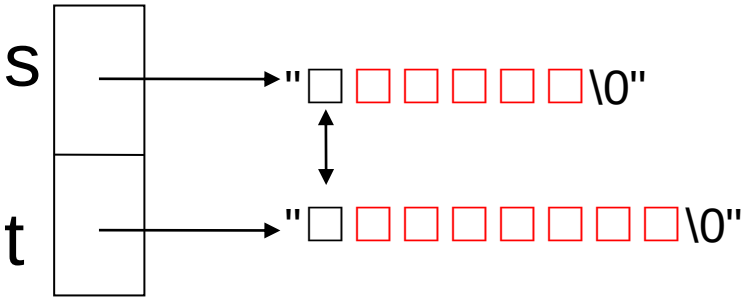



```
int strlen(char s[ ])
{
   if(s[0]=='\0')
      return 0;
   else
      return(1+strlen(s+1));
}
```

```
int strlen(char *s)
{
   if(*s=='\0')
      return 0;
   else
      return(1+strlen(++s));
}
```

【例12.2】 用递归实现标准库函数strcmp(s,t)

- 字符串看成“一个字符后面再跟一个字符串”



递归结束条件:

(1)两个串首字符不等

或

(2)两个串是空串

```
int strcmp_rec(char *s,char *t)
{
    if(*s!=*t||*s=='\0')
        return(*s-*t);
    else
        return(strcmp(++s,++t));
}
```

例 验证密码

```
int main(void)
{
      char pw[]="1234",s[20];
      int  count=3;
      do {
          printf("Input  password\n"）；
           gets(s);
           count--;
       } while(strcmp_rec(pw,s)&&count));
        if(strcmp_rec(pw,s))
             return 1;
         ....
        return 0;
}
```

例8.15 计算两个向量的数量积（向量的数乘）

- 向量：n个实数组成的有序数组

  a =（a1，a2,….,an）

  b =（b1，b2,….,bn）

- 向量的数量积等于它们对应分量的乘积的和 $\sum_{i=1}^{n} a_i b_i$

- 将计算两个向量的数量积定义成函数scalar

  形式参数：向量a，double *a

  向量b，double *b

  向量的维数n，int n

  返回值：数量积

例8.15 计算两个向量的数量积（向量的数乘）

$$\sum_{i=1}^{n} a_i b_i$$

```
double scalar(double *a,double *b,int n)
{
    double s,i;
    for(s=0,i=0;i<n;i++) {
        s += *a * *b;
        a++;
        b++;
     }
    return s;
}
```

s += ((*a++) * (*b++));

例8.15 计算两个向量的数量积（向量的数乘）。

```
#include <stdio.h>
#define SIZE 3
double scalar(double *a,double *b,int n);
void main(void)
{
    double x[SIZE],y[SIZE],*px,*py;
    for(px=x;px<x+SIZE;px++)   /* x+SIZE是元素x[SIZE]的地址 */
        scanf("%lf",px);
    getchar();
    for(py=y;py<&y[SIZE];)     /* &y[SIZE]就是y+SIZE */
        scanf("%lf",py++);
    px=x;py=y;
    printf("the scalar is %f\n", scalar(px,py,SIZE));
}
```

8.4.3 用指向基本数组元素的指针表示多维数组

```
#define M  3
#define N  2
int a[M][N]={ {1，  3},
              {4，  6},
              {7，  9}
             };
int *p;
```

数组a的元素的数据类型为int，则指向数组a基本元素的指针为int *。

(1) 使该指针指向一个数组元素；

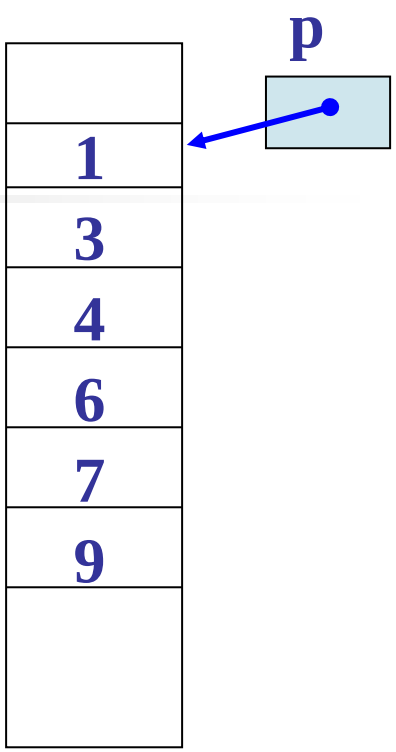
（2）用该指针表示数组元素；

（3）对该指针还可以施行运算，使它指向数组中的其它元素。

思考：如何用指针p逐行输出数组a的所有元素？

```
for(p=a[0]; p<a[0]+M*N;p++){
    if (!( (p-a[0])%N ))   printf("\n");
     printf("%5d",*p);
}
```

```
int a[M][N]={ {1，  3},
              {4，  6},
              {7，  9}
            };
int *p;
```



如何用指针p逐行输出数组a的所有元素？

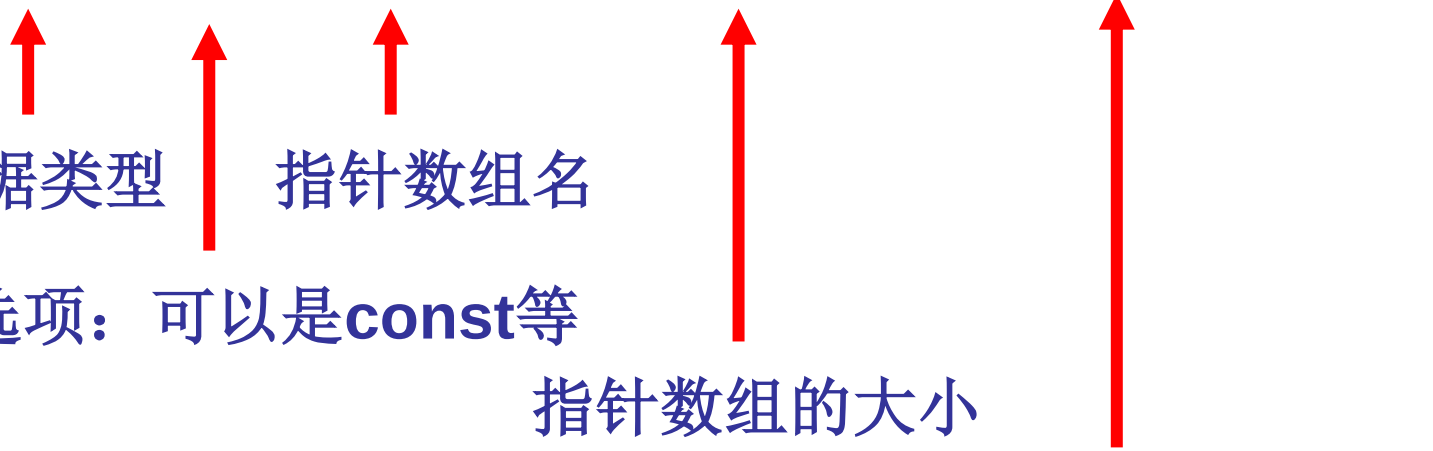
```
p=a[0];  /*  等价于 p=&a[0][0] */
for(i=0; i<M;i++)  {
   printf(“\n”);
   for(j=0; j<N;j++)
     printf(“%5d”, *(p+i*N+j )  );
 }
```

8.5 指针数组

8.5.1 指针数组的声明及使用

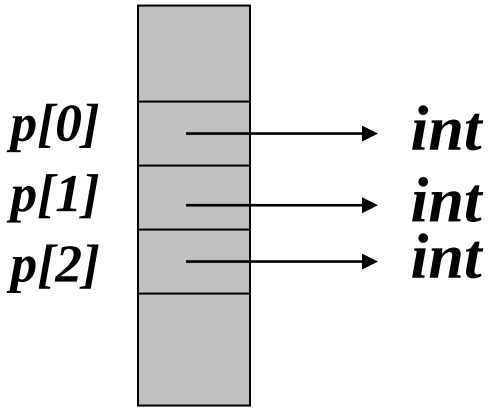
1. 指针数组的声明

cv T * cv 标识符[常量表达式]={初值表};



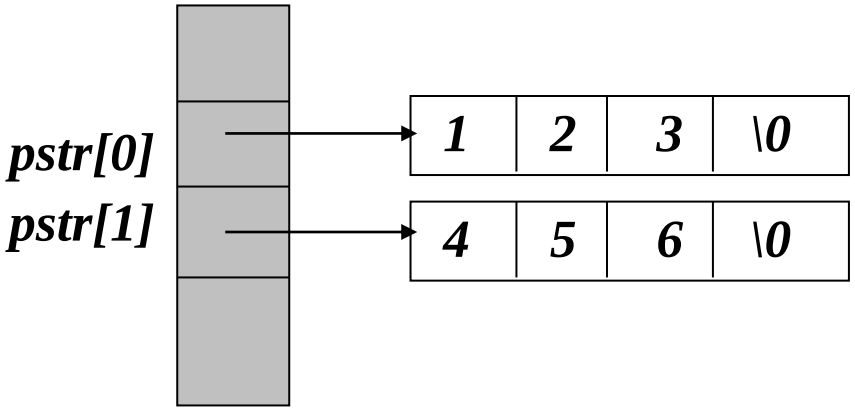
int *p[3]; /* p是一个有3个元素的整型指针数组

即每个元素是指向整型变量的指针 */

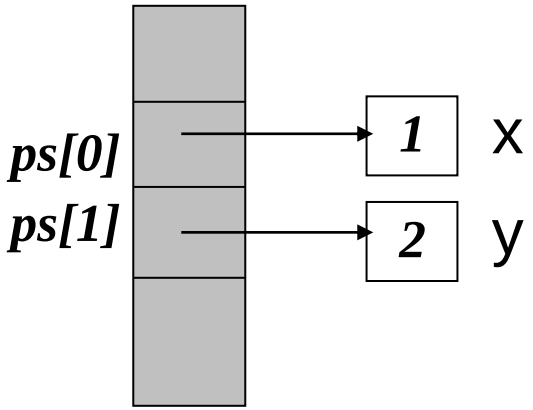


char *pstr[2]={ “123”, “456” };

/* pstr 是一个有2个元素的字符指针数组
pstr[0]指向字符串“123”
pstr[1]指字符串“456” */



```
const  int x=1,y=2;     /*  x, y 值不可更改  */
const int *ps[2];
        /* ps是由2个指向整型常量的指针组成的指针数组;
           简称为指向常量的整型指针数组。*/
ps[0]=&x; ps[1]=&y;   /* ps[0]指向x，ps[1]指向y */
*ps[0]=4; /* 错误,指针数组ps的元素所指的对象不能修改 */
ps[0]=&y;  /*正确,ps的元素本身可以修改,使之指向其他对象*/
```



```
const  char * const  p[2]={ “abc”,  “123”};
    /* p是一个指向常量字符串的字符型指针常量数组。p的元素本身不能修改，p的元素所指对象也不能修改。*/
```

2. 指针数组的使用

指针数组的应用：

描述二维数组（数值型二维数组,字符串数组)

尤其是每行元素个数不相同的二维数组

（*如三角矩阵*,有不同长度的字符串组成的字符串数组)

例1 输入N本书名。

如何存储读入的N本书名？

（1）二维数组

（2）指针数组

如何解决

```
#define N 3
int main( )
{
    int i;
    char *s[N];
    for(i=0;i<N;i++)
      fgets(s[i],20,stdin);
    return 0;
}
```

错误，使用了悬挂指针

```
#define N 3
#include<stdio.h>
#include<stdlib.h>
int main( )
{
   int i;
   char *s[N], t[50];
   for(i=0;i<N;i++)   {
      gets(t);
      s[i] = (char *)malloc(strlen(t)+1);
      strcpy(s[i],t);
   }
   ……
}
```

在该头文件中给出函数原型：
void *malloc(unsigned size);

分配size字节的存储区。
返回所分配单元的起始地址。
如不成功，返回NULL(空指针)

动态存储分配函数malloc

- 动态存储分配函数是C的标准函数，函数的原型声明在头文件<stdlib.h>中给出。

  void * malloc(unsigned size);

  功能：分配size字节的存储区。

  返回值：所分配单元的起始地址。如不成功，返回NULL

无值型指针与空指针

类型为void *的指针称为无值型指针或void指针。不能对void指针执行间访操作，即对void指针施行“*”操作属于非法操作。

指针值为0的指针称为空指针，0在C中往往用符号常量NULL表示并被称为空值。

```
scanf（”%d”,&n）;
 /* 建立大小为n的int型数组 */
p=(int *)malloc( n*sizeof(int));
if(p==NULL)   exit(-1);
```

例　用字符指针数组构造程序的菜单系统 menu.c

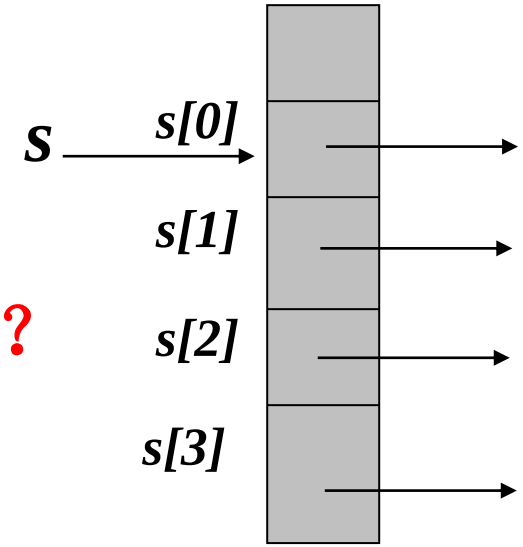
```
/* 菜单函数，函数返回值是整形，代表所选的菜单项 */
int MenuSelect(void )
{   char *menu[ ]={
            "1:Enter record",
            "2:Search record on name",
            "3:Delete a record"
            "4:Add a record",
            "5:Quit"        };
    int i;
    system("cls");    /*  清屏 */
    do {
          puts(menu[i]);
          printf("\n Enter your choice(1-5):");
          scanf("%d",&i);   /* 输入选择项 */
    } while( i<1 || i>5 ); /* 选择项不在1-5之间重输 */
    return c; /* 返回选择项，调用函数根据该数调用相应的函数 */
}
```

例 判断字符串是否关键字。

```
/*  若s是关键字，函数返回1；否则,返回0   */
int iskey(char *s)
{   char *keyword[]={  /*keyword中的每个元素都指向特定的关键字字符串*/
    "auto", "_Bool",   "break",  "case",   "char", "_Complex",
    "const","continue","default","restrict","do",  "double",
    "else", "enum",    "extern", "float",  "for",   "goto",
    "if", "_Imaginary","inline", "int",    "long", "register",
    "return", "short", "signed", "sizeof", "static","struct",
    "switch","typedef","union", "unsigned","void", "volatile",
    "while","" };
    int i;
    for(i=0;*keyword[i]!='\0';i++)  /*将标识符s与每个关键字比较，是返回1*/
        if(!strcmp(s,keyword[i])) return 1;
    return 0;  /*否返回0*/
}
```

例 判断字符串是否合法的用户标识符。

```
/*  t是标识符，函数返回1；否则，返回0  */
int isid(char * t)
{   char *s=t;
    if( !isalpha(*t) && *t!= '_')  /* 首字符不是字母或下划线，返回0 */
        return 0;
    for(t++;*t!='\0';t++)/* 后续字符不是字母、数字、或下划线，返回0  */
        if(!(isdigit(*t)||isalpha(*t)||*t=='_'))
            return 0;
    if(iskey(s))  /* s是关键字，返回0 */
           return 0;
    return 1;
}
```



char *s[4];

问：s[0][0]，s[0]，s类型分别是什么？

s[0][0]《==》*s[0]　　类型 char

s[0],指针数组的元素，类型 char *

s，指针数组名，指向s[0],类型：char **（字符型的二级指针）

二级指针的应用：作函数的形参

8.5.2 多重指针

```
int x, *p=&x;
```

x有地址（&x），那 p有地址吗？

&p 的类型？

&p—〉p□—〉x□

欲保存p的地址，应该定义一个什么类型的变量？

```
int x, *p=&x, **pp=&p;/* pp是int的二级指针 */
```

pp□—〉p□—〉x□

```
**pp=5;    /* x=5 */
*p=8;    /* x=8 */
```

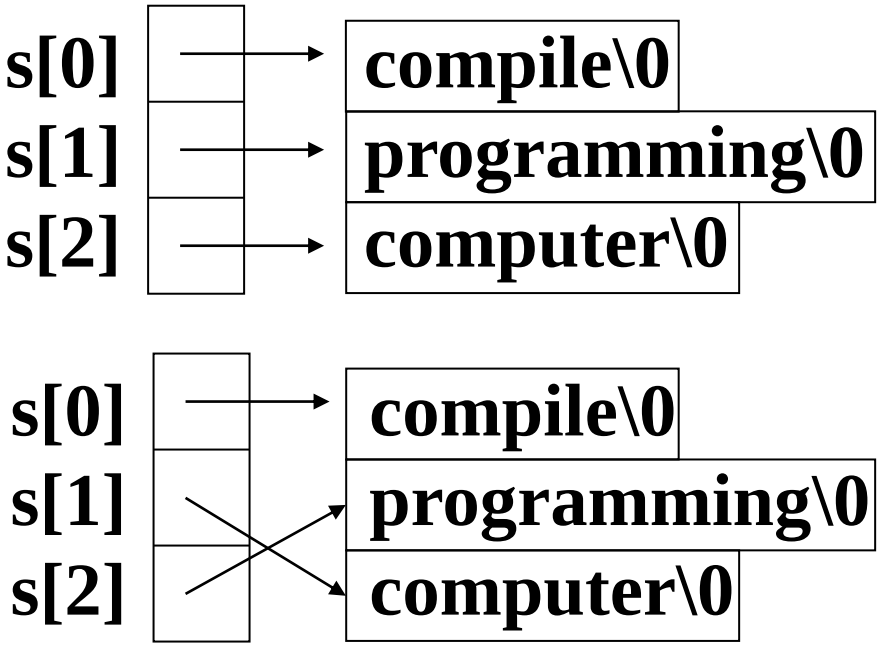
定义函数strsort对字符串按字典序升序排序

```
#include<string.h>
void strsort ( char *s[ ]   , int size )
{
  char *temp;
  int i, j ;
  for(i=0; i<size-1; i++)
     for(j=0; j<size-i-1; j++)
          if  ( strcmp(s[j],s[j+1] ) >0 ) {
              temp=s[j];                /* 移动指针变量 */
              s[j]=s[j+1];
                s[j+1]=temp;
          }
}
```

char **s

例　输入N本书名，排序后输出。

```
#define N 3
#include<stdio.h>
#include<stdlib.h>
void main( )
{
   int i;
   char *s[N], t[10];
   for(i=0;i<N;i++)   {
      gets(t);
      s[i] = (char *)malloc(strlen(t)+1);
      strcpy(s[i],t);
   }
   strsort(s,N);
   for(i=0;i<N;i++)  puts(s[i]);
}
```



户输入

时所需的数据何时输入

```c
#include<stdio.h>

int main(void)
{
    int n,sum,i;
    scanf("%d",&n);
    for(sum=0,i=1;i<=n;i++)
        sum+=i;
    printf("1+2+...+%d=%d\n",n,sum);
    getchar();
    return 0;
}
```

```c
#include<stdio.h>
#include<stdlib.h>
int main(int argc,char *argv[])
{
    int n,sum,i;
    if(argc!=2) {
        printf("Command line error!\n");
        return -1;
    }
    n=atoi(argv[1]);
    for(sum=0,i=1;i<=n;i++)
        sum+=i;
    printf("1+2+...+%d=%d\n",n,sum);
    return 0;
}
```

8.6 带参数的main函数

8.6.1 命令行参数

- 在命令行中给定的参数就是命令行参数。

sum_arg 4

(以上sum_arg参数是必须的，否则不能运行)

- 命令行的参数由谁来接收

运行程序时操作系统将命令行参数传给main函数的形式参数

带参数的main函数的一般形式

```
int main(int argc,char *argv[])
{
   …
}
```

2

={ “sum” ,
“10” }

实参由命令行提供。

argc：argument count，

代表命令行中参数的个数（包括文件名）

argv：argument value，为字符指针数组，

argv[i]指向命令行的第 i+1个参数（字符串）的首字符

```
#include<stdio.h>
#include<stdlib.h>
int main(int argc,char *argv[])
{
    int n,sum,i;
    if(argc!=2) {
        printf("Command line error!\n");
        return -1;
    }
    n=atoi(argv[1]);
    for(sum=0,i=1;i<=n;i++)
        sum+=i;
    printf("1+2+...+%d=%d\n",n,sum);
    return 0;
}
```

C:\> sum 11

argv[0] argv[1]

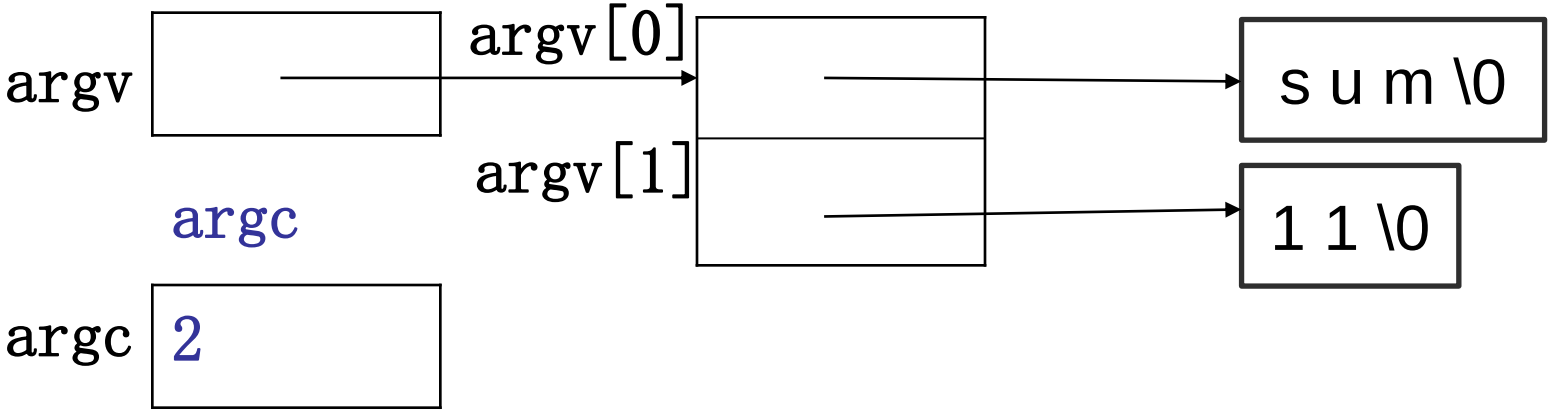
命令行中参数的个数（包括文件名）

长度为argc的字符指针数组
每个参数是一个字符串，
有argc个字符串

C:\> sum 11

```
int main(int argc,char *argv[])
{
    ......
}
```

char **argv



```
#include<stdio.h>
#include<stdlib.h>
int main(int argc,char **argv)
{
    int n,sum,i;
    if(argc!=2) {
        printf("Command line error!\n");
        return 1;
     }
    n=atoi(*++argv);
    for(sum=0,i=1;i<=n;i++)
          sum+=i;
    printf("1+2+...+%d=%d\n",n,sum);
    return 0;
}
```

在集成开发环境下调试程序时命令行所带参数如何输入？(只输入文件名后的参数)

1) 在C::B下：

选择菜单“project/set programs’ arguments…”，在“Program arguments”文本框中输入main函数的参数。

2）Dev： Execute/Parameters...

3) 在VC下：

工程（project）->设置（setting）->调试（debug）-> 程序变量（program arguments）

8.7 指针函数

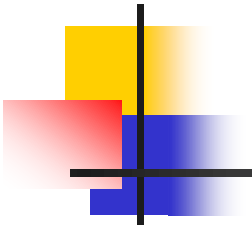
在C语言中，函数返回的只能是值。这个值可以是一般的数值，也可以是某种类型的指针值。如果函数的返回值是指针类型的值，该函数称为指针函数。

类型 *函数名（形参表）；

如：char *strcpy(char *t,const char *s);

函数strcpy是一个字符指针函数。即：该函数的返回值是字符指针。

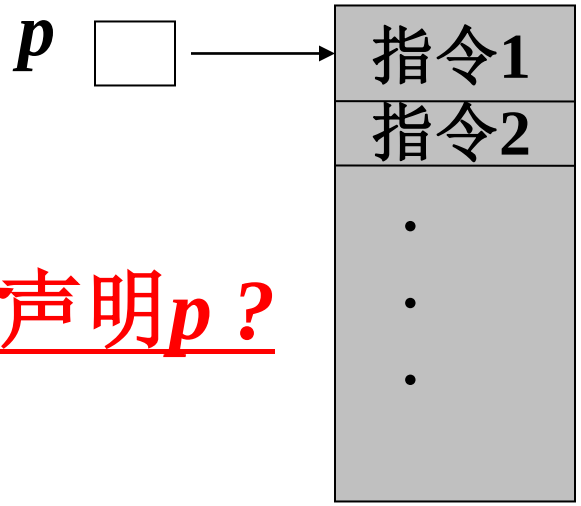
```
#include<stdio.h>
char  *strcpy( char *t, const char *s )
{
   char *p=t ;
   while(*t++ = *s++)
      ;
   return(p) ;  /* 返回第1个串的首地址 */
}
int main( )
{
  char st1[40]=” abcd”, st2[ ]=” hijklmn”;
  printf( “%s”, strcpy( st1,st2));
  return 0;
}
```



如果函数返回的指针指向一个数组的首元素，就间接解决了函数返回多值的问题。

8.8 函数的指针

每个函数都占用一段内存单元，有一个起始地址。



函数

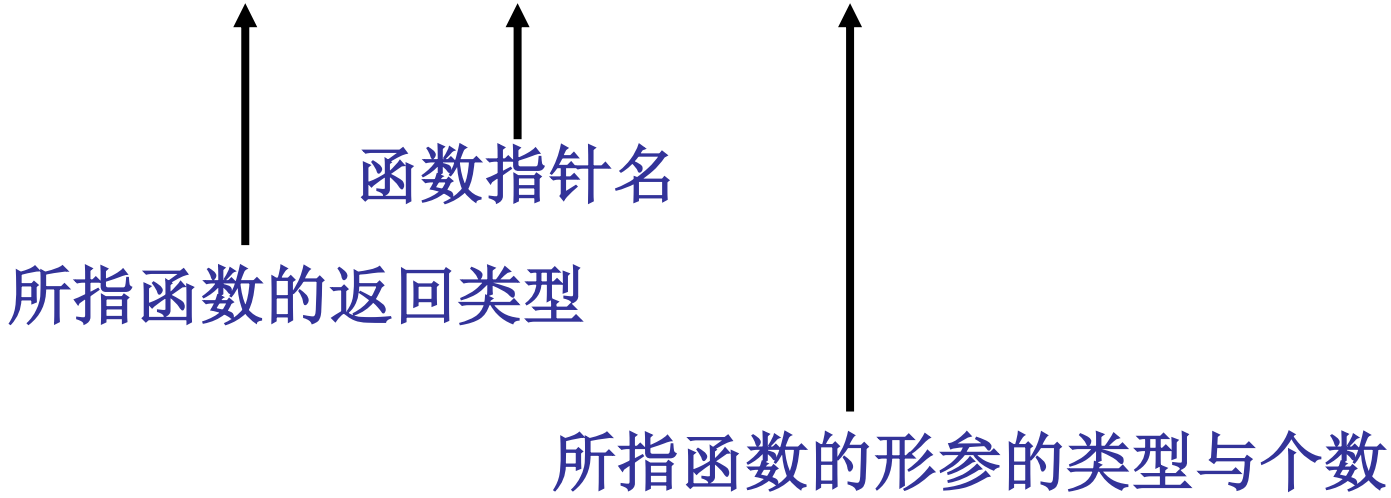
max(int x,int y)

{ … }

如何声明p？

8.8.1 函数指针的声明

类型 (*标识符)(形参表);




```
int  (*p) ( int, int);
  /* p 是指向有两个 int参数的int函数的指针 */
```

函数指针的应用举例

```
#include "stdio.h"
void f1(int x)
{    printf("x=%d\n",x); }
void f2(int x,int y)
{   printf("x=%d\ty=%d\n",x,y);}
int main(void)
{   void (*pf1)(int x);
    void (*pf2)(int x,int y);
    pf1=f1;
    pf2=f2;
    pf1(5);        /*  等价于(*pf1)(5);*/
    pf2(10,20);  /*  等价于(*pf2)(10，20);*/
    return 0;
}
```

函数指针的使用：

（1）通过初始化或赋值使其指向特定的函数；

函数指针名=函数名；

（2）通过函数指针来调用它所指的函数

定义一个通用的整数排序函数

- 既能实现升序排序，也能实现降序排序

  函数名称：**sort**

  函数参数：

  v--待排序数组的首地址

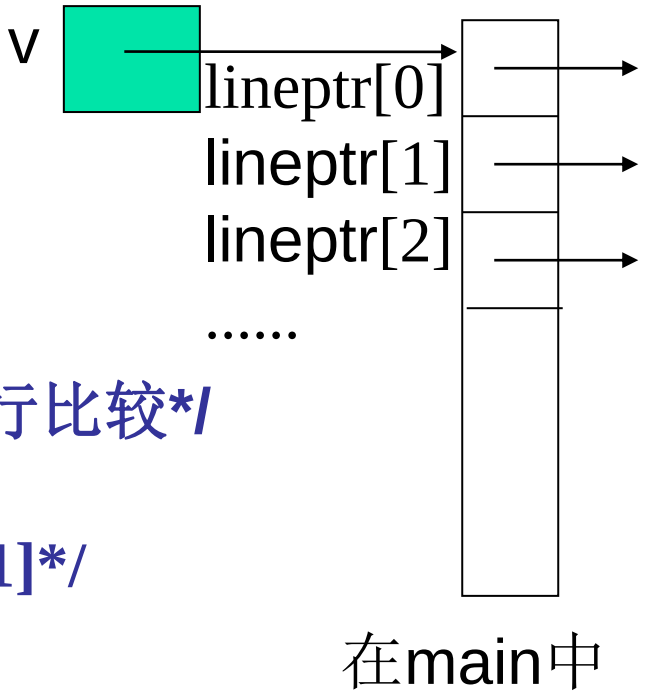
  n--数组中待排序元素数量

  comp--指向函数的指针，用于确定排序的规则

  函数返回值：无

/*对指针v指向的n个整数按comp规则排序 */

```
void sort ( int *v, int n, int (*comp)(int,   int) )
{
   int  i, j ;

   for(i=1; i<n; i++)        /*冒泡法*/
     for(j=0;j<n-i; j++)
          /*对v[j]和v[j+1]按照comp的规则进行比较*/
          if ( (*comp)(v[j],v[j+1])>0 )
              swap(v, j , j+1) ;  /*交换v[j]和v[j+1]*/
}
```



规则：按升序

```
int asc(int x, int y)
{
   if(x>y)  return 1;
   else return 0;
  }

// caller
int a[6]={4,6,3,9,7,2};
sort(a,6,asc);
// 思考：如果要降序排呢？如何定义描述规则的函数
```

自学教材例8-30

进阶：定义更通用的排序函数

- 能够对int、char、double、字符串、struct类型的数据排序。
- 既能实现升序排序，也能实现降序排序
- 函数参数

  void *v，待排序数组首地址

  int n，　数组中待排序元素数量

  int size，　各元素的占用空间大小（字节）

  int (*fcmp)(const void *,const void *)，指向函数的指针，用于确定排序的规则
- stdlib.h中的标准库函数qsort----万能数组排序函数

  void qsort(void *base, int nelem, int width,

  int (*fcmp)(const void *,const void *));
- P269（快速排序），p296例13.6（通用的排序函数定义）
- 思考：如何调用qsort对字符串数组排序。

C语言与程序设计

The C Programming Language

第9章 结构与联合

毛伏兵

华中科技大学计算机学院

9.1 结构与联合

结构与联合都属于C的构造类型。

- 某种具体的结构说明（关键字struct）
- 结构变量的定义
- 结构成员的引用（.运算符）
- 通过结构指针引用结构成员（-〉运算符）
- 字段结构
- 联合类型（union）

例1、输入平面上两个点，求它们之间的长度。

```
#include "stdio.h"
#include "math.h"
struct point{   /* 平面上点的结构类型说明 */
    int    x;
    int    y;      /* x,y是点的坐标 */
};
int main(void)
{   struct point start,  end;    /* 声明点结构变量 */
    double dx,dy,length;
    printf("输入线段的起点和终点坐标：\n");
    scanf("%d%d%d%d",&start.x,&start.y，&end.x,&end.y);
    dx=(end.x-start.x)*(end.x-start.x);
    dy=(end.y-start.y)*(end.y-start.y);
    length=sqrt(dx+dy);                    /*计算线段长度*/
    printf("the length is %f\n",length);      /*输出线段长度*/
    return 0;
}
```

结构成员的访问：

结构变量名. 成员名

例2、改写例1将求两点间的长度定义为函数

```
double  linelen(struct point  pt1,struct point  pt2)
{
  double dx,dy,length;
  dx=(pt2.x-pt1.x)*(pt2.x-pt1.x);
  dy=(pt2.y-pt1.y)*(pt2.y-pt1.y);
  length=sqrt(dx+dy);      /*计算线段长度*/
  return length;
}
```

结构变量作为函数的参数

```
#include "stdio.h"
#include "math.h"
struct point{   /* 平面上点的结构类型 */
    int    x;
    int    y;      /* x,y是点的坐标 */
};
int main(void)
{   struct point start，end；   /* 声明点结构变量 */
    double length;
    printf("输入线段的起点和终点坐标：\n");
    scanf("%d%d%d%d",&start.x,&start.y，&end.x,&end.y);
    length=linelen(start，end);                    /*计算线段长度*/
    printf("the length is %f\n",length);       /*输出线段长度*/
    return 0;
}
```

例3、改写例2将函数参数改为结构类型的指针

```
double  linelen(struct point  *p1,struct point  *p2)
{
   double dx,dy,length;
   dx=(p2->x-p1->x)*(p2->x-p1->x);
   dy=(p2->y-p1->y)*(p2->y-p1->y);
   length=sqrt(dx+dy);   /*计算线段长度*/
   return length;
}
```

- 结构类型的指针作为函数的参数
- 结构类型的指针也可以作为函数的返回值

```
#include "stdio.h"
#include "math.h"
struct point{   /* 平面上点的结构类型 */
    int    x;
    int    y;      /* x,y是点的坐标 */
};
int main(void)
{   struct point start，end；   /* 声明点结构变量 */
    double length;
    printf("输入线段的起点和终点坐标：\n");
    scanf("%d%d%d%d",&start.x,&start.y，&end.x,&end.y);
    length=linelen(&start，&end);              /*计算线段长度*/
    printf("the length is %f\n",length);       /*输出线段长度*/
    return 0;
}
```

结构类型的变量作为函数的参数时：

1、将实参结构拷贝给形参，占用内存较多，耗费时间较长。

——— 适用于较小的结构

2、值传递，形参结构不影响实参结构的值。

结构类型的指针作为函数的参数时：

1、只将实参指针的值拷贝到形参指针单元。占用内存少，耗费时间短。

——— 适用于较大的结构

2、在函数内部对形参指针的间访操作会影响实参指针所指结构变量的值。

———适合于要修改实参指针所指的结构变量值的情况。

通过结构指针访问结构成员

- struct point a,b={10,20}, *p=&a;
  p=b; / 与a=b;等价 */
- 通过“*”用结构指针访问结构变量的成员
  (*结构指针).成员名
  (*p).x <==> a.x
- 通过运算符“->”访问结构变量的成员
  结构指针名->结构成员名
  p->y <==> (*p).y

结构变量的赋值操作

- 当两个结构变量的类型相同时，它们之间可以直接相互赋值。例如：

```
static struct point {
        int  x;
        int  y;
} a={1,2},   b;
```

则 b=a; /* 合法,对应的各个成员赋值 */

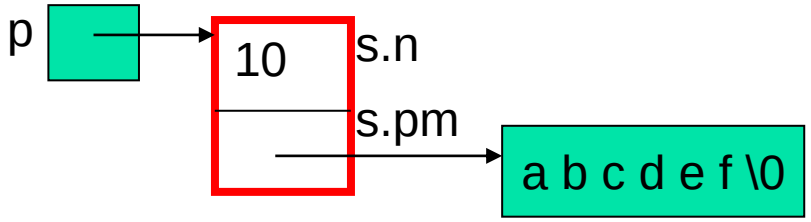
b={1,2}; 对的 /* 错*/??

定义一个初始化点结构变量的函数

- 结构成员作为函数的参数
- 结构变量作为函数的返回值

```
point makepoint(int x, int y)
  {
     struct point temp;
     temp.x = x;
     temp.y = y;
     return temp;
   }
```

```
#include "stdio.h"
#include "math.h"
struct point{   /* 平面上点的结构类型 */
    int    x;
    int    y;      /* x,y是点的坐标 */
};
int main(void)
{   struct point start，end；   /* 声明点结构变量 */
    double length;
    start=makepoint（0，0）；
    end=makepoint（2，3）；
    length=linelen(&start，&end);              /*计算线段长度*/
    printf("the length is %f\n",length);       /*输出线段长度*/
    return 0;
}
```



设有：

```
struct T{
    int  n;
    char *pm;
}s={10,“abcdef ”},*p=&s;
```

写出以下表达式的值和类型。（各表达式相互无关）

1） ++p->n

2） p->n++

3) *p->pm

4) *p->pm++

5) *++p->pm

表达式	值与类型	表达式执行的操作	等价的表达式
++p->n	11，int	访问 n 并使其增 1	++s.n 和 ++(*p).n
p->n++	10，int	访问 n，取其原值参与运算，再使 n 增 1	s.n++ 和 (*p).n++
*p->pm	'a'，char	访问 pm 所指字符'a'	*s.pm 和 *(*p).pm
*p->pm++	'a'，char	访问 pm 所指字符'a'后 pm 增 1 指向'b'	*s.pm++和*(*p).pm++
*++p->pm	'b'，char	先访问 pm，然后 pm 增 1，再访问 pm 所指字符'b'	*++s.pm 和*++(*p).pm

所有运算符的优先级和结合性规则

优先级（从高到低）	运算符	运算符名称	结合性
1	( ) [ ] ->> .	圆括号 下标 间接引用结构体成员	左结合
单目运算符	! ~ ++、--	逻辑非 按位取反 自增、自减	
2	+、-	取正、取负	右结合
	(数据类型) &、* sizeof	强制类型转换 取地址、间接引用 数据长度	
3	*、/、%	乘、除、求余数	左结合
4	+、-	加、减	左结合
5	<<、>>	左移、右移	左结合
6	<、> >=、<=	大于、小于 大于等于、小于等于	左结合
7	==、!=	等于、不等于	左结合
8	&	按位与	左结合
9	^	按位异或	左结合
10	|	按位或	左结合
11	&&	逻辑与	左结合
12	||	逻辑或	左结合
13	? :	条件	右结合
14	=、+=、-=、*=、/=、%=、>>=、<<=、&=、^=、|=	赋值	右结合
15	,	逗号	左结合

- **例4 编程输入商品信息（包括商品编码、名称、价格），并且按照价格排序并显示排序后的结果。**

商品编码	名称	价格（元）
1	笔	2.0
2	毛巾	10.5

```
#include<stdio.h>
#define N 3
struct GOODS {
    long code;              /* 货物编码 */
    char name[20];     /* 名称 */
    float price;        /* 价格 */
};
/* 输入n件物品的信息  */
void input(struct GOODS *p,int n);
/*  对n件物品按价格降序排序  */
void sort(struct GOODS *p,int n);
/*显示n件物品的信息  */
 void display(struct GOODS *p,int n);
```

```
int main(void)
{
    struct GOODS g[N];

    input(g,N);    /* 结构数组名作为实参 */
    display(&g[0],N);
    sort（g,N);
    display(g,N);
    return 0;
}
```

```
/* 输入n件物品的信息 */
void input(struct GOODS *p,int n)
{
    int i;
    for(i=0;i<n;i++){
        scanf("%ld",&p[i].code);
        scanf("%s",(p+i)->name);
        scanf("%f",&(p+i)->price;
    }
}
```

```
/*显示n件物品的信息 */
void display(struct GOODS *p,int n)
{
    int i;
    for(i=0;i<n;i++){
        printf("%ld\t",(*(p+i)).code);
        printf("%s\t",(p+i)->name);
        printf("%f\n",p[i].price);
    }
}
```

```
/*  对n件物品按价格降序排序  */
void sort(struct GOODS *p,int n)
{    int i,j;    struct GOODS t;
     for(i=0;i<n-1;i++)
          for(j=i+1;j<n;j++)
               if((p+i)->price<(p+j)->price){
                    t=*(p+i);
                    *(p+i)=*(p+j);
                    *(p+j)=t;
          }
}
```

设计一个能够描述学生成绩的结构类型，然后声明对应的结构数组，描述以下成绩表。

成绩表

学号	姓名	性别	入学时间			计算机原理	英语	数学	音乐
			年	月	日				

定义

```
struct date
{
    int month;
    int day;
    int year;
};

struct  STUDENT
{
        int             ID;
        char            Name[10];
        char            Sex[4];
        struct date timeOfEnter;
        int             Computer;
        int             English;
        int             Math;
        int             Music;
};

struct STUDENT stu[30];
```

初始化

```
struct STUDENT stu[30] = {
    {1," Tom  ",'M',{1999,12,20},90,83,72,82},
    {2," Nick ",'M',{1999,07,06},78,92,88,78},
    {3," Sue  ",'F',{1999,07,06},89,72,98,66},
    {4," May  ",'M',{1999,07,06},78,95,87,90}
  };
```

嵌套结构中结构成员的访问

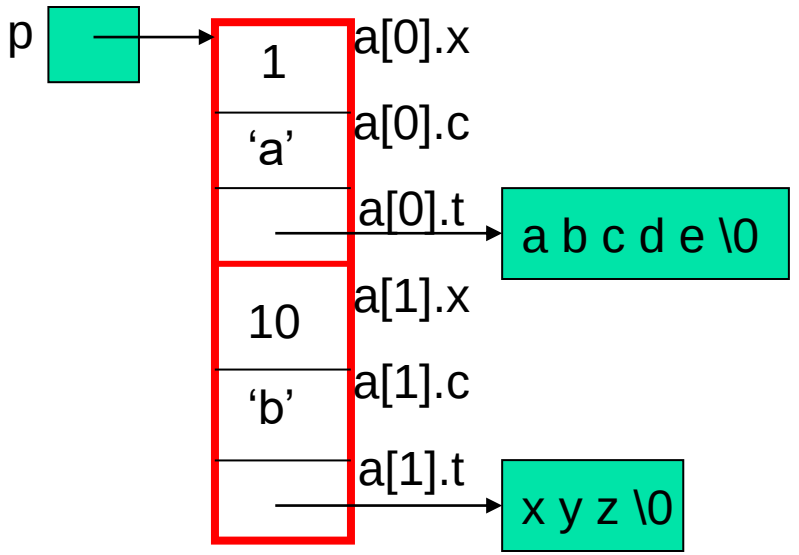
- 结构变量名.结构成员名.成员名
- 理解为:

  （结构变量名.结构成员名）.成员名

  stu[0].timeofenter.year

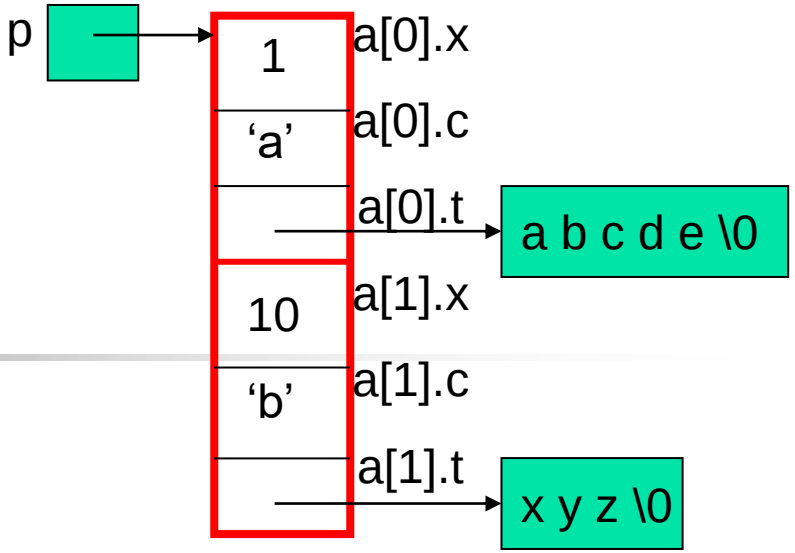
- 注意“.”、“->”、“*”、“++或--”运算符 优先级和结合性
- 设有说明：

```
char u[]="abcde";
char v[]="xyz";
struct T{
        int x;
        char c;
        char *t;
}a[]={{1,’a’,u},{10,‘b’,v}},*p=a;
```



- 若执行代码片段：

```
printf(“%d\n”,(++p)->x);     /* 输出 10 */
printf("%c\n",p->c);          /* 输出 b */
```



- 若执行代码片段：

```
p=a;
printf("%d\n",(p++)->x);    /*  输出 1  */
printf("%d\n",p->x);         /*  输出 10  */
```

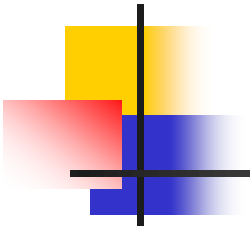
若执行代码片段：

```
p=a;
printf("%c\n",*p++->t);     /*  输出 a
*/??Sequence?
printf("%c\n",*p->t);       /*  输出 x  */
```

比较下面各表达式的异同：

- (++p)->x 与 ++p->x　　/* 不同 */优先级？
- (p++)->x 与 p++->x　　/* 相同 */
- *(p++)->t 与 *p++->t　　/* 相同 */
- *(++p)->t 与 *++p->t　　/* 不同 */
- *(++p->t) 与 *++p->t　　/* 相同 */
- *++p->t 与 ++*p->t　　/* 不同 */

用sizeof计算数组元素的个数



设有数组a，表达式:

sizeof a/sizeof a[0]

计算a数组中元素的个数。

*9.8 字段（bit field）结构

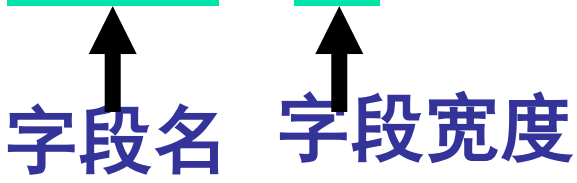
【例2.16】压缩和解压。把表示21世纪日期的日、月和年3个整数压缩成1个16位的整数。

分析：因为日有31个值，月有12个值，年有100个值，所以可以在一个整数中用5b表示日，用4b表示月，用7b表示年.

用字段结构实现

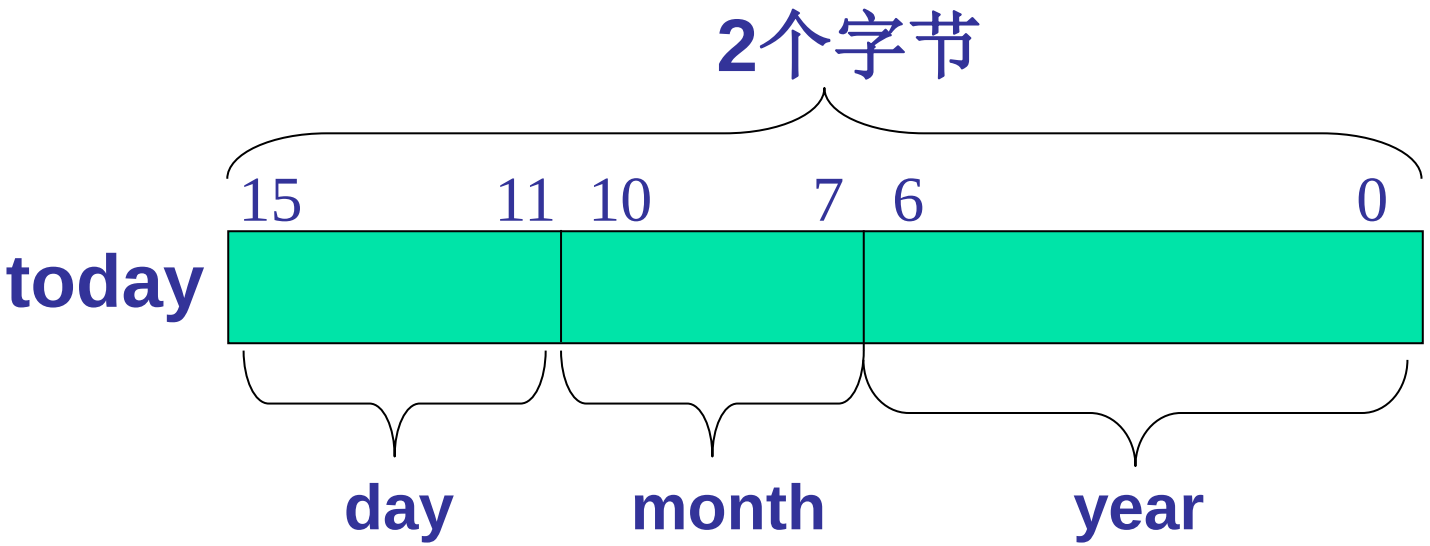
字段结构的声明：

```
struct date {
   unsigned short year  : 7 ;    /* 年 */
   unsigned short month : 4 ;    /* 月 */
   unsigned short day   : 5 ;    /* 日 */
};
```



字段的存储结构

```
struct date {
  unsigned short year  :  7 ;   /* 年 */
  unsigned short month : 4 ;    /* 月 */
  unsigned short day   :  5 ;   /* 日 */
};
struct  date  today;   /*  today是 date 字段结构变量 */
```



引用字段

- 与结构完全相同：　.　　或　->
- 字段就是一个小整数，它可以出现在其它整数可以出现的任何地方。字段在参与运算时被自动转换为int或unsigned int类型的整数。

```
today.year=2013;
today.month=5;
today.day=9;
```

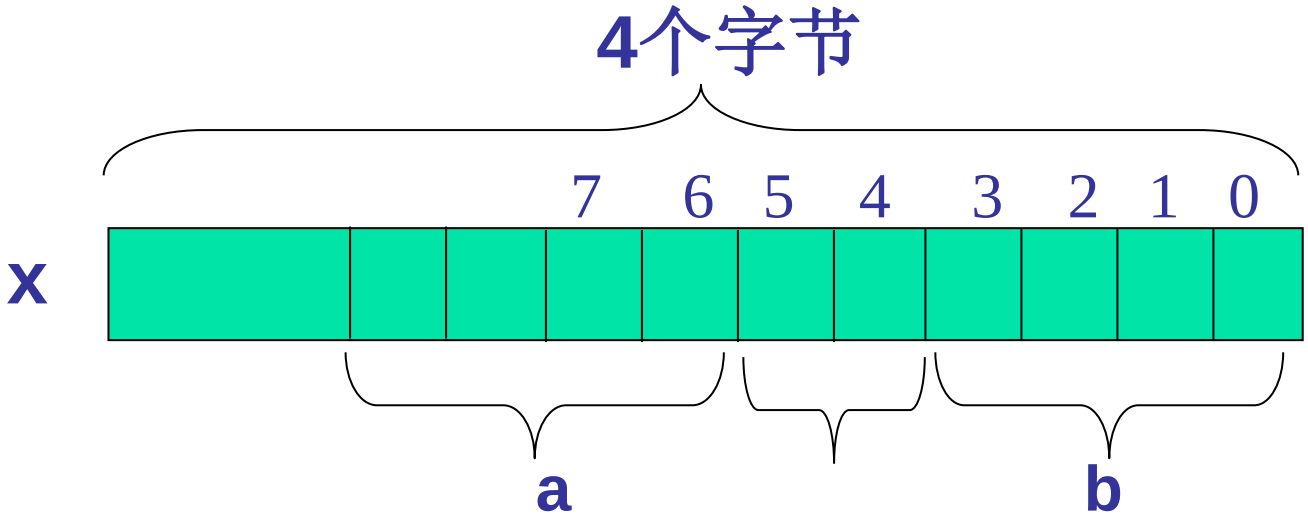
字段：字中一组相邻的二进制位，是指定了存储位数的，**unsigned int**（或**unsigned short**）的结构成员。

字段的宽度：组成字段的二进制位的数目，是一个非负的整型常量表达式。

字段结构在操作系统，编译程序，计算机接口的C语言编程方面使用较多。
可以用一个字段结构变量表示多个值很小的整数，通过字段名对相应的小整数进行操作。

无名位段（匿名）

```
struct  {
   unsigned  a:   4 ;
                 :   2 ;
   unsigned  b:   4 ;
} x ;
```



9.7 联 合

- **结构变量**是占据各自不同空间的各成员变量的集合。
- **如何让几个变量共享同一存储区?**
- **联合变量**是占用同一块内存空间的各成员变量的集合。

- 与结构类似，联合类型也是一种构造类型。一个联合类型中包含有多个成员，这些成员共享共同的存储区域，但这些成员并不同时存在，联合存储区域的大小由各个成员中所占字节数最大的成员决定，在任何时刻，各个成员中只能有一个成员拥有该存储。
- 除了用关键字union取代struct之外，联合类型的定义、联合变量的声明、以及联合成员的引用在语法上与结构完全相同。

- 联合变量的声明、初始化及联合成员的引用与结构完全相同，但是只能对联合的第1个成员进行初始化.

```
union chl {
        char  c;
        short  h;
        long  l;
} v = {'9'} ;
union chl  w = {'a'} ;
```

注意：

程序员必须确保访问的联合成员的类型的一致性。即，数据是以何种类型写入的，读取时就要以那种类型来读。

联合的指针

可以声明联合类型的指针。如：

union chl v,*pv=&v;

- 说明了一个union chl类型的指针pv，并且取出v地址对pv进行初始化，使联合指针pv指向了联合变量v。
- 值得注意的是，联合所有成员的地址和联合变量的地址都是相同的。因为所有成员都是从同一存储空间的边界（低地址）开始存放。
- 但是，不同成员指针值（地址值）的类型是不相同的。

&u　　&u.c　　&u.h　　&u.l　　值相同

union chl *　　char *　　short *　　long *

字段结构与联合的应用

如何访问16位字中的高低字节和各二进制位？

- 定义8位宽的字段byte0、byte1

  ---- 表示一个16位字中的高/低字节

- 定义1位宽的字段b0~b15

  ---- 表示一个16位字中的bit

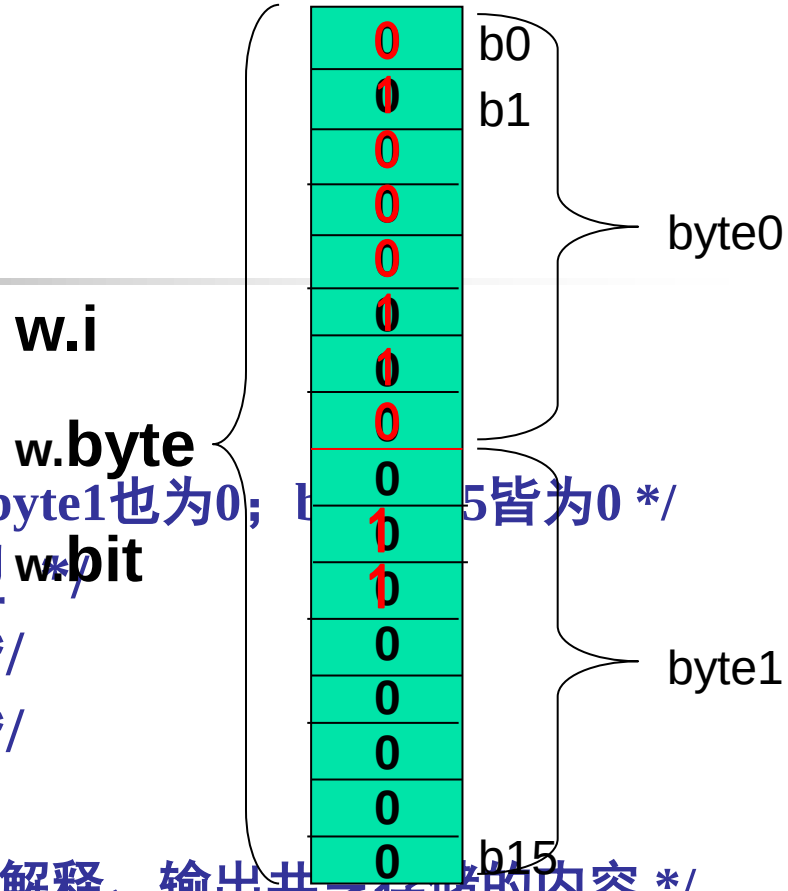
- 定义联合类型

  ---- 使一个short变量、2个byte字段与16个bit字段共享存储。

例9.12　用字段和联合访问一个16位字中的高低字节和各二进制位。

```
#include<stdio.h>
#define CHAR_BIT 8
struct w16_bytes{
        unsigned short byte0:8;    /* byte0：低字节*/
        unsigned short byte1:8;    /* byte1：高字节 */
} ;
struct w16_bits{
        unsigned short b0:1,b1:1,b2:1,b3:1,b4:1,b5:1,b6:1,b7:1,
             b8:1,b9:1,b10:1,b11:1,b12:1,b13:1,b14:1,b15:1;
} ;
```

```
union w16{ /* 短整型变量、结构成员byte、结构成员bit共享共同的存储 */
        short                   i;
        struct w16_bytes        byte;
        struct w16_bits         bit;
};
void main(void)
{
    union w16  w={0};   /* i为0；byte0和byte1也为0；bit0~bit15皆为0 */
    void bit_print(short);   /* 函数原型 */
    w.bit.b9=1;      /* 相当于byte1为2 */
    w.bit.b10=1;     /* 相当于byte1为6 */
    w.byte.byte0='b';
    printf("w.i=0x%x\n",w.i); /* 按整型解释、输出共享存储的内容 */
    bit_print(w.i); /* 从高到低，逐位输出共享存储的各bit值 */
}
```



运行结果：

```
w.i=0x662
00000110 01100010
```

C语言与程序设计

The C Programming Language

第10章 文件的输入与输出

毛伏兵

华中科技大学计算机学院

第10章 文件的输入与输出

- 存储在变量和数组中的数据是临时的
  程序结束后都会消失
- 文件用来永久地保存大量数据
- 文件被存储在外存储设备中（如硬盘）
- 本章讨论怎样用C程序建立、更新、处理数据文件

例1、从键盘输入若干行字符，保存到 d:\a.txt 文件中，直到遇到ctrl+z结束.

文件操作的步骤：

（1）打开文件——建立文件指针(或文件描述符)与文件间联系；

（2）通过文件指针对文件进行读写操作；

（3）关闭文件——取消文件指针(或文件描述符)与文件间的联系。

提示：打开文件时，确定了文件读写格式和读写方式！

定义函数实现：将键盘输入存盘到磁盘文件中

- 函数名： copy_term
- 函数参数：

  filename：存盘的文件名， char *
- 函数返回值：无

```
/*  将键盘输入存盘到 filename 文件中  */
void copy_term (const char * filename)
{
   FILE *fp;
   char ch;
   if ( (fp = fopen(filename,"w") ) == NULL){
        printf("can't open the file!");
        exit(-1);
   }
   while((ch = getchar( ) ) != EOF)
        fputc(ch,fp);
   fclose(fp);
}
```

```
#include<stdio.h>
#include<stdlib.h>
void copy_TERM(const char * filename);
int main(void)
{
   copy_TERM ("d:\\a.txt");
   return 0;
}
```

\源程序\ L1.c

打开文件函数fopen()

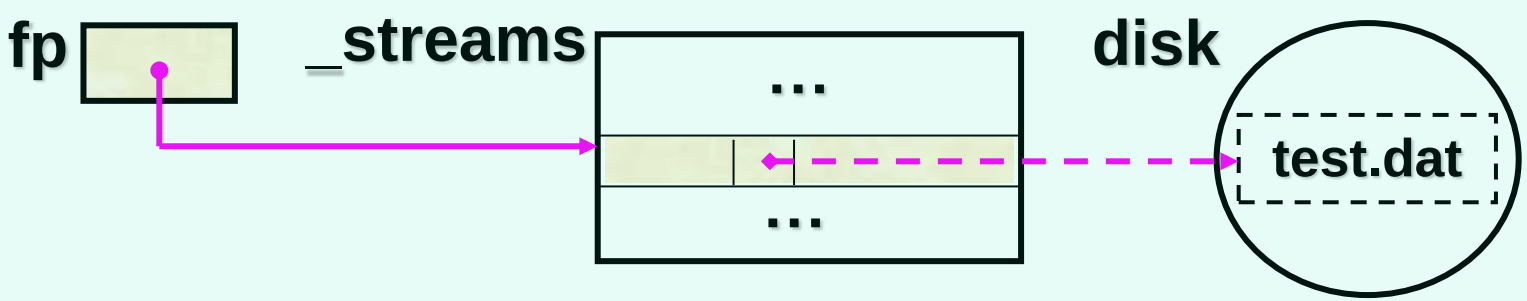
```
FILE * _Cdecl fopen(const char * filename, const char * mode);
```

按照mode方式，打开文件filename。最多可打开OPEN_MAX个文件

成功：返回一个指向filename的文件指针

不成功：返回空指针NULL

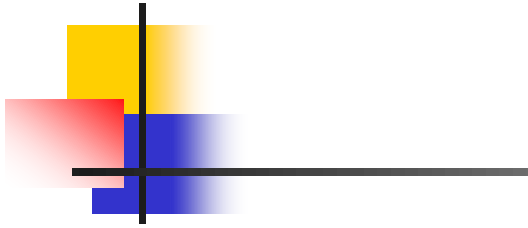
```
FILE * fp;
fp＝fopen(“c:\\test.txt”,”w”);
```



打开方式 mode

r —— 只读（read）
w —— 只写（write）
a —— 添加（append）
+ —— 更新
b —— 二进制文件（binary）

这些mode符，可以组合；如果其中不含有“b”，则表示以文本文件方式打开。



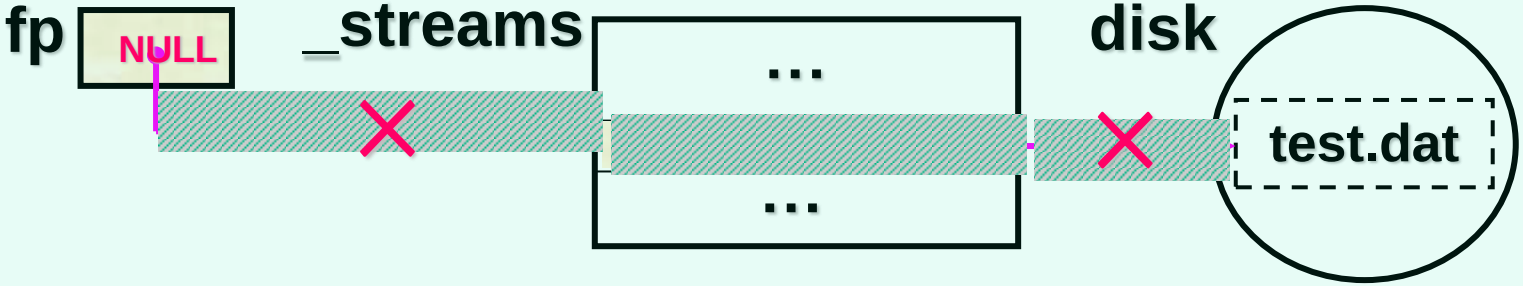
r	以读方式打开文本文件
w	以写方式打开或创建文本文件(清除原内容)
a	以写方式打开或创建文本文件(尾部添加)
rb	以读方式打开二进制文件
wb	以写方式打开或创建二进制文件(清除原内容)
ab	以写方式打开或创建二进制文件(尾部添加)

r+	以读写方式打开文本文件
w+	以读写方式打开或创建文本文件(清除原内容)
a+	以读写方式打开或创建文本文件(尾部添加)
r+b或rb+	以读写方式打开二进制文件
w+b或wb+	以读写方式打开或创建二进制文件(清除原内容)
a+b或ab+	以读写方式打开或创建二进制文件(尾部添加)

关闭文件函数fclose

```
int    _Cdecl  fclose(FILE *stream);
```

关闭文件指针stream“指向”的文件，
正常：返回0
出错：EOF（-1）。



基于字符的文本文件读写 fgetc和fputc

int fgetc(FILE *stream);

从文件指针stream联系的文件中读取一个字符，并将该字符作为unsigned char类型转换为整型值返回。到文件尾或读操作出错时返回EOF。

int fputc(int c, FILE *stream);

参数c转换成为unsigned char类型然后写到文件指针stream指向的输出流的当前位置处。返回值是被写字符；如果写操作出错或遇到文件尾函数返回EOF 。

FILE结构类型(记载文件的全部信息)

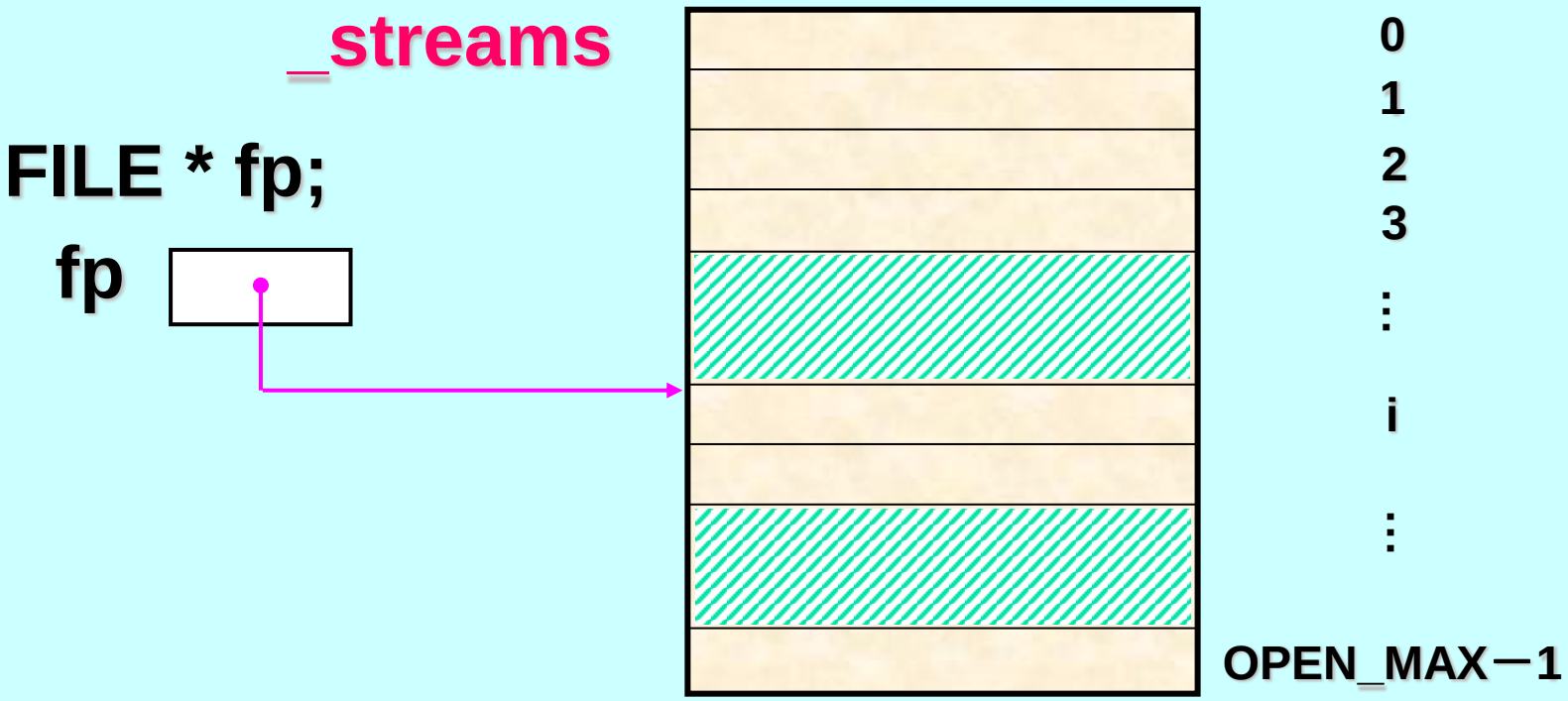
FILE是C语言标准中定义的一种结构类型名称。

```
struct _iobuf {
    char *_ptr;   /*当前读写位置指针，即缓冲区中偏移量*/
    int   _cnt;    /*缓冲区空满*/
    char *_base;     /*指向缓冲区基地址指针*/
    int   _flag;       /*文件状态标志*/
    int   _file;       /*文件描述符*/
    int   _charbuf;   /*非缓冲回送字符*/
    int   _bufsiz;     /*缓冲区大小*/
    char *_tmpfname;/*临时文件描述符*/
 };                  /* 结构类型命名       */
 typedef struct _iobuf FILE;
```

文件描述符是个简单的整数，用以标明每一个被打开的文件。第一个打开的文件是0，第二个是1，依此类推。

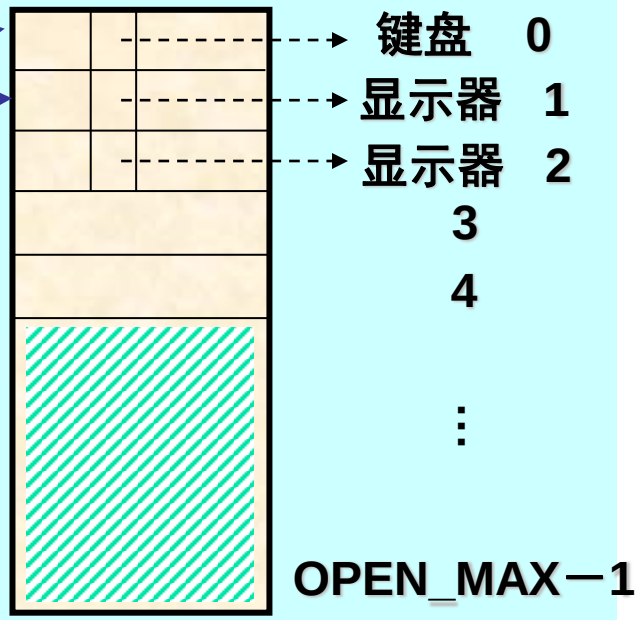
文件指针

程序中仅使用FILE*型变量，通常简称为FILE指针。



自动打开的三个标准流文件

```
#define stdin  (&_streams[0])
/*标准输入设备，通常是键盘 */
#define stdout  (&_streams[1])
/*标准输出设备，通常是显示器 */
#define stderr  (&_streams[2])
/*标准出错设备----显示器 */
```



stdin称为标准输入文件，stdout称为标准输出文件 。

getchar、gets和scanf函数是从stdin文件读数据；

putchar、puts和printf函数则是向stdout 文件写数据。

freopen函数可以重定向stdin和stdout。如，定向到磁盘文件。

文件的重定向函数freopen()

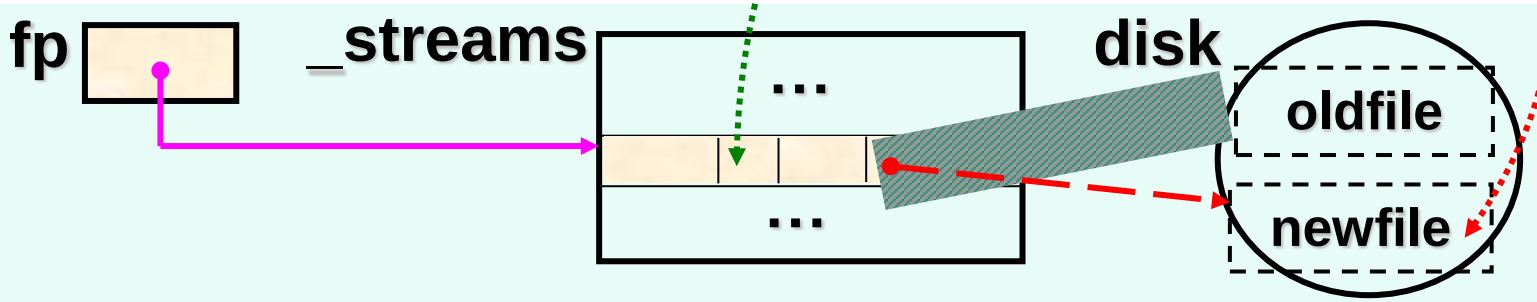
FILE * _Cdecl freopen(const char * filename,const char *mode,FILE *fp);

（1）fclose（fp）；

（2）fp=fopen（filename，mode）；

成功：返回fp

失败：返回NULL。



例2、读入d:\aaa.dat 文件中的内容在屏幕上显示出来

定义函数实现：从磁盘文件中读数据并显示

- 函数名： mytype
- 函数参数：

  filenane：待读数据的文件名， char *
- 函数返回值：无

```
/*  将filename文件中的内容在屏幕上显示出来  */
void mytype(const char * filename)
{
    char ch;
    FILE *fp;
    if((fp = fopen(filename,"r")) == NULL){
        printf("can't open the file!");
        exit(-1);
    }
    while( (ch = fgetc(fp)) != EOF)
        putchar ( ch );
    fclose(fp);
}
```

```
#include<stdio.h>
#include<stdlib.h>
void mytype(const char * filename);
int main(void)
{
    mytype ("d:\\a.dat");
   return 0;
}
```

文件类型

文件按照数据格式分为文本文件和二进制文件两类。

- 文本文件（ASCII文件）

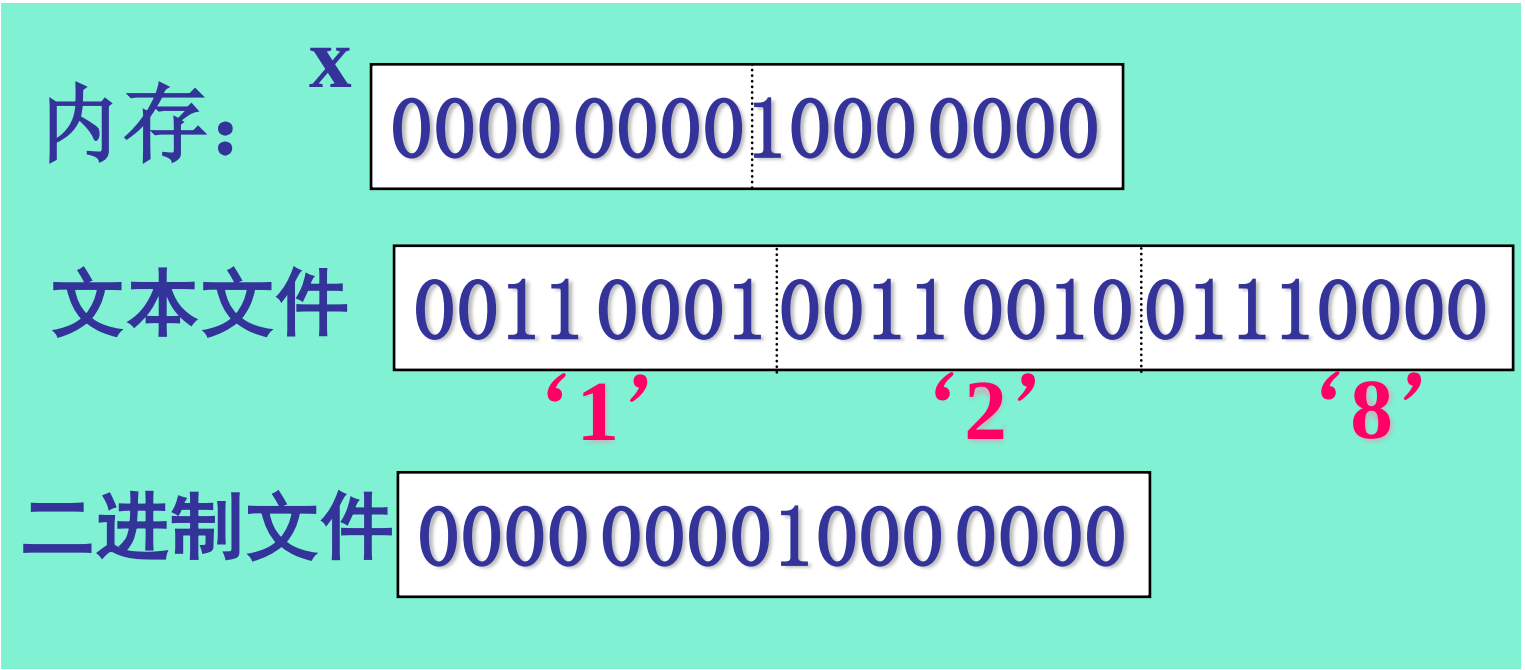
  由字符的ASCII码组成的数据文件（’\n’ -> ‘\r’ 和 ’\n’）。

- 二进制文件

  内存表示的数据序列组成的数据文件

文本文件和二进制文件的存储

短整数x=128以文本文件和二进制文件在磁盘上分别占多少字节？



文本文件的读写

- 基于字符：fputc函数和 fgetc函数
- 基于字符串：fputs函数 和 fgets函数
- 格式化：fprintf 函数和 fscanf函数

getchar和putchar由下面的宏定义。

```
#define getchar( )  getc(stdin)
#define putchar(c) putc((c), stdout)
```

基于字符串的文件读写

调用标准C 提供的字符串输入输出函数每次从输入文件中读或者向输出文件写一个字符串。

```
char * gets(char *s);
int  puts(const char *s);
char * fgets(char *s, int n, FILE *stream);
int  fputs(const char *s, FILE *stream);
```

char * fgets(char *s, int n, FILE *stream);

从stream指向的流中读一行（行长度<n）或至多读n-1个字符（行长度≥n）到s指向的字符数组中。fgets函数正常返回s，出错或遇文件尾返回NULL。

当遇到文件尾且无字符读入s指向的数组时，数组保持原有值不变。

```
char s[10];
fgets（s，10，stdin）；/* 从键盘输入*/
```

int fputs(const char *s, FILE *stream) ;

将指针s 指向的字符串写入stream指向的流中，终结符不写。函数返回一个非负值；如果写操作出错，fputs返回EOF 。

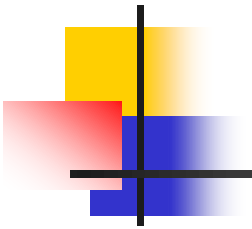
fputs函数与fgets配对使用。

```
fputs（s，stdout）；
```

文件的格式读写

```
int  printf(const char *format, ...);
int  scanf(const char *format, ...);
int  fprintf(FILE *stream, const char *format, ...);
int  fscanf(FILE *stream, const char *format, ...);
int  sprintf(char *buffer, const char *format, ...);
int  sscanf(const char *buffer, const char *format, ...);
```

输出函数正常时都返回输出字符个数，写操作出错时返回负值。输入函数时都返回已输入项数，如果读操作出错都返回EOF。



三对函数的共同特点是它们的功能相同，输入输出都受format指针所指的格式字符串控制， 且使用方法与第3章介绍的printf、scanf用法相同。

三对函数的相异之处在于它们输入输出的对象不同。

教材【例10.6】 文本文件的分解。

- 将一个大的文本文件以行为单位分解成为若干个较小的文本文件，文件名和分解的行数都由用户从命令行输入。

/*将文件abc.dat分解为a.dat，b.dat 和c.dat三个文件，分解的行数为10*/

parts abc.dat a.dat b.dat c.dat 10

演示：parts.exe

parts abc.dat a.dat b.dat c.dat 10

（1）len=命令行中分解的行数

atoi（argv[argc-1]）

（2）以读方式打开源文件

fopen（“argv[1]”，“r”）

（3）依次打开目标文件，从源文件读len行写入目标文件（命令行中给出几个源文件？）

for（i=2；i<argc-1;i++){

- 以写方式打开文件argv[i]
- 从argv[1]读1行写入argv[i]直到写了len行或源文件到文件尾。
- 关闭 文件argv[i]

}

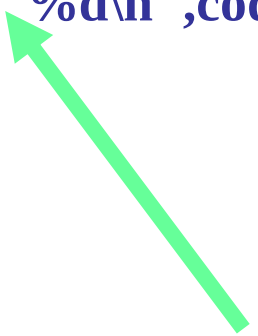
（4）关闭源文件argv[1]

\源程序\ parts.c

教材【例11.9】 编写一个数据采集和处理程序。暂定数据从键盘输入手工采集，采集某种商品到货批次（code）和数量（number）。

```
#include<stdio.h>
#include <stdlib.h>
#define True   1
void data_write(char *);        /*  数据采集并存盘*/
void data_cal(char * ,float );    /*从文件读入数据并进行计算*/
void main(void)
{
    char a[20] = "data.dat";
    data_write(a);
    data_cal(a,  31.4);     /* 31.4是单价 */
}
```

```
/*data_write：从键盘采集某种商品到货批次和数量，存于filename文件中
   */
void data_write(char * filename)
{
    FILE * out;
    char code[5];
    int number = 1;
    if((out = fopen(filename,"w")) == NULL)
        exit(-1);
    printf("input code and number please!\n");
    while( True ){
        scanf("%s%d",code, &number); /* 输入批次，数量 */
        if(number == 0) /* 如果数量为0，结束输入 */
            break;
        fprintf(out,"%s  %d\n",code,number);
    }
    fclose(out);
}
```



必须有空格，便于能够正确进行读

```
/*data_cal: 从filename文件中读入某种商品到货批次和数量，按单价price计算
    总金额，并显示计算结果 */
void data_cal(char * filename,float price)
{
    FILE * in;
    char code[5];
    int number;
    if((in = fopen(filename,"r")) == NULL)
        exit(-1);
    printf("batch\tamount\tsum \n");
    while( fscanf(in,"%s %d ",code,&number) != EOF {
        /* fscanf函数从in指向的流中读数据到code和number中 */
        printf("%s\t%d\t%8.2f\n",code,number,price*number);
    }
    fclose(in);
}
```

演示\源程序\ ex10-9.c

文本文件数据的间隔符

- 当将内存多个数据写入文本文件的时候，特别注意的是：在两个数据之间增加必需的**间隔符**。
- 目的在于读文本文件时，能够保证获得数据转换的正确性。

```
fprintf(out,"%s   %d\n",code,number);
```

文件的直接输入输出-二进制文件的读写

文件直接输入输出又称为**文件成组输入输出**。标准C为文件的直接输入输出提供了两个函数fread和fwrite，**适用于二进制形式文件的读写**。

```
size_t  fread(void *ptr, size_t size, size_t n, FILE *stream);
size_t  fwrite(const void *ptr, size_t size, size_t n,FILE *stream);
```

size_t fread(void *ptr, size_t size, size_t n, FILE *stream);

从stream指向的输入文件中读取至多n个大小为size的记录到指针ptr指向的内存单元中。返回值是实际读取记录数。

文件尾测试函数feof()，如果到文件尾，函数返回非0值，否则返回0。 ferror()测试出错函数，如果出错，ferror函数返回非0值，否则返回0。

size_t fwrite(const void *ptr, size_t size, size_t n,FILE *stream);

从指针ptr指向的内存缓冲区中取n条，大小为size的记录写到stream指向的输出文件中。返回值是写出的记录数， 当返回值小于n时，这种情况只有在写操作出错时出现。

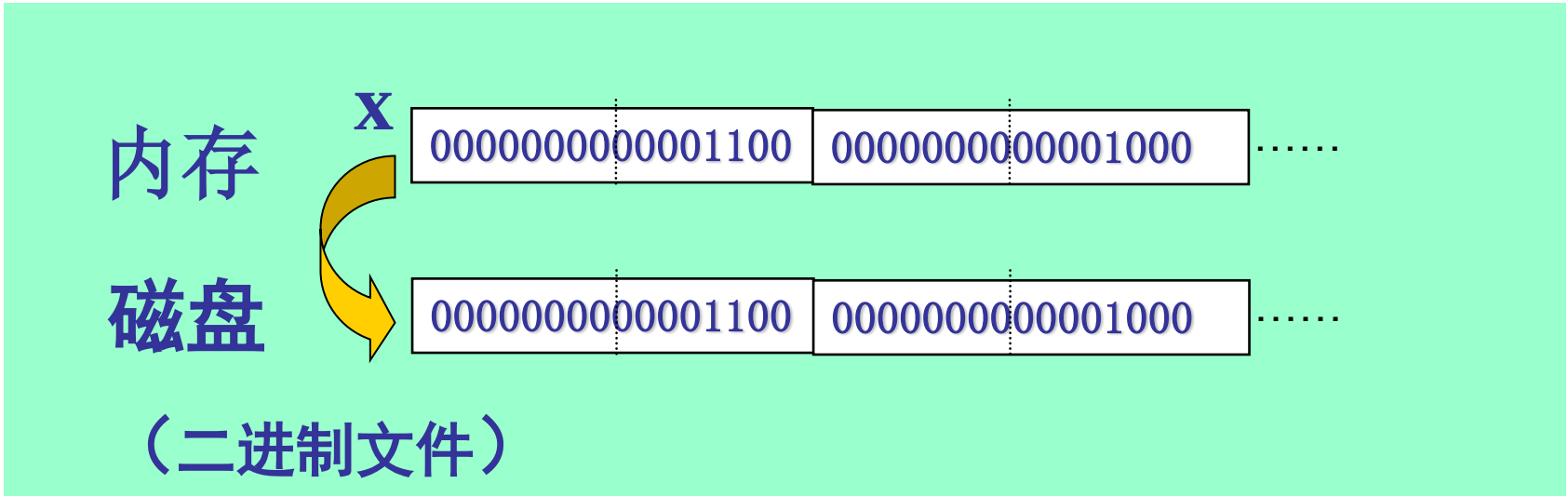
参数的含义同fread， 不同的是stream指向的输出文件。

如何将int x[]={12, 8, 34, 421}输出到磁盘（二进制文件）？

(1) fwrite(x, sizeof(int), 4, fp);

(2) for(i=0;i<4;i++)

fwrite(&x[i], sizeof(int), 1, fp);



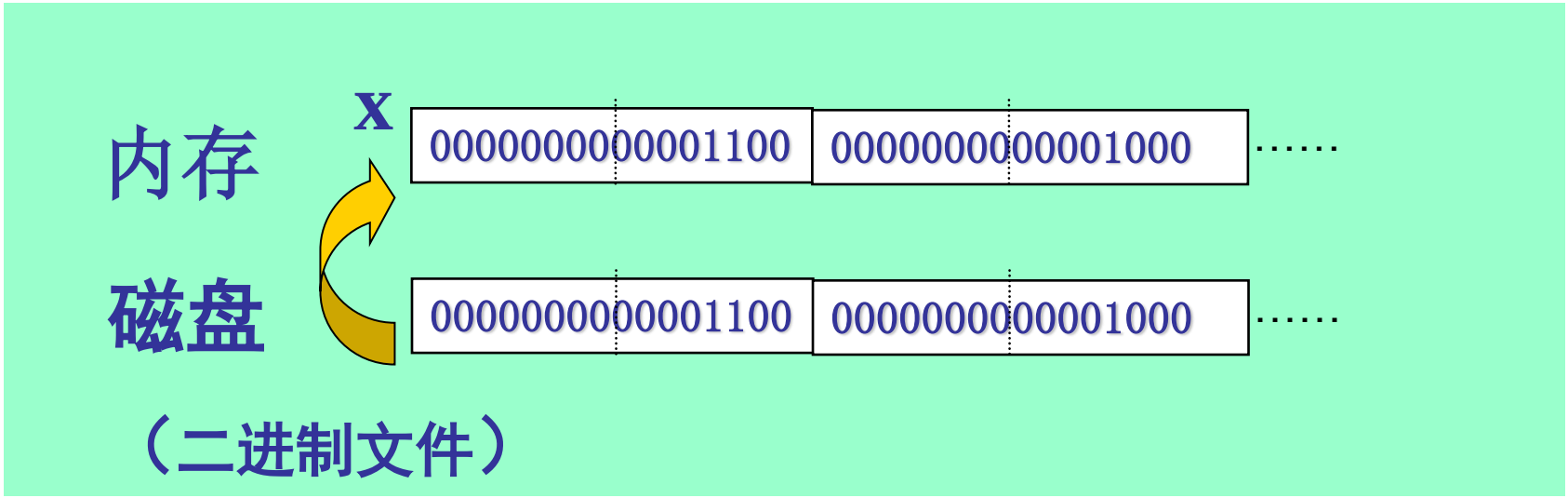
二进制文件数据不需要间隔符

如何从磁盘文件（二进制文件）读数据到内存？

```
int  fread(void *ptr, size_t size, size_t n,
           FILE *stream);
```

——— 二进制文件

```
int i=0, x[10];
while( fread(x+i,sizeof(int), 1,fp) ==1)  i++;
```

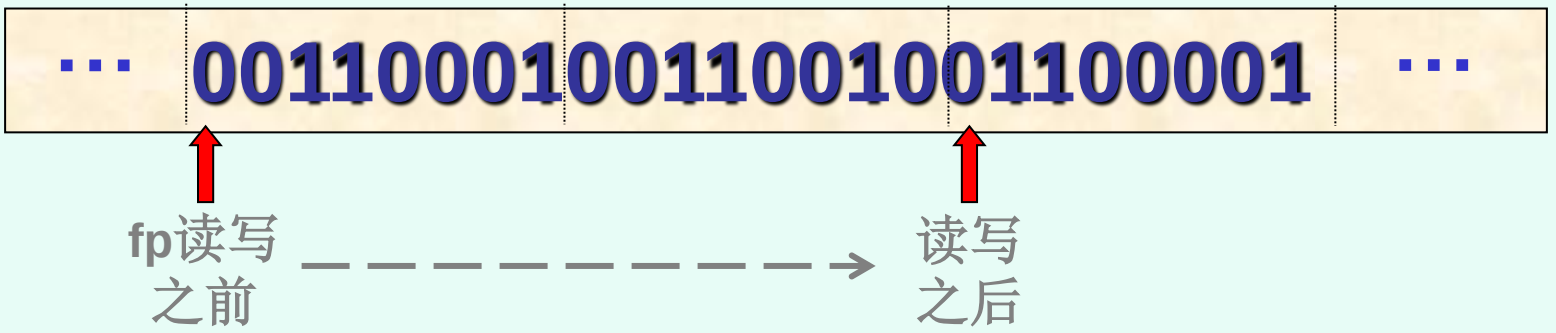


feof()

文件尾测试函数feof()，如果到文件尾，函数返回非0值，否则返回0。 ferror()测试出错函数，如果出错，ferror函数返回非0值，否则返回0。

```
#define  _IOEOF  0x0010

#define  feof(_stream)  ((_stream) ->_flag & _IOEOF)
```

二进制文件数据不需要间隔符



按二进制格式，读入1个int数据：

```
int x;
fread(&x, sizeof(int), 1, fp);
```

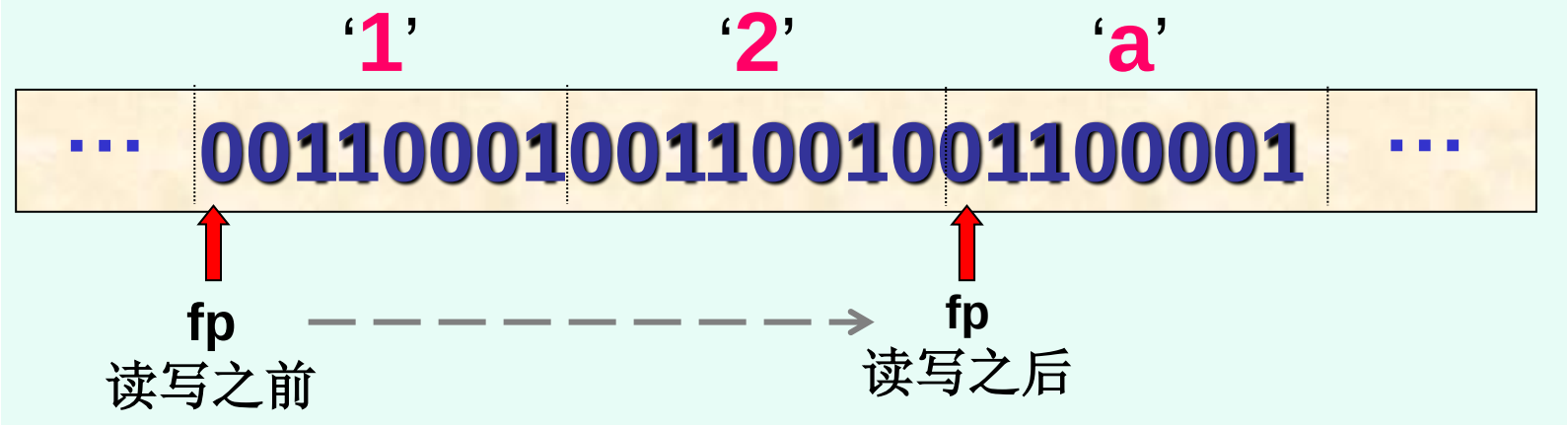
X:12594

文件的读写方式

C语言中，对文件的读写方式有两种：**顺序读写**和**随机读写**。

文件的**顺序读写**：**打开文件**时，读写指针指向**文件头**；读写一个“数据”后，读写指针自动指向下一个“数据”；**程序不用修改读写指针**。数据是从文件头到文件尾，顺序读写。

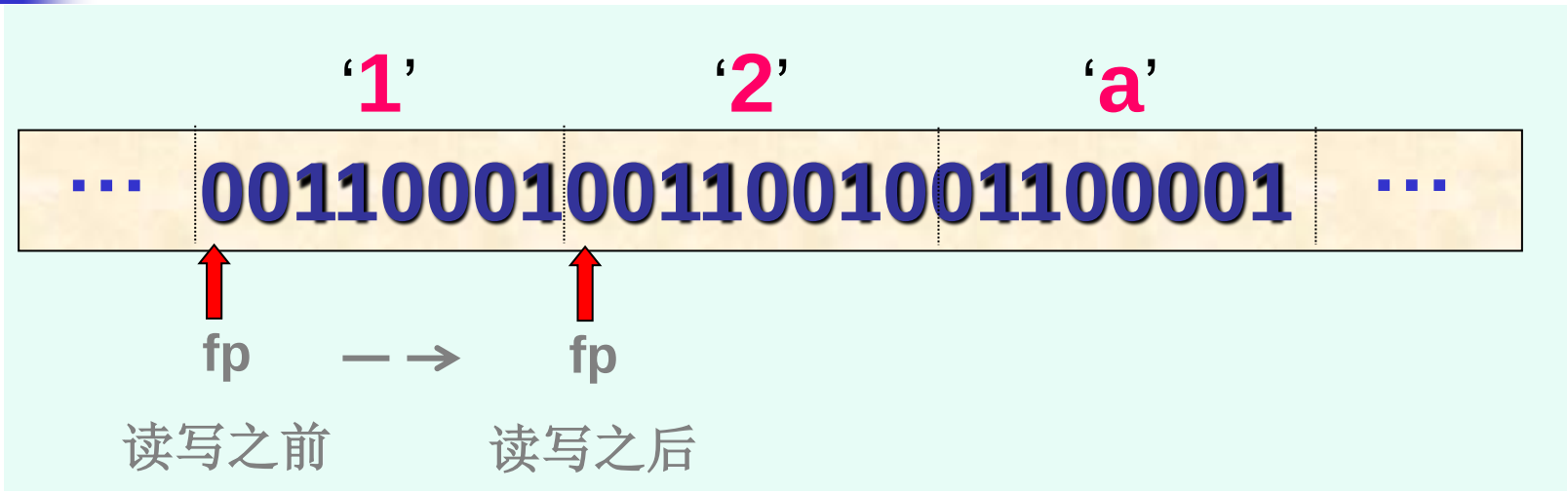
顺序读写



按文本格式，读入1个int数据：

fscanf(fp,”%d”,&x);

x:12， 内存中为：00000000 00001100



按文本格式，读入1个char数据：

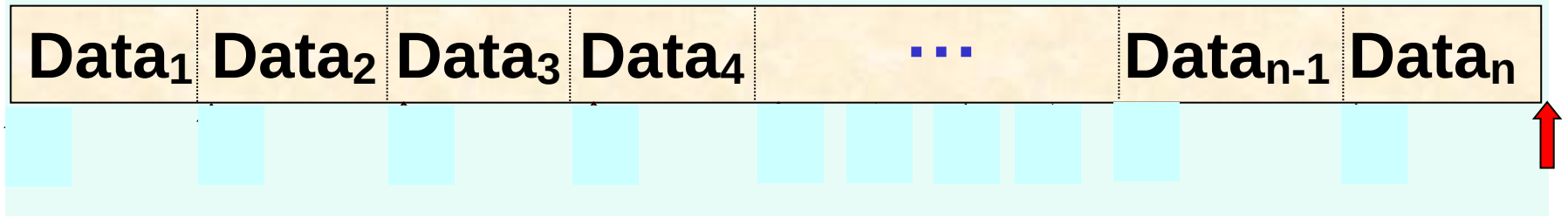
fscanf(fp,”%c”,&x);

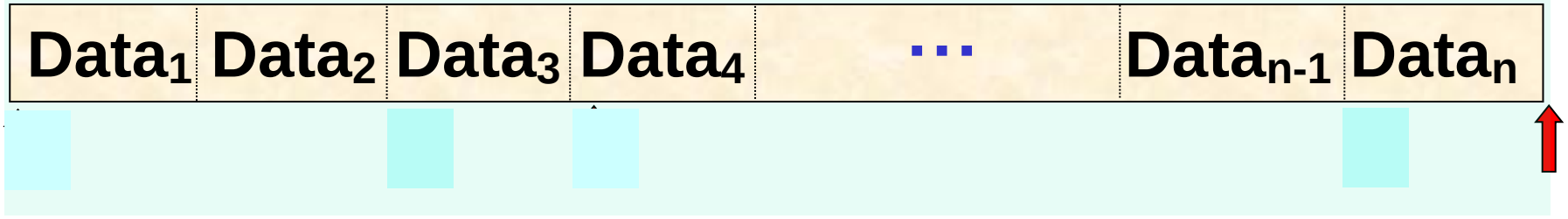
X: ‘1’，内存中为： 00110001

文本文件只能顺序读写
二进制文件能顺序读写，也能随机读写

文本文件（用fprintf建立）数据不一定具有同样的长度

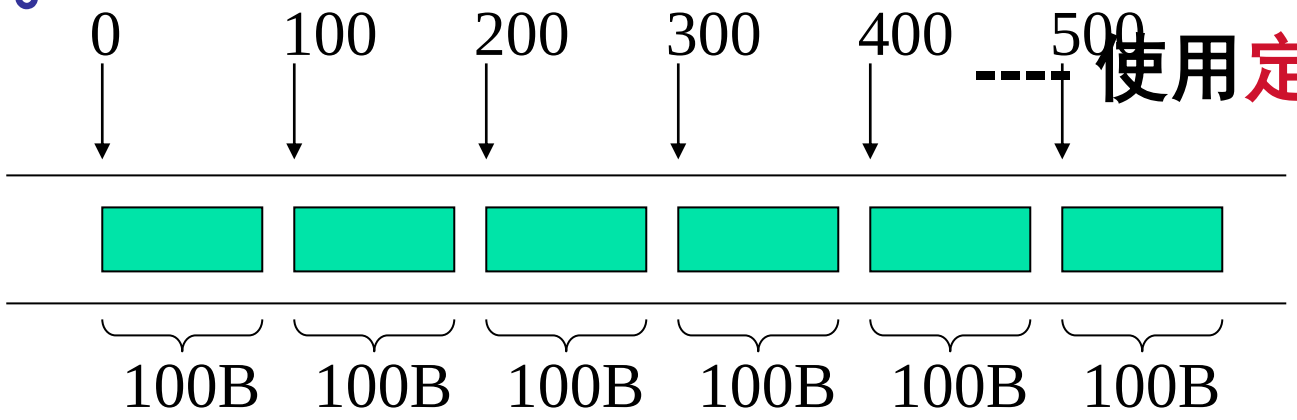
二进制文件（用fwrite建立）数据具有同样的长度





随机读写

随机读写方式：打开文件时，读写指针指向文件头；读写一个“数据”后，读写指针自动指向下一个“数据”；程序能改变读写指针。数据不必从文件头到文件尾顺序读写。



具有定长记录的随机存取文件

流式文件的随机输入输出

1. 文件定位函数

流式文件的随机输入输出是指对文件的读写可以从文件内指定的位置进行，而不必每次从头顺序开始。

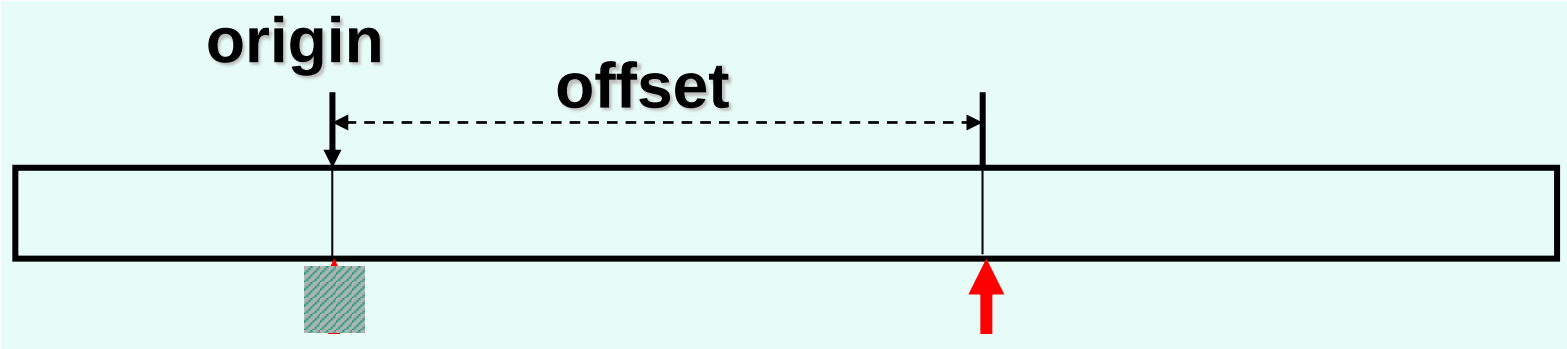
```
int fseek(FILE *stream, long offset, int origin);
long ftell(FILE *stream);
int fgetpos(FILE *stream, fpos_t *pos);
int fsetpos(FILE *stream, const fpos_t *pos);
void rewind(FILE *stream);
```

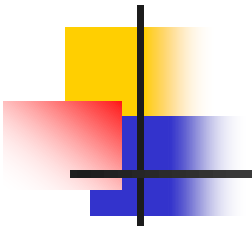
fseek

int fseek(FILE *stream, long offset, int origin);

将stream指向的文件之当前读写指针，定位到origin+offset。如果正常定位，fseek函数返回0，否则返回非0值。 。

origin和offset分别称为基准点和偏移量。





基准点有三种取值：SEEK_SET、SEEK_CUR和SEEK_END

#define SEEK_SET 0 文件起始位置为基准点

#define SEEK_CUR 1 文件当前位置为基准点

#define SEEK_END 2 文件尾部位置为基准点

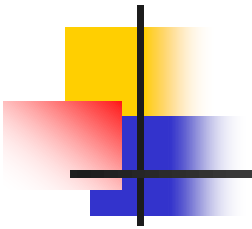
C标准规定：对二进制文件，基准点的取值与偏移量无关；对文本文件基准点只能取0，而偏移量可以取0或ftell函数的返回值。

long ftell(FILE *stream);

读取文件读写指针的当前值返回，出错时返回-1L。

int fgetpos(FILE *stream, fpos_t *pos);

将stream指向文件的读写指针当前值，保存到 pos指针所指的 fpos_t类型的对象中。成功保存，fgetpos返回0，否则返回非0值。



int fsetpos(FILE *stream, const fpos_t *pos);

用fgetpos函数保存到pos指针所指对象中的值来设置stream所指向文件的当前位置。设置成功函数返回0，否则返回非0值。

void rewind(FILE *stream);

将文件指针stream指向文件的读写指针重新定位到文件的起始位置，同时清除文件结束标志和出错标志。

2. 文件随机读写

利用文件的定位函数和文件的读写函数，即可实现文件的随机读写。

显然，文件中的数据应该按照某种规律进行组织存储，才便于程序按照这种规律对文件进行读写指针的定位。

定位函数中参数值，依赖于具体的编译程序之约定。在软件开发时，需要认真阅读所使用的开发工具的相关技术资料！

【例10.14】用随机读写方式编制填写班级成绩单文件的程序。

- 假设某班级的学生参加了数学（Math）、物理（Physics）、英语（English）C语言程序设计（C）四门课程的考试，学生试卷按交卷顺序随机排列，然后在试卷不按照学号排序的方式下进行阅卷、登分。
- 分析：由于学生交卷先后顺序有随机性，且各门课程的登分先后也会有随机性，因此比较合理的方式是根据班级学生人数按照学号顺序建立一张空的成绩单文件，然后根据试卷登分时的情况选用单科成绩登分或四科成绩同时登分。随机定位的依据是试卷上学生的学号。
- 源程序\ex10_14.c

例10.18 （使用底层接口函数）读取指定汉字在16×16点阵汉字库中的点阵字模信息，并将汉字点阵字模信息以文本方式显示。

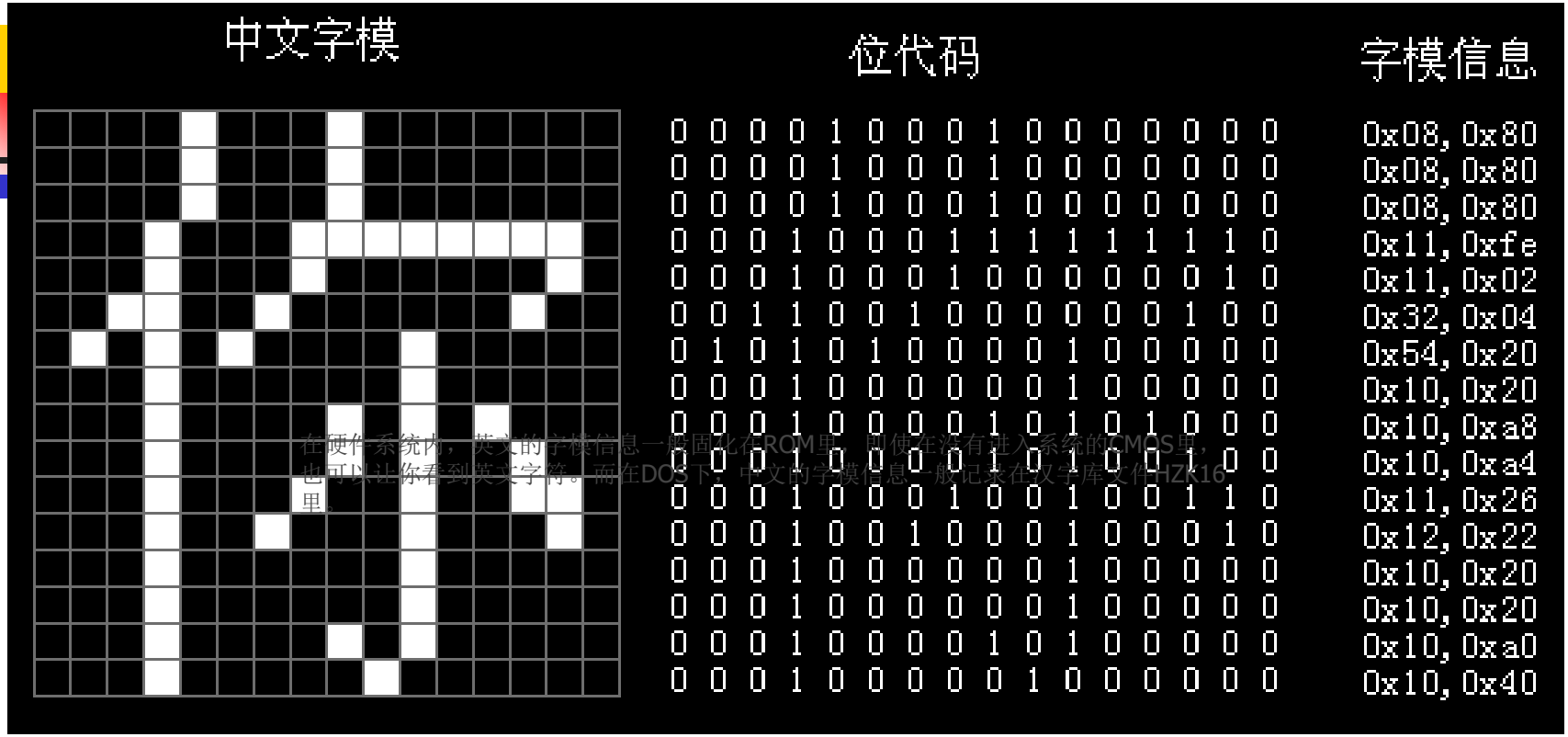
汉字的内码：

中文或西文信息在计算机系统中的代码表示称为机内码。ASCII码是一种西文机内码（1B）。汉字机内码用连续2B表示，每个字节的最高位是1。

机内码 = 国标码+8080H =区位码$_{16}$+A0A0H

- 国标码（2121H～7E7EH）
- 区位码（0101～9494）
- 机内码（A1A1H～FEFEH）
- 汉字内码仅是一组数字，又如何在屏幕上去显示呢？这就涉及到文字的字模，字模也是一组数字，但它的意义却与数字的意义有了根本的变化，它是用数字的各位信息来记载英文或汉字的形状.

汉字字模



在硬件系统内，英文的字模信息一般固化在**ROM**里。而在**DOS**下，中文的字模信息一般记录在汉字库文件**HZK16**里。如何得到汉字的字模信息。

汉字库文件（HZK16）

- HZK16是16*16。还有HZK24，HZK32和HZK48字库及黑体、楷体和隶书字库。它们都是按照区位的顺序排列 的。
- 汉字在汉字库中的具体位置（序号：从0开始）

  index=94*(区号-1)+位号-1。
- 汉字点阵字模在汉字库中的位置（B）：

  index*一个汉字字模占用字节数

源程序\ex10_18_stream.c

其它文件操作函数(自学）

1. 文件访问函数

C标准规定了五个文件操作函数如下，其中前两个已经讨论过。

```
FILE * _Cdecl fopen(const char * filename, const char * mode);
int    _Cdecl  fclose(FILE *stream);
int   fflush(FILE *stream);
int   setvbuf(FILE *stream, char *buf, int mode, size_t size);
void  setbuf(FILE *stream, char *buf);
```

11.5 其它文件操作函数

int fflush(FILE *stream);

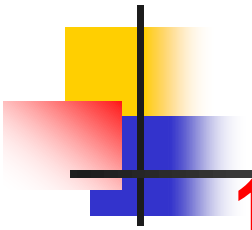
将stream指向的输出缓冲区或更新缓冲区（用追加方式打开）中尚未存盘的数据强制性的全部写到介质文件中。如果stream为NULL，执行fflush操作将使所有缓冲区的数据存盘。写操作成功，fflush函数返回0，否则置出错标志并返回EOF。

11.5 其它文件操作函数

int setvbuf(FILE *stream, char *buf, int mode, size_t size);

在打开文件之后，其他任何有关文件的操作被执行之前调用；setvbuf函数用buf指向的存储区作为stream指向流的缓冲，缓冲方式由mode参数决定，缓冲大小由size决定。调用成功，函数返回0，否则返回非0值。

标准C为mode规定了三种模式：_IOFBF(全缓冲)、_IOLBF(行缓冲)和_IONBF(无缓冲)。



11.5 其它文件操作函数

void setbuf(FILE *stream, char *buf);

功能与 setvbuf函数完全一样。不同的是该函数采用全缓冲(_IOFBF)，缓冲大小用BUFSIZ，函数没有返回值。

缓冲大小值BUFSIZ，在stdio.h定义，一般为512个字节。

11.5 其它文件操作函数

2. 文件操作函数

int remove(const char * filename);

int rename(const char * oldname, const char * newname);

FILE * tmpfile(void);

char * tmpnam(char *s);

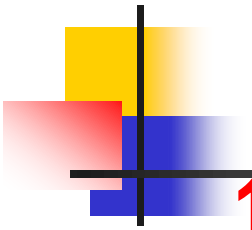
11.5 其它文件操作函数

int remove(const char * filename);

删除以filename指向的字符串为文件名的文件。如果操作成功，remove函数返回0，否则返回非0值。

如果对一个已经打开的文件进行remove操作，实际结果由各个C编译器自行决定。

实际上，remove函数就是DOS的del操作之功能。



11.5 其它文件操作函数

int rename(const char * oldname, const char * newname);

以oldname指向的字符串为文件名，更换成以newname指向的字符串为文件名。如果操作成功，函数返回0，否则返回非0值。

实际上，rename函数就是DOS的ren操作之功能。

11.5 其它文件操作函数

FILE * tmpfile(void);

以"wb+"方式创建一个临时二进制文件，该临时文件在文件关闭或程序结束时被自动清除。如果调用成功，tmpfile函数返回一个指向它所创建的临时文件的文件指针，否则返回NULL。

char * tmpnam(char *s);

产生一个与系统中已存在的文件不相同的文件名。当参数非空，函数将产生的文件名存入s指针指向的字符数组，并将s的值作为返回值；如果操作失败，返回一个空指针。

11.5 其它文件操作函数

3. 出错检测处理函数

```
void clearerr(FILE *stream);
int feof(FILE *stream);
int ferror(FILE *stream);
void perror(const char *s);
```

11.5 其它文件操作函数

clearerr函数清除stream 所指流的EOF标志和出错标志，无返回值。

feof函数检测stream所指流是否到达尾。如果到达，返回非0值，否则返回0值。

ferror函数检测stream所指流的出错，如果出错，ferror函数返回非0值，否则返回0值。

perror函数将出错信息(全局变量errno的值)写入字符指针s所指的字符数组中，无返回值。

11.6 输入输出的低层接口(io.h)

与流式文件使用标准C 定义的基于缓冲的函数不同，输入输出的低层接口中使用的函数不是由标准C 定义，而是通过操作系统提供的功能调用(也称为接口)实现。

流式文件使用通过FILE结构定义的缓冲，而输入输出的低层接口不使用FILE文件指针，所使用的是整型文件说明符。

对于不同的操作系统，编译器提供的低层接口形式是统一的。差异之处在于对读写方式、文件属性的规定和符号常量的定义有所不同。

11.6 输入输出的低层接口

1. 文件的顺序输入输出

```
int  creat(const char *path, int amode);
int  open(const char *path, int access,
unsigned mode);
int  read(int handle, void *buf, unsigned
len);
int  write(int handle, void *buf, unsigned
len);
int  close(int handle);
```

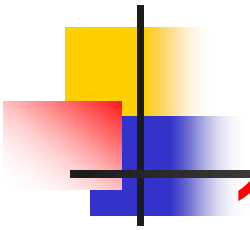
11.6 输入输出的低层接口

2. 文件的随机输入输出

long lseek(int handle, long offset, int origin);

long tell(int handle);

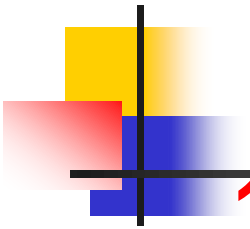
int eof(int handle);



11.7 流式文件系统中库函数的设计

1. fopen函数的设计

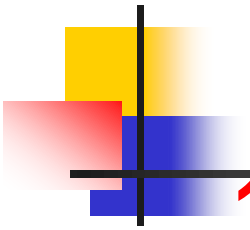
（略）



11.7 流式文件系统中库函数的设计

2. mygetfp函数和my_openfp函数的设计

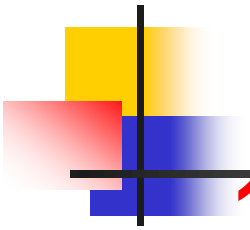
（略）



11.7 流式文件系统中库函数的设计

3. myCheckOpenType函数的设计

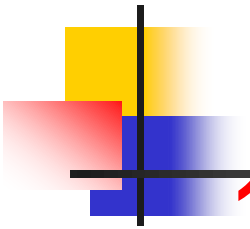
（略）



11.7 流式文件系统中库函数的设计

4. myfread函数的设计

（略）



11.7 流式文件系统中库函数的设计

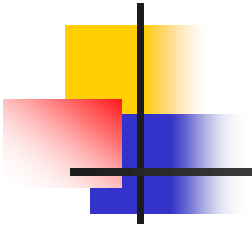
5.myfwrite函数的设计

（略）

11.7 流式文件系统中库函数的设计

6. 使用自己编写的库函数

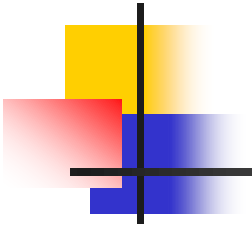
（略）



本章小结

首先介绍了文件的有关概念，包括了文本文件和二进制文件的含义，顺序文件和随机文件的文件的读写方式，以及C程序输入输出的实现方法。

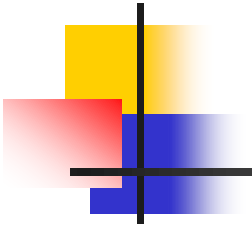
在FILE指针和标准流式文件方面重点介绍了FILE结构类型和FILE指针，标准流式文件，这些概念要熟练掌握。



本章小结

在流式文件的顺序输入输出方面介绍了文件的打开与关闭，文件的重定向，基于字符和基于字符串的文件读写，以及文件的格式读写和文件的直接输入输出。

在流式文件的随机输入输出方面，介绍了文件定位函数，并以随机读写方式编写填写班级成绩单文件的程序介绍了流式文件的随机读写。



本章小结

其次，对C标准规定的其它文件操作函数，如文件访问函数、文件操作函数、出错检测处理函数进行了介绍。

介绍了输入输出的低层接口中的文件顺序输入输出和文件的随机输入输出。

最后，介绍了在流式文件系统中，库函数的设计方法。

C语言与程序设计

The C Programming Language

第11章 复杂类型的指针

毛伏兵

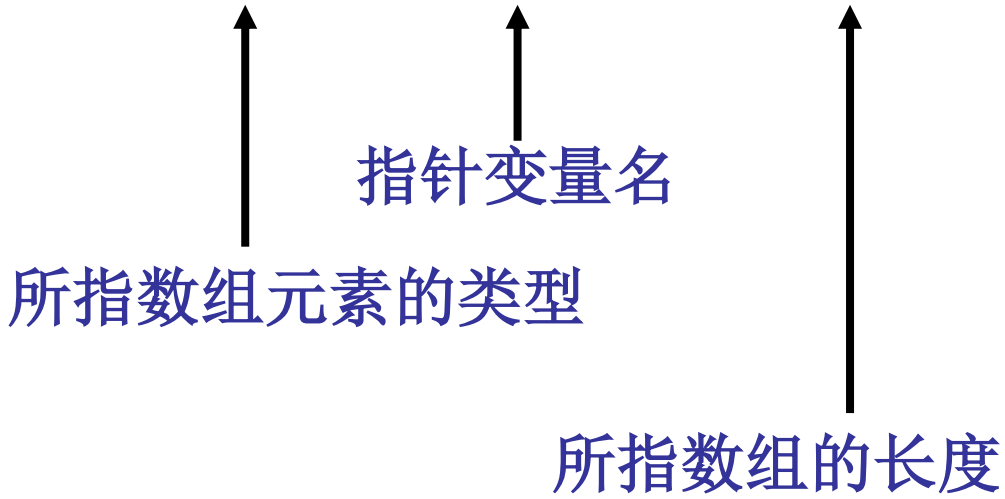
华中科技大学计算机学院

11.1 指向数组的指针

11.1.1 指向数组的指针的声明与定义

指向数组的指针又称为数组的指针。

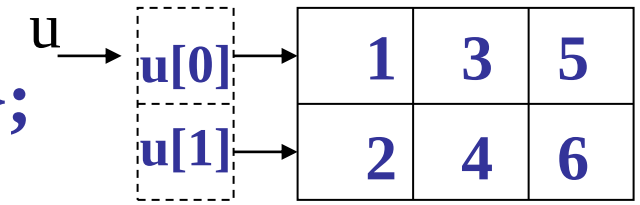
类型名 （*标识符）[常量];



11.1.2 用数组名间访二维数组的元素

二维数组被看成以1维数组（行）为元素的一维数组；

```
int u[2][3]={{1，3，5}
             {2，4，6} };
```



u被看成有两个1维数组（行）元素(u[0],u[1])组成的一维数组

u[0] 第0行首地址 即 u[0]= = &u[0][0]

u[1] 第1行首地址 即 u[1]= = &u[1][0]

i行j列的元素的地址？

（1）用指向数组元素的指针表示：u[i]+j

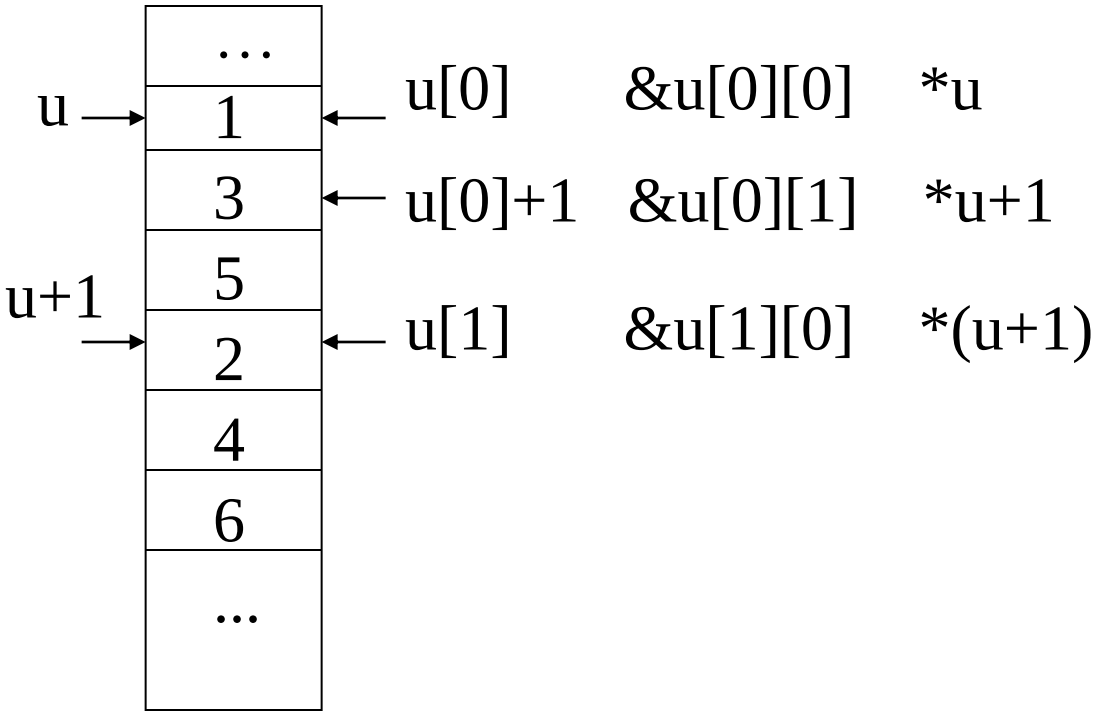
（2） 用指向一维数组的指针表示： *(u+i)+j

如何间访i行j列的元素？

（1）*(u[i]+j） （2）*(*(u+i)+j）

用指针变量描述二维数组元素

```
int u[2][3]={
   {1，3，5}
   {2，4，6}
};
```



1、将指针定义为指向数组元素

见8.4.3

2、将指针定义为指向由m个元素组成的数组的指针

```
int a[3][2]={{1，3}，{4，6}，{7，9} };
int （*p)[2];
```

p是指向数组的指针，该数组有2个int型元素

```
p=a;
```

*(*p) 或 (*p)[0]	*(*p+1) 或 (*p)[1]
((p+1)) 或 (*(p+1))[0]	*(*(p+1)+1)或(*(p+1))[1]
((p+2)) 或 (*(p+2))[0]	*(*(p+2)+1)或(*(p+2))[1]

例　二维数组元素的输入/输出

```
#include <stdio.h>
#define I 2
#define J 3
void main(void)
{    int  u[I][J], (*p)[J]=u;
     int j;
     for(j=0;j<J;j++)     /* 用指向数组元素的指针完成第0行元素的输入 */
          scanf("%d",(u[0]+j));
     for(p++,j=0;j<J;j++)  /* 用指向数组的指针完成第1行元素的输入 */
          scanf("%d",(*p+j));
     for(j=0;j<J;j++) /* 用指向数组的指针完成第0行元素的输出 */
          printf("%6d",*(*u+j));
     printf("\n");
     for(j=0;j<J;j++)     /* 用指向数组元素的指针完成第1行元素的输出 */
          printf("%6d",*(u[1]+j));
     printf("\n");
}
```

11.1.4 二维数组作函数参数

- 形参说明为数组

```
fun(int x[ ][4])
 {
   ...
 }
```

- 形参说明为指针
  - 指向数组元素的指针
  - 指向下一级数组的指针

```
fun(int *x)
 {
   ...
 }
```

```
fun(int (*x)[4] )
 {
   ...
 }
```

11.2 用typedef定义类型表达式

11.2.1 类型表达式

C中的表达式可以分成两类。

(1)值表达式，由运算符和操作数组成，可被CPU处理和计算

(2)类型表达式,由类型说明符和数据类型名组成,类型说明符有：()、[]、*

```
int (*)[5]
```

2) typedef定义中

typedef是关键字，为一个类型表达式定义一个别名。

typedef 类型区分符 说明符；

基本类型 结构 联合
也可以是由 typedef 定义的类型名

(1) typedef unsigned int size_t;

size_t定义为unsigned int类型

```
size_t  x,y; /* unsigned int x,y; */
```

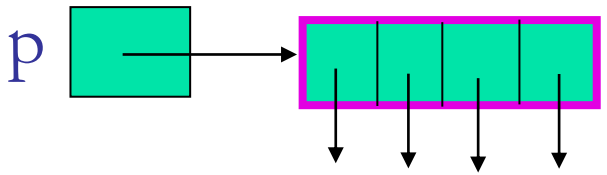
(4) typedef char * (*p_to_fun)(char * ,char *);

p_to_fun 定义为 char *(*)(char * ,char *)

p_to_fun fptr;

11.3 复杂说明的解释

unsigned *(*p)[4]

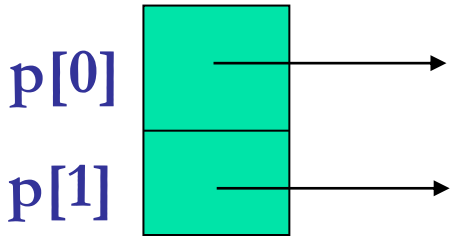


p是指向数组的指针，
该数组有4个无符号整型指针元素。
或

p是有4个无符号整型指针元素的数组的指针。

char *(*p[2])(char *, int);

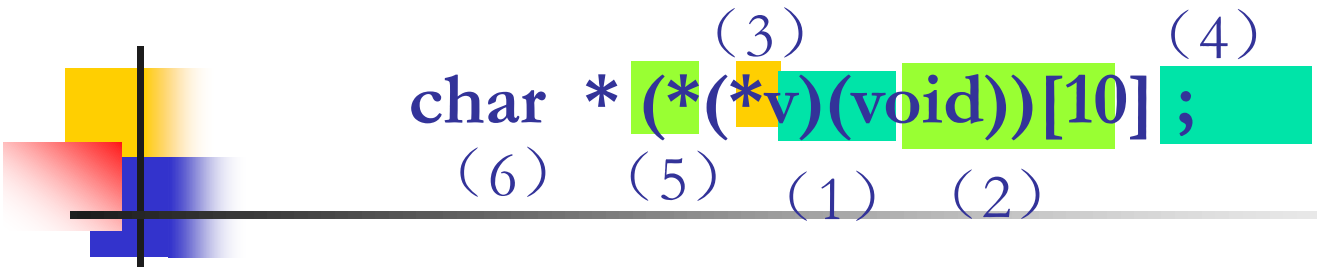
p是含有2个指针元素的数组，每个指针指向有一个字符指针参数和一个整型参数，返回值为字符型指针的函数。



函数： char *(char *, int)

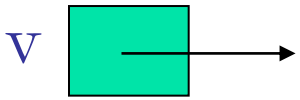
例11.7 有两个元素的函数指针数组的复杂说明的使用举例。

```
#include "stdio.h"
int a1(int);
int a2(int);
int main(void)              /*fpa是有2个元素的函数指针数组，每个元素指向*/
{   int i,(*fpa[2])(int);/*的函数有一个整型形参，返回整型值*/
    fpa[0]=a1;    /*第1个（下标为0）元素指向a1函数*/
    fpa[1]=a2;    /*第2个（下标为1）元素指向a2函数*/
    for(i=0;i<2;i++)
      printf("%d\n",fpa[i]((i+1)*5));/*依次调用fpa[0]、fpa[1]所指函数*/
    return 0;
}
int a1(int x){ printf("in function a1,x=%d\n",x); return 2*x; }
int a2(int y){ printf("in function a2,y=%d\n",y);  return 2*y; }
```

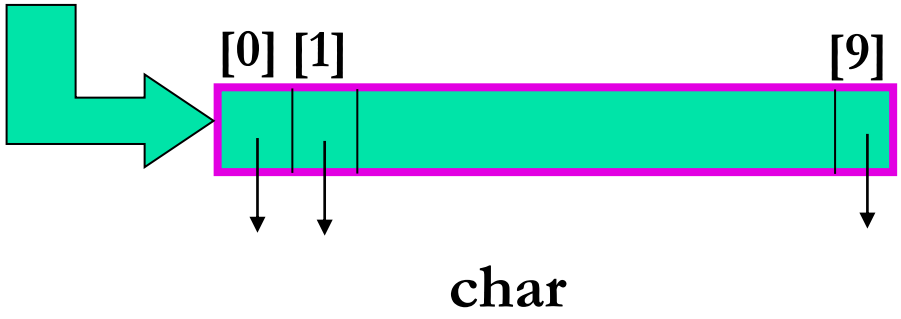


v 是函数的指针，该函数没有参数，

返回值是指向有10个元素的字符指针数组的指针。



无参函数
函数的返回值是指针类型



int (*f(char *(*)(int)))[10];

f是一个指针函数,

*f*函数的形参为一个指向函数的指针，所指函数有一个整型形参且返回值类型为char *;

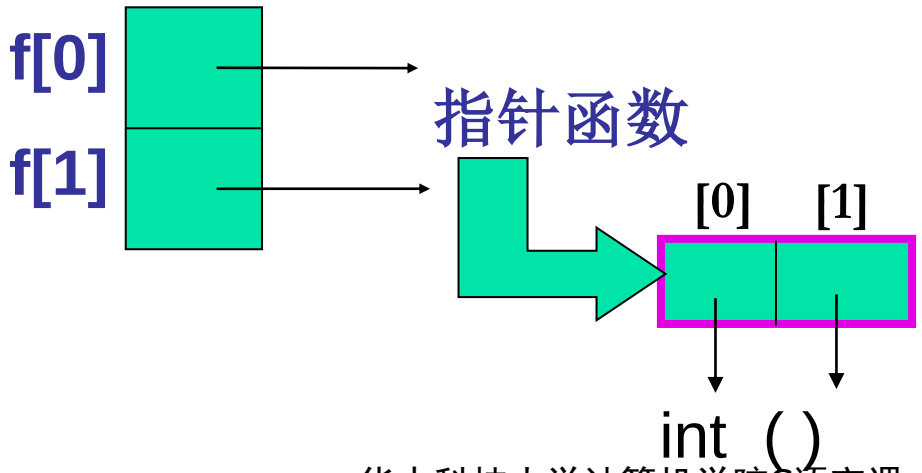
f函数的*返回值是指向有10个整型元素的数组的指针。*

f 的形参： char *(*)(int)

f是一个指针函数

int (*(*(*f[2])())[2])();

f是一个有两个元素的函数指针数组；所指函数的返回值是指向有两个元素的指针数组的指针，指针数组的每个指针元素指向一个无参整型函数。（无参整型函数指函数没有参数且返回值为整型值）。



**11.4 复杂说明的应用

- 例11.6 设p是一个函数指针，所指向的函数无参，且返回值为一个函数指针；该（返回的）函数指针指向的函数有两个字符指针形参，且返回值为字符指针值。请写出该声明语句并编写使用p的应用程序。
- 相应的声明语句为：

```
char *(*(*p)(void))(char *,char *);
```

```
#include "stdio.h"
char *(*(*p)(void))(char *,char *);/*f1是指针函数，返回的指针指向有两个*/
char *(*f1(void))(char *,char *);/*字符指针形参，返回值为字符指针值的函数*/
char *strcpy(char *,char *);/*有两个字符指针形参，返回值为字符指针值的函数*/
int main(void)
{   char a[80],b[]="aaa,bbb,ccc",*str;
    p=f1;/*p指向f1，下面语句先计算(*p)()，即f1函数*/
    str=(*p)()(a,b);/*返回strcpy；然后调用strcpy(a,b)，结果赋给str*/
    printf("%s\n",str);
    return 0;
}
char *(*f1(void))(char *,char *)
{   /*f1(void)表示f1是无参函数，其余部分都是描述它的返回值的类型*/
    return strcpy; /*返回指向有两个字符指针形参，返回值为字符指针值的函数*/
}
char *strcpy(char *t,char *s)
{
    char *p=t;
    while((*t++=*s++)!='\0') ;
    return p;
}
```

演示： 源程序\ex11_6.c

复杂类型的使用

- 下面进一步就复杂说明：

  int (*fpa[2])(int);

  和int (*(*(*f[2])())[2])(int,int);

  为例，说明复杂类型的使用。
- 对于复杂说明int (*fpa[2])(int);要从理解fpa的含义，对fpa数组的元素进行赋值，以及通过fpa去调用fpa数组元素所指函数三个方面加以把握。
- 由于fpa是一个有两个元素的函数指针数组，因此构造两个函数a1和a2，并通过fpa[0]=a1;和fpa[1]=a2;使fpa[0]指向a1，fpa[1]指向a2。
- 同时，通过表达式fpa[i]((i+1)*5)，当i=0调用函数a1，当i=1调用函数a2，而(i+1)*5是实参。

例11.7 有两个元素的函数指针数组的复杂说明的使用举例。

```
#include "stdio.h"
int a1(int);
int a2(int);
int main(void)              /*fpa是有2个元素的函数指针数组，每个元素指向*/
{   int i,(*fpa[2])(int);/*的函数有一个整型形参，返回整型值*/
    fpa[0]=a1;    /*第1个（下标为0）元素指向a1函数*/
    fpa[1]=a2;    /*第2个（下标为1）元素指向a2函数*/
    for(i=0;i<2;i++)
      printf("%d\n",fpa[i]((i+1)*5));/*依次调用fpa[0]、fpa[1]所指函数*/
    return 0;
}
int a1(int x){ printf("in function a1,x=%d\n",x); return 2*x; }
int a2(int y){ printf("in function a2,y=%d\n",y);  return 2*y; }
```

C语言与程序设计

The C Programming Language

第14章 线性数据结构

毛伏兵

华中科技大学计算机学院

第14章　线性数据结构

- C语言的动态存储分配函数，进行动态数组设计。
- 然后从自引用结构、动态创建结点和相关操作等方面介绍单向链表、双向链表和十字交叉链表。
- 并进一步介绍堆栈、队列，以及基于线性数据结构的深度优先搜索、广度优先搜索等各种算法。
- 这些数据结构和算法在程序设计中起着基础性作用，需要熟练掌握。

14.1 动态存储分配

14.1.1 静态数据结构和动态数据结构

- 到目前为止，教材中介绍的各种基本类型和导出类型的数据结构都是静态数据结构。

  数据结构是计算机存储、组织数据的方式。数据结构是指相互之间存在一种或多种特定关系的数据元素的集合。

- **静态数据结构**指在变量声明时建立的数据结构。在变量声明时变量的存储分配也确定了，并且在程序的执行过程中不能改变。------ **内存固定，连续**

- **动态数据结构**是在程序运行过程中通过调用系统提供的动态存储分配函数，向系统申请存储而逐步建立起来的。在程序运行过程中，动态数据结构所占存储的大小可以根据需要调节，使用完毕时可以通过释放操作将所获得的存储交还给系统供再次分配。------ **内存可变，不连续**

- 数据结构可以分为线性数据结构和非线性数据结构。
- 前者涉及链表、栈、队列等；
- 后者主要涉及树、图等。
- 简单情况下，链表、栈、队列、树、图等数据结构可以用数组等静态数据结构模拟，但作用有限。
- 更一般情况下，它们往往需要调用C的动态存储分配函数向系统申请存储而逐步建立的动态数据结构描述。
- 采用动态存储分配构造的链表、栈、队列等数据结构称为线性动态数据结构；
- 而采用动态存储分配构造的树、图等数据结构称为非线性动态数据结构。

14.1.2 C的动态存储分配函数

- 动态存储分配函数是C的标准函数，函数的原型声明在头文件<stdlib.h>中给出。
- 使用动态存储分配函数必须先使用#include <stdlib.h>编译预处理命令。
- C提供下列与动态存储分配相关的函数。

```
void  * malloc(size_t size);
void  * calloc(size_t n, size_t size);
void  * realloc(void * p_block, size_t size);
void    free(void * p_block);
```

其中，size_t表示unsigned int，即无符号整型。它是在<stdio.h>中通过typedef unsigned size_t;定义的。

C的动态存储分配函数解释

void * malloc(size_t size);

void * calloc(size_t n, size_t size);

- calloc在动态分配完内存后，自动初始化该内存空间为零，而malloc不做初始化，分配到的空间中的数据是随机数据。calloc返回的是一个数组，而malloc返回的是一个对象

void * realloc(void * p_block, size_t size);

- 指针名=（数据类型*）realloc（要改变内存大小的指针名，新的大小）
- 传递给realloc的指针必须是先前通过malloc(), calloc(), 或realloc()分配的

void free(void * p_block);

Alignment(内存地址对齐)

- 一般情况下计算机对内存是按字进行读写的，32位计算机每次读写4字节的一个字，并且都是从0,4,8等4的倍数地址开始读写一个字。
- 一般地，将整型变量从4的倍数地址处对齐存放，其读写速度是最快的。

*14.2 动态数组设计

- 在没有学习动态存储分配之前，如果要设计某班的C语言课程成绩单，首先必须知道人数，并且要按照可能出现的最多人数进行相关数组的声明。**C99支持动态数组**，例如：

```
int n, i;
scanf("%d",&n);
int a[n];
```

可以动态创建大小为n的数组。

- 但是，如果要求学号、姓名等数据项的长度也不必事先规定，而是根据各个班上的实际人数的多少、学号、姓名等数据项的实际长度，来恰当、无冗余地表示，使用动态存储分配函数创建动态数组也许更加合适。

例14.2　用动态存储分配方法设计某班的C语言课程成绩单。

```
#include "stdio.h"
#include "stdlib.h"
#include "string.h"
#include "assert.h"
```

成绩单包含学号字符指针num和姓名字符指针name，它们将分别指向动态分配的学号存储区和姓名存储区。其优点是可以应用于不同长度的学号、姓名，并且根据学号、姓名实际长度分配存储，适应面广，且不会存在多余的空闲存储。

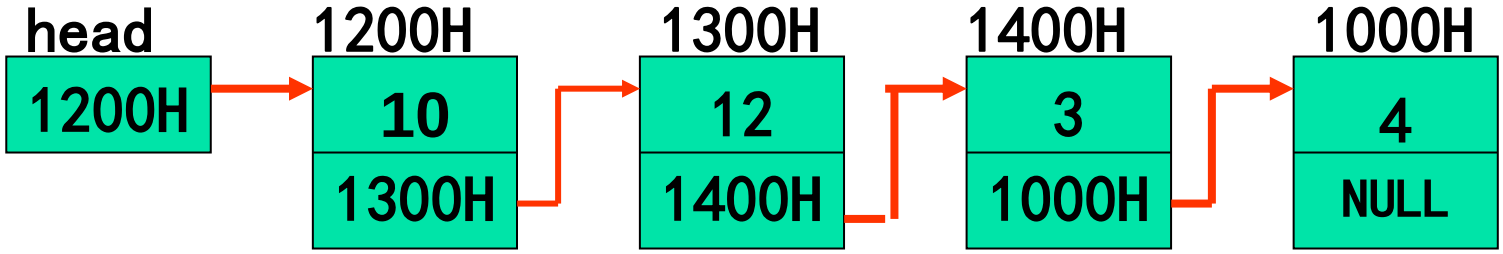
```
struct c_score_tab{
        char *num;              /* 学号 */
        char *name;             /* 姓名 */
        int     c;              /* C语言课程成绩 */
};
```

```
void dynamic_input(char *s1,char **s2)
                    /*字符指针形参s1指向提示性的实参字符串
                      输入字符串到 (*s2) 指向的存储区                */
{       char  pc[40];
        int len;
       printf(s1);    /* 根据需要输出不同的提示 */
        gets(pc);     /* 从键盘接收输入的字符串 */
        len=strlen(pc);    /* 计算输入的字符串的长度 */
        (*s2)=(char *)malloc(len*sizeof(char)+1);
                /*根据输入串长度适当分配存储，
                  动态分配len+1个字节，并由(*s2)指向
                  实际上是有len+1个元素的动态字符数组*/
        assert((*s2));   /*  断言（*s）非零 */
        strcpy((*s2),pc); /* 将输入串拷贝到动态申请的存储区 */
}
```

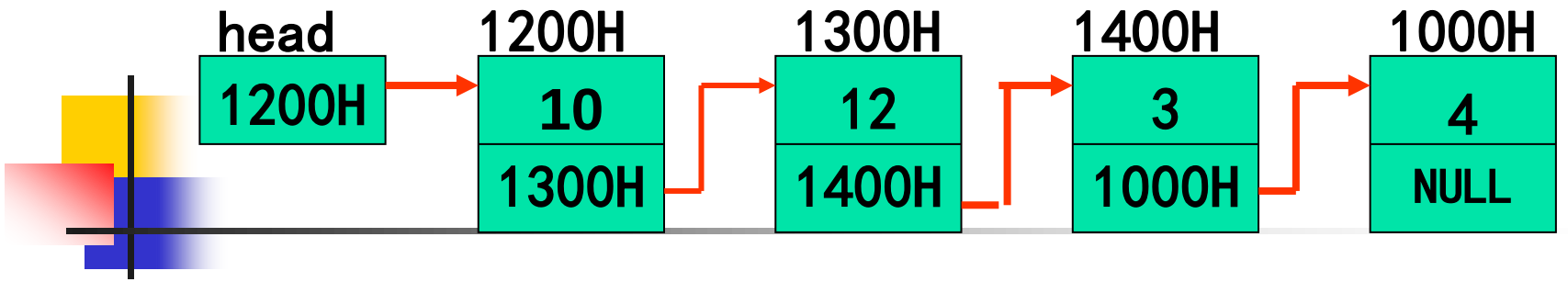
```
void main(void)
{       int n,i;   struct c_score_tab *p;
        printf("input the number of students please!\n");
        scanf("%d",&n);
        getchar(); /* getchar用于读结束scanf输入的回车符 */
        p=(struct c_score_tab *)
            malloc(n*sizeof(struct  c_score_tab));
        assert(p);
        for(i=0;i<n;i++){
            dynamic_input("input num!\n",&(p+i)->num  );
            dynamic_input("input name!\n",&(p+i)->name);
            printf("input score!\n");
            scanf("%d",&(p+i)->c);    /* 输入C语言课程成绩 */
            getchar();
        }
        printf("\n");
        for(i=0;i<n;i++) /* 打印成绩单 */
                printf("unm=%s\tname=%s\tscore=%d\n",
                        (p+i)->num, (p+i)->name, (p+i)->c);
        printf("\n");
        free(p); /* 释放成绩单占据的存储 */
}
```

14.3 链 表

- 链表是一种常用的动态数据结构，它由一系列包含数据域和指针域的结点组成。
- 如果结点的指针域中只包含一个指向后一个结点指针，这种链表称为单向链表。



单向链表结构



单向链表结构

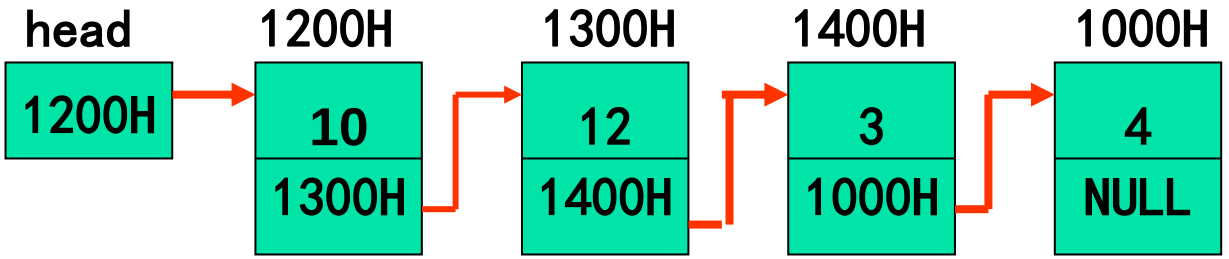
head：头指针，存放一个地址，指向第一个元素

结点：链表中的元素，包括 数据域，用户数据
指针域，放下一个结点的地址

链尾：最后一个结点，指针域为NULL

链表结点的结构类型定义

```
struct s_list{
    int data;
    struct s_list *next;   /*指向该结构自身的指针*/
};
```



头指针说明

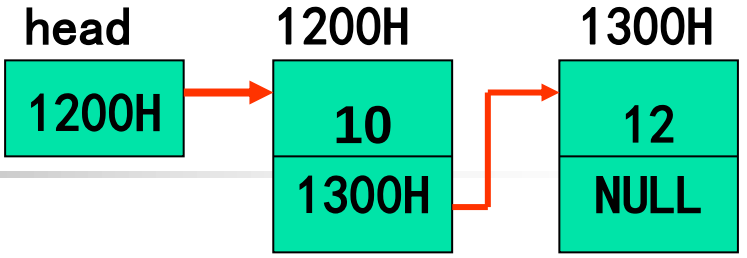
```
struct s_list  *head;
```

14.3.1 自引用结构

如果一个结构中包含一个指向该结构自身的指针，称该结构类型为自引用结构类型

```
struct s_list node1={ 10,NULL};
                /*  node1为自引用结构变量  */
struct s_list  *head;
                /*  head自引用结构类型的指针 */
```

14.3.2 动态创建结点



- 可以通过malloc的方式来动态创建结点。
- 如：

```
head=(struct s_list *)malloc(sizeof(struct s_list)) ;
head->data=10;
head->next=(struct s_list *)malloc(sizeof(struct s_list));
head->next->data=12;
head->next->next=NULL;
```

14.3.3 单向链表

链表的建立

先进后出链（用链表实现栈）：

结点的排列顺序和数的输入顺序相反

先进先出链（用链表实现队列）：

结点的排列顺序和数的输入顺序相同

1. 用循环方式建立先进先出链表

建立一个非空先进先出链表相关算法步骤如下：

（1）声明头指针，尾指针。

```
struct s_list * loc_head=NULL,*tail;
```

（2）创建第一个结点。包括：

① 给第一个结点动态分配存储并使头指针指向它。

```
loc_head=(struct s_list *)malloc(sizeof(struct s_list));
```

② 给第一个结点的数据域中成员赋值。

```
loc_head->data=*p++;
```

③ 使尾指针也指向第一个结点。

```
tail=loc_head;
```

（3）循环建立后续结点

如果没遇到结束标志，进行下列操作：

① 给后继结点动态分配存储并使前驱结点的指针指向它。

```
tail->next=(struct s_list *)malloc(sizeof(struct s_list));
```

② 使尾指针指向新建立的后继结点

```
tail=tail->next;
```

③ 给后继结点的数据域中成员赋值。

```
tail->data=*p++;
```

（4）给尾结点（最后一个结点）的指针赋NULL值。

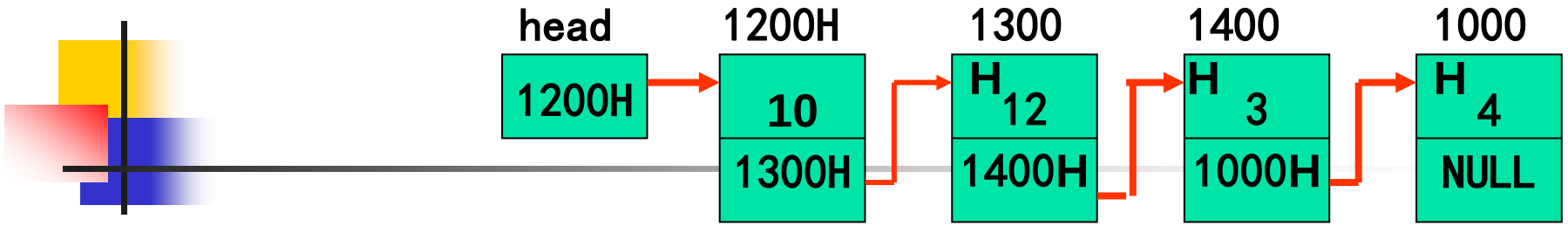
```
tail->next=NULL;
```

```
/*create_list_v1：用循环方式建立先进先出链表 *headp，
  将指针p所指的整数依次存放到链表的每个节点。*/
void create_list_v1(struct s_list **headp, int *p)
{   struct s_list * loc_head=NULL,*tail;
    if(p[0]==0) ；/* 相当于*p==0 */
    else {      /* loc_head指向动态分配的第一个结点 */
        loc_head=(struct s_list *)malloc(sizeof(struct s_list));
        loc_head->data=*p++;       /* 对数据域赋值 */
        tail=loc_head;                /* tail指向第一个结点 */
        while(*p){      /* tail所指结点的指针域指向动态创建的结点 */
            tail->next=(struct s_list *)malloc(sizeof(struct s_list));
            tail=tail->next;      /* tail指向新创建的结点 */
            tail->data=*p++;   /* 向新创建的结点的数据域赋值 */
         }
        tail->next=NULL;     /* 对指针域赋NULL值 */
    }
    *headp=loc_head;     /* 使头指针headp指向新创建的链表 */
}
```

在main函数中调用create_list_v1

```
void main(void)
{
   struct s_list *head=NULL,*p;
   int s[]={1,2,3,4,5,6,7,8,0}; /* 0为结束标记 */
   create_list_v1(&head,s); /* 创建新链表 */
   print_list(head); /* 输出链表结点中的数据 */
}
```

遍历链表的算法步骤



（1）初始化，使遍历指针p指向头结点。 p=head;

（2）如果链表非空（即遍历指针p非空，p！=NULL）

（2-1）输出结点数据域中成员的值。

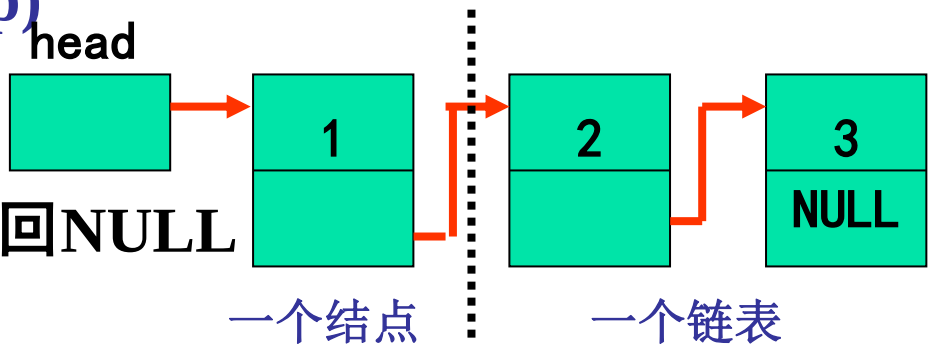
printf("%d\t",p->data);

（2-2）使遍历指针p指向下一个结点。p=p->next;

```
void print_list( struct s_list *head)
 {   struct s_list * p=head;  /*遍历指针p指向链头 */
     while(p){
       printf("%d\t",p->data);
       p=p->next; /*遍历指针p指向下一结点 */
     }
 }
```

2．用递归方式建立先进先出链表

```
/*create_list_v2：用递归方式建立先进先出链表，将指针p所指
    的整数依次存放到链表的每个结点。返回该链表的头指针 */
struct s_list *create_list_v2(int *p)
{
struct s_list * loc_head=NULL;
if(p[0]==0) /* 遇到结束标记，返回NULL
    return NULL;
else { /* loc_head指向新创建的结点 */
    loc_head=(struct s_list *)malloc(sizeof(struct s_list));
    loc_head->data=*p;          /* 对新创建结点的数据域赋值 */
    loc_head->next=create_list_v2(p+1); /* 递归创建下一结点 */
    return loc_head;            /* 返回链头地址 */
 }
}
```



14.3.4 链表的相关操作

- 链表的相关操作包括：
- 创建链表；√
- 遍历链表；
- 在链表中插入一个结点；
- 删除链表中的一个结点；
- 同类型链表的归并；
- 链表的排序等。

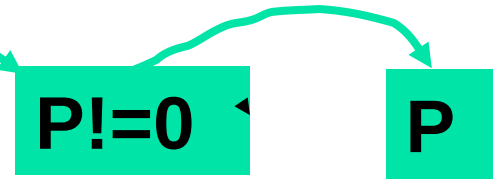
1．遍历链表

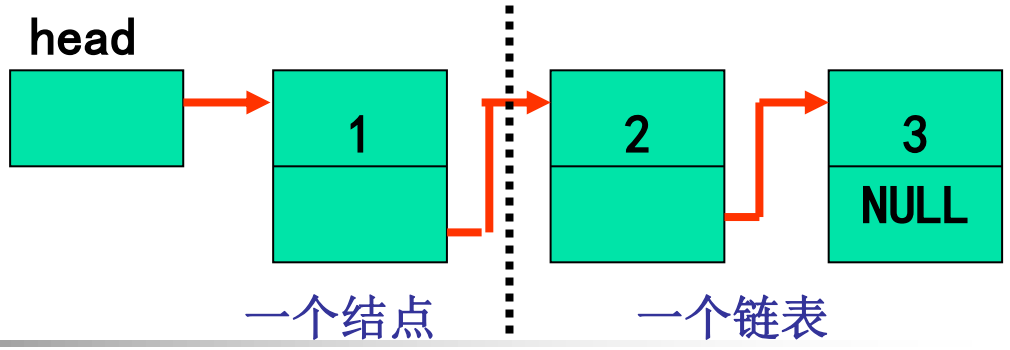
- 遍历链表可以进一步分为：
  - 输出链表中各个结点的数据域成员； √
  - 统计链表中结点的数目；
  - 查找链表中符合条件的某个结点等。

其中输出链表中各个结点的数据域成员已经介绍。

例　用循环遍历的方式统计链表中结点的数目（即链表的长度）。

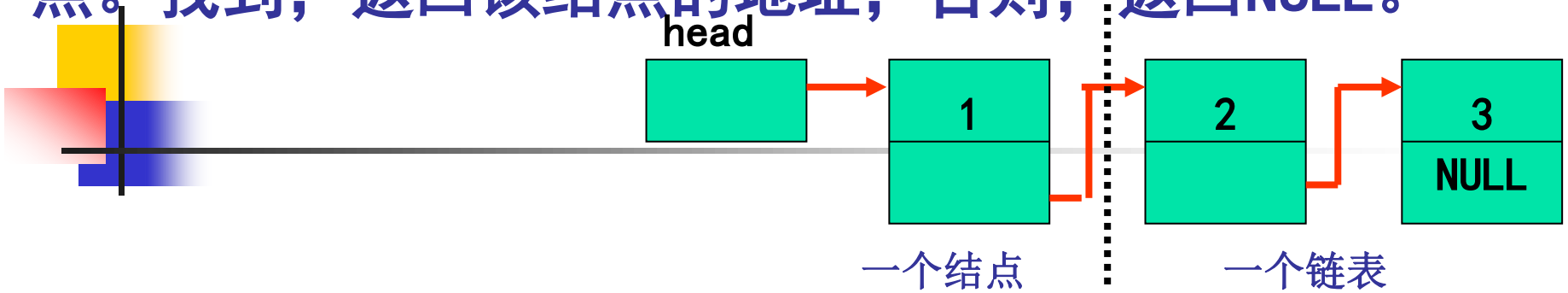
```
/*count_nodes用循环遍历法统计链表head中结点的数目*/
int count_nodes(struct s_list * head)
{
   struct s_list *p=head;
   int num=0;
   while ( p!=NULL ){
        num++;
        p=p->next;
   }
   return num;
}
```






```
/* 用递归法统计链表head中结点的数目 */
int count_nodes_recursive(struct s_list * head)
{
    struct s_list * p=head;
    if(p)
        return (1+count_nodes_recursive(p->next));
    else
        return 0;
}
```

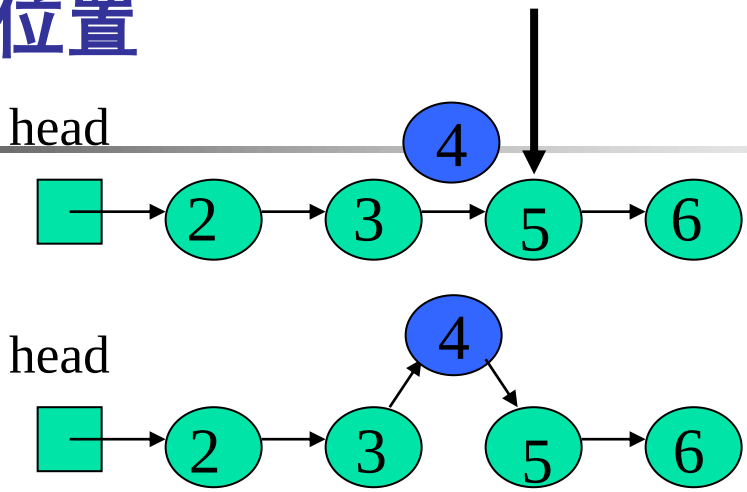
例　用递归法查找链表中数据成员值与形参n相同的结点。找到，返回该结点的地址，否则，返回NULL。




```
struct s_list * find_nodes_recursive(struct s_list * head, int n)
{
    struct s_list * p=head;
    if(p) {  /* 链表非空，查找 */
        if(p->data==n)
            return p; /* 找到，返回该结点的地址 */
        else
            return (find_nodes_recursive(p->next,n));  /* 递归查找 */
    }
    else  return NULL;
}
```

2、插入结点

(1) 寻找插入位置



插入点的位置可能是链头、链尾、或者链中。

插入方式有将加入结点作为插入点的新后继结点和插入点的新前驱结点两种。

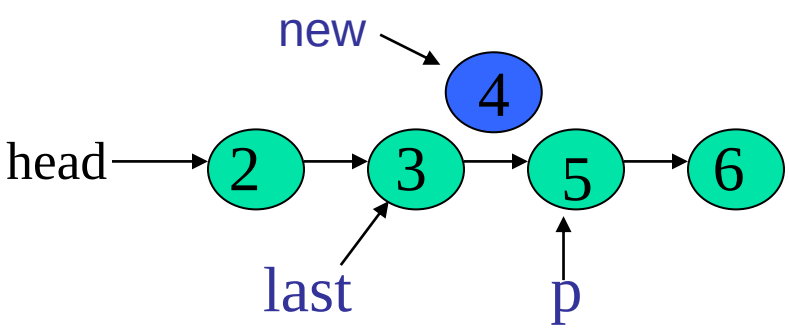
教材介绍将加入结点作为插入点的新后继结点（自学）

本节介绍将加入结点作为插入点的新前驱结点。

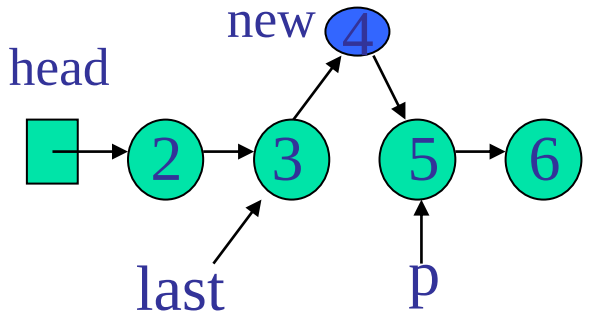
（2）插入结点

插入点是链中（ p 和last之间）

需要两个遍历指针，一个是指向插入点的遍历指针p，另一个是指向插入点前驱结点的指针last。设新结点由new指针所指，它将插入在p和last所指结点之间。



插入前结点间的指向关系

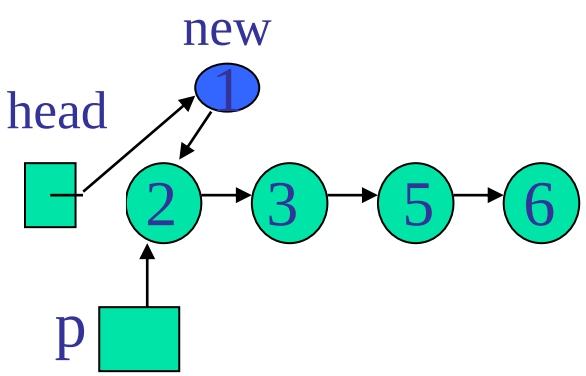


插入后结点间的指向关系

last->next=new;

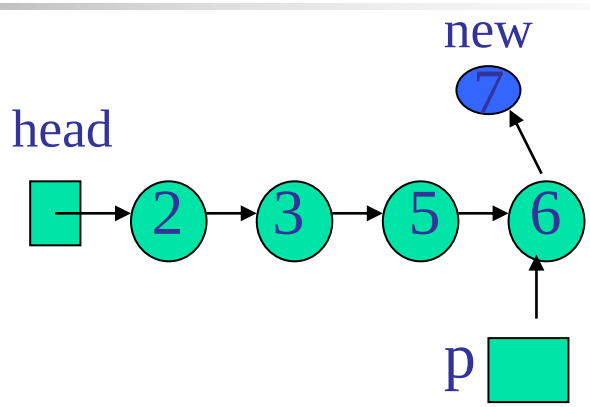
new->next=p;

插入点是链头



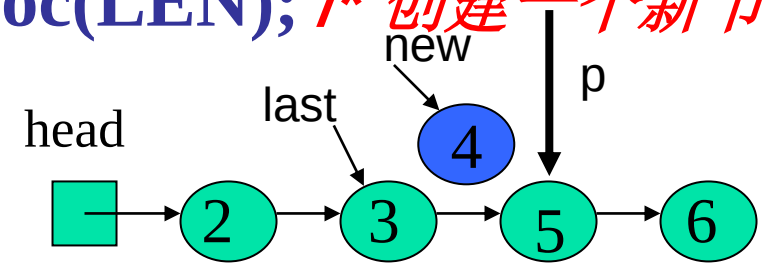
head=new;
new->next=p;

插入点是链尾

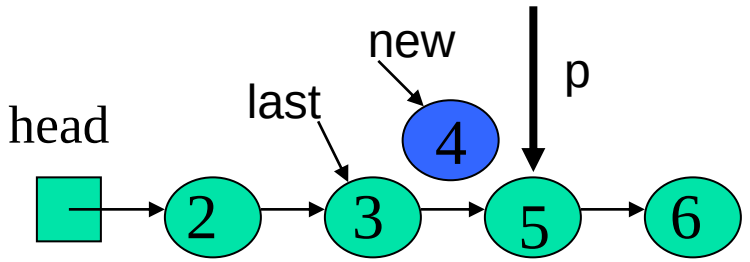


new->next=NULL;
p->next=new;

```
/* insertnode： 在一个升序链表head中插入值为x的结点，返回链表第一个结点的地址。 */
#define LEN  sizeof(struct s_list)
struct s_list *insertnode ( struct s_list *head, int x  )
{
   struct s_list *p, *last，  *new;
   new = (struct  s_list *)malloc(LEN); /* 创建一个新节点 */
   new->data=x;
   p=head;
        /* 寻找插入位置 */
   while( x > p->data  && p->next != NULL )   {
      last=p;
      p=p->next;
    }
```



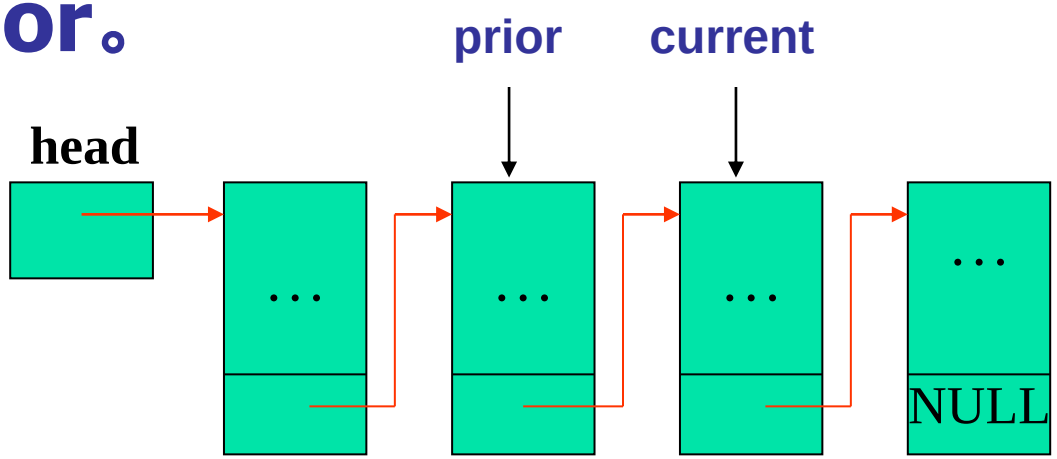
```
if ( x< p->data ) {  /* 插入点不是链尾 */
    if ( p = = head )  head=new;   /* 新节点为链头*/
    else last->next= new;          /* 链中*/
    new->next=p;
  }
  else {      /*新节点为 链尾*/
      new->next=NULL;
      p->next=new;
    }
 return  head;
}
```



3．删除结点

(1)查找被删结点

需要指向被删除结点的遍历指针current和指向被删除结点的前驱结点的遍历指针prior。

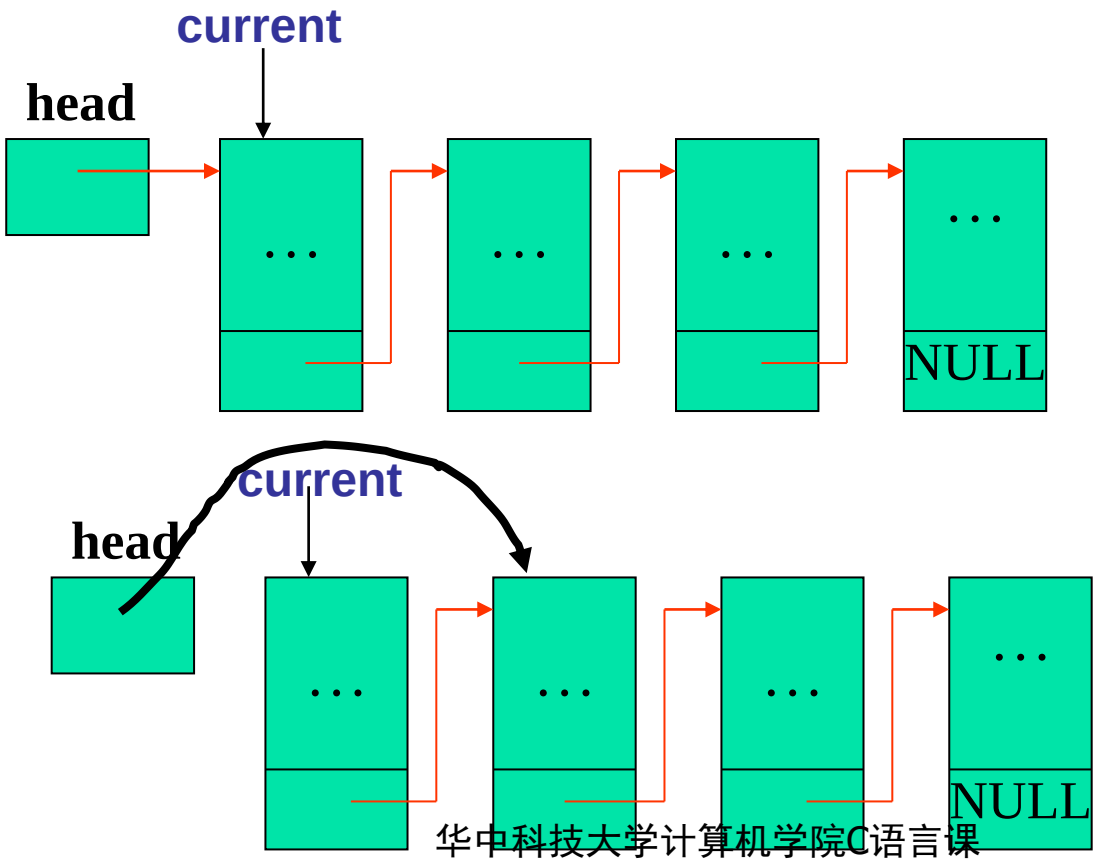


(2) 删除current所指结点

即改变连接关系

链头

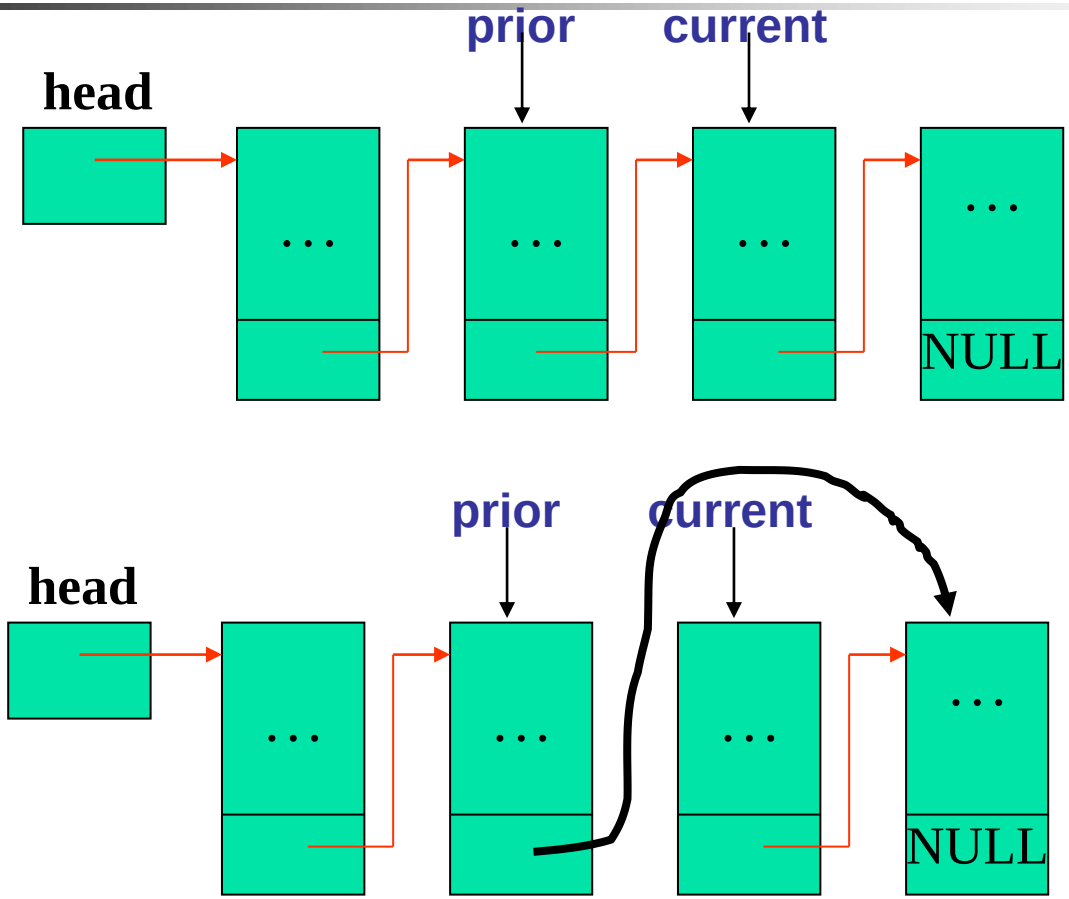
修改 head：head= current ->next



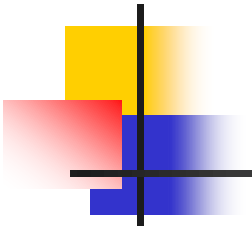
非链头

修改前驱结点的指针域：

prior ->next= current ->next;



(3) 释放 current *所指的存储区*

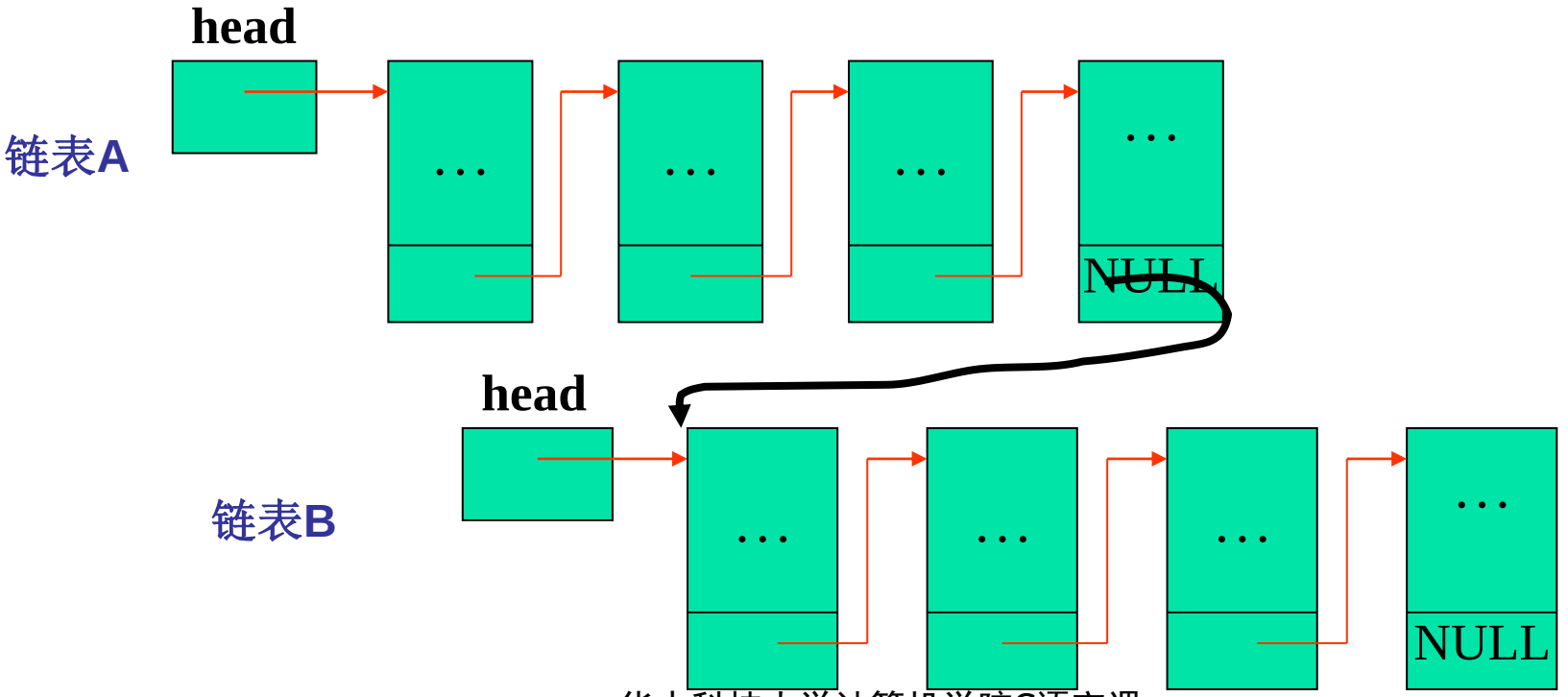


```
free(current);
```

```
/*  查找链表中数据成员的值与形参n相等的结点，如果找到，删除该结点，
    并返回插入点的地址；如果没有找到，返回NULL。*/
struct s_list *delete_nodes(struct s_list **headp,int n)
{   struct s_list * current=*headp,*prior=*headp;
    /* 查找成员值与n相等的结点 */
    while(current !=NULL&&current->data!=n){
        prior=current;   /* prior指向当前结点 */
        current=current->next;   /* current指向下一结点 */
    }
    if(!current)      /*   current==NULL，没有符合条件的结点 */
        return NULL;
    if(current==*headp)   /* 被删结点是链头*/
        *headp=current->next;
    else                                /* 被删结点不是链头 */
        prior->next= current->next;
    free(current);              /* 释放被删结点的存储 */
    return current;
}
```

4. 归并链表

对于非空链表A、B,将链表B归并到链表A，指将链表A的链尾指向链表B的链头所形成的一个新的链表。



归并链表的算法

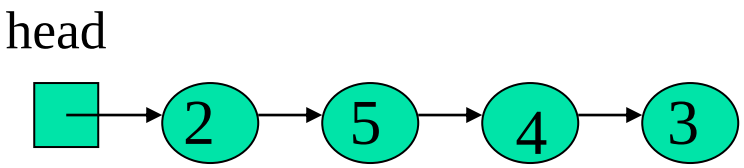
（1）遍历链表A找到其链尾

（2）将链表B的头指针值赋给链表A链尾的指针域

```
void concatenate_lists ( struct s_list *headA,
                         struct s_list *headB   )
{
   struct s_list *current=headA;
   while(current-> next != NULL ) {
         current=current->next;
   }
   current->next=headB;
}
```

5. 链表排序

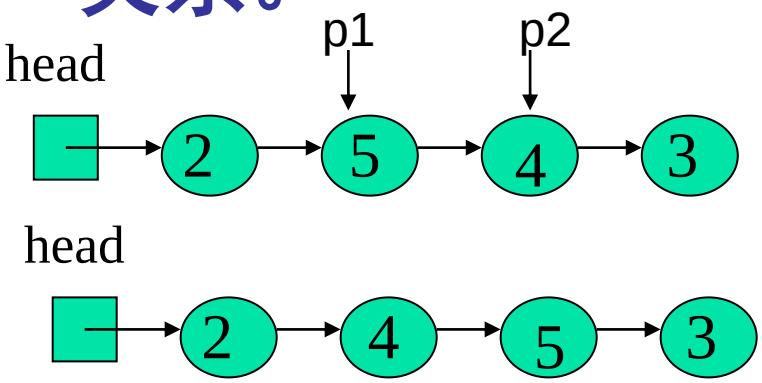
- 链表排序是指将链表的结点**按某个数据域**从小到大（升序）或从大到小（降序）的顺序连接。



两种方法

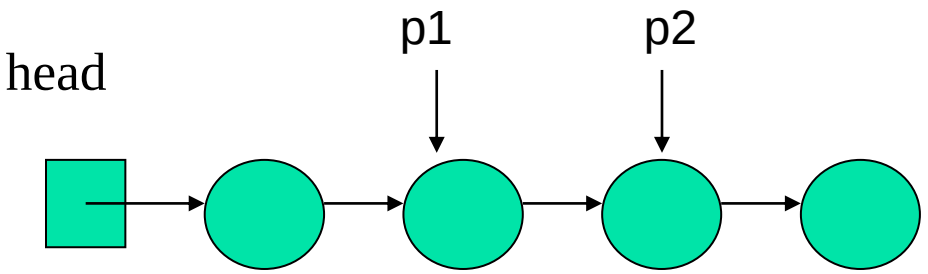
排序中对链表结点的交换有**两种方法**，

（1）**交换结点的数据域**，不改变结点间的指向关系。



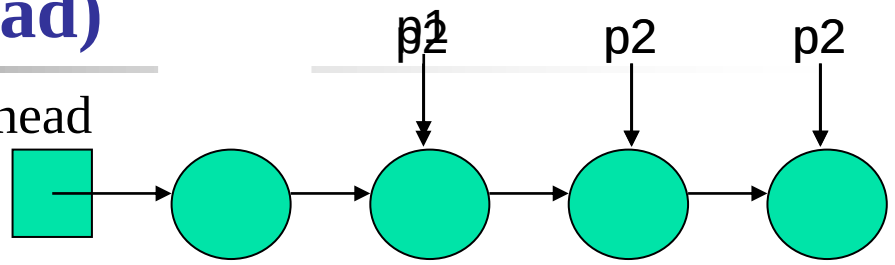

```
if(p1->data > p2->data){
    int t;
    t=p1->data; /* 交换数据域 */
    p1->data=p2->data;
    p2->data=t;
}
```

```
struct std {
        char num[12];
        char name[9];
        int score;
};
struct  s_list {
    struct std  data;
    struct  s_list *next;
};
if(p1->data->score > p2->data->score) {
    struct std  t;
    t=p1->data; /* 交换数据域 */
    p1-> data =p2-> data;
    p2-> data =t;
}
```



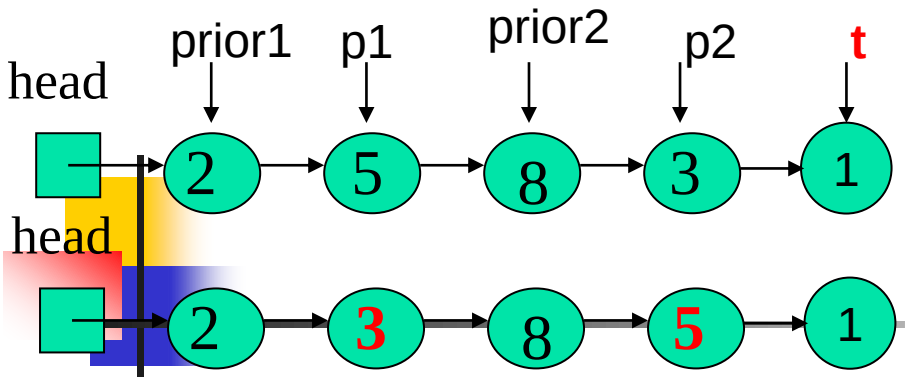
例 写一个函数，采用交换结点数据域的方法实现对链表的升序排序。

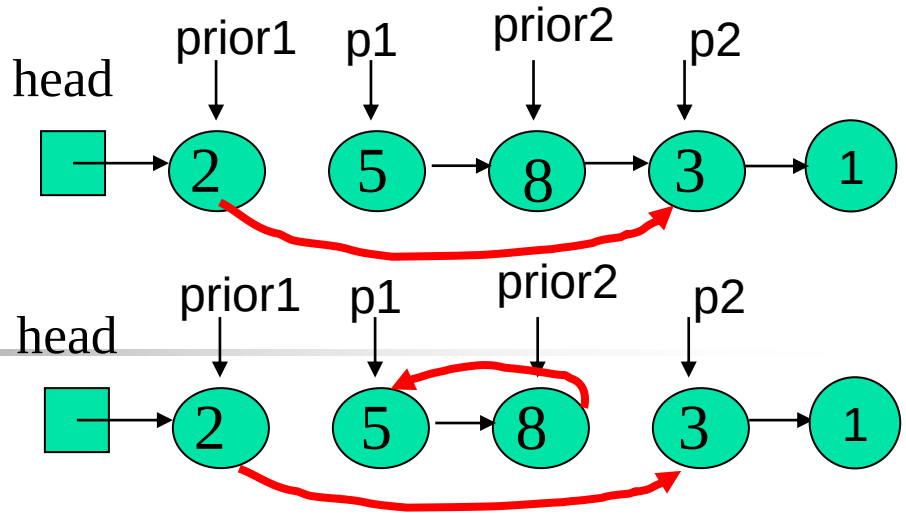
```
void sort_lists(struct s_list *head)
{
   struct s_list *p1=head,*p2;
   int t;
   for(p1=head;p1!=NULL;p1=p1->next)
        for(p2=p1->next;p2!=NULL;p2=p2->next)
            if(p1->data > p2->data){
               t=p1->data;                    /* 交换数据域 */
               p1->data=p2->data;
               p2->data=t;
            }
}
```



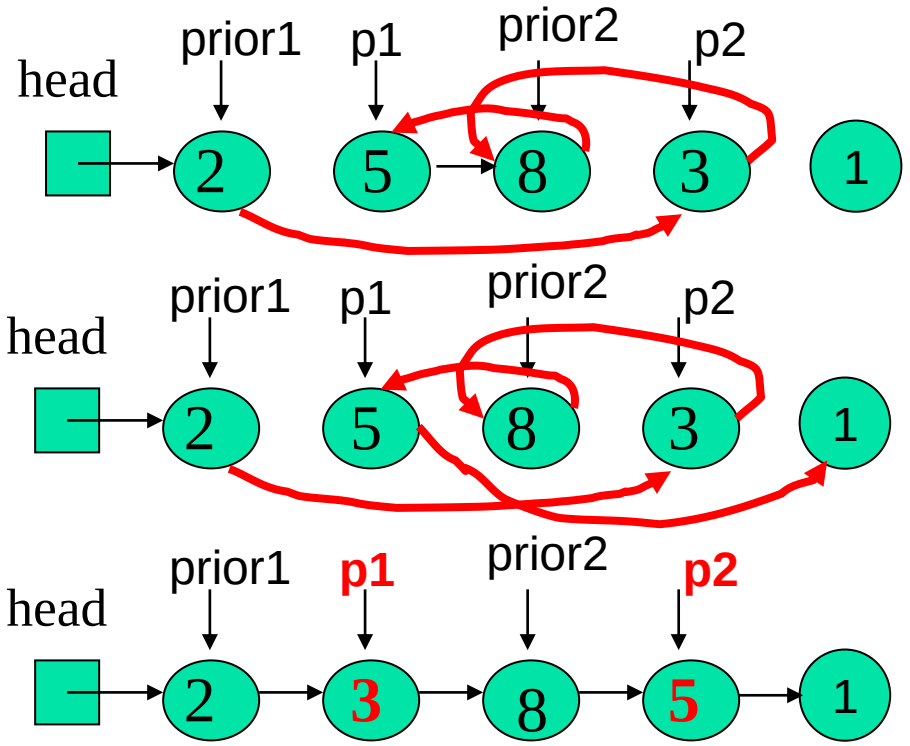
每轮得到最小的值（在最开始的位置上）

- **(2) 改变结点的连接，操作较为复杂。**
- **在数据域较为简单的情况下，多采用交换结点的数据域的方法进行链表排序。**
- **在数据域较为复杂，成员较多，采用改变链表结点之间的连接实现结点的交换的排序方法则效率较高。**



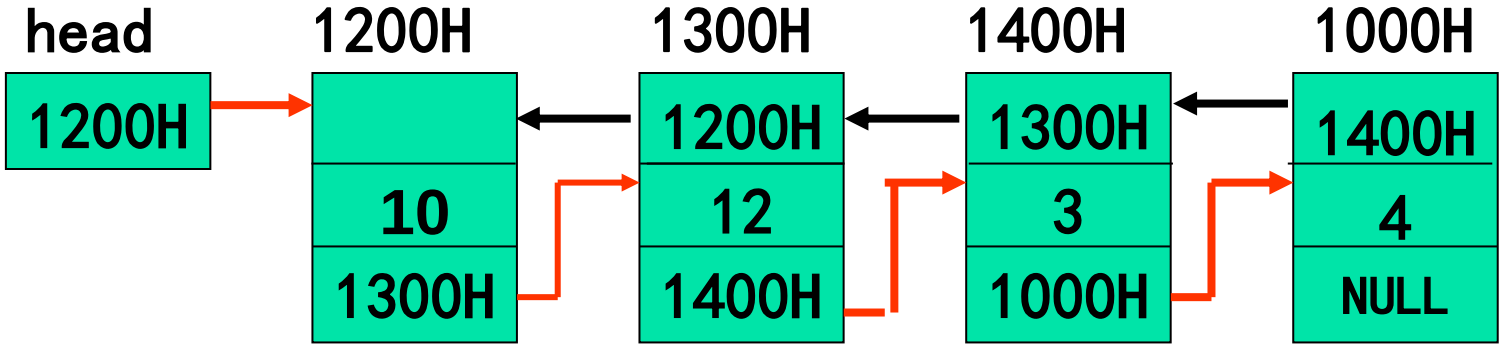



```
if(p1->data>p2->data){
        t=p2->next;
        prior1->next=p2;
        prior2->next=p1;
        p2->next=p1->next;
        p1->next=t;
        p2=p1;
        p1=prior1->next;
}
```



14.3.5 双向链表

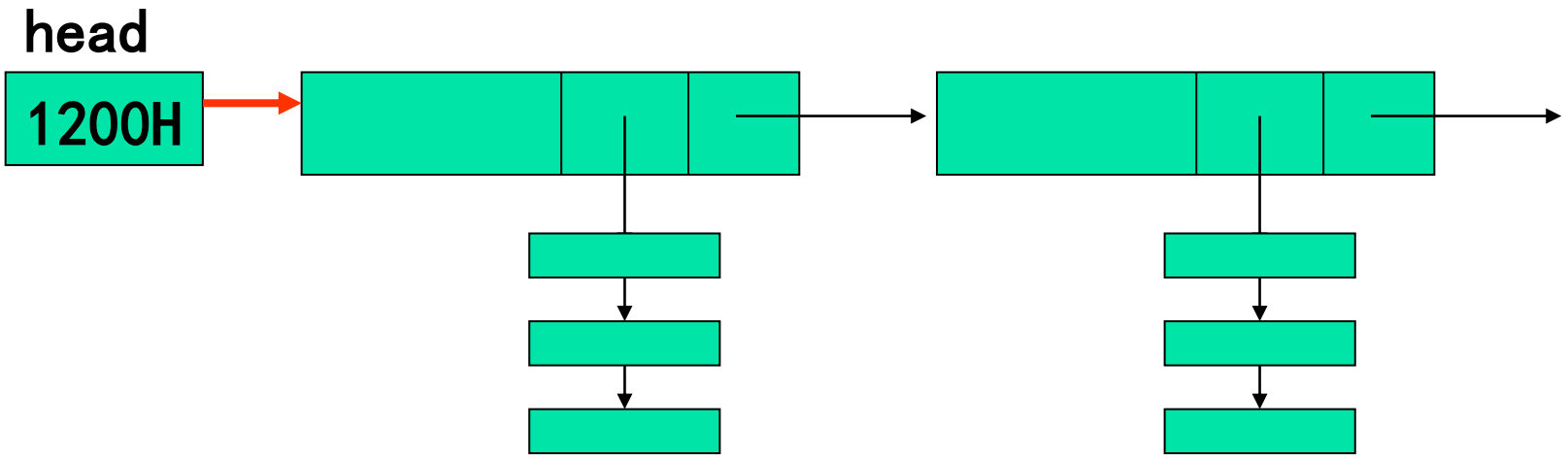
- 如果结点的指针域包含两个指针，且一个指向前一个结点，另一个指向后一个结点，这种链表称为**双向链表**。



双向链表结构

14.3.6 十字交叉链表

- 如果结点的指针域包含两个指针，且一个指向后一个结点，另一个指向另外一个链表，这种链表称为**十字交叉链表**。



十字交叉链表结构

例14.18 利用十字交叉链表和fread、fwrite函数实现学生基本信息和学生学习成绩的录入，遍历，存盘，加载等操作功能。

学生基本信息表

学号	姓名	性别	年龄	家庭住址	联系电话	备注
0001	aaa	m	18	hubei,wuhan	12345678	Abc def
0001	bbb	f	18	hunan,changsha	76545678	Aaaaa bbb
…						
00xx	zzz	m	19	hubei,honghu	32145678	Nnn kkk

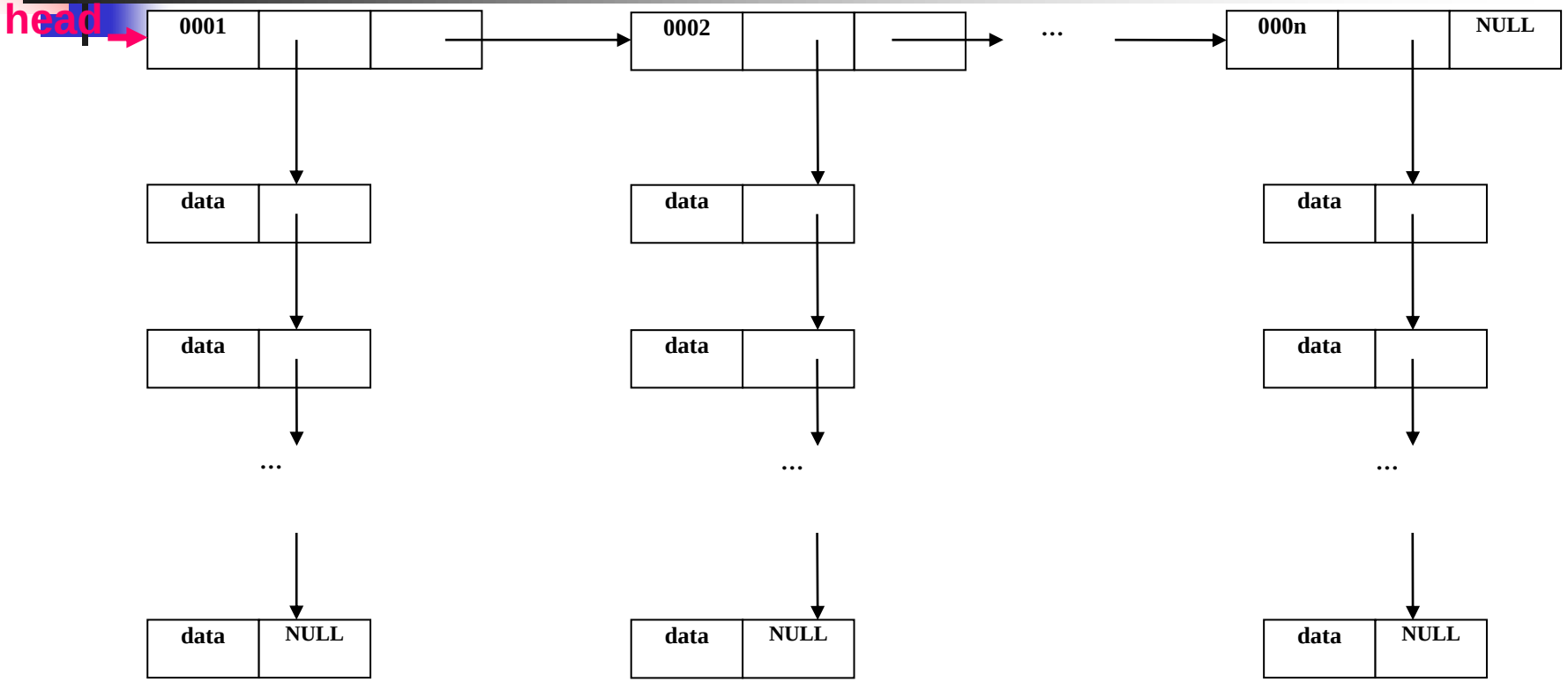
学生学习成绩表

学号	高等数学	普通物理	电工基础	演讲与口才	欧洲文化	论语初探
0001	86	85	73	×	80	×
0002	77	83	76	82	87	75
0003	87	82	81	×	×	86
…						
00xx	89	87	85	×	×	×

数据结构需求分析

- 用结构数组，编程处理比较方便。但存在明显不足：
- 1.学生基本信息表中的备注项长度越长，能够记载的内容就越丰富，但多数人没有备注项就会浪费大量存储。如果长度取得过短，描述能力有限，有时不能满足记载要求。
- 2. 学生选修的课程不一样，有的选课门数多，有的选课门数少；如果采用数组就必须按照选课最多的同学考虑数组的大小。同时，对课程设置的变化缺乏适应能力。编好的程序这学期可能能够满足要求，如果下个学年课程设置变了，程序也许需要进行修改。
- 3. 处理学生的增减、课程的增减效率很低。
- 综上所述，用十字交叉链表比较合理。十字交叉链表中由学生基本信息构成一个单向链表，链表中的每个结点描述一个同学的基本信息；同时，每个结点都有一个头指针，它指向该生的学习成绩链。

十字交叉链表



水平方向:学生基本信息链;　　垂直方向:各学生课程成绩链

学生课程成绩结构类型courses的定义

该类型的结构变量可以构成纵向单向链表的结点。

```
typedef struct score_tab{
    char        num[5];          /* 学号 */
    char        course[20];      /* 课程名称 */
    int         score;           /* 成绩 */
    struct      score_tab *   next;
} courses;
```

学生基本信息结构类型studs的定义

该类型的结构变量可以构成十字交叉链表中横向学生基本信息链的结点。

```
typedef struct student_tab{
    char    num[5];              /* 学号 */
    char    name[10];            /* 姓名 */
    char    sex;                 /* 性别 */
    int     age;                 /* 年龄 */
    char    addr[30];            /* 家庭住址 */
    char    phone[12];           /* 联系电话 */
    char*   memo;                /* 备注字段 */
    courses  *head_score;        /* 指向成绩链的头指针 */
    struct   student_tab   *next; /* 指向下一个结点的自引用指针 */
} studs;
```

函数原型声明

```
/* 输入数据并创建十字交叉链表 */
void create_cross_list(studs **head);
 /* 遍历十字交叉链表 */
void traverse_cross_list(studs *head);
/* 将十字交叉链表中结点数据存盘 */
void save_cross_list(studs *head);
/*读入已经保存的数据，创建新的十字交叉链表 */
void load_cross_list(studs **head);
```

十字交叉链表的创建

- 算法思路：先创建学生基本信息链，创建完毕后通过(*head) = hp使main函数中的头指针指向学生基本信息链；然后遍历学生基本信息链，对链每个结点，询问用户是否创建该结点对应学生的课程成绩链，如果得到肯定应答，则以该结点的head_score指针为该生课程成绩链的头指针，并创建该生的课程成绩链。因此，当遍历学生基本信息链后，每个学生的课程成绩链也相应创建完毕，十字交叉链表动态创建完毕。学生基本信息链和课程成绩链都是后进先出链（栈式链），即头指针始终指向最后加入的结点。另外，由于需要修改头指针，因此必须传递头指针的地址。

```
void create_cross_list(studs **head)    /* 创建十字交叉链表的函数 */
{
    studs *hp=NULL,*p;
    courses * pcrs;
    char s[80]="",ch;
loop:
    p = (studs *)malloc(sizeof(studs));  /* 创建学生基本信息的第一个结点 */
    printf("input num,name,sex,age,addr,phone\n");
    scanf("%s %s %c %d %s %s",p->num,p->name,&p->sex,&p->age,
                              p->addr,p->phone);/* 输入各项基本数据 */
    getchar(); /* 专门用于读scanf 输入中的换行符 */
    p->head_score = NULL; /* 置成绩链头指针为NULL */
    p->next = hp; /* 学生基本信息链头指针值赋给结点的next 域 */
    hp = p; /* 头指针指向新创建的结点 */
    printf("continue input data of students? yes or no\n");
    ch = getchar();        getchar(); /*是否继续创建学生基本信息结点 */
    if(ch == 'y' || ch =='Y')
        goto loop; /* 是，创建下一个学生基本信息结点 */
    (*head) = hp; /* 调用函数中的head 头指针指向新创建的学生基本信息链 */
```

```
    while(p != NULL){ /* 循环创建每个学生的学习成绩链 */
        printf("input %s student's score? yes or no?\n",p->num);
        ch = getchar(); getchar();
        while(ch == 'y' || ch =='Y'){
            pcrs = (courses *)malloc(sizeof(courses));/* 创建成绩结点 */
            printf("input course and score\n");
            scanf("%s %d", pcrs->course,&pcrs->score);/* 输入课程名和成绩 */
            getchar();  /* fflush(stdin);  */
            strcpy(pcrs->num, p->num); /* 学号成员赋值 */
            pcrs->next = p->head_score; /* 头指针值赋给新结点的指针域 */
            p->head_score = pcrs; /* 头指针指向新结点 */
            printf("input %s student's score? yes or no?\n",p->num);
            ch = getchar(); getchar();/*是否继续创建成绩结点 */
        }
       p = p->next; /* p指向学生基本信息链的下一个学生 */
    }
}
```

十字交叉链表的遍历

算法思路：首先将头指针值赋给学生基本信息链的遍历指针p；先遍历学生基本信息链，在遍历过程中，对链中每个结点，先输出结点数据域的内容，然后从该结点的head_score开始，将p->head_score赋给学生课程成绩链的遍历指针pcrs，让pcrs从课程成绩链的链头开始遍历课程成绩链的每个结点并输出课程成绩链结点数据域的内容。因此，当学生基本信息链遍历完毕时，链中每个结点指向的课程成绩链也全部遍历完毕。

```
void traverse_cross_list(studs *head) /* 遍历十字交叉链表的函数 */
{
    studs * p = head;
    courses  * pcrs;
    while(p != NULL){ /* 遍历学生基本信息链 */
        printf("%s\t%s\t%c\t%d\t%s\t%s\n",p->num,p->name,p->sex,
                p->age,p->addr,p->phone);
        pcrs = p->head_score;
        while(pcrs != NULL){ /* 遍历学生成绩链 */
          printf("%s\t%s\t%d\n",pcrs->num,pcrs->course,pcrs->score);
          pcrs = pcrs->next;
        }
        p =p->next;
    }
}
```

学生基本信息和学生课程成绩的保存

创建两个数据文件student.dat和score.dat,
学生基本信息保存在student.dat中,
学生课程成绩保存在score.dat中

（1）说明两个文件指针out1 和 out2

两个遍历指针p（遍历学生基本信息链）
pcrs（遍历学生课程成绩链）

（2）以二进制写方式打开两个数据文件

（3）对十字交叉链表进行遍历。

对学生基本信息链的每个结点，

①将p指向的结点内容写入文件student.dat；

②pcrs = p->head_score将该结点指向的学生课程成绩链的头指针赋给pcrs

③让pcrs遍历学生课程成绩链，将pcrs指向的学生课程成绩链结点内容写入到score.dat文件

（4）关闭两个文件

```
void save_cross_list(studs *head) /* 保存十字交叉链表的函数 */
{
    FILE *out1,*out2;
    studs * p = head;
    courses  * pcrs;
    if((out1 = fopen("c:\\student.dat","wb")) == NULL) /* 打开基本信息文件 */
        exit(-1);
    if((out2 = fopen("c:\\score.dat","wb")) == NULL) /* 打开成绩文件 */
        exit(-1);
    while(p != NULL){
        fwrite(p,sizeof(studs),1,out1);/* 写学生基本信息记录 */
        pcrs = p->head_score; /* 成绩遍历指针指向成绩链链头 */
        while(pcrs != NULL){ /* 遍历学生成绩链 */
                fwrite(pcrs,sizeof(courses),1,out2); /* 写成绩记录 */
                pcrs = pcrs->next; /* 指向下一个成绩结点 */
        }
        p =p->next; /* 指向下一个学生基本信息结点 */
    }
    fclose(out1); /* 关闭基本信息文件 */
    fclose(out2); /* 关闭成绩文件 */
}
```

读取student.dat和score.dat，构造十字交叉链表

（1）说明两个文件指针in1 和 in2
两个遍历指针p（遍历学生基本信息链）
pcrs（遍历学生课程成绩链）

（2）以二进制读方式打开两个数据文件

（3）读学生基本信息文件student.dat，创建横向的学生基本信息链

（4）创建课程成绩链的结点并使pcrs指向它，再从score.dat文件中读一条记录并将其赋给pcrs所指向的结点；然后遍历学生基本信息链，查找链中学号值与pcrs所指向结点的学号值相等的结点。找到后，以该结点的head_score指针成员为课程成绩链的头指针，以后进先出的方式将课程成绩新结点加入到以head_score为头指针的课程成绩链中。

```
void load_cross_list(studs **head) /* 读入十字交叉链表的函数 */
{
    FILE *in1,*in2;
    studs *hp=NULL,*p;
    courses  * pcrs;
    if((in1 = fopen("c:\\student.dat","rb")) == NULL) /* 打开基本信息文件
    */
        exit(-1);
    if((in2 = fopen("c:\\score.dat","rb")) == NULL) /* 打开成绩文件 */
        exit(-1);
    while(!feof(in1)){
        p = (studs *)malloc(sizeof(studs)); /* 创建基本信息结点 */
        fread(p,sizeof(studs),1,in1); /* 读一条基本信息记录到结点中 */
        if(!feof(in1)){
            p->head_score = NULL; /* 置成绩链头指针为NULL */
            p->next = hp; /* 学生基本信息链头指针值赋给结点的next 域 */
           hp = p;         /* 头指针指向新创建的结点 */
        }
    }
```

```
    (*head) = hp; /* 调用函数中的head 头指针指向新创建的学生基本信息链 */
    while(!feof(in2)){
        pcrs = (courses *)malloc(sizeof(courses)); /* 创建新成绩结点 */
        fread(pcrs,sizeof(courses),1,in2); /* 读一条成绩记录到新结点 */
        if(!feof(in2)){
            p = (*head);/*遍历指针p指向学生基本信息链的链头 */
            while(p != NULL){ /*遍历学生基本信息链 */
                if(!strcmp(p->num , pcrs->num)){ /* 查找相同学号的结点 */
                    pcrs->next = p->head_score; /* 找到，新结点加入成绩链 */
                    p->head_score = pcrs;
                    break;
                 }
                else
                    p = p->next; /* 没找到，转下一结点 */
            }
        }
    }
    fclose(in1); /* 关闭基本信息文件 */
    fclose(in2); /* 关闭成绩文件 */
}
```